UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

Macroarea di Ingegneria



TESI DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

Difesa Adversarial per Sistemi di Rilevamento delle Intrusioni di Rete basati su Modelli di Deep Learning: un Approccio Layer-wise

Relatore:
Dott. Gabriele Russo Russo

Correlatore:
Prof. Giuseppe Bianchi

Laureando:
Alessandro Finocchi
matricola: 0340543

Anno Accademico 2025/2026

A chi ha creduto in me quando io non ci credevo.

Alla mia famiglia, per i suoi sacrifici e il supporto costanti.

Ai miei amici, che vedevano in me più di quanto io riuscissi a vedere.

A me stesso e alla mia determinazione, al coraggio del passato e ai sogni del futuro.

Indice

1 Introduzione **1**

1.1 Contesto e motivazioni 1

1.2 Obiettivi e contributi 2

1.3 Struttura della tesi 4

2 Background teorico **5**

2.1 Network Intrusion Detection Systems (NIDS) 5

2.1.1 Collocazione architetturale e tassonomia 5

2.1.2 Paradigmi di rilevamento 6

2.1.3 Granularità dell'analisi: dal pacchetto al flusso 7

2.1.4 Formulazione del task di apprendimento 8

2.1.5 Vincoli di dominio 8

2.1.6 Vincoli di deployment 9

2.2 Reti Neurali Artificiali e Rappresentazioni Latenti 9

2.2.1 Dal percettrone alle architetture feedforward 10

2.2.2 Propagazione in avanti e rappresentazioni latenti 11

2.2.3 Funzioni di attivazione 12

2.2.4 Livello di uscita e funzione di loss 13

2.2.5 L'addestramento come problema di ottimizzazione 13

2.2.6 Discesa del gradiente e varianti stocastiche 14

2.2.7 Algoritmo di addestramento: Backpropagation 15

2.2.8 Stabilità della propagazione e implicazioni per la robustezza 16

2.3 Adversarial Machine Learning e Attacchi di Evasione 18

2.3.1 Origine e natura del fenomeno . . . 18
2.3.2 Tassonomia e modello delle minacce . . . 18
2.3.3 Attacchi white-box . . . 20
2.3.4 Attacchi black-box e trasferibilità . . . 21
2.3.5 Panorama delle contromisure . . . 22
2.3.6 Vincoli del dominio nei flussi di rete . . . 24
2.4 L'Intersezione dei Tre Ambiti . . . 24

3 Stato dell'arte **27**
3.1 Deep Learning nei Network Intrusion Detection Systems . . . 27
3.1.1 Modelli feed-forward e classificazione di istanze . . . 28
3.1.2 Approcci semi-supervisionati e non supervisionati . . . 28
3.1.3 Modelli convoluzionali . . . 29
3.1.4 Modelli ricorrenti e classificazione di sequenze . . . 29
3.1.5 Rappresentazioni a grafo . . . 30
3.1.6 Transformer, pre-training e rappresentazioni trasferibili . . . 30
3.1.7 Sintesi critica . . . 30
3.2 Adversarial Machine Learning . . . 31
3.2.1 L'escalation tra attacchi e difese . . . 31
3.2.2 Il paradigma black-box dominante e i suoi limiti . . . 32
3.2.3 Verso una prospettiva layer-wise . . . 32
3.2.4 Detection multi-livello . . . 33
3.2.5 Estensione del problema ad altre modalità . . . 34
3.2.6 Sintesi del gap individuato . . . 34

4 Dataset **35**
4.1 Criteri di selezione dei dataset . . . 35
4.2 Criticità dei dataset per il rilevamento delle intrusioni . . . 38
4.3 UNSW-NB15 . . . 39
4.4 CIC-IDS2017 . . . 41
4.5 CSE-CIC-IDS2018 . . . 42
4.6 CTU-13 . . . 43

4.7 Pipeline di preprocessing . . . 45
4.7.1 Lettura e pulizia delle catture . . . 46
4.7.2 Fusione, valori non validi e duplicati . . . 46
4.7.3 Partizionamento . . . 47
4.7.4 Trasformazione delle distribuzioni a coda pesante . . . 47
4.7.5 Standardizzazione . . . 48
4.7.6 Feature perturbabili . . . 49

5 Metodologia e modellazione delle soluzioni 51

5.1 Modello della rete . . . 52
5.1.1 Formulazione del problema . . . 52
5.1.2 Architettura della backbone . . . 53
5.1.3 Rappresentazione osservata . . . 53
5.1.4 Livelli estesi e stato del flusso . . . 54
5.2 Metrica di regolarizzazione layer-wise . . . 55
5.2.1 Definizione della Single Layer Adversarial Vulnerability . . . 55
5.2.2 Interpretazione . . . 56
5.2.3 Ruolo della metrica nella metodologia . . . 57
5.3 Estensione dei livelli della rete . . . 58
5.3.1 Architettura biforcata per detection multi-layer . . . 58
5.3.2 Architetture per la manipolazione geometrica dello spazio delle attivazioni . . . 62
5.3.3 Flusso dei gradienti . . . 67
5.4 Definizione degli attacchi adversarial . . . 80
5.4.1 Modello di minaccia . . . 80
5.4.2 Fast Gradient Sign Method . . . 81
5.4.3 Projected Gradient Descent . . . 81
5.4.4 PGD adattivo . . . 82
5.5 Dettagli di addestramento . . . 83
5.5.1 Funzione di loss complessiva . . . 83
5.5.2 Configurazioni di modello . . . 84
5.6 Metriche di valutazione . . . 84

6 Esperimenti e Analisi dei Risultati **87**

6.1 Domande di Ricerca . . . 87

6.2 Stadio 1: baseline e test di V&V . . . 88

6.2.1 Impostazione sperimentale . . . 88

6.2.2 Risultati delle configurazioni . . . 89

6.2.3 Suite di verifica e validazione . . . 94

6.2.4 Sintesi dello stadio 1 . . . 101

6.3 Stadio 2: esplorazione delle architetture . . . 101

6.3.1 Impostazione sperimentale . . . 102

6.3.2 Criterio di selezione . . . 103

6.3.3 Effetto dei fattori architetturali . . . 104

6.3.4 Risultati per famiglia . . . 106

6.3.5 Architetture selezionate . . . 109

6.3.6 Sintesi dello stadio 2 . . . 112

6.4 Stadio 3: flusso dei gradienti e modalità di detection . . . 113

6.4.1 Impostazione sperimentale . . . 113

6.4.2 Regime del flusso dei gradienti . . . 114

6.4.3 Politica di aggregazione degli score . . . 117

6.4.4 Configurazioni selezionate . . . 117

6.4.5 Sintesi dello stadio 3 . . . 117

6.5 Stadio 4: ottimizzazione della loss complessiva . . . 118

6.5.1 Impostazione sperimentale . . . 118

6.5.2 Loss del task sui campioni adversarial . . . 120

6.5.3 Peso della loss di detection . . . 120

6.5.4 Peso e margine della loss repulsiva . . . 121

6.5.5 Peso della loss attrattiva . . . 122

6.5.6 Configurazioni selezionate . . . 122

6.5.7 Sintesi dello stadio 4 . . . 123

6.6 Stadio 5: analisi del caso peggiore . . . 123

6.6.1 Impostazione sperimentale . . . 124

6.6.2 Il caso peggiore al budget di addestramento . . . 124

6.6.3 Effetto dei parametri dell'attacco 126
6.6.4 Robustezza al variare del budget 127
6.6.5 Sintesi dello stadio 5 129
6.7 Risposte alle domande di ricerca 129

7 Conclusioni e sviluppi futuri **133**
7.1 Considerazioni finali 133
7.2 Limiti e sviluppi futuri 134

Bibliografia **140**

Elenco delle figure

4.1 Distribuzione delle categorie d'attacco in UNSW-NB15 40

4.2 Distribuzione delle categorie d'attacco in CIC-IDS2017 42

4.3 Distribuzione delle categorie d'attacco in CSE-CIC-IDS2018 43

4.4 Distribuzione delle categorie d'attacco in CTU-13 45

5.1 Class diagram delle estensioni dei moduli PyTorch definiti in `lwad_wrapper.py`. `LWADSequential` incapsula automaticamente ogni `nn.Module` non esteso in un `PassThrough`. 59

5.2 Architettura biforcata nel caso in cui tutti i livelli nascosti siano estesi. Le probabilità stimate dai detector vengono aggregate in un unico sistema di allarme. 60

5.3 Rappresentazione schematica delle due strategie nello spazio delle attivazioni del livello l. A sinistra, la loss attrattiva avvicina tra loro le attivazioni pulite (•) e adversarial (◦), riducendone la distanza rapportata alla scala delle attivazioni pulite. A destra, la loss repulsiva le allontana, finché la stessa distanza non raggiunge il margine. 62

6.1 Score di test medio $\bar{S}$ in funzione del numero di parametri, per tutte le 121 configurazioni dello stadio 2. 110

6.2 $\bar{S}$ di ciascuna configurazione nei due regimi del flusso dei gradienti, per taglia e politica di aggregazione. Le barre indicano la deviazione standard delle medie per seed. 116

6.3 $\bar{S}$ in funzione dei pesi della loss, per taglia: (b) e (c) riportano FurtherAL con PGD adattivo e, tratteggiato, con PGD; in (d) $\lambda_{act} = 0$ è l'adversarial training. Nei modelli con detector la loss del task è calcolata sui soli campioni puliti. 121

6.4 Accuratezza end-to-end robusta dei modelli con detector contro l'attacco adattivo con $\varepsilon = 0.2$, in media su taglie e dataset; la legenda indica l'attacco di addestramento. (a) Al variare di β, con $T = 100$ e $\alpha = 2\varepsilon/T$. (b) Al variare del passo, considerando il β peggiore per ciascun modello: $\alpha = \varepsilon/5 = 0.04$ con $T = 100$, e $\alpha = 2\varepsilon/T$ con $T = 20$ e $T = 100$. 127

6.5 Accuratezza end-to-end robusta al variare del budget, con $T = 100$ e $\alpha = 2\varepsilon/T$, in media su taglie e dataset; la legenda indica il modello e il suo attacco di addestramento. (a) Contro PGD. (b) Contro l'attacco adattivo con il β peggiore per ciascun modello; CloserAL e il solo adversarial training, privi di detector, sono riportati con PGD, che per loro è l'attacco white-box completo. 128

Elenco delle tabelle

4.1 Proprietà dei dataset selezionati secondo lo schema di classificazione di [29]. Il numero di feature include le etichette. Le categorie d'attacco sono indicate con le iniziali dello schema: B brute force, C botnet command and control, D denial of service, R reconnaissance, V exploit di vulnerabilità, O altri, ? attacchi non esplicitamente dichiarati. 37

4.2 Distribuzione delle categorie d'attacco in UNSW-NB15 40

4.3 Distribuzione delle categorie d'attacco in CIC-IDS2017 41

4.4 Distribuzione delle categorie d'attacco in CSE-CIC-IDS2017 43

4.5 Distribuzione delle categorie d'attacco in CTU-13 44

5.1 Configurazioni di modello considerate. La colonna relativa alla loss del task indica se questa è calcolata anche sui campioni adversarial: se ci sono due scelte, la prima è quella di default e la seconda è configurabile. 84

6.1 Configurazioni dello stadio 1. Le colonne Larghezze e Est. riportano le larghezze dei livelli nascosti della backbone e i livelli estesi; tra parentesi i livelli CloserAL con $\lambda_{act} = 0$, che non contribuiscono alla loss. L'ultima colonna indica i test della suite di V&V che impiegano i checkpoint della configurazione. 90

6.2 Metriche di test dello stadio 1 su UNSW-NB15. $\mathrm{Acc}_{task}^{clean}$ e $\mathrm{Acc}_{task}^{adv}$ sono le accuratezze sui campioni puliti e su quelli attaccati con PGD ($\varepsilon = 0.2$); TNR e TPR sono le frazioni di campioni puliti non segnalati e di campioni adversarial segnalati dall'allarme; lo Score è la media di $\mathrm{Acc}_{clean}^{e2e}$ e Acc_{rob}^{e2e}, anch'esso calcolato sul test set. 91

6.3 Grandezze operative dello stadio 1: epoche eseguite prima dell'arresto anticipato, soglia di allarme τ calibrata sul validation set, latenza media di inferenza per campione sui campioni puliti e adversarial, frazione di campioni adversarial che escono in anticipo e durata complessiva della run. 93

6.4 Test di verifica e relativo esito, espresso come numero di controlli superati sul totale. 95

6.5 Test di validazione degli attacchi e relativo esito, espresso come numero di controlli superati sul totale. 97

6.6 Modelli senza detector: accuratezza della backbone sotto attacco (m_{task}) al crescere della forza dell'attacco. Salvo l'ultima colonna, $\varepsilon = 0.2$; 200×5 indica 200 passi e 5 partenze casuali con $\alpha = 0.01$, e l'attacco con $\varepsilon = 5$ impiega 1000 passi. 97

6.7 Modelli con detector: accuratezza end-to-end robusta (m_{e2e}) al crescere della forza dell'attacco. β^* è riportato per ciascuno dei tre seed. Le colonne 1 passo e 20 passi impiegano l'attacco di base; 40×1 e 200×5 indicano passi e partenze casuali con $\alpha = 0.01$; l'attacco con $\varepsilon = 5$ impiega 1000 passi e tutti i β della griglia. 97

6.8 Test A2: robustezza al variare del budget ε (media sui tre seed), con $\alpha = 0.01$ e 200 passi. Per i modelli con detector l'attacco adattivo impiega il β^* selezionato a $\varepsilon = 0.2$. 98

6.9 Test di validazione dei meccanismi e relativo esito. 99

6.10 Effetto della profondità. A sinistra la SLAV_{rel} del modello non difeso a quattro livelli e il guadagno $\sqrt{\mathrm{SLAV}_{rel}^{(l)}/\mathrm{SLAV}_{rel}^{(l-1)}}$ tra livelli consecutivi; a destra l'AUC dei detector del modello DetectorLayer a quattro livelli nel regime isolato. 100

6.11 Test M6: tempo di inferenza per campione di `further-exit-deep` su 500 campioni puliti e 500 adversarial, 30 ripetizioni. I tempi sono mediane; il risparmio è la mediana, sui campioni del gruppo, della riduzione relativa rispetto al forward completo con i detector. 100

6.12 Architetture esplorate dello stadio 2. Ogni riga è un prodotto cartesiano dei valori elencati; le 26 configurazioni di ciascuno dei due blocchi dei modelli con detector sono ripetute per DetectorLayer e FurtherAL, per un totale di $4 \times 26 + 17 = 121$ configurazioni per dataset. I livelli sono numerati a partire da 1 come nello stadio 1. 102

6.13 Effetti dei fattori architetturali sui modelli con detector: $\bar{S}$ medio sulle configurazioni che condividono il valore indicato del fattore. Nelle backbone a tre livelli i detector hanno sempre dimensione (64, 32). 105

6.14 CloserAL con PGD: Score di test per dataset e media $\bar{S}$ delle 17 configurazioni. P è il numero di parametri per UNSW-NB15 ($d = 42$). In grassetto le architetture selezionate. 106

6.15 Modelli con detector con PGD: $\bar{S}$ di ciascuna configurazione. P è il numero di parametri per UNSW-NB15 ($d = 42$) ed è lo stesso per DetectorLayer e FurtherAL. In grassetto le architetture selezionate. 107

6.16 Modelli con detector con PGD adattivo: $\bar{S}$ di ciascuna configurazione. Le backbone coincidono con quelle di CloserAL; P è il numero di parametri per UNSW-NB15 ($d = 42$) ed è lo stesso per DetectorLayer e FurtherAL. In grassetto le architetture selezionate. 108

6.17 Metriche di test medie di ciascuna famiglia, su tutte le configurazioni e dataset. 109

6.18 Architetture selezionate. $P(d)$ è il numero di parametri in funzione della dimensione dell'ingresso secondo la (6.1) e P il suo valore per UNSW-NB15 ($d = 42$). Per i modelli con detector l'architettura è la stessa per DetectorLayer e FurtherAL, e le backbone dei modelli adattivi coincidono con quelle di CloserAL allo stesso livello. 112

6.19 Metriche di test delle architetture selezionate: Score per dataset e, mediati sui quattro dataset, $\bar{S}$ e accuratezze end-to-end. 113

6.20 Score di test dello stadio 3 nei due regimi del flusso dei gradienti, isolato e cooperativo, per taglia e politica di aggregazione (Aggr.): media sui tre seed per ciascun dataset e $\bar{S}$, con la deviazione standard delle medie per seed. In grassetto le configurazioni selezionate. 114

6.21 Metriche di test medie nei due regimi, per le configurazioni con aggregazione tramite la media, su tutte le taglie e i dataset. $\mathrm{SLAV}^{(1)}_{rel}$ è la vulnerabilità relativa del primo livello, esteso in tutte le architetture. 116
6.22 Meccanismi di detection selezionati, come politica di aggregazione e regime del flusso dei gradienti, e relativo $\bar{S}$; per CloserAL non si applica nessuno dei due fattori. 118
6.23 $\bar{S}$ delle configurazioni dello stadio 4 per taglia, e media $\langle \bar{S} \rangle$ sulle tre taglie. La colonna $\boldsymbol{x}^{adv}$ indica se la loss del task è calcolata anche sui campioni adversarial; i pesi non indicati hanno il valore di default ($\lambda_{det} = \lambda_{act} = 1$, $\kappa = 5$). In grassetto le configurazioni selezionate, in corsivo le varianti di controllo che le affiancano nello stadio 5. 119
6.24 Metriche di test medie su taglie e dataset. Sono indicati i soli valori diversi dal default; $\boldsymbol{x}^{adv}$ indica la loss del task calcolata anche sui campioni adversarial, e per CloserAL $\lambda_{act} = 0$ è l'adversarial training. 120
6.25 Configurazioni della loss selezionate per lo stadio 5, comuni alle tre taglie di ciascun modello e attacco, e relativo $\bar{S}$, insieme alle varianti di controllo che le affiancano; $\boldsymbol{x}^{adv}$ ha lo stesso significato della Tabella 6.23. Tra parentesi i pesi ininfluenti nel regime isolato; i meccanismi di detection sono quelli della Tabella 6.22. 123
6.26 Attacchi dello stadio 5 e numero di run per modello. Gli attacchi contrassegnati da * sono eseguiti soltanto contro i modelli con detector. 124
6.27 Attacco di addestramento e caso peggiore al budget $\varepsilon = 0.2$, in media sui dataset. TPR e $\mathrm{Acc}^{adv}_{task}$ si riferiscono all'attacco che realizza il caso peggiore; $\boldsymbol{x}^{adv}$ indica le varianti di controllo con la loss del task calcolata anche sui campioni adversarial. In grassetto il miglior Score nel caso peggiore per ogni taglia. 125

Abstract

I sistemi di rilevamento delle intrusioni di rete (Network Intrusion Detection Systems, NIDS) monitorano il traffico alla ricerca di attività malevole. Il traffico è generalmente analizzato per flussi: ciascuno riassume una comunicazione tra due host in un vettore di caratteristiche numeriche (*feature*), come la durata o il numero di byte scambiati. Negli ultimi anni, il deep learning si è affermato come strategia di riferimento in questo ambito. Grazie alla capacità delle reti neurali profonde di apprendere rappresentazioni dei dati e individuare pattern complessi, le soluzioni basate su tali tecniche hanno raggiunto prestazioni molto elevate sui benchmark di riferimento nel dominio NIDS.

Tali prestazioni sono però spesso misurate in assenza di un attaccante. L'Adversarial Machine Learning, che studia il comportamento dei modelli di apprendimento sotto attacco, ha mostrato che queste reti sono vulnerabili ad attacchi di evasione: tali attacchi puntano a perturbare le feature di un flusso malevolo, entro un limite detto budget di perturbazione, con l'obiettivo di indurre il sistema NIDS a classificarlo come lecito. Il flusso modificato è detto campione adversarial, quello originale campione pulito. Nello scenario più sfavorevole, detto white-box, l'attaccante ha conoscenza completa del modello di classificazione, inclusi quindi i parametri e il gradiente della sua funzione di costo rispetto all'input. Diversi attacchi proposti in letteratura sfruttano proprio il gradiente per indurre il modello in errore: il Fast Gradient Sign Method (FGSM) compie un unico passo nella direzione del gradiente, il Projected Gradient Descent (PGD) ne compie molti di ampiezza ridotta. Questi attacchi funzionano perché calcolano delle perturbazioni impercettibili che, sfruttando la vulnerabilità caratteristica della rete, i livelli nascosti tendono ad amplificare dall'ingresso lungo la profondità, fino a provocare una decisione errata.

Le difese esistenti si dividono in quattro famiglie. L'*adversarial training*, la più solida sul piano empirico, addestra il modello anche su campioni adversarial, generati con PGD, perché impari a classificarli correttamente; la *robustezza certificata* offre garanzie formali, ma con forti limiti di scalabilità; la *purificazione* tenta di ripulire l'ingresso dalla perturbazione; il *rilevamento* (*detection*) si limita a riconoscere e scartare il campione attaccato, ed è particolarmente adatto ai NIDS, dove rifiutare un flusso sospetto è operativamente accettabile. Molte difese sono state però aggirate da attacchi più forti, spesso adattivi, cioè

costruiti conoscendo la difesa stessa: una difesa va quindi valutata contro un attaccante che la conosce.

Lo stato dell'arte presenta tre limiti. Quasi tutte le difese trattano la rete come un unico blocco: la robustezza è valutata solo sull'uscita e le contromisure agiscono sul rapporto tra ingresso e uscita, o su tutti i livelli della rete neurale in modo uniforme, ignorando dove la perturbazione cresce e diventa decisiva. I metodi di rilevamento che osservano le rappresentazioni latenti, più informative dell'ingresso grezzo, collegano per lo più un detector, ossia un classificatore che stima se il campione è adversarial, a un unico punto della rete, oppure calcolano statistiche a posteriori su una rete addestrata normalmente, e molti sono stati aggirati da attacchi adattivi. Infine, questi lavori riguardano quasi sempre immagini, non dati tabulari eterogenei come i flussi di rete.

Questa tesi adotta un approccio *layer-wise*, che tratta ogni livello nascosto come un punto in cui la deviazione tra le rappresentazioni di un campione pulito e della sua variante adversarial può essere misurata, rilevata e modellata. Definita backbone la rete che classifica i flussi, il lavoro innesta detector in più punti della backbone, addestrandoli insieme a essa anziché a posteriori, misura e modella la deviazione livello per livello anziché valutarla solo all'uscita, controlla esplicitamente l'influenza dei detector sulla backbone e valuta il tutto sui flussi di rete contro un attaccante adattivo.

La metodologia introduce tre strumenti. La Single Layer Adversarial Vulnerability (SLAV), una metrica che misura in ciascun livello la distanza media tra le rappresentazioni dei campioni puliti e delle loro varianti adversarial, anche in versione relativa, cioè rapportata alla scala delle rappresentazioni pulite. Un'architettura biforcata che collega a uno o più livelli della backbone un piccolo detector, i cui score, stime della probabilità che il campione sia adversarial, sono aggregati in un unico allarme. Infine, due strategie opposte aggiungono alla funzione obiettivo dell'addestramento (loss) un termine basato sulla SLAV relativa: CloserAL avvicina le rappresentazioni adversarial a quelle pulite, estendendo l'adversarial training, mentre FurtherAL le allontana fino a una distanza minima (margine) calibrata per livello, per renderle più riconoscibili ai detector. Un'analisi formale del flusso dei gradienti, cioè di come l'addestramento dei detector influenza la backbone, distingue un regime isolato, in cui i detector la osservano senza modificarla, da uno cooperativo, in cui ne plasmano le rappresentazioni, e mostra che solo la distanza relativa impedisce

alla rete di soddisfare queste loss riscalando i propri valori interni, fenomeno che blocca le strategie proposte senza modificare il funzionamento della rete.

Il modello di minaccia assunto è white-box, in cui l'attaccante è a conoscenza sia della backbone sia dei detector: oltre a FGSM e PGD, che ignorano i detector, si impiega una variante adattiva di PGD, che ricerca l'errore di classificazione evitando contemporaneamente l'allarme del detector. Le difese sono confrontate con uno *score*, media delle accuratezze sui flussi puliti, dove un falso allarme conta come errore, e sui flussi attaccati, dove l'attacco è sventato se il flusso è classificato correttamente o segnalato. La validazione si articola in cinque stadi sui dataset UNSW-NB15, CIC-IDS2017, CSE-CIC-IDS2018 e CTU-13, per un totale di circa 1400 addestramenti e oltre 3000 valutazioni, fino a un'analisi del caso peggiore con attacchi più forti e diversi da quelli visti in fase di addestramento.

I risultati mostrano che la perturbazione, pur crescendo di quasi sei volte verso l'uscita, è più rilevabile nel primo livello nascosto, dove un detector è indispensabile. Contro l'attacco di addestramento i modelli con detector superano di gran lunga l'adversarial training, e FurtherAL rafforza il rilevamento senza costi per la classificazione. Il caso peggiore ridimensiona però questo vantaggio: i detector addestrati contro PGD sono elusi dall'attacco adattivo: i detector imparano a riconoscere un attaccante che li ignora, non la perturbazione in sé. Anche quelli addestrati con l'attacco adattivo scendono al livello dell'adversarial training o sotto, perché un attaccante libero di scegliere il peso di evasione e un passo più fine raggiunge regioni non esplorate in addestramento. Resiste invece la combinazione di rilevamento e adversarial training: con una backbone addestrata anche sui campioni adversarial, FurtherAL ottiene nel caso peggiore lo score più alto, e un'accuratezza sui flussi attaccati del 91%, perché la backbone classifica correttamente gran parte dei campioni sfuggiti all'allarme. CloserAL, infine, equivale all'adversarial training entro il budget di addestramento, ma degrada più lentamente oltre di esso.

Nel complesso, l'osservazione delle rappresentazioni interne fornisce strumenti sia di difesa sia di interpretazione, rilevamento e adversarial training risultano complementari più che alternativi, e una difesa basata sul rilevamento va valutata con un attaccante adattivo i cui parametri siano esplorati, non fissati.

Capitolo 1

Introduzione

1.1 Contesto e motivazioni

Negli ultimi anni il Deep Learning ha assunto un ruolo centrale nell'elaborazione di dati complessi e nell'automazione decisionale in ambiti critici, tra cui la sicurezza informatica. Questo vale in particolare per i sistemi di rilevamento delle intrusioni di rete (Network Intrusion Detection Systems, NIDS) [27, 80], dispositivi o software incaricati di monitorare il traffico alla ricerca di attività malevole. In questo contesto le reti neurali profonde si sono dimostrate molto capaci di apprendere pattern complessi, fino a raggiungere sui dataset di riferimento accuratezze prossime alla saturazione.

L'adozione di queste architetture in scenari ostili è tuttavia ostacolata dalla loro intrinseca vulnerabilità agli attacchi adversarial di evasione. Iniettando perturbazioni limitate nello spazio delle feature, un attaccante è in grado di alterare la predizione del classificatore e di far passare un flusso malevolo per traffico lecito, senza che venga sollevato alcun allarme. La ragione dell'efficacia di questi attacchi risiede nella dinamica interna delle reti profonde: una perturbazione trascurabile all'ingresso viene trasformata, e tipicamente amplificata, dalla successione dei livelli nascosti, finché la deviazione delle rappresentazioni latenti non conduce a una decisione errata.

La quasi totalità delle difese proposte in letteratura ignora però questa struttura: la robustezza è valutata soltanto all'uscita della rete, e le contromisure, a partire dall'adversarial training, agiscono sulla relazione tra ingresso e uscita o applicano regolarizzazioni uniformi su tutti i livelli, perdendo l'informazione su dove la perturbazione cresce e diventa decisiva. Le difese basate sul rilevamento hanno in parte sfruttato le rappresentazioni intermedie,

ma quasi sempre collegando un detector a un unico punto della rete o calcolando statistiche a posteriori su una rete addestrata in modo standard, e molte di esse sono state aggirate da attaccanti che ne conoscevano lo score. Gran parte di questi studi, infine, riguarda le immagini, mentre i flussi di rete sono dati tabulari eterogenei, in un dominio nel quale scartare un flusso sospetto è un'azione accettabile, e dunque l'adozione di un meccanismo di rilevamento rappresenta una difesa particolarmente adatta.
Ne nasce la motivazione di un'indagine layer-wise, che tratti i livelli nascosti come stadi in cui la minaccia può essere monitorata, contrastata e spiegata, e che valuti le difese ottenute contro un attaccante che le conosce.

1.2 Obiettivi e contributi

Il presente lavoro studia la robustezza dei modelli neurali per il rilevamento delle intrusioni da una prospettiva layer-wise, con un duplice obiettivo: comprendere come la perturbazione adversarial si propaga nei singoli livelli, e sfruttare tale comprensione per costruire difese, basate sul rilevamento o sulla classificazione robusta, da valutare nel caso peggiore. I principali contributi sono i seguenti.

1. **Una metrica di vulnerabilità per livello**. La Single Layer Adversarial Vulnerability (SLAV) misura la deviazione indotta dall'attacco nello spazio delle attivazioni di ciascun livello. Poiché la metrica non è invariante rispetto a riscalature che lasciano inalterata la funzione calcolata, è introdotta una versione relativa, che rapporta la deviazione alla scala delle attivazioni pulite e su cui operano tutte le loss proposte.
2. **Un'architettura biforcata per il rilevamento multi-livello**. Detector di piccole dimensioni, addestrati congiuntamente alla backbone – il classificatore alla base – osservano le pre-attivazioni dei livelli estesi in posizioni arbitrarie, e i loro score vengono aggregati, con la possibilità di scegliere tra due strategie diverse, in un unico allarme. L'implementazione arricchisce i moduli di PyTorch permettendo di estendere qualunque livello senza modificare gli altri.
3. **Due strategie di manipolazione geometrica dello spazio latente**. La strategia attrattiva (CloserAL) affianca all'adversarial training una loss che avvicina le attivazioni adversarial a quelle pulite; la strategia repulsiva (FurtherAL) affianca a ciascun detector una loss che le allontana fino a un margine, calibrato livello per

livello come multiplo della vulnerabilità naturale.

4. **Un'analisi formale del flusso dei gradienti**. Attraverso l'operatore di arresto del gradiente vengono distinti un regime isolato, in cui i detector osservano passivamente la backbone, e uno cooperativo, in cui la loss di detection la plasma. L'analisi mostra inoltre che, nelle loss sulle attivazioni, soltanto la distanza relativa con il riferimento pulito lasciato libero esclude soluzioni degeneri basate sul rimpicciolimento o sull'inflazione dei livelli, e che l'attacco adattivo deve rimuovere ogni arresto del gradiente all'ingresso dei detector, per non sovrastimare la robustezza.
5. **Un'ampia campagna sperimentale**. Cinque stadi di sperimentazione sui dataset UNSW-NB15, CIC-IDS2017, CSE-CIC-IDS2018 e CTU-13, per circa 1400 addestramenti e oltre 3000 valutazioni, comprensiva di una suite di verifica e validazione, per l'esplorazione di architetture migliori e un'analisi del caso peggiore che varia parametri e budget di attacchi white-box completi contro il sistema di difesa.

Principali risultati. La campagna sperimentale porta ai seguenti risultati.

- *La perturbazione è più rilevabile all'inizio della rete.* La deviazione relativa cresce dal primo all'ultimo livello, ma, contrariamente all'ipotesi iniziale, il detector più accurato è quello del primo livello, e un detector in questa posizione è indispensabile.
- *Rilevamento e classificazione robusta si completano.* La migliore difesa nel caso peggiore combina i due meccanismi: FurtherAL con la loss del task calcolata anche sui campioni adversarial, perché la backbone robusta classifica correttamente gran parte dei campioni attaccati quando l'allarme viene eluso. Con budget molto superiori a quello di addestramento, inoltre, il rilevamento resta l'unica difesa efficace.
- *La loss attrattiva agisce sulla geometria, non sulla robustezza entro il budget.* CloserAL riduce fino a sette volte la deviazione relativa del primo livello, ma entro il budget di addestramento equivale, in termini di prestazioni, al solo adversarial training; oltre quel budget degrada invece più lentamente.
- *Il passo dell'attacco conta più del numero di iterazioni.* Contro i detector l'attacco adattivo è efficace soltanto con un passo fine, e il peso di evasione migliore varia da modello a modello: una difesa basata sul rilevamento va valutata con parametri d'attacco esplorati, non fissati.

1.3 Struttura della tesi

Il seguito della tesi è organizzato come segue:

Capitolo 2 Background teorico: introduce i NIDS e i loro vincoli, i fondamenti delle reti neurali feedforward e della stabilità della propagazione, e l'adversarial machine learning, con gli attacchi di evasione e le contromisure.

Capitolo 3 Stato dell'arte: passa in rassegna i modelli di Deep Learning per i NIDS e l'evoluzione di attacchi e difese adversarial, con attenzione agli approcci layer-wise e al rilevamento multi-livello, e individua il gap affrontato.

Capitolo 4 Dataset: descrive i quattro dataset impiegati e la pipeline di preprocessing, fino alla definizione delle feature perturbabili.

Capitolo 5 Metodologia e modellazione delle soluzioni: formalizza la metrica SLAV, l'architettura biforcata, le strategie CloserAL e FurtherAL, l'analisi del flusso dei gradienti, il modello di minaccia con gli attacchi FGSM, PGD e PGD adattivo, e le metriche di valutazione.

Capitolo 6 Esperimenti e analisi dei risultati: formula le domande di ricerca e presenta i cinque stadi della sperimentazione, fino all'analisi del caso peggiore.

Capitolo 7 Conclusioni e sviluppi futuri: riassume le risposte alle domande di ricerca e discute limiti ed estensioni del lavoro.

Capitolo 2

Background teorico

Per contestualizzare adeguatamente le problematiche affrontate e le soluzioni metodologiche introdotte in questa tesi, è necessario prima definire i concetti fondamentali sui quali si fonda questo lavoro. La tesi si colloca all'intersezione tra i domini del Deep Learning, la sicurezza delle reti mediante Network Intrusion Detection e la robustezza contro Adversarial Machine Learning.

2.1 Network Intrusion Detection Systems (NIDS)

Nella sicurezza informatica, un Network Intrusion Detection System (NIDS) è un'infrastruttura software o hardware preposta al monitoraggio e all'analisi costante del traffico di rete, con l'obiettivo di intercettare attività anomale, violazioni delle policy di sicurezza o tentativi di attacco in corso. L'idea di sottoporre l'attività di un sistema a un monitoraggio continuo per individuare comportamenti riconducibili a un abuso risale ai lavori fondativi sulla sorveglianza degli audit trail [1] e alla prima formalizzazione di un modello di rilevamento delle intrusioni basato su profili statistici di comportamento [20]; tali principi, pur nati in un contesto tecnologico profondamente diverso, restano concettualmente alla base dei sistemi odierni.

2.1.1 Collocazione architetturale e tassonomia

I sistemi di rilevamento delle intrusioni si distinguono anzitutto in base alla sorgente della telemetria analizzata. Gli Host-based IDS (HIDS) operano sul singolo nodo, ispezionando system call, log applicativi, integrità del file system e comportamento dei processi; gli approcci di deep learning applicati a questa classe di segnali hanno prodotto risultati

significativi nell'analisi di log di sistema [22]. I Network-based IDS (NIDS), oggetto di questa tesi, monitorano invece il traffico che attraversa un punto strategico della rete, e mirano a rilevare attacchi diretti a un insieme di host piuttosto che a una singola macchina [27].

Una seconda distinzione riguarda la modalità di gestione del flusso dei dati da monitorare: un sistema reattivo quale un NIDS osserva una copia del traffico e genera allarmi, mentre un sistema proattivo come un Intrusion Prevention System (IPS) opera online ed è in grado di scartare o riscrivere i pacchetti ritenuti malevoli. Quest'ultima modalità impone però vincoli di latenza molto più stringenti e amplifica il costo operativo dei falsi positivi, poiché ogni errore si traduce direttamente in un'interruzione di servizio per traffico legittimo.

Dal punto di vista del deployment, un NIDS può essere collocato sul perimetro di rete, in posizione distribuita sui gateway, all'interno del piano di controllo di reti programmabili — dove la centralizzazione delle informazioni di flusso offerta dal paradigma Software-Defined Networking semplifica la raccolta della telemetria [37] — oppure direttamente su dispositivi edge a risorse limitate, scenario sempre più rilevante con la diffusione di reti IoT e IIoT [46].

2.1.2 Paradigmi di rilevamento

I sistemi di rilevamento tradizionali sono storicamente suddivisi in due grandi categorie [27]:

1. Sistemi basati su firme (Signature-based): operano confrontando il payload dei pacchetti o la telemetria monitorata in ingresso con un archivio di pattern noti, relativi a minacce già documentate. Pur garantendo tassi di falsi positivi estremamente bassi, risultano del tutto inefficaci nei confronti di vulnerabilità sconosciute (zero-day) o di varianti offuscate. A questa famiglia appartengono sistemi di rilevamento quali Snort [72], Suricata e la piattaforma di analisi real-time Bro [65]. La manutenzione dell'archivio di firme rappresenta inoltre un costo ricorrente: la copertura di una nuova minaccia è possibile solo dopo la sua caratterizzazione da parte di un analista.
2. Sistemi basati su anomalie (Anomaly-based): definiscono un modello di comportamento del traffico di rete, e segnalano come potenziale intrusione qualsiasi deviazione statisticamente significativa rispetto alla normalità. In questo modo si permette

l'intercettazione di attacchi mai visti prima; tuttavia, siccome ogni anomalia viene segnalata, si è soggetti ad un alto numero di falsi positivi. La definizione stessa di "comportamento normale" è inoltre instabile nel tempo, poiché il traffico di una rete evolve in funzione di nuovi servizi, aggiornamenti applicativi e variazioni stagionali del carico.

L'integrazione del Machine Learning e del Deep Learning ha trasformato radicalmente i NIDS basati su anomalie, spostando il focus dalla definizione manuale di regole e soglie all'apprendimento automatico a partire da grandi volumi di telemetria [80].

2.1.3 Granularità dell'analisi: dal pacchetto al flusso

Nei moderni sistemi di rilevamento, l'analisi non viene eseguita a livello di singolo pacchetto grezzo, bensì a livello di flusso di rete. Un flusso aggrega una sequenza bidirezionale di pacchetti scambiati tra due endpoint, ed è identificato univocamente dalla classica quintupla di trasporto:

$$\text{Flow-ID} = \langle \text{IP sorgente}, \text{Porta sorgente}, \text{IP destinazione}, \text{Porta destinazione}, \text{Protocollo} \rangle$$

La scelta del flusso come unità di analisi risponde a tre esigenze concrete. In primo luogo, la progressiva cifratura end-to-end (es. TLS 1.3 o QUIC) ha reso il payload applicativo inaccessibile all'ispezione, lasciando disponibili quasi esclusivamente i metadati di connessione e le statistiche della comunicazione [44]. In secondo luogo, l'aggregazione riduce di ordini di grandezza il volume di dati da processare, rendendo il monitoraggio sostenibile su collegamenti ad alta velocità. In terzo luogo, diverse classi di attacco — scansioni, esfiltrazione, comunicazione con un centro di comando e controllo — si manifestano come pattern di connessione piuttosto che come contenuti anomali del singolo pacchetto.
L'estrazione delle feature dal traffico grezzo è demandata a strumenti di flow export quali NetFlow, IPFIX, Argus o CICFlowMeter, che producono record contenenti ad esempio: durata della connessione, volume di byte e pacchetti trasmessi in ciascuna direzione, valori dei flag TCP, dimensioni delle finestre di congestione, rapporti fra byte inviati e ricevuti. È importante osservare che tale stadio non è neutrale: la scelta del flow exporter, dei timeout di sessione e dell'insieme di attributi esportati condiziona in modo diretto e non trascurabile le prestazioni misurate del classificatore a valle, al punto che è stata proposta

l'adozione di insiemi di feature standardizzati e indipendenti dallo strumento per rendere confrontabili i risultati fra dataset differenti [75].

2.1.4 Formulazione del task di apprendimento

Una volta fissata la rappresentazione, il task del NIDS si formalizza come un problema di classificazione supervisionata:

- Binaria: discriminare tra traffico Benigno e Malevolo;
- Multi-classe: identificare la specifica tassonomia della minaccia (es. DoS/DDoS, Botnet, MitM, Phishing, SQL injection o altri Advanced Persistent Threats (APTs)).

In alternativa, il problema può essere posto come anomaly detection, ovvero come stima di deviazione dalla distribuzione del traffico benigno: in questo regime il modello viene addestrato esclusivamente su dati relativi a traffico normale, e ogni campione che produca un errore di ricostruzione superiore a una soglia viene segnalato. Tale formulazione ha il vantaggio operativo di non richiedere etichette per il traffico malevolo — molto costose da produrre — ed è alla base degli approcci basati su autoencoder [54].

2.1.5 Vincoli di dominio

L'applicazione dell'adversarial machine learning alle reti neurali in questo contesto presenta una specificità critica: a differenza di domini continui come la computer vision (in cui ogni pixel può essere perturbato arbitrariamente), i dati tabulari dei flussi di rete combinano variabili continue, discrete e categoriche, molte delle quali soggette a rigidi vincoli di protocollo, necessari a mantenere intatta la semantica di un flusso. Tali vincoli possono essere classificati come segue:

- Vincoli di dominio sulla singola feature: numerosi attributi sono per costruzione non negativi (conteggi di pacchetti, durate), interi (flag TCP osservati) o limitati superiormente da parametri di protocollo (dimensione massima del segmento, ampiezza della finestra di ricezione);
- Vincoli categorici: gli attributi come il protocollo di trasporto o lo stato della connessione hanno un insieme discreto e non ordinato di valori ammissibili, tipicamente codificato one-hot; una perturbazione continua su tali componenti produce vettori privi di significato;

- Vincoli di inter-dipendenza: molte feature sono legate da relazioni aritmetiche o logiche — il numero totale di byte è la somma dei byte nelle due direzioni, la velocità media è il rapporto fra volume e durata — cosicché una modifica indipendente di una singola componente genera un record internamente incoerente;

La rilevanza di questa caratterizzazione non è solo descrittiva perchè definisce la superficie di attacco effettivamente disponibile a un attaccante e, aspetto da non sottovalutare, separa gli attacchi adversarial realizzabili nello spazio del problema da quelli formulati nel solo spazio delle feature [78].

2.1.6 Vincoli di deployment

Un ultimo insieme di requisiti, spesso trascurato nella letteratura accademica, riguarda la sostenibilità del rilevamento. Un NIDS deve garantire un throughput di elaborazione superiore al tasso di arrivo dei pacchetti, pena la formazione di code e la perdita di visibilità sul traffico, deve operare con latenza compatibile con un'eventuale risposta online, e, nel caso di deployment distribuito su gateway o dispositivi edge, deve rispettare severi vincoli di memoria e capacità di calcolo. Questi requisiti motivano linee di ricerca dedicate a modelli a bassa complessità, in grado di operare su hardware di fascia bassa elaborando un campione alla volta senza necessità di accumulare dati [54], e a schemi di rilevamento che operano su rappresentazioni compatte del traffico per sostenere velocità di linea elevate [26]. Tali vincoli sono direttamente rilevanti ai fini di questa tesi, poiché rendere robusta una rete contro attacchi adversarial introduce un sovraccarico computazionale [50], aprendo al contempo la possibilità di terminare anticipatamente l'inferenza sui campioni riconosciuti come adversarial.

2.2 Reti Neurali Artificiali e Rappresentazioni Latenti

Le reti neurali artificiali (Artificial Neural Networks, ANN), e in particolare le architetture feedforward profonde (Deep Neural Networks, DNN, ma anche CNN), rappresentano modelli computazionali non lineari capaci di apprendere approssimazioni di funzioni complesse a partire da dati strutturati o non strutturati. La loro diffusione nei sistemi di rilevamento delle intrusioni deriva da due proprietà complementari: la capacità di approssimare strutture decisionali arbitrariamente complesse senza richiedere una modellazione esplicita

del fenomeno osservato, e la possibilità di apprendere automaticamente rappresentazioni intermedie dei dati progressivamente più astratte e caratteristiche. Proprio queste rappresentazioni intermedie, che costituiscono il nucleo dell'indagine condotta in questa tesi, nascono dal prodotto della struttura a strati della rete e dell'addestramento che ne determina i parametri. Nel seguito si richiamano quindi sia la formulazione del modello, sia l'algoritmo che ne governa l'apprendimento, poiché entrambi concorrono a definire la geometria dello spazio latente su cui operano gli attacchi adversarial e le difese proposte. Si rimanda a [31] per ulteriori approfondimenti.

2.2.1 Dal percettrone alle architetture feedforward

Il modello elementare da cui discendono le reti neurali moderne è il percettrone, un'unità di calcolo che riceve in ingresso un vettore di numeri reali, ne effettua una combinazione lineare pesata e applica al risultato una funzione di attivazione. Oltre agli ingressi veri e propri si considera un termine di bias, che rappresenta la componente invariante della predizione, ossia il valore prodotto quando tutti gli ingressi sono nulli:

$$y = \sigma\left(\sum_{j=1}^{d} w_j x_j + b\right) = \sigma\left(\boldsymbol{w}^\top \boldsymbol{x} + b\right) = \operatorname{sgn}\left(\boldsymbol{w}^\top \boldsymbol{x} + b\right)$$

dove nella formulazione originale la funzione di attivazione coincide con la funzione segno. L'addestramento del percettrone si basa su un'euristica di correzione dell'errore: i pesi vengono inizializzati arbitrariamente, si prende un'istanza $\boldsymbol{x}^{(i)}$ del training set e si confronta la predizione $y^{(i)}$ con il target $t^{(i)}$; in caso di errore il peso viene aggiornato nella direzione che minimizza la differenza tra $y^{(i)}$ e $t^{(i)}$,

$$w_j \leftarrow w_j + \left(t^{(i)} - y^{(i)}\right) x_j^{(i)}$$

iterando la procedura per un arbitrario numero di epoche.

La stessa regola può essere derivata in modo più rigoroso con la discesa del gradiente, basata però su una funzione di loss differenziabile. Infatti, la funzione di loss 0-1 per l'istanza $(\boldsymbol{x}^{(i)}, t^{(i)})$ con $t^{(i)} \in \{-1, +1\}$ può essere riscritta come

$$\mathcal{L}_{0/1}^{(i)} = \frac{1}{2}\left(t^{(i)} - \operatorname{sgn}\{\boldsymbol{w}^\top \boldsymbol{x}^{(i)}\}\right)^2 = 1 - t^{(i)} \operatorname{sgn}\{\boldsymbol{w}^\top \boldsymbol{x}^{(i)}\}$$

dove si osserva che la presenza della funzione segno ne impedisce la differenziabilità. Si

adotta allora il surrogato

$$\mathcal{L}^{(i)} = \max\{0, -t^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)})\}, \qquad \mathcal{L} = \sum_{i=1}^{N} \max\{0, -t^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)})\}$$

il cui gradiente rispetto ai pesi restituisce la regola di aggiornamento precedente:

$$\nabla_{\boldsymbol{w}} \mathcal{L}^{(i)} = \left(y^{(i)} - t^{(i)}\right) \boldsymbol{x}^{(i)}, \qquad \boldsymbol{w} \leftarrow \boldsymbol{w} - \nabla_{\boldsymbol{w}} \mathcal{L}^{(i)} = \boldsymbol{w} + \left(t^{(i)} - y^{(i)}\right) \boldsymbol{x}^{(i)}$$

Il limite strutturale di questo modello risiede nel fatto che un percettrone dispone di un solo livello di calcolo e può pertanto approssimare unicamente funzioni lineari: un modello lineare fallisce nell'apprendere relazioni non linearmente separabili, come la funzione XOR. Il superamento di questo vincolo si ottiene componendo più neuroni in strutture stratificate, dando origine alle reti neurali artificiali; tra queste, si definiscono feedforward le architetture il cui grafo computazionale è privo di cicli, ossia in cui il flusso informativo procede in un'unica direzione dall'ingresso all'uscita.
I neuroni di una rete feedforward sono organizzati in livelli:

- un livello di input, privo di capacità computazionale, che si limita a rappresentare il vettore delle feature;
- uno o più livelli nascosti, attraversati sequenzialmente, all'interno dei quali avviene la computazione;
- un livello di output, che produce il risultato finale a partire dall'ultimo livello nascosto.

Nel seguito si assume, come avviene nelle architetture considerate in questa tesi, che livelli adiacenti siano completamente connessi, ovvero che ogni unità del livello $l-1$ sia collegata a ogni unità del livello l, e considereremo solo modelli di classificazione.

2.2.2 Propagazione in avanti e rappresentazioni latenti

Formalmente, un classificatore basato su DNN a L livelli può essere espresso come una composizione sequenziale di funzioni parametriche a più variabili:

$$f(\boldsymbol{x}; \boldsymbol{\theta}) = \left(f^{(L)} \circ f^{(L-1)} \circ \cdots \circ f^{(1)}\right)(\boldsymbol{x})$$

dove $\boldsymbol{x} \in \mathcal{X} \subseteq \mathbb{R}^d$ rappresenta il vettore delle feature di ingresso e $\boldsymbol{\theta} = \{\boldsymbol{W}^{(l)}, \boldsymbol{b}^{(l)}\}_{l=1}^{L}$ il vettore di tutti i parametri addestrabili (pesi e bias).

A ciascun livello intermedio $l \in \{1, \dots, L\}$, la trasformazione affine è seguita dall'applicazione di una funzione di attivazione non lineare componente per componente $\sigma^{(l)}(\cdot)$ (quali ad esempio ReLU, Sigmoide, Tangente iperbolica, SeLU etc...). È utile, in vista della derivazione dell'algoritmo di Backpropagation, distinguere esplicitamente il valore di pre-attivazione $\boldsymbol{a}^{(l)}$ dall'attivazione vera e propria $\boldsymbol{h}^{(l)}$:

$$\boldsymbol{a}^{(l)}(\boldsymbol{x}) = \boldsymbol{W}^{(l)}\boldsymbol{h}^{(l-1)}(\boldsymbol{x}) + \boldsymbol{b}^{(l)}, \qquad \boldsymbol{h}^{(l)}(\boldsymbol{x}) = \sigma^{(l)}\left(\boldsymbol{a}^{(l)}(\boldsymbol{x})\right)$$

con $\boldsymbol{h}^{(0)}(\boldsymbol{x}) = \boldsymbol{x}$, dove $\boldsymbol{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ e $\boldsymbol{b}^{(l)} \in \mathbb{R}^{d_l}$, mentre d_l denota la larghezza del livello l. Il numero di livelli L definisce la profondità della rete.
Il vettore $\boldsymbol{h}^{(l)}(\boldsymbol{x}) \in \mathbb{R}^{d_l}$ costituisce la rappresentazione latente (o embedding) dell'input allo strato l. Dal punto di vista geometrico, ogni strato successivo opera una proiezione dello spazio delle feature originale in un altro spazio, in cui la separabilità delle classi risulta progressivamente più marcata, fino a giungere all'ultimo strato (che opera la classificazione), il quale produce i logit

$$\boldsymbol{z}(\boldsymbol{x}) = \boldsymbol{W}^{(L)}\boldsymbol{h}^{(L-1)}(\boldsymbol{x}) + \boldsymbol{b}^{(L)}$$

e, tramite una normalizzazione probabilistica (funzione Softmax o Sigmoide), assegna la probabilità di appartenenza alle classi di destinazione.
La sequenza $\{\boldsymbol{h}^{(1)}(\boldsymbol{x}), \boldsymbol{h}^{(2)}(\boldsymbol{x}), \dots, \boldsymbol{h}^{(L-1)}(\boldsymbol{x})\}$ definisce dunque una traiettoria delle trasformazioni che l'input attraversa: è precisamente su questa traiettoria, e sulle deviazioni che essa subisce in presenza di perturbazioni adversarial, che si concentra l'analisi layer-wise proposta in questa tesi.

2.2.3 Funzioni di attivazione

La non linearità introdotta dalle funzioni di attivazione è condizione necessaria affinché la profondità della rete si traduca in un incremento di capacità espressiva: in assenza di tali funzioni, la composizione di trasformazioni affini collasserebbe in un'unica trasformazione affine equivalente, riducendo la capacità della rete a quella di un modello lineare, indipendentemente dal numero di livelli. Le scelte più diffuse sono le seguenti.

- ReLU (Rectified Linear Unit), che azzera la componente negativa del segnale:

$$\sigma(z) = \max\{0, z\}$$

- Parametric ReLU, che introduce una pendenza non nulla nel semiasse negativo, governata dal parametro p:

$$\sigma(z) = \begin{cases} z & z > 0 \\ pz & z \leq 0 \end{cases}$$

- Sigmoide logistica, che comprime il segnale nell'intervallo $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Tangente iperbolica, analoga alla precedente ma a codominio simmetrico $(-1, 1)$:

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

2.2.4 Livello di uscita e funzione di loss

La struttura del livello di uscita e la corrispondente funzione di loss dipendono dalla natura del task:

- per un problema di regressione si adotta solitamente un'uscita lineare, $\boldsymbol{y} = \boldsymbol{W}^{(L)}\boldsymbol{h}^{(L-1)} + \boldsymbol{b}^{(L)}$, tipicamente accoppiata all'errore quadratico $\mathcal{L} = \frac{1}{2}\|\boldsymbol{t} - \boldsymbol{y}\|_2^2$;
- per un problema di classificazione binaria, come nel dominio di questo studio per la discriminazione tra traffico benigno e malevolo, si applica una sigmoide ai logit, $y = \sigma(\boldsymbol{W}^{(L)}\boldsymbol{h}^{(L-1)} + \boldsymbol{b}^{(L)})$, interpretando l'uscita come $P(t = 1 \mid \boldsymbol{x})$;
- per un problema di classificazione multiclasse, quale l'identificazione della specifica categoria della minaccia, si adotta un livello softmax:

$$\boldsymbol{z} = \boldsymbol{W}^{(L)}\boldsymbol{h}^{(L-1)} + \boldsymbol{b}^{(L)}, \qquad y_k = \frac{e^{z_k}}{\sum_{k'=1}^{K} e^{z_{k'}}}$$

La softmax approssima l'operatore argmax, poiché se il modello rende z_k sensibilmente maggiore delle altre componenti si ha $y_k \approx 1$, ma a differenza di quest'ultimo è differenziabile e dunque compatibile con l'ottimizzazione basata sul gradiente.

2.2.5 L'addestramento come problema di ottimizzazione

Addestrare una rete neurale significa determinare il valore dei parametri $\boldsymbol{\theta}$, comprensivo delle matrici dei pesi $\boldsymbol{W}^{(l)}$ e dei vettori di bias $\boldsymbol{b}^{(l)}$ per ogni $1 \leq l \leq L$, che minimizza una

funzione di costo definita su un dataset $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$:

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) = \frac{1}{N} \sum_{i=1}^{N} \ell(f(\boldsymbol{x}_i; \boldsymbol{\theta}), y_i)$$

dove $\ell(\cdot, \cdot)$ denota la loss calcolata sul singolo campione. Alla funzione di costo si aggiunge in genere un termine di regolarizzazione $\Omega(\boldsymbol{\theta})$, pesato da un coefficiente λ, che esprime una preferenza a priori su determinate configurazioni dei parametri:

$$\tilde{\mathcal{L}}(\boldsymbol{\theta}; \mathcal{D}) = \mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) + \lambda\Omega(\boldsymbol{\theta})$$

Rispetto a un problema di ottimizzazione puro, l'apprendimento presenta due differenze sostanziali. La prima è che esso agisce in modo indiretto: la quantità effettivamente minimizzata è il rischio empirico calcolato sul training set, mentre l'obiettivo reale è la minimizzazione dell'errore atteso su dati non osservati. La seconda è che la procedura non viene arrestata al raggiungimento di un minimo locale, bensì quando un criterio di convergenza definito sul validation set risulta soddisfatto (es. la logica dell'early stopping). La non linearità delle reti neurali rende inoltre la funzione di costo non convessa, per cui i metodi basati sul gradiente non offrono alcuna garanzia di convergenza al minimo globale e il punto raggiunto dipende dall'inizializzazione. Nella pratica questo si rivela meno penalizzante di quanto si potrebbe temere: si osserva infatti che individuare un minimo locale di qualità sufficiente offre ugualmente una soluzione adeguata.

2.2.6 Discesa del gradiente e varianti stocastiche

La discesa del gradiente è un algoritmo iterativo impiegato quando la soluzione analitica del problema di minimizzazione non è disponibile oppure è computazionalmente troppo onerosa. La strategia consiste nell'inizializzare i parametri a un valore ragionevole e nel correggerli ripetutamente nella direzione opposta a quella di massima crescita del costo:

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial \mathcal{L}}{\partial \theta_j}, \qquad \text{ovvero} \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$$

dove α è il learning rate e descrive una misura del passo di aggiornamento compiuto ad ogni iterazione di training. Il segno della derivata parziale determina infatti il verso della correzione: se $\partial\mathcal{L}/\partial\theta_j > 0$ un incremento di θ_j fa crescere il costo, e viceversa.
Il calcolo del gradiente sull'intero dataset, noto come batch training, risulta impraticabile

per grandi moli di dati. La discesa del gradiente stocastica (Stochastic Gradient Descent, SGD) stima invece il gradiente su un singolo campione estratto casualmente, mentre nella pratica si adotta una soluzione intermedia basata su un sottoinsieme $\mathcal{B}$ del training set, detto minibatch:

$$\boldsymbol{g} \leftarrow \frac{1}{|\mathcal{B}|}\nabla_{\boldsymbol{\theta}}\mathcal{L}^{\mathcal{B}}(\boldsymbol{\theta}), \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \boldsymbol{g}$$

L'uso dei minibatch offre un importante vantaggio, ovvero di vettorializzare le operazioni e sfruttare il parallelismo hardware, oltre a favorire la convergenza con l'impiego di più campioni simultaneamente per un'iterazione di aggiornamento. La dimensione del minibatch costituisce un ulteriore iperparametro, e viene convenzionalmente scelta come potenza di due per allinearsi all'organizzazione delle unità di calcolo parallele. Si definisce infine epoca un insieme di iterazioni dell'algoritmo di aggiornamento entro il quale ogni istanza del training set viene osservata esattamente una volta.

Anche la scelta del learning rate incide in modo determinante sulla convergenza: un valore troppo elevato può impedire il raggiungimento del minimo, facendo oscillare i parametri attorno ad esso o divergere, mentre un valore troppo ridotto richiede un numero eccessivo di iterazioni. L'approccio ragionato consiste nel partire da un valore elevato, per avvicinarsi rapidamente alla regione di interesse, e nel decrementarlo progressivamente per aumentare la precisione degli ultimi passi.

2.2.7 Algoritmo di addestramento: Backpropagation

Il punto critico dell'applicazione dei metodi a gradiente alle reti profonde consiste nel calcolo efficiente di $\nabla_{\boldsymbol{\theta}}\mathcal{L}$: mentre in un modello a singolo livello la derivazione è immediata, in una rete a L livelli la funzione di costo è una funzione composta di tutti i livelli precedenti. L'algoritmo di retropropagazione dell'errore (backpropagation) risolve il problema applicando sistematicamente la regola di derivazione delle funzioni composte, e si fonda sull'assunzione che il grafo computazionale della rete sia ordinato topologicamente, ossia che l'esistenza di un arco da h_i a h_j implichi che h_i preceda h_j nell'ordinamento. L'algoritmo si articola in due fasi: fase in avanti (Forward) e fase all'indietro (Backward).

Nella fase in avanti l'istanza di training viene propagata attraverso i livelli della rete, memorizzando i valori intermedi necessari alla fase successiva, e l'uscita finale è impiegata per calcolare la funzione di costo:

Algoritmo 1 Forward pass

$\boldsymbol{h}^{(0)} \leftarrow \boldsymbol{x}$
for $k = 1, \dots L$ **do**
  $\boldsymbol{a}^{(k)} \leftarrow \boldsymbol{W}^{(k)}\boldsymbol{h}^{(k-1)} + \boldsymbol{b}^{(k)}$
  $\boldsymbol{h}^{(k)} \leftarrow \sigma^{(k)}\left(\boldsymbol{a}^{(k)}\right)$
end
$\boldsymbol{y} \leftarrow \boldsymbol{h}^{(L)}$
return $\tilde{\mathcal{L}} = \mathcal{L}(\boldsymbol{y}, \boldsymbol{t}) + \lambda\Omega(\boldsymbol{\theta})$

Nella fase all'indietro si calcola il gradiente della funzione di costo rispetto a tutti i parametri, attraversando la rete in direzione opposta. Introdotta la variabile ausiliaria $\boldsymbol{g}$, che trasporta il gradiente accumulato dai livelli successivi, l'algoritmo procede come segue:

Algoritmo 2 Backward pass

$\boldsymbol{g} \leftarrow \nabla_{\boldsymbol{y}}\tilde{\mathcal{L}} = \nabla_{\boldsymbol{y}}\mathcal{L}(\boldsymbol{y}, \boldsymbol{t})$
for $k = L, \dots 2, 1$ **do**
  $\boldsymbol{g} \leftarrow \nabla_{\boldsymbol{a}^{(k)}}\tilde{\mathcal{L}} = \boldsymbol{g} \odot \sigma^{(k)\prime}\left(\boldsymbol{a}^{(k)}\right)$
  $\nabla_{\boldsymbol{b}^{(k)}}\tilde{\mathcal{L}} = \boldsymbol{g} + \lambda\nabla_{\boldsymbol{b}^{(k)}}\Omega(\boldsymbol{\theta})$
  $\nabla_{\boldsymbol{W}^{(k)}}\tilde{\mathcal{L}} = \boldsymbol{g} \cdot \boldsymbol{h}^{(k-1)\top} + \lambda\nabla_{\boldsymbol{W}^{(k)}}\Omega(\boldsymbol{\theta})$
  $\boldsymbol{g} \leftarrow \nabla_{\boldsymbol{h}^{(k-1)}}\tilde{\mathcal{L}} = \boldsymbol{W}^{(k)\top}\boldsymbol{g}$ `// Propaga il gradiente`
end
return $\nabla_{\boldsymbol{W}^{(k)}}\tilde{\mathcal{L}},\ \nabla_{\boldsymbol{b}^{(k)}}\tilde{\mathcal{L}},\ \forall\ k = 1, \dots L$

dove $\odot$ denota il prodotto componente per componente.

È importante osservare che l'algoritmo di backpropagation calcola, come sottoprodotto, anche il gradiente della funzione di costo rispetto all'ingresso, $\nabla_{\boldsymbol{x}}\mathcal{L}$, ottenuto semplicemente proseguendo la ricorsione fino a $k = 1$. Questa quantità non ha alcun ruolo nell'addestramento, dal momento che $\boldsymbol{x}$ non è un parametro addestrabile, ma costituisce esattamente l'informazione sfruttata dagli attacchi di evasione white-box descritti nella Sezione 2.3: la stessa differenziabilità che rende addestrabile una rete profonda ne rende al tempo stesso sistematicamente attaccabile la superficie decisionale.

2.2.8 Stabilità della propagazione e implicazioni per la robustezza

L'analisi della dinamica dei gradienti nelle reti profonde mette in luce un fenomeno che assume un ruolo centrale nell'impostazione di questa tesi. Si consideri una matrice $\boldsymbol{W}$ ripetutamente moltiplicata lungo il grafo computazionale: dopo t passi il suo effetto equivale a $\boldsymbol{W}^t$, e detta $\boldsymbol{V}$ la matrice degli autovettori e $\mathrm{diag}(\boldsymbol{\lambda})$ quella degli autovalori, si ottiene

$$\boldsymbol{W}^t = \boldsymbol{V} \cdot \mathrm{diag}(\boldsymbol{\lambda})^t \cdot \boldsymbol{V}^{-1}$$

Per $t \to \infty$ il comportamento è interamente governato dal modulo degli autovalori: se $|\lambda_i| < 1$ il contributo associato si estingue, dando luogo al fenomeno del gradiente evanescente (Vanishing Gradient), mentre se $|\lambda_i| > 1$ cresce senza limite, producendo l'esplosione del gradiente (Exploding Gradient). Nel primo caso i livelli iniziali della rete ricevono un segnale di aggiornamento trascurabile e l'addestramento si arresta di fatto; nel secondo gli aggiornamenti divergono e rendono instabile l'ottimizzazione.

Il medesimo meccanismo, letto nella direzione opposta, descrive la propagazione in avanti di una perturbazione applicata all'ingresso [81]. Si denoti con $f^{(l)}$ la funzione che compone la rete al livello l e $\boldsymbol{h}^{(l)}$ l'uscita del livello nascosto l. Detta $\boldsymbol{\delta}$ una perturbazione iniziale e $\boldsymbol{\delta}^{(l)} = f^{(l)}(\boldsymbol{h}^{(l-1)} + \boldsymbol{\delta}^{(l-1)}) - f^{(l)}(\boldsymbol{h}^{(l-1)})$ la deviazione da essa indotta sulla rappresentazione latente allo strato l, uno sviluppo al primo ordine fornisce

$$f^{(l)}(\boldsymbol{h}^{(l-1)} + \boldsymbol{\delta}^{(l-1)}) \approx f^{(l)}(\boldsymbol{h}^{(l-1)}) + \nabla f^{(l)}(\boldsymbol{h}^{(l-1)}) \cdot \boldsymbol{\delta}^{(l-1)}$$

e sostituendolo nella formula di $\boldsymbol{\delta}^{(l)}$ il risultato è

$$\boldsymbol{\delta}^{(l)} \approx \boldsymbol{J}^{(l)} \boldsymbol{\delta}^{(l-1)}$$

con $\boldsymbol{\delta}^{(0)} = \boldsymbol{\delta}$ e dove $\boldsymbol{J}^{(l)}$ è il jacobiano del livello l-esimo, da cui, iterando nella profondità della rete si ottiene

$$\|\boldsymbol{\delta}^{(L-1)}\| \leq \left(\prod_{l=1}^{L-1} \|\boldsymbol{J}^{(l)}\| \right) \|\boldsymbol{\delta}\|$$

La norma del prodotto delle matrici jacobiane di ciascun livello fornisce dunque un limite superiore all'amplificazione che una perturbazione può subire attraversando la rete. Poiché tale prodotto cresce in modo moltiplicativo con la profondità, una deviazione impercettibile nello spazio di input può tradursi in uno spostamento considerevole nello spazio delle attivazioni degli ultimi livelli, sufficiente ad attraversare la frontiera decisionale e a determinare una classificazione errata.

Da questa osservazione discendono due conseguenze che motivano direttamente l'impostazione metodologica adottata nel seguito. La prima è che il contributo all'amplificazione complessiva non è distribuito uniformemente lungo la rete, ma dipende dalle proprietà di ciascuno strato. La seconda è che la deviazione adversarial è osservabile nelle rappresentazioni latenti prima che essa raggiunga il livello di classificazione: le attivazioni intermedie contengono pertanto informazione sfruttabile a fini di rilevamento, infor-

mazione che un'analisi limitata alla relazione ingresso-uscita, ossia di tipo black-box, scarta integralmente.

2.3 Adversarial Machine Learning e Attacchi di Evasione

2.3.1 Origine e natura del fenomeno

Nonostante l'elevata capacità espressiva, le reti neurali profonde manifestano un'intrinseca instabilità quando sottoposte a manipolazioni dell'input. L'Adversarial Machine Learning (AML) è la disciplina che analizza le vulnerabilità dei modelli di apprendimento e studia metodologie per quantificare e migliorare la loro robustezza, collocandosi di fatto all'intersezione tra Machine Learning, Computer Science e Cybersecurity [66].

La consapevolezza che il fenomeno interessasse le reti neurali profonde emerge con l'osservazione che ogni rete, anche se addestrata fino a raggiungere accuratezze elevatissime, possiede regioni di input (dette "blind spots") in cui una perturbazione di norma trascurabile è sufficiente a ribaltare la predizione [81]. Lo stesso lavoro evidenzia una proprietà molto particolare: i campioni adversarial si trasferiscono tra modelli addestrati indipendentemente, anche da dataset differenti ma sullo stesso task, riuscendo comunque a ingannare il classificatore.

L'interpretazione del fenomeno inizialmente proposta attribuisce la causa alla non linearità e all'overfitting; è stato successivamente argomentato che la causa dominante sia invece la natura eccessivamente lineare delle reti in spazi ad alta dimensionalità, dove una perturbazione infinitesima per componente produce, sommata su migliaia di dimensioni, una variazione macroscopica dell'attivazione [32]. Una lettura complementare e più recente considera i campioni adversarial non come un difetto dell'ottimizzazione ma come la conseguenza dell'esistenza di feature non robuste, predittive sul piano statistico ma fragili rispetto a perturbazioni limitate [38].

2.3.2 Tassonomia e modello delle minacce

La rilevanza operativa del problema è oggi riconosciuta anche a livello normativo e di standardizzazione: il National Institute of Standards and Technology (NIST) ha prodotto una tassonomia e una terminologia unificate degli attacchi e delle mitigazioni nel ciclo di vita dei sistemi di apprendimento, affiancata da una raccolta di risorse applicative dedicate

([58], [83]).

Seguendo la tassonomia NIST, un attacco può essere caratterizzato lungo quattro assi [83]:

1. Obiettivo dell'attaccante: violazione dell'integrità (indurre una classificazione errata), della disponibilità (degradare le prestazioni complessive del sistema) o della riservatezza (estrarre informazioni sul modello o sui dati di addestramento).
2. Fase del ciclo di vita colpita: attacchi in fase di training (poisoning) oppure in fase di inferenza (evasion). La presente tesi si concentra esclusivamente sul secondo caso, sebbene esistano recenti framework di certificazione formale che trattano in modo unificato entrambe le famiglie [82].
3. Conoscenza dell'attaccante: dal regime white-box, in cui architettura, iperparametri, pesi e grafo di calcolo dei gradienti sono noti, al regime black-box, in cui il modello è accessibile solo tramite query; esiste anche un range di attacchi gray-box in cui l'attaccante ha una conoscenza parziale del modello.
4. Capacità dell'attaccante: insieme delle manipolazioni ammissibili (tipicamente formalizzate mediante un vincolo di norma L_∞ o L_2), controllo dei dati di training (backdoor poisoning), controllo del modello stesso (ad esempio in contesti di federated learning iniettando aggiornamenti malevoli), accesso tramite query o al codice sorgente (in contesti di MLaaS).

Nell'ambito dei sistemi di classificazione, la minaccia primaria in fase di inferenza è costituita dagli attacchi di evasione (evasion attacks), nei quali l'attaccante genera campioni adversarial. Dato un campione $\boldsymbol{x} \in \mathcal{X}$ con etichetta reale y, un campione adversarial $\boldsymbol{x}_{adv}$ è formalmente definito come:

$$\boldsymbol{x}_{adv} = \boldsymbol{x} + \boldsymbol{\delta}, \quad \text{tale che } f(\boldsymbol{x}_{adv}) \neq y \ \wedge \ \|\boldsymbol{\delta}\|_p \leq \varepsilon$$

dove $\boldsymbol{\delta}$ rappresenta il vettore di perturbazione calcolato dall'attaccante, $\varepsilon > 0$ è un budget di perturbazione e $\|\cdot\|_p$ indica una norma geometrica (generalmente la norma infinito L_∞ per limitare l'alterazione massima della singola feature, oppure la norma euclidea L_2).

È utile distinguere due formulazioni duali del problema di generazione. Nella formulazione a perdita massima l'attaccante fissa il budget ε e massimizza la funzione di costo entro la sfera ammissibile; è il paradigma adottato da FGSM e PGD. Nella formulazione a norma

minima l'attaccante fissa l'obiettivo di misclassificazione e minimizza la norma della perturbazione necessaria a raggiungerlo, ottenendo una misura diretta della distanza dal confine decisionale ([9], [15]).
Infine, si distingue tra attacchi non mirati (untargeted), che richiedono soltanto $f(\boldsymbol{x}_{adv}) \neq y$, e attacchi mirati (targeted), che impongono $f(\boldsymbol{x}_{adv}) = t \neq y = f(\boldsymbol{x})$ per una classe bersaglio t scelta dall'attaccante. Nel dominio del rilevamento delle intrusioni la distinzione tende a collassare, dato che il task binario rende l'attacco non mirato coincidente con l'attacco mirato verso la classe benigna, ovvero con la generazione di un falso negativo, e questo è infatti il caso discusso in questa tesi.

2.3.3 Attacchi white-box

Tra le formulazioni fondamentali per la generazione di campioni adversarial in regime white-box, assumono particolare rilievo:

- Fast Gradient Sign Method (FGSM): tecnica one-step che perturba l'input in modo da massimizzare la funzione di costo, e dunque l'errore di classificazione, lungo la direzione di massima pendenza, ma entro un intorno limitato in norma L_∞ [32]:

  $$\boldsymbol{x}_{adv} = \boldsymbol{x} + \varepsilon \cdot \operatorname{sign}\left(\nabla_{\boldsymbol{x}} \mathcal{L}(f(\boldsymbol{x}), y)\right)$$

  Il metodo, estremamente economico poiché richiede una sola iterazione di back-propagation, è stato inizialmente proposto non solo come attacco ma anche come termine di regolarizzazione per l'adversarial training.
- Basic Iterative Method (BIM): applicazione ripetuta di FGSM con passo ridotto e clipping della perturbazione nell'intorno ammissibile, che migliora sensibilmente l'efficacia a parità di budget ε [42].
- Projected Gradient Descent (PGD): estensione iterativa a passo multiplo e inizializzazione casuale all'interno della sfera ammissibile, è considerato l'attacco di primo ordine standard per la valutazione di robustezza [50]:

  $$\boldsymbol{x}^{(t+1)} = \Pi_S\left(\boldsymbol{x}^{(t)} + \alpha \cdot \operatorname{sign}\left(\nabla_{\boldsymbol{x}^{(t)}} \mathcal{L}(f(\boldsymbol{x}^{(t)}), y)\right)\right)$$

  dove Π è l'operatore di proiezione sulla sfera ammissibile $S = \{\boldsymbol{z} \mid \|\boldsymbol{z} - \boldsymbol{x}\|_p \leq \varepsilon\}$ e α è la dimensione del passo di aggiornamento. Il ruolo di PGD come riferimento non è

arbitrario: gli autori mostrano sperimentalmente che, nonostante la non concavità del problema interno, le ripetizioni con punto di partenza casuale convergono a valori di loss concentrati attorno a un massimo comune, il che consente di trattare PGD come un approssimatore affidabile dell'attaccante di primo ordine universale.

- DeepFool: metodo iterativo che approssima localmente il confine decisionale con un iperpiano e calcola la proiezione ortogonale minima su di esso, fornendo una stima efficiente della distanza dal confine [55].
- Attacco di Carlini e Wagner: formulato come un problema di minimizzazione non vincolata: si cerca la minima perturbazione capace di cambiare il risultato di predizione. Le tre varianti in norma L_0, L_2 e L_∞ hanno dimostrato tassi di successo del 100 percento contro la defensive distillation, all'epoca considerata una difesa efficace, riducendone la robustezza apparente a zero ([9]; [64]).
- AutoAttack: ensemble che combina quattro attacchi complementari parameter-free, ovvero APGD-CE e APGD-DLR (varianti di PGD), FAB (attacco a norma minima) e Square Attack (attacco black-box gradient-free, basato su ricerca casuale). L'eliminazione degli iperparametri da tarare manualmente rende la valutazione riproducibile e ha permesso di individuare numerose difese la cui robustezza dichiarata era sovrastimata ([16]; [15]; [2]).

2.3.4 Attacchi black-box e trasferibilità

Il regime black-box è operativamente più realistico nei contesti di sicurezza, dove il difensore raramente espone i pesi del modello. Le strategie si dividono in due famiglie.

- La prima sfrutta la trasferibilità [81]: l'attaccante addestra un modello surrogato su un dataset proprio, interroga il modello bersaglio un numero contenuto di volte, genera campioni adversarial in white-box sul surrogato e li riusa contro il bersaglio vero e proprio [63]. L'efficacia di questa strategia è stata spiegata in termini di allineamento dei gradienti tra i modelli, complessità del modello surrogato e varianza della soluzione ottima: modelli meno complessi e con superfici di loss più regolari producono perturbazioni che trasferiscono meglio [19]. Anche l'aggiunta di un termine funzione del momento nel calcolo del campione adversarial stabilizza significativamente la trasferibilità [21].

- La seconda famiglia stima direttamente l'informazione necessaria interrogando il modello bersaglio dell'attacco: metodi a ordine zero che approssimano il gradiente tramite differenze [11], metodi decision-based che richiedono soltanto la label predetta [6] e metodi basati su ricerca casuale strutturata [2].

2.3.5 Panorama delle contromisure

Le difese proposte in letteratura possono essere ricondotte a quattro macro-famiglie.

- Adversarial training. È la contromisura empiricamente più solida e consiste nel risolvere il problema di ottimizzazione a sella (Saddle Point Problem)

$$\min_{\boldsymbol{\theta}} \mathbb{E}_{(\boldsymbol{x},y)\sim\mathcal{D}} \left[\max_{\|\boldsymbol{\delta}\|_p \leq \varepsilon} \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{x} + \boldsymbol{\delta}, y) \right]$$

  dove $\boldsymbol{\theta}$ sono i parametri del modello e $\mathcal{L}$ la sua loss function, $(\boldsymbol{x}, y)$ sono le feature e le corrispettive label estratte dalla distribuzione di dati $\mathcal{D}$, e $\boldsymbol{\delta}$ è una perturbazione ammissibile (ovvero limitata in norma L_p).

  La natura minmax dell'adversarial training può essere osservata sotto altra luce se scomposta nei due sottoproblemi: il problema interno, quello di massimizzazione, che corrisponde all'atto di attaccare la rete neurale con una perturbazione che massimizza l'errore; il problema esterno, di minimizzazione, che equivale invece all'operazione di difendere il modello attaccato, trovando i parametri che minimizzano il valore atteso della loss massimizzata nel problema interno, di fatto minimizzando la minaccia adversarial sulla distribuzione dei dati complessiva.

  Il problema interno di massimizzazione viene approssimato da un attacco iterativo, tipicamente PGD [50]. Questo metodo presenta però due limiti noti: un costo computazionale che cresce linearmente con il numero di iterazioni interne all'attacco, e una tendenza al robust overfitting, per cui la robustezza sul test set degrada dopo un certo punto dell'addestramento [71].

  Il problema a sella è tuttavia NP-hard: già il solo problema interno è NP-hard per reti con attivazioni ReLU [88]. Non esistendo metodi esatti in grado di scalare a reti di dimensioni realistiche, nella pratica oggi il problema viene affrontato esclusivamente tramite euristiche, che forniscono soluzioni approssimate e prive di garanzie di ottimalità. Di conseguenza, la robustezza di una difesa è sempre misurata rispetto a

un attacco specifico e non in assoluto, ed è per questo che molte difese sono state rapidamente aggirate da attacchi più forti ([4], [8]).

- Robustezza certificata. A differenza delle difese empiriche, forniscono una garanzia formale che la predizione rimanga invariata entro un raggio calcolabile. Gli approcci deterministici procedono tramite il rilassamento convesso del "politopo adversarial" [89] o per altri tipi di rilassamento del problema [70], ma comportano il problema di scalare con grave difficoltà oltre reti di dimensioni contenute. Gli approcci probabilistici invece, basati su randomized smoothing, costruiscono un classificatore derivato addestrato sotto rumore gaussiano, e derivano un raggio di certificazione sufficientemente stretto in norma L_2, con garanzie che si applicano a reti di profondità arbitraria [13].

  Il limite principale di questa famiglia di difese è la dimensionalità, che fa collassare il raggio certificato al crescere di d. Lavori recenti affrontano il problema, proiettando lo spazio delle feature in un sottospazio lineare a bassa dimensionalità prima di applicare lo smoothing, ottenendo regioni certificate di volume superiore di diversi ordini di grandezza e robustezza ad attacchi L_∞-bounded [67].
- Purificazione. Questa difesa non modifica il classificatore ma agisce sull'input, rimuovendo la componente adversarial prima dell'inferenza. Gli approcci basati su modelli di diffusione aggiungono del rumore controllato per poi invertire il processo, riportando il campione sulla distribuzione dei dati puliti [59]. Varianti successive eliminano la necessità di calcolare gradienti nel processo di purificazione [18]. Va tuttavia osservato che anche questa famiglia è stata successivamente attaccata con metodi appositamente progettati per aggirare il processo di purificazione [40].
- Rilevamento (detection). Anziché classificare correttamente il campione adversarial, questa famiglia si limita a riconoscerlo e a rifiutarlo, delegando la decisione a un modulo dedicato o a una politica di astensione. Questo approccio è particolarmente adatto al dominio NIDS, dove il rifiuto di un flusso sospetto è un'azione operativamente accettabile e un falso allarme costa meno di un falso negativo. Numerose proposte operano direttamente sull'input, mediante classificatori binari addestrati sui dati grezzi [30], test statistici sulla loro distribuzione [33] o tramite l'analisi delle componenti principali [36]. Tali metodi, che non accedono agli stati interni della rete,

sono stati sistematicamente aggirati [8]. L'analisi di questi fallimenti ha mostrato che l'input grezzo offre un segnale troppo povero. Da qui la scelta, adottata in questa tesi, di approfondire il rilevamento sulle rappresentazioni latenti interne ai modelli.

2.3.6 Vincoli del dominio nei flussi di rete

Gran parte della letteratura sopra richiamata è stata sviluppata e validata nel dominio della computer vision, in cui la nozione di impercettibilità è ben definita rispetto alla percezione umana e ogni pixel è liberamente perturbabile. Nel dominio dei flussi di rete questa assunzione decade: le feature combinano variabili continue, discrete e categoriche, molte delle quali legate da dipendenze funzionali o vincolate dalla semantica dei protocolli, e l'attaccante controlla soltanto un sottoinsieme delle grandezze osservate dal difensore. La definizione stessa di perturbazione ammissibile richiede quindi di sostituire il vincolo di norma con un insieme di vincoli di dominio.

2.4 L'Intersezione dei Tre Ambiti

La convergenza di questi tre ambiti delinea il problema cardine di questa tesi:

1. Il NIDS neurale richiede modelli profondi ed espressivi per gestire la complessità e la variabilità statistica dei moderni flussi di traffico;
2. L'attaccante sfrutta la natura differenziabile della rete per generare vettori di attacco in grado di mascherare un flusso malevolo facendolo classificare come traffico lecito (false negative), eludendo i controlli di sicurezza;
3. Il limite degli approcci attuali risiede nel fatto che la quasi totalità delle contromisure valuta l'interazione tra perturbazione $\boldsymbol{\delta}$ e predizione finale unicamente come un fenomeno ingresso-uscita (black-box).

In realtà, l'efficacia di un'evasione scaturisce dalla dinamica interna di propagazione: una deviazione controllata $\boldsymbol{\delta}$ iniettata nello spazio di input può essere progressivamente amplificata dalle successive trasformazioni non lineari $\{f^{(1)}, f^{(2)}, \dots, f^{(L)}\}$, distorcendo la traiettoria degli stati latenti $\boldsymbol{h}^{(l)}(\boldsymbol{x})$ verso il collasso della decisione finale in una arbitraria. È proprio da questa osservazione che trae origine la motivazione di un'indagine layer-wise: analizzare lo spazio delle attivazioni nascoste non come una black-box, ma come una

sequenza divisibile di stadi computazionali differenziati, per monitorare, contrastare e spiegare l'evoluzione della minaccia adversarial direttamente all'interno della struttura della rete.

Capitolo 3

Stato dell'arte

3.1 Deep Learning nei Network Intrusion Detection Systems

L'adozione di modelli neurali profondi nel rilevamento delle intrusioni di rete costituisce oggi un filone di ricerca molto ricco. La motivazione di fondo è duplice: da un lato la capacità delle architetture profonde di apprendere funzioni non lineari complesse e rappresentazioni direttamente dai dati; dall'altro, la disponibilità di volumi di telemetria di rete che eccedono largamente la capacità di analisi umana, e che si prestano dunque in maniera naturale ai metodi basati sul machine learning [27]. Molti Intrusion Detection System in commercio fanno uso di questa tipologia di strumenti già da tempo, come ad esempio le piattaforme di sicurezza Cisco Stealthwatch e Microsoft Azure Sentinel ([12], [53]), o prodotti più recenti e aggiornati come Cortex XSIAM di Palo Alto Networks, CrowdStrike Falcon o Fortinet ([60], [17], [25]).

Rassegne del settore [80] concordano nel collocare l'origine di questa transizione nel momento in cui i metodi di machine learning tradizionali — SVM, alberi decisionali, k-NN, Naïve Bayes — pur restando competitivi su dataset di dimensione contenuta, sono diventati inadeguati, siccome saturano rapidamente le proprie capacità all'aumentare della dimensionalità e dell'eterogeneità dello spazio delle feature.

Di seguito, la letteratura viene organizzata considerando al tempo stesso due filoni ortogonali: la rappresentazione dell'input adottata (istanza tabulare, sequenza, immagine, grafo, datagramma grezzo) e il paradigma di apprendimento (supervisionato, semi-supervisionato, non supervisionato, generativo).

3.1.1 Modelli feed-forward e classificazione di istanze

La formulazione più diretta tratta ciascuna istanza di flusso come un vettore indipendente, classificato da una rete neurale feed-forward. Nonostante la semplicità concettuale, questa famiglia costituisce tuttora una baseline estremamente competitiva: valutazioni su larga scala di architetture profonde su più dataset eterogenei riportano prestazioni allo stato dell'arte con costi di addestramento e inferenza contenuti [85]. Lo studio comparativo condotto da [27] è in questo senso particolarmente istruttivo, poiché mostra che su quattro dataset — due storici e due moderni — una rete feed-forward supervisionata domina, in termini di accuratezza, F1-score, tempi di addestramento e inferenza, sia gli autoencoder, sia le deep belief network, sia le reti LSTM, e che architetture profonde a struttura "a imbuto" (strati progressivamente più stretti) superano sistematicamente reti larghe e poco profonde. Analoghe conclusioni emergono da lavori applicativi recenti che confrontano modelli profondi e algoritmi classici sugli stessi benchmark [69].

3.1.2 Approcci semi-supervisionati e non supervisionati

La scarsità e il costo delle etichette hanno motivato un'ampia produzione di modelli in grado di sfruttare traffico non annotato. Gli autoencoder occupano una posizione centrale: addestrati a ricostruire il proprio input, apprendono una rappresentazione compressa del traffico e segnalano come anomalo ogni campione caratterizzato da un elevato errore di ricostruzione ([39], [79]).

Il contributo più influente sul piano sistemistico è Kitsune [54], che rinuncia a un singolo modello profondo in favore di un ensemble di autoencoder di piccole dimensioni, ciascuno assegnato a un sottoinsieme correlato di feature; gli errori di ricostruzione dei componenti dell'ensemble vengono aggregati da un autoencoder di uscita che funge da meccanismo di voto finale, e l'intero sistema opera in modalità online, completamente non supervisionata, conservando in memoria un solo campione alla volta e risultando eseguibile persino su un Raspberry Pi. Questo lavoro è rilevante per la tesi in discussione anche sul piano metodologico, poiché mostra come la scomposizione di un modello in componenti specializzate su porzioni distinte dello spazio di rappresentazione possa migliorare simultaneamente efficienza e capacità discriminativa.

Va tuttavia considerato che, come citato nel paragrafo 3.1.1, le valutazioni più recenti non

confermano alcun vantaggio del pre-training non supervisionato rispetto all'addestramento supervisionato diretto sui medesimi dati [27].

3.1.3 Modelli convoluzionali

Le reti convoluzionali sono state introdotte nel dominio attraverso due strategie distinte. La prima converte il traffico grezzo in rappresentazioni bidimensionali in scala di grigi, trattando i byte dei pacchetti come pixel e riconducendo la classificazione del traffico a un problema di riconoscimento di immagini [86]. La seconda, più coerente con la natura del dato, applica convoluzioni lungo la sequenza di byte del payload o lungo il vettore di feature, con l'obiettivo ulteriore di catturare dipendenze spaziali di byte vicini [47]. Valutazioni recenti riportano per modelli convoluzionali accuratezze superiori al 98% su benchmark standard, con un distacco in performance significativo rispetto agli algoritmi di machine learning tradizionali costruiti sulle stesse feature [69]. Occorre al tempo stesso osservare che l'ipotesi di località spaziale su cui si fonda la convoluzione non ha giustificazione nel caso di vettori di feature di flusso, il cui ordinamento delle colonne è arbitrario: parte del vantaggio riportato è quindi attribuibile alla capacità del modello più che alla attinenza dell'architettura.

3.1.4 Modelli ricorrenti e classificazione di sequenze

Il traffico di rete possiede una struttura temporale intrinseca, sia all'interno del singolo flusso sia nella successione di flussi generati da uno stesso host. Le reti neurali ricorrenti sono state infatti impiegate per classificare sequenze di record relativi a flussi temporalmente adiacenti, piuttosto che studiarli come istanze isolate [92]. Lavori più recenti combinano reti ricorrenti, dotate di meccanismi di gate, con pipeline di normalizzazione e riduzione della dimensionalità a monte, migliorando al contempo accuratezza e tempi di convergenza [57]. Il limite operativo principale di questa famiglia di classificatori risiede nel costo di inferenza: la valutazione comparativa in [27] riporta per le architetture ricorrenti tempi di classificazione superiori di oltre un ordine di grandezza rispetto alle reti feed-forward, tempi dovuti alla complessità di queste reti, a fronte di un guadagno in accuratezza marginale se non nullo; si tratta di un aspetto rilevante, poiché in contesti critici come quello in esame la tempestività degli allarmi costituisce un fattore preponderante.

3.1.5 Rappresentazioni a grafo

Un flusso di rete è per definizione una relazione fra due endpoint, e un insieme di flussi induce naturalmente un grafo in cui gli host sono nodi e le comunicazioni archi. Le Graph Neural Network (GNN) sfruttano esplicitamente questa struttura, aggregando le informazioni dei nodi adiacenti di ciascun elemento. Il contributo di riferimento in ambito NIDS estende l'algoritmo GraphSAGE [35] al caso in cui l'informazione discriminante risiede sugli archi anziché sui nodi, effettuando la classificazione di flussi malevoli mediante classificazione degli edge e superando lo stato dell'arte su quattro benchmark NIDS recenti [46]. Un'ulteriore estensione di quest'ultimo lavoro applica una versione self-supervised delle GNN con l'idea di eliminare la dipendenza dalle label, massimizzare l'uso delle informazioni sugli archi e aumentare la capacità di riconoscere attacchi mai visti prima in fase di addestramento [10]. L'interesse di questa linea per il problema della robustezza è duplice: l'informazione sulla topologia della rete fornisce un segnale che l'attaccante controlla solo parzialmente, poiché modificare la reale struttura delle connessioni di rete ha un costo operativo non indifferente, ed è stato argomentato che tale proprietà conferisce alle GNN una resistenza intrinseca superiore agli attacchi di evasione basati sulla sola manipolazione delle feature di flusso [68].

3.1.6 Transformer, pre-training e rappresentazioni trasferibili

L'ultima generazione di modelli importa nel dominio della rete il paradigma basato sul meccanismo di attenzione affermatosi nell'elaborazione del linguaggio naturale [84]. ET-BERT rappresenta il riferimento di questa categoria: trattando i datagrammi come sequenze di token, il modello apprende rappresentazioni contestualizzate da grandi volumi di traffico cifrato e non etichettato, ottenendo miglioramenti sostanziali su cinque differenti task di classificazione del traffico e mantenendo l'efficacia su TLS 1.3, dove i metodi basati su ispezione del plaintext falliscono [45].

3.1.7 Sintesi critica

Dall'analisi della letteratura emergono chiaramente tre limiti condivisi dai moderni NIDS basati su Deep Learning. In primo luogo, le metriche riportate sui dataset storici sono ormai saturate, con accuratezze stabilmente superiori al 98–99%. A fronte di questi numeri,

l'impiego di architetture complesse non offre un reale vantaggio: studi comparativi controllati [27] mostrano che una rete feed-forward ben regolarizzata è comparabile o supera modelli convoluzionali o ricorrenti, garantendo latenze di inferenza decisamente inferiori. In secondo luogo, la letteratura soffre di una marcata carenza sul piano della riproducibilità sperimentale. Troppi lavori omettono la documentazione puntuale dell'addestramento, delle strategie di partizionamento dei dataset e delle pipeline di preprocessing, impedendo di fatto una replica affidabile dei risultati o una comparazione rigorosa tra approcci diversi. Infine, la letteratura si limita quasi sempre a testare i modelli in scenari non ostili, ignorando la presenza di un attaccante attivo: tale omissione preclude la formulazione e lo sviluppo di architetture dotate di un'effettiva robustezza contro tentativi di evasione mirati.
Le sezioni seguenti approfondiscono proprio questo divario, introducendo la robustezza adversarial e le metodologie per resistere a tali manipolazioni.

3.2 Adversarial Machine Learning

3.2.1 L'escalation tra attacchi e difese

L'evoluzione dello stato dell'arte in adversarial machine learning è descrivibile come una successione di cicli in cui una difesa viene proposta, valutata contro l'attacco più forte disponibile al momento, e successivamente aggirata da un attacco costruito appositamente. La defensive distillation, proposta come contromisura generale [64], è stata annullata dall'attacco di Carlini e Wagner [9]; un'intera generazione di difese basate su trasformazioni dell'input e su randomizzazione è stata aggirata tramite BPDA ed EOT [4]; dieci metodi di rilevamento sono stati sconfitti da funzioni di loss adattive [8]; più recentemente, le difese basate su purificazione con modelli di diffusione sono state a loro volta attaccate [40].
La reazione della comunità scientifica è stata la standardizzazione dei protocolli di valutazione ([16], [14], [7]) e lo studio delle cause della sovrastima della robustezza, tra cui ad esempio i fenomeni dei gradienti offuscati [4] o sbilanciati [48]. Sul fronte opposto, la ricerca continua a produrre attacchi sempre più efficienti in termini di perturbazione e costo computazionale, anche con l'obiettivo esplicito di rendere praticabile la valutazione su larga scala ([94]; [91]).
Questo quadro giustifica la scelta, adottata in questa tesi, di non limitare la validazione

sperimentale agli attacchi canonici, ma di includere una formulazione adattiva costruita sulla conoscenza completa della difesa proposta.

3.2.2 Il paradigma black-box dominante e i suoi limiti

La maggior parte dello stato dell'arte in tema di robustezza adversarial affronta il problema trattando la rete neurale come una black-box. In tale paradigma:

1. La nozione di robustezza viene valutata unicamente sull'accuratezza delle predizioni allo strato di output;
2. Le tecniche di difesa (come il classico adversarial training) e le regolarizzazioni standard vengono spesso applicate a livello globale o uniformemente su tutti i layer, ignorando le caratteristiche dei singoli strati.

Ciò vale per la quasi totalità delle formulazioni di riferimento ([50], [59], [18], [13], [93], [87]). Tale approccio presenta un limite metodologico fondamentale: concentrandosi esclusivamente sulla relazione input-output, viene completamente persa l'informazione sul percorso e sulla traiettoria che la perturbazione adversarial segue lungo i livelli intermedi. Di conseguenza, risulta impossibile diagnosticare con precisione in quale punto della rete si collochi l'anello debole della vulnerabilità e in quale stadio avvenga la deviazione critica della rappresentazione. Senza una comprensione profonda delle dinamiche interne, le difese rischiano di essere inefficaci.

3.2.3 Verso una prospettiva layer-wise

L'intuizione che la vulnerabilità agli attacchi adversarial sia legata alla semantica delle singole unità dei livelli nascosti è presente fin dallo studio precursore della disciplina [81], il quale lega l'entità alle costanti di Lipschitz dei singoli strati, suggerendo una forma di regolarizzazione per tale quantità. Da questa osservazione, uno dei filoni di ricerca che vi ha preso ispirazione, direttamente collegato a questa tesi, studia proprio la geometria dello spazio delle attivazioni. L'idea di applicare esplicitamente una regolarizzazione tra l'embedding del campione pulito e quello del corrispondente campione adversarial è introdotta nell'Adversarial Metric Learning [51]; l'antecedente diretto della strategia attrattiva adottata in questa tesi; un altro lavoro propone una riformulazione della loss di cross-entropy che mira a produrre rappresentazioni latenti in cui i campioni adversarial

sono più facilmente distinguibili dai campioni normali [61]. Un approccio complementare interviene sul flusso delle attivazioni sopprimendo i canali che risultano ridondanti sui dati puliti ma vengono attivati dalle perturbazioni [5], oppure inserendo blocchi di denoising direttamente all'interno della rete, sulla base dell'osservazione che le mappe di feature intermedie sotto attacco presentano rumore strutturato assente nei campioni benigni [90]. Sul versante della regolarizzazione layer-wise, esistono lavori che applicano l'adversarial training a perturbazioni iniettate direttamente nelle attivazioni intermedie anziché nell'input [74], e framework che introducono un'analisi delle rappresentazioni intermedie con tanto di regolarizzazione per ciascun livello [41]. Questa linea rappresenta il precedente più prossimo alla metrica di Layer Adversarial Vulnerability introdotta nel Capitolo 5.

3.2.4 Detection multi-livello

Parallelamente alle difese che mirano alla classificazione corretta, si è sviluppata anche una linea di ricerca che utilizza le rappresentazioni intermedie come sorgente di segnale per il rilevamento di un attacco adversarial. Chiameremo "backbone" il modello che si occupa di risolvere il task, e "detector" il modello che si occupa del rilevamento di attacchi adversarial.

I primi lavori collegano un detector in un punto specifico della backbone [52]. Sono anche stati proposti detector basati su metriche di natura statistica, calcolate sull'intera pila delle attivazioni: stime di densità dei livelli di dropout e incertezza bayesiana sulle feature dell'ultimo livello nascosto [23], oppure la dimensionalità intrinseca locale (LID) dei livelli, capace di caratterizzare le regioni adversarial [49], o ancora la distanza di Mahalanobis [43]. Altri approcci sostituiscono l'ultimo livello softmax con un criterio di inferenza basato sull'algoritmo Deep k-Nearest Neighbors che applica il famoso algoritmo kNN alle rappresentazioni imparate nei livelli nascosti della rete, puntando non solo alla robustezza ma anche all'interpretabilità [62], oppure applicano test statistici sull'effetto di perturbazioni casuali ai logit delle predizioni [73].

Il limite comune a questa famiglia è la fragilità rispetto ad attaccanti che conoscano il detector e ne includano lo score nella funzione obiettivo [8]. Ciò non invalida l'approccio, ma impone che ogni proposta di rilevamento venga valutata contro un attacco adattivo esplicito, requisito che la presente tesi assume.

3.2.5 Estensione del problema ad altre modalità

La natura sistemica della vulnerabilità agli attacchi adversarial è confermata dalla sua ricomparsa in domini molto distanti da quello visivo. Nei modelli linguistici di grandi dimensioni, procedure di ottimizzazione discreta su suffissi di prompt sono in grado di produrre stringhe universali e trasferibili capaci di aggirare lo strato di sicurezza di diversi modelli, inclusi modelli commerciali a cui l'attaccante non ha accesso diretto, a tal punto che la superficie d'attacco diventa molto ampia come documentato in diverse rassegne ([95], [77]). Il richiamo a questo filone narrativo, seppur esterno agli obiettivi della tesi, è utile a motivare la generalità del problema affrontato e la trasversalità degli strumenti metodologici adottati [66].

3.2.6 Sintesi del gap individuato

Emergono dunque tre elementi convergenti. In primo luogo, la vulnerabilità agli attacchi adversarial è riconosciuta come proprietà stratificata almeno dal 2014, ma la quasi totalità delle difese continua a operare su una misura di robustezza definita all'uscita. In secondo luogo, le rappresentazioni intermedie sono già state sfruttate come sorgente di segnale per il rilevamento, ma quasi sempre mediante descrittori statistici calcolati a posteriori su una rete addestrata in modo standard. In terzo luogo, i pochi lavori che uniscono analisi layer-wise e difesa sono stati sviluppati prevalentemente su dati visivi, e per giunta con scarsa attenzione verso una valutazione contro attacchi adattivi. Il presente lavoro si colloca in questo spazio, proponendo una metrica di vulnerabilità per strato, due strategie contrapposte di modellazione dello spazio delle attivazioni guidate da tale metrica, un'architettura di detector innestati lungo la dorsale con controllo esplicito del flusso dei gradienti, e una validazione sotto attacco adattivo nel dominio del traffico di rete.

Capitolo 4

Dataset

La valutazione di un sistema di rilevamento delle intrusioni basato su deep learning dipende in modo determinante dai dati su cui il modello viene addestrato. La rappresentatività del traffico, la correttezza delle etichette e il modo in cui i record vengono preparati condizionano le prestazioni misurate almeno quanto le scelte architetturali, e diversi studi hanno mostrato come errori nella costruzione o nell'impiego dei dataset possano condurre a risultati eccessivamente ottimistici [3]. Il problema assume un rilievo ancora maggiore in un lavoro dedicato alla robustezza adversarial, dove la nozione stessa di perturbazione ammissibile dipende dalla rappresentazione adottata.

Il capitolo descrive i quattro dataset impiegati nella sperimentazione, ovvero UNSW-NB15, CIC-IDS2017, CSE-CIC-IDS2018 e CTU-13, e la procedura che li trasforma negli insiemi di addestramento, validazione e test utilizzati. Dopo aver discusso i criteri che hanno guidato la scelta e le criticità note dei dataset pubblici per il rilevamento delle intrusioni, ciascun dataset viene caratterizzato in termini di modalità di generazione del traffico, feature disponibili, tipologie di attacco e distribuzione delle classi. La seconda parte del capitolo illustra la pipeline di preprocessing, ponendo il focus su ciascun passaggio rispetto ai dettagli implementativi.

4.1 Criteri di selezione dei dataset

Una recente revisione sistematica della letteratura, che ha analizzato 89 dataset pubblici, dimostra che i dataset per il deep learning dei NIDS esistono e sono numerosi, ma la comunità ne conosce solo una parte e manca di pratiche condivise per la loro selezione e il loro utilizzo [29]. La stessa rassegna, analizzando 45 lavori pubblicati tra il 2020 e il 2023

nelle principali conferenze di sicurezza, mostra che non esiste un benchmark di riferimento universalmente adottato: la maggior parte degli studi raccoglie dati propri, spesso non condivisi, e tra i dataset pubblici il più utilizzato, CIC-IDS2017, compare soltanto nel 29% dei lavori. Seguono, con più di cinque utilizzi ciascuno, CTU-13, il Kitsune Network Attack Dataset e UNSW-NB15, mentre CSE-CIC-IDS2018 si colloca immediatamente dopo.
La scelta dei dataset impiegati in questa tesi si fonda su tre criteri.

1. Rappresentatività: adottare i dataset più utilizzati consente di collocare i risultati nel contesto dei lavori esistenti e di confrontare le prestazioni dei modelli con quelle riportate in letteratura.
2. Features Flow-based: i dataset selezionati offrono record di flusso bidirezionali con feature già estratte ed etichette associate direttamente a ciascun record. Il formato a flussi è d'altra parte il più diffuso, essendo presente nel 60% dei dataset censiti, di cui l'87% in forma bidirezionale [29].
3. Eterogeneità: i quattro dataset differiscono per ambiente di generazione del traffico, strumento di estrazione delle feature, dimensionalità dello spazio di ingresso e copertura degli attacchi. Questa varietà permette di verificare che le proprietà delle difese proposte non siano legate alle peculiarità di un singolo dataset. La dimensionalità assume inoltre un ruolo specifico nel contesto adversarial: l'effetto di una perturbazione limitata in norma L_∞ cresce con il numero di componenti perturbabili, e valutare le difese su spazi di ingresso che vanno da circa una decina a circa settanta feature consente di osservarne il comportamento in contesti differenti.

Il dataset Kitsune, pur figurando tra i più utilizzati, è stato escluso poichè le sue feature sono statistiche per pacchetto su finestre temporali, non omogenee con la rappresentazione a flussi adottata per gli altri dataset. Tutti i dataset sono stati ottenuti attraverso la piattaforma Kaggle, che la rassegna indica tra i principali repository pubblici di dati per la sicurezza.
La Tabella 4.1 riassume le principali proprietà dei dataset selezionati, in accordo con lo studio della review.

Proprietà	UNSW-NB15	CIC-IDS2017	CSE-CIC-IDS2018	CTU-13
Anno di cattura	2015	2017	2018	2011
Ambito	generale, reti locali	generale, reti locali	generale	traffico di botnet
Traffico normale	emulato, fisico e virtuale	emulato, fisico	emulato, virtuale	reale
Traffico malevolo	emulato, fisico e virtuale	emulato, fisico	emulato, virtuale	reale
Attacchi distinguibili	9	14	14	7
Categorie d'attacco	D, R, V, O	B, C, D, R, V	B, C, D, R, V	C, D, R, O, ?
Formati	pacchetti, flussi, log di rete	pacchetti, flussi	pacchetti, flussi, log degli host	pacchetti, flussi
Numero di feature	49	79	84	12 mono, 15 bi
Volume	2,54 milioni di flussi	3,1 milioni di flussi	16,2 milioni di flussi	20,6 milioni di flussi
Durata	1 g 7 h	5 giorni da 8 h	10 giorni, non continua	5 g 23 h, non continua
Rete	piccola	piccola	grande	grande, accademica
Partizioni predefinite	sì	no	no	no

Tabella 4.1. Proprietà dei dataset selezionati secondo lo schema di classificazione di [29]. Il numero di feature include le etichette. Le categorie d'attacco sono indicate con le iniziali dello schema: B brute force, C botnet command and control, D denial of service, R reconnaissance, V exploit di vulnerabilità, O altri, ? attacchi non esplicitamente dichiarati.

4.2 Criticità dei dataset per il rilevamento delle intrusioni

La rassegna [29] riconduce le limitazioni dei dataset per il rilevamento delle intrusioni alle proprietà specifiche del dominio: la variabilità del traffico all'interno di una stessa rete e tra reti differenti, la presenza di un attaccante attivo, l'elevato costo degli errori, l'incertezza delle ground truth e la riservatezza dei dati. Alcune delle limitazioni che ne derivano sono intrinseche al dominio e difficilmente eliminabili, altre sono dovute a errori nella generazione o nella gestione dei dati e possono essere mitigate. Si richiamano di seguito quelle che hanno orientato le scelte di preprocessing descritte nella Sezione 4.7.

- Sbilanciamento delle classi: il traffico di rete è naturalmente dominato dalla componente benigna, e la maggior parte dei dataset riflette questo squilibrio, lasciando agli utilizzatori il compito di gestirlo. Lo squilibrio riguarda anche le singole categorie di attacco, alcune delle quali sono rappresentate da poche decine di record. La raccomandazione è di addestrare i modelli su dati bilanciati, osservando al tempo stesso che la distribuzione di test rispecchi quella reale.
- Assenza di partizionamenti predefiniti: solo il 23% dei dataset censiti fornisce una suddivisione ufficiale in training e test. In sua assenza ciascun lavoro costruisce partizioni proprie, aumentando la difficoltà di confronto dei risultati.
- Errori nella produzione dei dati: difetti degli strumenti di estrazione e della fase di elaborazione producono frequentemente record e feature duplicati e valori mancanti o non validi (come nel caso del dataset CIC-IDS2017), in aggiunta senza che questi problemi vengano documentati.
- Feature con effetti negativi per l'apprendimento: attributi quali indirizzi IP, porte, timestamp o altri identificativi di flusso possono risultare fortemente correlati con l'etichetta. In realtà tali feature non sono caratteristiche del flusso stesso, comportando una sovrastima delle prestazioni.
- Realismo: la maggior parte dei dataset contiene traffico emulato in ambienti di laboratorio, la cui componente benigna tende a essere meno varia e complessa di quella reale e quindi più facilmente separabile da quella malevola; inoltre, data l'evoluzione continua delle reti e delle minacce, i dataset statici perdono progressivamente rappresentatività con il passare del tempo.

Le prime quattro criticità possono essere affrontate in fase di preprocessing dei dati. I limiti del realismo sono invece proprietà dei dati, che il preprocessing non può correggere.

4.3 UNSW-NB15

UNSW-NB15 è stato generato nel 2015 presso il Cyber Range Lab dell'Australian Centre for Cyber Security, UNSW Canberra, con l'obiettivo di superare i limiti dei benchmark della famiglia KDD, ormai non più rappresentativi del traffico e delle minacce contemporanee [56]. Il traffico è stato prodotto mediante la piattaforma IXIA PerfectStorm, che combina attività normali realistiche con comportamenti d'attacco sintetici tratti da un archivio di vulnerabilità costantemente aggiornato, ed è stato catturato con tcpdump in due sessioni di circa 16 e 15 ore, per un totale di circa 100 GB di traffico grezzo.
Dai pacchetti sono state estratte 49 feature, comprese le etichette. Le feature sono organizzate in gruppi: gli identificativi del flusso, cioè indirizzi e porte di sorgente e destinazione e protocollo; le feature di base, che descrivono la connessione nel suo complesso, come stato, durata e pacchetti per direzione; le feature di contenuto, relative ai parametri TCP e ai protocolli applicativi; le feature temporali, come jitter e tempi di interarrivo; infine, le feature generate artificialmente.
Il traffico malevolo è suddiviso in nove famiglie:

- Fuzzers: invio di dati generati casualmente a un programma o a un servizio con l'obiettivo di provocarne il malfunzionamento o l'interruzione;
- Analysis: attività di penetrazione, tra cui scansioni di porte, spam e attacchi basati su file HTML;
- Backdoor: tecniche che aggirano in modo furtivo i meccanismi di sicurezza per ottenere accesso a un sistema o ai suoi dati;
- DoS: tentativi di rendere indisponibile un server o una risorsa di rete;
- Exploits: sfruttamento di vulnerabilità note di sistemi operativi o applicazioni;
- Generic: attacchi che operano contro qualunque cifrario a blocchi, indipendentemente dalla sua struttura;
- Reconnaissance: raccolta di informazioni sui potenziali bersagli;
- Shellcode: frammenti di codice utilizzati come payload nello sfruttamento di una vulnerabilità;

- Worms: codice malevolo che si replica per propagarsi ad altri host.

Oltre all'insieme completo, di circa 2,54 milioni di record, gli autori distribuiscono una suddivisione ufficiale in una partizione di training di 175.341 record e una di test di 82.332 record, circostanza che rende UNSW-NB15 l'unico dei quattro dataset dotato di partizioni predefinite.

La distribuzione delle classi nelle due partizioni è riportata nella Tabella 4.2 e in Figura 4.1.

Categoria	Training	%	Test	%
Normal	56.000	31,94	37.000	44,94
Generic	40.000	22,81	18.871	22,92
Exploits	33.393	19,04	11.132	13,52
Fuzzers	18.184	10,37	6.062	7,36
DoS	12.264	6,99	4.089	4,97
Reconnaissance	10.491	5,98	3.496	4,25
Analysis	2.000	1,14	677	0,82
Backdoor	1.746	1,00	583	0,71
Shellcode	1.133	0,65	378	0,46
Worms	130	0,07	44	0,05
Totale	175.341	100,00	82.332	100,00

Tabella 4.2. Distribuzione delle categorie d'attacco in UNSW-NB15

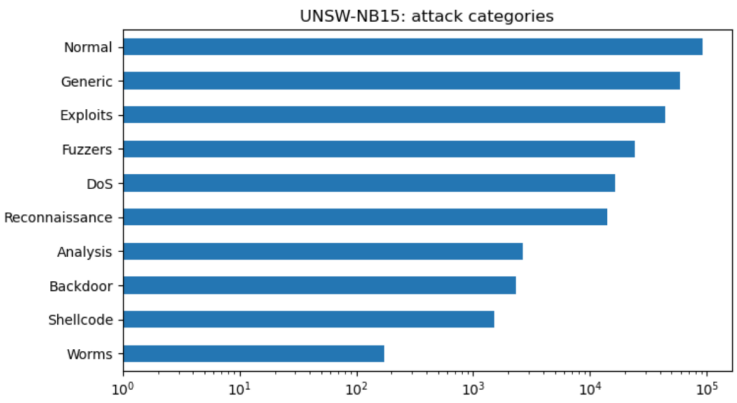


Figura 4.1. Distribuzione delle categorie d'attacco in UNSW-NB15

4.4 CIC-IDS2017

CIC-IDS2017 è il dataset pubblico più utilizzato nella ricerca recente sui NIDS [76]. Il traffico è stato catturato in una network fisica composta da una rete vittima, dotata di firewall, router, switch e host con sistemi operativi eterogenei, e da una rete attaccante separata.

Le feature sono state estratte con CICFlowMeter, strumento che ricostruisce flussi bidirezionali e ne calcola circa 80 statistiche. Il feature set di CICFlowMeter si è affermato come uno dei più diffusi, essendo adottato dal 31% dei dataset a flussi studiati in [29].

La distribuzione delle classi, riportata nella Tabella 4.3 e in Figura 4.2, è fortemente sbilanciata: il traffico benigno costituisce circa l'80% dei flussi, e tre categorie – DoS Hulk, PortScan e DDoS – raccolgono oltre il 90% del traffico malevolo, mentre Infiltration, SQL Injection e Heartbleed contano rispettivamente 36, 21 e 11 flussi.

Nonostante la sua diffusione, CIC-IDS2017 presenta numerosi difetti documentati, tra cui feature e record duplicati, valori infiniti o indefiniti, feature costanti e timestamp incoerenti. Queste anomalie vengono gestite dalla pipeline di preprocessing. Gli errori di costruzione dei flussi e di etichettatura, che richiederebbero una nuova estrazione dai pacchetti, non sono invece correggibili a questo livello.

Categoria	Flussi	%
BENIGN	2.273.097	80,30
DoS Hulk	231.073	8,16
PortScan	158.930	5,61
DDoS	128.027	4,52
DoS GoldenEye	10.293	0,36
FTP-Patator	7.938	0,28
SSH-Patator	5.897	0,21
DoS slowloris	5.796	0,20
DoS Slowhttptest	5.499	0,19
Bot	1.966	0,07
Web Attack Brute Force	1.507	0,05
Web Attack XSS	652	0,02
Infiltration	36	<0,01
Web Attack Sql Injection	21	<0,01
Heartbleed	11	<0,01
Totale	2.830.743	100,00

Tabella 4.3. Distribuzione delle categorie d'attacco in CIC-IDS2017

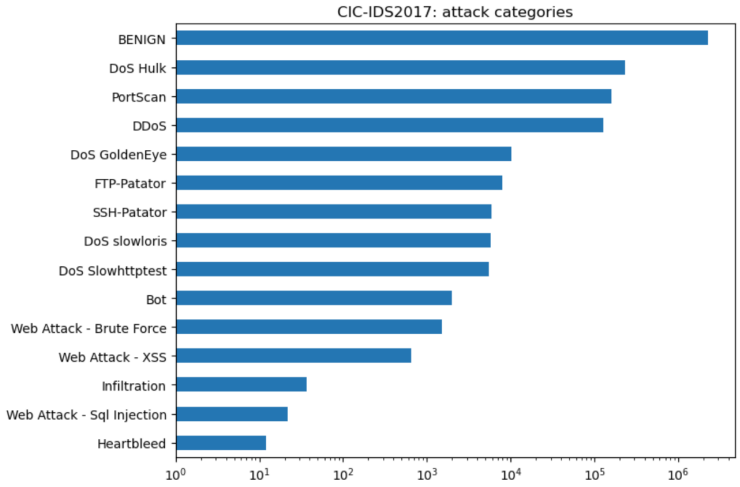


Figura 4.2. Distribuzione delle categorie d'attacco in CIC-IDS2017

4.5 CSE-CIC-IDS2018

CSE-CIC-IDS2018 è il risultato della collaborazione tra il Communications Security Establishment (CSE) e il CIC e ripropone su scala più ampia la metodologia basata su profili di CIC-IDS2017. Il traffico è stato generato in un ambiente virtualizzato sull'infrastruttura cloud di Amazon Web Services, comprendente centinaia di host. Oltre ai pacchetti e ai flussi, il dataset include i log di sistema degli host. Le feature, circa 80, sono state estratte con una versione successiva di CICFlowMeter e coincidono in larga parte con quelle di CIC-IDS2017.

Gli scenari d'attacco comprendono brute force contro FTP, SSH e ambienti web, quattro varianti di DoS, tre varianti di DDoS, cross-site scripting e SQL injection, infiltration e attività di una botnet, come mostrano la Tabella 4.4 e la Figura 4.3.

Con circa 16,2 milioni di flussi, CSE-CIC-IDS2018 è tra i più voluminosi dei dataset a flussi considerati e presenta uno squilibrio ancora più accentuato di CIC-IDS2017.

Categoria	Flussi	%
Benign	13.484.708	83,07
DDoS attacks-LOIC-HTTP	686.012	4,23
DDOS attack-HOIC	576.191	3,55
DoS attacks-Hulk	461.912	2,85
Bot	286.191	1,76
Infilteration	193.360	1,19
SSH-Bruteforce	187.589	1,16
DoS attacks-GoldenEye	161.934	1,00
DoS attacks-Slowloris	139.890	0,86
DDOS attack-LOIC-UDP	41.508	0,26
Brute Force -Web	10.990	0,07
Brute Force -XSS	1.730	0,01
SQL Injection	611	<0,01
DoS attacks-SlowHTTPTest	230	<0,01
FTP-BruteForce	87	<0,01
Totale	16.232.943	100,00

Tabella 4.4. Distribuzione delle categorie d'attacco in CSE-CIC-IDS2017

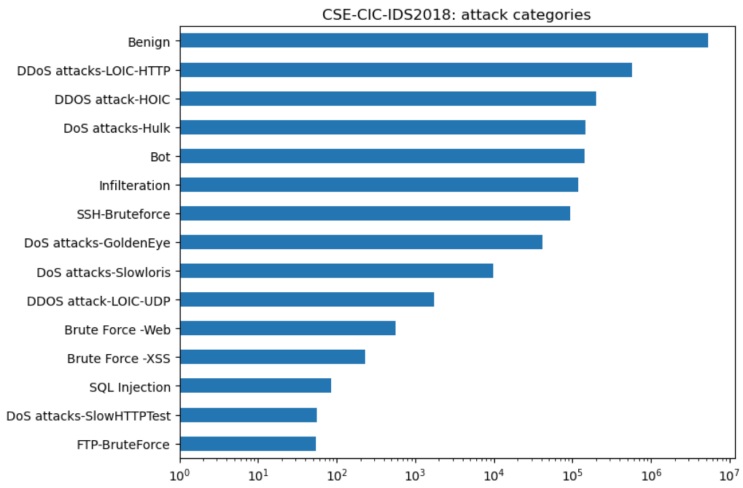


Figura 4.3. Distribuzione delle categorie d'attacco in CSE-CIC-IDS2018

4.6 CTU-13

CTU-13 è stato catturato nel 2011 presso la Czech Technical University di Praga ed è uno dei principali dataset dedicati al rilevamento di botnet, nonché uno dei più utilizzati nella ricerca moderna ([28], [29]). Il dataset è composto da tredici scenari, in ciascuno dei quali uno o più host della rete universitaria sono stati infettati con un malware reale, eseguito senza modifiche in un ambiente controllato ma connesso a Internet. Gli scenari coprono

sette famiglie di malware, Neris, Rbot, Virut, Menti, Sogou, Murlo e NSIS.ay, alcune delle quali ripetute con configurazioni differenti.

Il traffico di ciascuno scenario è etichettato a livello di flusso in tre classi: botnet, per i flussi generati dagli host infetti; normal, per i flussi di un insieme di host verificati come non compromessi; background, per tutto il resto del traffico della rete, la cui natura non è nota e che può contenere sia comunicazioni legittime sia attività malevole non attribuite. Quest'ultima classe costituisce la netta maggioranza dei flussi, mentre il traffico di botnet ne rappresenta una frazione dell'ordine di pochi punti percentuali.

I flussi sono forniti in formato NetFlow bidirezionale prodotto con Argus, e ciascun record contiene soltanto una quindicina di campi. Nel complesso il dataset comprende circa 20,6 milioni di flussi bidirezionali suddivisi come mostrano la Tabella 4.5 e la Figura 4.4.

Il problema viene ricondotto a una classificazione binaria tra traffico normale e traffico di botnet, scartando i flussi di background: la loro etichetta costituisce una ground truth inaffidabile, e includerli come traffico benigno significherebbe addestrare e valutare i modelli su record la cui natura è incerta. Le diverse famiglie di malware e i diversi comportamenti vengono aggregati in un'unica classe malevola.

Categoria	Flussi	%
Background	10.133.649	95.612
Botnet-Neris	210.070	1.982
Normal	202.549	1.911
Botnet-Rbot	25.142	0.237
Botnet-Virut	21.236	0.200
Botnet-Menti	2.667	0.025
Botnet-NsisAy	2.000	0.019
Botnet-Murlo	1.395	0.013
Botnet-Sogou	63	0.001
Totale	10.598.771	100,00

Tabella 4.5. Distribuzione delle categorie d'attacco in CTU-13

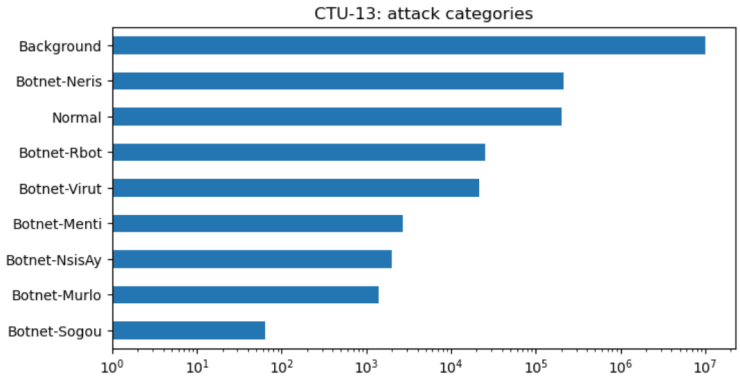


Figura 4.4. Distribuzione delle categorie d'attacco in CTU-13

4.7 Pipeline di preprocessing

I quattro dataset differiscono per formato, organizzazione dei file, nomenclatura delle feature e convenzioni di etichettatura. La pipeline di preprocessing ha lo scopo di ricondurli a una rappresentazione comune, costituita da feature numeriche, da un'etichetta binaria che vale 0 per il traffico benigno e 1 per quello malevolo e da una categoria d'attacco ausiliaria, e di produrre partizioni di addestramento, validazione e test riproducibili. La categoria d'attacco non viene mai fornita in ingresso al modello, il quale risolve il task di classificazione binaria, ma è impiegata per stratificare le partizioni e per bilanciare le classi tra le tre partizioni.

La pipeline si articola in due fasi. La prima trasforma i file di cattura grezzi in una coppia di partizioni di training e di test; viene eseguita una sola volta per ciascun dataset e il suo risultato viene memorizzato, così che tutte le configurazioni sperimentali siano addestrate e valutate esattamente sugli stessi record. La seconda fase viene eseguita a ogni caricamento del dataset e comprende le operazioni che dipendono da parametri sperimentali o che devono essere stimate sui soli dati di addestramento. Tutte le operazioni di campionamento utilizzano un seed fisso, rendendo l'intero processo deterministico. I passi della pipeline sono i seguenti:

1. lettura e pulizia di ciascun file di cattura;
2. fusione dei file, rimozione dei record non validi e dei duplicati;
3. partizionamento stratificato in training e test;
4. codifica degli attributi categorici;

5. trasformazione delle feature con distribuzione a coda pesante;
6. standardizzazione;
7. estrazione del validation set e definizione delle feature perturbabili.

I primi tre passi costituiscono la prima fase e si applicano a CIC-IDS2017, CSE-CIC-IDS2018 e CTU-13, distribuiti come raccolte di catture prive di una suddivisione ufficiale; per UNSW-NB15 sono sostituiti dall'impiego diretto delle partizioni predefinite. I passi successivi sono invece comuni a tutti i dataset.

4.7.1 Lettura e pulizia delle catture

Ciascun file di cattura viene elaborato separatamente prima della fusione. Oltre a semplificare la gestione delle differenze tra file, questa scelta contiene l'occupazione di memoria. Su ciascun file vengono eseguite le seguenti operazioni.

- Pulizia dello schema: i nomi delle colonne vengono ripuliti dai caratteri non-ASCII e le colonne ripetute vengono eliminate.
- Costruzione delle etichette: dall'etichetta originale si ricavano la categoria d'attacco e l'etichetta binaria, che vale 0 per il traffico benigno e 1 per qualunque categoria d'attacco.
- Rimozione degli identificativi: gli attributi che identificano gli host o il momento della cattura, anziché descrivere il comportamento del traffico, vengono eliminati. La rimozione ha una duplice motivazione: da un lato previene che i modelli imparino a classificare i flussi in funzione di feature non rappresentative di un attacco, come discusso anche nella Sezione 4.2; dall'altro, un attributo come il timestamp renderebbe ogni record formalmente unico, vanificando la rimozione dei duplicati.

4.7.2 Fusione, valori non validi e duplicati

I file così ottenuti vengono concatenati. Poiché catture diverse non condividono necessariamente lo stesso insieme di colonne, la fusione avviene sull'intersezione degli schemi: una colonna presente solo in alcuni file risulterebbe altrimenti indefinita per tutti i record degli altri, e la sua presenza porterebbe all'eliminazione di interi file nel passo successivo. Sul dataset fuso vengono quindi eseguite tre operazioni di pulizia.

- Rimozione dei record a valori non validi o infiniti.

- Rimozione dei duplicati, per evitare che la stessa istanza sia presente sia nel training che nel testing set.
- Rimozione delle feature costanti (a varianza nulla), non informative ai fini dell'apprendimento.

4.7.3 Partizionamento

Per UNSW-NB15 si adottano le partizioni in training e test set ufficiali.
Per gli altri tre dataset, privi di una suddivisione ufficiale, il dataset ripulito viene diviso in una partizione di training contenente il 70% dei record e una di test contenente il restante 30%, con un campionamento casuale stratificato rispetto alla categoria d'attacco. La stratificazione garantisce che ogni categoria, comprese quelle rappresentate da poche decine di record, compaia in entrambe le partizioni con la stessa proporzione.
La suddivisione è casuale e non temporale. La letteratura raccomanda, in generale, di rispettare l'ordine cronologico dei dati, addestrando i modelli su traffico che precede quello di valutazione, per evitare che il modello benefici di informazioni provenienti dal futuro (Data Snooping, [3]). Nei dataset considerati, tuttavia, ciascuna categoria d'attacco è confinata in una o più giornate consecutive di cattura, dunque una suddivisione cronologica non ha rilevanza nel contesto di questa tesi.
Dalla partizione di training viene infine estratto un validation set pari al 30% dei record, anch'esso stratificato rispetto alla categoria d'attacco.

4.7.4 Trasformazione delle distribuzioni a coda pesante

Le feature di flusso che misurano volumi, durate, tassi e tempi di interarrivo presentano distribuzioni fortemente asimmetriche e heavy tailed, che si estendono su diversi ordini di grandezza: la grande maggioranza dei flussi è breve e trasporta pochi byte, mentre una piccola frazione, come i trasferimenti di file, le sessioni di lunga durata o gli attacchi ad alto volume di traffico, assume valori estremi. Una semplice standardizzazione non è adeguata a questo tipo di distribuzione, poiché media e deviazione standard risultano dominate dai valori estremi e la quasi totalità dei record viene compressa in un intervallo molto ristretto attorno alla media. Nel contesto di questa tesi, ciò rende non omogeneo il significato di un budget di perturbazione: uno stesso spostamento ε corrisponderebbe, rispetto alla distribuzione tipica dei valori, a variazioni trascurabili per alcune feature e

molto ampie per altre.

Per ciascuna feature numerica j viene quindi calcolato, sulla partizione di training, il coefficiente di asimmetria (o skewness) campionario

$$\hat{\gamma}_j = \frac{\frac{1}{N}\sum_{i=1}^{N}\left(x_{ij} - \bar{x}_j\right)^3}{\left(\frac{1}{N}\sum_{i=1}^{N}\left(x_{ij} - \bar{x}_j\right)^2\right)^{3/2}},$$

e le feature per le quali $|\hat{\gamma}_j| > 2$ vengono sottoposte a una trasformazione per quantili verso la distribuzione normale standard, riducendo l'effetto degli outliers:

$$\tilde{x}_{ij} = \Phi^{-1}\left(\hat{F}_j\left(x_{ij}\right)\right),$$

dove $\hat{F}_j$ è la funzione di distribuzione cumulativa della feature, stimata sulla partizione di training, e Φ^{-1} è l'inversa della funzione di distribuzione della normale standard. La trasformazione è monotona, e dunque preserva l'ordinamento dei valori, ma ridistribuisce i campioni in modo che la feature trasformata abbia distribuzione approssimativamente gaussiana; i valori del test set esterni all'intervallo osservato in addestramento vengono ricondotti agli estremi, limitando l'effetto di eventuali valori anomali.

Sono esclusi dalla trasformazione gli attributi categorici e quelli discreti, per i quali una ridistribuzione della massa non avrebbe significato, oltre alla label e agli attributi di servizio.

4.7.5 Standardizzazione

Infine, tutte le componenti del vettore di ingresso vengono standardizzate:

$$z_{ij} = \frac{\tilde{x}_{ij} - \mu_j}{\sigma_j},$$

con media μ_j e deviazione standard σ_j stimate sulla partizione di training. Come per le trasformazioni precedenti, i parametri sono stimati esclusivamente sui dati di addestramento, e applicati invariati al test set, così che nessuna informazione sulla distribuzione del test influenzi la preparazione dei dati.

Lo spazio così ottenuto è quello in cui opera il modello e, di conseguenza, quello in cui vengono calcolate le perturbazioni adversarial: un budget ε in norma L_∞ vincola ogni feature perturbabile a spostarsi, indipendentemente dalle altre, di al più ε nello spazio standardizzato, ovvero di al più ε deviazioni standard nello spazio trasformato.

Infatti, per le feature sottoposte alla trasformazione per quantili, la cui distribuzione è approssimativamente normale standard, il vincolo $\|\boldsymbol{\delta}\|_\infty < \varepsilon$ significa che $\max_j |\delta_j| < \varepsilon$, e considerando la standardizzazione $z_j = (x_j - \mu_j)/\sigma_j$, invertendo si ha che $x_j = \mu_j + \sigma_j z_j$, perciò uno spostamento Δz_j nello spazio delle feature standardizzato corrisponde nello spazio delle feature trasformato a $\Delta x_j = \sigma_j \cdot \Delta z_j$, dunque da $|\Delta z_j| < \varepsilon$ segue che $|\Delta x_j| < \sigma_j \cdot \varepsilon$. Grazie alla trasformazione per quantili, questo vincolo assume un significato comparabile tra feature con scale originali molto diverse.

4.7.6 Feature perturbabili

L'ultimo passo della pipeline riguarda la definizione, per ciascun dataset, delle feature che un attaccante può perturbare. Come discusso precedentemente, i dati dei flussi combinano variabili continue, discrete e categoriche, e una perturbazione continua applicata a un attributo categorico o discreto produce valori privi di significato. Per tenere conto di questi vincoli, a ciascun dataset è associato un vettore binario $\boldsymbol{m} \in \{0, 1\}^d$, nullo in corrispondenza delle feature non perturbabili, e il campione adversarial viene costruito come

$$\boldsymbol{x}_{adv} = \boldsymbol{x} + \boldsymbol{m} \odot \boldsymbol{\delta},$$

così che la perturbazione $\boldsymbol{\delta}$ agisca soltanto sulle feature continue.
Questa maschera rappresenta una prima approssimazione dei vincoli di dominio: esclude dalla perturbazione gli attributi categorici e discreti, ma non impone la non negatività delle grandezze strettamente positive, l'interezza dei conteggi né le relazioni aritmetiche tra feature dipendenti, come quella tra byte totali e byte inviati in ciascuna direzione. I campioni adversarial ottenuti sono quindi definiti nello spazio delle feature, e alcuni di essi possono non corrispondere a traffico realmente realizzabile.
Si tratta tuttavia di una scelta conservativa. Rilassando i vincoli di dominio, l'insieme delle perturbazioni ammesse resta un sovrainsieme di quello delle perturbazioni realizzabili, e l'attaccante dispone così di una capacità e di una superficie d'attacco maggiori di quelle che avrebbe in uno scenario reale. Poiché il caso peggiore calcolato su un insieme più ampio non può essere meno severo di quello calcolato su un suo sottoinsieme, la vulnerabilità dei modelli può risultare al più sovrastimata, e questa semplificazione non rende in alcun modo ottimistici i risultati. L'approccio adottato è dunque di natura più teorica:

non pone il focus sulla fattibilità degli attacchi, ma sullo studio della propagazione delle perturbazioni attraverso i livelli della rete, scopo per il quale resta adeguato. La scelta di non modellare in modo completo i vincoli di dominio è motivata dalla complessità sia della loro caratterizzazione sia della loro implementazione: individuare per ciascun dataset le dipendenze tra feature e i domini ammissibili, e integrarli nel processo di generazione degli attacchi, avrebbe richiesto uno sforzo considerevole, deviando in maniera considerevole il focus dall'obiettivo della tesi.

Capitolo 5

Metodologia e modellazione delle soluzioni

Nei capitoli precedenti si è argomentato che la vulnerabilità di una rete neurale profonda agli attacchi di evasione non è una proprietà della sola relazione tra ingresso e uscita, ma si costruisce lungo la profondità del modello: una perturbazione adversarial di norma limitata nello spazio di input viene trasformata, e potenzialmente amplificata, dalla successione dei livelli (Paragrafo 2.2.8), e la deviazione che essa induce sulle rappresentazioni latenti può essere osservabile prima di raggiungere il livello di classificazione. Si è inoltre osservato come la quasi totalità delle contromisure proposte in letteratura trascuri questa struttura, valutando la robustezza soltanto all'uscita della rete o al più applicando regolarizzazioni uniformi su tutti i livelli (Paragrafo 3.2.2).

Questo capitolo traduce tali osservazioni in una metodologia per affrontarle. Il principio che accomuna tutte le soluzioni proposte consiste nel trattare ogni livello nascosto non soltanto come uno stadio di calcolo, ma come un punto dove si può intervenire: le rappresentazioni che esso produce su un campione pulito e sulla sua corrispondente versione adversarial possono essere confrontate, misurate e, durante l'addestramento, modellate. A partire da questo principio vengono sviluppati tre strumenti complementari:

1. una metrica di vulnerabilità definita per singolo livello, la Single Layer Adversarial Vulnerability (SLAV), che quantifica la deviazione indotta dall'attacco nello spazio delle attivazioni in modo confrontabile tra livelli di dimensione diversa;
2. un'architettura biforcata, nella quale micro-classificatori binari (detector) innestati lungo la backbone, in posizioni arbitrarie, osservano le rappresentazioni intermedie

e ne stimano la natura pulita o adversarial;

3. due strategie di manipolazione geometrica dello spazio delle attivazioni, fondate sulla misura di distanza della SLAV, ma di segno opposto: una strategia attrattiva (CloserAL), che avvicina le attivazioni adversarial a quelle pulite in funzione di una classificazione robusta, e una strategia repulsiva (FurtherAL), che le allontana ai fini di un rilevamento più efficace.

La metodologia è completata dall'analisi del flusso dei gradienti tra le diverse componenti del modello, della definizione del modello di minaccia e degli attacchi – inclusa una formulazione adattiva a conoscenza del detector – e di un insieme di metriche di valutazione che rendono confrontabili, sulla stessa scala, difese basate sul rilevamento e difese basate sulla classificazione robusta.

5.1 Modello della rete

5.1.1 Formulazione del problema

Si consideri un insieme di dati etichettati $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$, in cui $\boldsymbol{x}_i \in \mathcal{X} \subseteq \mathbb{R}^d$ è il vettore di feature che descrive un flusso di rete e $y_i \in \{0, 1\}$ ne indica la natura benigna ($y_i = 0$) o malevola ($y_i = 1$). Per il caso in esame, i vettori di feature sono il risultato della pipeline di preprocessing descritta nel Capitolo 4, alla quale si rimanda per i dettagli: ai fini della metodologia è sufficiente osservare che le feature numeriche con distribuzione a coda pesante vengono ricondotte a una forma approssimativamente gaussiana e che tutte le feature sono infine standardizzate con statistiche stimate sul solo training set. Ne segue che le componenti di $\boldsymbol{x}$ hanno scala confrontabile, proprietà che si rivelerà necessaria sia per dare significato a un budget di perturbazione uniforme sulle feature (Sezione 5.4), sia per evitare che le distanze misurate nei livelli della rete siano dominate dalle feature di ampiezza maggiore.

Il task del NIDS è formulato come classificazione binaria supervisionata (Paragrafo 2.1.4). Il classificatore è una funzione parametrica $f_{\boldsymbol{\theta}} : \mathbb{R}^d \to \mathbb{R}^2$ che associa a ogni flusso un vettore di logit $\boldsymbol{z} = f_{\boldsymbol{\theta}}(\boldsymbol{x})$, dal quale la predizione si ottiene come $\hat{y} = \arg\max_k z_k$. In questo studio le probabilità delle classi si ottengono applicando la softmax ai logit: l'adozione di due logit, anziché di un'unica uscita sigmoide, è equivalente nel caso binario e consente di estendere la formulazione al caso multiclasse senza modifiche.

5.1.2 Architettura della backbone

Seguendo la terminologia introdotta nel Paragrafo 3.2.4, è chiamata backbone la rete che risolve il task di classificazione. Essa è una rete feedforward completamente connessa, definita dalla composizione

$$\begin{aligned} \boldsymbol{h}^{(0)}(\boldsymbol{x}) &= \mathrm{LN}(\boldsymbol{x}) \\ \boldsymbol{a}^{(l)}(\boldsymbol{x}) &= \boldsymbol{W}^{(l)}\boldsymbol{h}^{(l-1)}(\boldsymbol{x}) + \boldsymbol{b}^{(l)}, \qquad \boldsymbol{h}^{(l)}(\boldsymbol{x}) = \mathrm{ReLU}\big(\boldsymbol{a}^{(l)}(\boldsymbol{x})\big), \qquad l = 1, \dots, L-1 \\ \boldsymbol{z}(\boldsymbol{x}) &= \boldsymbol{W}^{(L)}\boldsymbol{h}^{(L-1)}(\boldsymbol{x}) + \boldsymbol{b}^{(L)} \end{aligned}$$

con $\boldsymbol{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$, $d_0 = d$ e $d_L = 2$. Il primo stadio è di layer normalization, che normalizza il vettore di input rispetto alla media e alla deviazione standard delle sue stesse componenti e ne ripristina la scala tramite due vettori di parametri addestrabili $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$. Questo stadio stabilizza la scala dell'ingresso del primo livello e rende l'ottimizzazione meno sensibile a differenze di ampiezza residue tra i campioni.

La scelta di un'architettura feedforward è motivata da tre considerazioni. La prima è di natura empirica: come discusso nella Sezione 3.1, le reti feedforward risultano competitive contro architetture convoluzionali e ricorrenti, a fronte di latenze di inferenza sensibilmente inferiori [27], requisito rilevante per un NIDS. La seconda è di natura metodologica: in una rete feedforward ogni livello nascosto produce una e una sola rappresentazione vettoriale del campione, priva di condivisione di pesi o di stato temporale, e l'analisi layer-wise può quindi essere condotta senza fattori disorientanti. La terza riguarda la generalità: la costruzione proposta nel seguito richiede soltanto che la rete sia scomponibile in una successione di stadi, e si estende pertanto senza modifiche concettuali ad architetture più complesse; la rete feedforward rappresenta quindi il contesto minimo nel quale lo studio proposto può essere condotto.

5.1.3 Rappresentazione osservata

Per ciascun livello nascosto l la rete produce due rappresentazioni del campione: la pre-attivazione $\boldsymbol{a}^{(l)}(\boldsymbol{x})$ e l'attivazione $\boldsymbol{h}^{(l)}(\boldsymbol{x})$. Tutti gli strumenti introdotti in questo capitolo operano sulla pre-attivazione: il motivo è che $\boldsymbol{a}^{(l)}$ conserva l'intera informazione prodotta dalla trasformazione affine, mentre la ReLU azzera le componenti negative; due campioni che differiscono soltanto in componenti negative della pre-attivazione risultano indistin-

guibili dopo l'applicazione della funzione di attivazione, e se un detector osservasse $\boldsymbol{h}^{(l)}$ ne sarebbe cieco. Nel seguito, per comodità e salvo diversa indicazione, con il termine attivazioni del livello l ci si riferirà al vettore $\boldsymbol{a}^{(l)}(\boldsymbol{x}) \in \mathbb{R}^{d_l}$.

5.1.4 Livelli estesi e stato del flusso

Si indica con $\mathcal{I} \subseteq \{1, \dots, L-1\}$ l'insieme dei livelli estesi, ossia dei livelli nascosti nei quali la trasformazione affine standard viene sostituita da una sua versione modificata. Un livello esteso calcola la stessa pre-attivazione $\boldsymbol{a}^{(l)}$ del livello originale, e dunque non altera la funzione calcolata dalla backbone, ma esegue su di essa un'operazione aggiuntiva: la classificazione pulito/adversarial tramite un detector, il calcolo di un termine di loss sulle attivazioni, oppure entrambe. L'insieme $\mathcal{I}$ è un iperparametro: la sua scelta consente di studiare se l'intervento sia più efficace nei livelli iniziali o in quelli finali, e se convenga estendere tutti i livelli o soltanto alcuni di essi.

Affinché un livello esteso possa trovarsi in una posizione arbitraria della rete, le informazioni che esso produce devono poter raggiungere l'uscita senza modificare l'interfaccia dei livelli successivi. A tal fine, la forward pass viene arricchita con uno stato del flusso[1] S che viaggia insieme alle attivazioni. Lo stato contiene:

- il vettore $\boldsymbol{\eta} \in \{0,1\}^B$ che indica, per ciascuno dei B elementi del batch, se si tratta di un campione pulito ($\eta_j = 0$) o adversarial ($\eta_j = 1$), noto soltanto in fase di addestramento;
- la sequenza dei logit prodotti dai detector attraversati;
- i termini di loss ausiliari accumulati, relativi ai detector e alle attivazioni.

Ogni livello della rete viene quindi esteso per diventare una mappa

$$\Phi^{(l)} : \left(\boldsymbol{h}^{(l-1)}, S^{(l-1)}\right) \longmapsto \left(\boldsymbol{h}^{(l)}, S^{(l)}\right),$$

dove per i livelli non estesi lo stato viene semplicemente inoltrato ($S^{(l)} = S^{(l-1)}$) mentre per i livelli estesi lo stato viene arricchito con i contributi calcolati sulle attivazioni. Al termine della forward pass la rete restituisce la coppia $(\boldsymbol{z}, S^{(L)})$. Nell'implementazione, il livello

[1] Da non confondere con il flusso di traffico: in questo contesto il termine flusso indica il flusso computazionale della forward pass, ossia il percorso che le attivazioni e lo stato associato compiono attraverso i livelli della rete, dall'ingresso fino all'uscita. Esso non va confuso con i flussi di rete, ossia le sequenze di pacchetti scambiati tra due endpoint che costituiscono le istanze dei dataset, e dunque gli input della rete.

che inoltra lo stato senza modificarlo e lo stato stesso corrispondono rispettivamente alle astrazioni PassThrough e FlowState, dalle quali derivano per specializzazione tutti i livelli estesi descritti nella Sezione 5.3.

Con questa formulazione, una singola forward pass produce contemporaneamente la predizione del task e tutte le grandezze calcolate nei livelli intermedi, senza la necessità di un secondo passaggio o di meccanismi esterni di intercettazione delle attivazioni. Inoltre, i termini delle loss ausiliarie appartengono allo stesso grafo computazionale della loss del task, per cui un'unica backward pass distribuisce a ciascun parametro il gradiente di tutti gli obiettivi da cui esso dipende. Infine, la costruzione è indipendente dalla posizione: qualunque livello può essere esteso senza modificare gli altri. In fase di inferenza il vettore $\boldsymbol{\eta}$ non è disponibile, le loss ausiliarie non vengono calcolate e lo stato trasporta soltanto i logits dei detector.

5.2 Metrica di regolarizzazione layer-wise

5.2.1 Definizione della Single Layer Adversarial Vulnerability

Sia $\mathcal{B} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{n}$ un minibatch e sia $\boldsymbol{x}_i^{adv} \in \Delta_\varepsilon(\boldsymbol{x}_i)$ il campione adversarial generato a partire da $\boldsymbol{x}_i$ da un attacco ammissibile (Sezione 5.4). Il confronto che interessa non è tra la distribuzione dei campioni puliti e quella dei campioni adversarial, ma tra le rappresentazioni dello stesso vettore di feature nella versione originale e in quella perturbata. Per ogni livello esteso si definisce la distanza per campione

$$s^{(l)}(\boldsymbol{x}, \boldsymbol{x}^{adv}) = \frac{1}{d_l} \left\| \boldsymbol{a}^{(l)}(\boldsymbol{x}^{adv}) - \boldsymbol{a}^{(l)}(\boldsymbol{x}) \right\|_2^2, \tag{5.1}$$

e la Single Layer Adversarial Vulnerability del livello l sul minibatch come la sua media campionaria:

$$\mathrm{SLAV}^{(l)}(\mathcal{B}) = \frac{1}{n} \sum_{i=1}^{n} s^{(l)}\left(\boldsymbol{x}_i, \boldsymbol{x}_i^{adv}\right). \tag{5.2}$$

La vulnerabilità complessiva della rete, relativa ai livelli estesi, è la somma dei contributi dei singoli livelli:

$$\mathcal{L}_{LAV}(\mathcal{B}) = \sum_{l \in \mathcal{I}} \mathrm{SLAV}^{(l)}(\mathcal{B}). \tag{5.3}$$

Il fattore $1/d_l$ in (5.1) ha un ruolo preciso. La norma euclidea al quadrato di un vettore è la somma dei quadrati delle sue componenti, per cui, per la linearità del valore atteso,

$\mathbb{E}\|\boldsymbol{a}^{(l)}(\boldsymbol{x}^{adv})-\boldsymbol{a}^{(l)}(\boldsymbol{x})\|_2^2 = \sum_{j=1}^{d_l} \sigma_j^2$, dove σ_j^2 è il momento secondo della deviazione dell'unità j. Se tali momenti sono confrontabili, $\sigma_j^2 \approx \sigma^2$, la somma vale approssimativamente $d_l\,\sigma^2$ e cresce quindi linearmente con la larghezza del livello.

In assenza di normalizzazione, la somma (5.3) sarebbe dominata dai livelli più larghi, indipendentemente dalla loro effettiva vulnerabilità, contraddicendo l'intento di far contribuire ogni livello nella stessa misura. Dividendo per la larghezza, $s^{(l)}$ diventa la deviazione quadratica media per unità, una grandezza che non dipende dalla dimensione del livello e che è quindi confrontabile lungo tutta la rete.

5.2.2 Interpretazione

La metrica può essere letta alla luce dell'analisi sulla propagazione della perturbazione del Paragrafo 2.2.8. Lo sviluppo al primo ordine $\boldsymbol{\delta}^{(l)} \approx \boldsymbol{J}^{(l)}\boldsymbol{\delta}^{(l-1)}$ mostra che la deviazione indotta dall'attacco evolve in profondità secondo il prodotto dei jacobiani dei livelli attraversati; il limite superiore $\prod_l \|\boldsymbol{J}^{(l)}\|$ che ne deriva descrive però il caso peggiore su tutte le direzioni possibili ed è in generale molto lasco. La SLAV misura invece la deviazione effettivamente realizzata lungo le direzioni scelte dall'attaccante, che sono esattamente quelle rilevanti ai fini della sicurezza dalla minaccia adversarial. Il profilo dei valori $\mathrm{SLAV}^{(l)}$ al variare di l costituisce dunque una stima empirica di come la perturbazione adversarial si propaga nella rete, e il rapporto $\sqrt{\mathrm{SLAV}^{(l')}/\mathrm{SLAV}^{(l)}}$ tra due livelli estesi consecutivi $l' = l + 1$ fornisce una misura del guadagno effettivo: valori maggiori di uno indicano che quello stadio amplifica la perturbazione.

La metrica presenta tuttavia un limite. La ReLU è positivamente omogenea, per ogni $c > 0$ $\mathrm{ReLU}(c\,\boldsymbol{x}) = c\,\mathrm{ReLU}(\boldsymbol{x})$; di conseguenza, moltiplicando $\boldsymbol{W}^{(l)}$ e $\boldsymbol{b}^{(l)}$ per c e dividendo $\boldsymbol{W}^{(l+1)}$ per lo stesso fattore si ottiene una rete che calcola esattamente la stessa funzione, ma per la quale $\mathrm{SLAV}^{(l)}$ risulta moltiplicata per c^2. Il valore assoluto della metrica non è quindi invariante rispetto a riparametrizzazioni che lasciano inalterato il comportamento e la robustezza della rete, e una sua variazione può riflettere un semplice cambiamento di scala delle attivazioni anziché un reale cambiamento della loro geometria. Per questo motivo, nell'analisi e nell'implementazione, la SLAV viene affiancata dalla sua versione relativa

$$\mathrm{SLAV}_{rel}^{(l)}(\mathcal{B}) = \frac{\mathrm{SLAV}^{(l)}(\mathcal{B})}{\rho^{(l)}(\mathcal{B})}, \qquad \rho^{(l)}(\mathcal{B}) = \frac{1}{n\,d_l}\sum_{i=1}^{n}\left\|\boldsymbol{a}^{(l)}(\boldsymbol{x}_i)\right\|_2^2, \tag{5.4}$$

che rapporta la deviazione alla scala media delle attivazioni pulite ed è invariante rispetto alla riscalatura descritta, poiché numeratore e denominatore risultano moltiplicati per lo stesso fattore c^2. La versione relativa può essere definita anche per singola coppia, rapportando la distanza (5.1) alla scala del batch:

$$s_{rel}^{(l)}\left(\boldsymbol{x}_i, \boldsymbol{x}_i^{adv}\right) = \frac{s^{(l)}\left(\boldsymbol{x}_i, \boldsymbol{x}_i^{adv}\right)}{\rho^{(l)}(\mathcal{B})}. \tag{5.5}$$

Poiché il denominatore è comune a tutte le coppie del batch, la media delle distanze relative coincide con $\mathrm{SLAV}_{rel}^{(l)}(\mathcal{B})$. La normalizzazione rispetto alla scala media del batch, anziché rispetto alla norma del singolo campione, evita inoltre che campioni con attivazioni di norma prossima a zero producano rapporti arbitrariamente grandi.

Le due versioni hanno, nel seguito, impieghi diversi. La versione relativa è quella su cui operano le loss di entrambe le strategie geometriche: come discusso nella Sezione 5.3.2, una loss costruita sulla distanza assoluta può essere soddisfatta da una semplice riscalatura del livello – un rimpicciolimento nella strategia attrattiva e un'inflazione in quella repulsiva – senza alcuna modifica effettiva della geometria delle attivazioni. La versione assoluta resta invece uno strumento diagnostico.

5.2.3 Ruolo della metrica nella metodologia

La SLAV svolge tre ruoli distinti nel resto del capitolo. É anzitutto uno strumento diagnostico, che localizza gli stadi della rete in cui la perturbazione cresce e rende la vulnerabilità del modello una grandezza distribuita lungo la profondità anziché un singolo numero calcolato all'uscita. É poi, nella sua versione relativa, direttamente il termine di regolarizzazione minimizzato dalla strategia attrattiva (Sezione 5.3.2), e infine, sempre nella versione relativa, interviene nella strategia repulsiva sia come distanza su cui opera la loss sia come scala di riferimento rispetto alla quale vengono calibrati i margini di separazione di ciascun livello. In tutti e tre i casi, la metrica è definita livello per livello, ed è proprio questa granularità a far distinguere l'approccio proposto: poiché la vulnerabilità non è distribuita uniformemente lungo la rete, neanche l'intervento può esserlo.

5.3 Estensione dei livelli della rete

A partire dal livello che si limita a inoltrare lo stato di flusso vengono definiti, per specializzazione, tre tipi di livello esteso. Il livello di detection (DetectorLayer) affianca alla trasformazione affine un detector che classifica le attivazioni del livello stesso come pulite o adversarial. Il livello con la loss sulle attivazioni è invece un'astrazione che, disponendo delle coppie di campioni puliti e adversarial, calcola per ognuna una distanza $\tilde{s}^{(l)}$ e la converte in un termine di loss attraverso una funzione scalare φ:

$$\mathcal{L}_{act}^{(l)}(\mathcal{B}) = \frac{1}{n}\sum_{i=1}^{n} \varphi\Big(\tilde{s}^{(l)}\big(\boldsymbol{x}_i, \boldsymbol{x}_i^{adv}\big)\Big), \qquad \mathcal{L}_{act}(\mathcal{B}) = \sum_{l\in\mathcal{I}} \mathcal{L}_{act}^{(l)}(\mathcal{B}). \tag{5.6}$$

La distanza $\tilde{s}^{(l)}$ è di default la (5.5) per entrambe le strategie geometriche discusse nella Sezione 5.3.2, che differiscono per la sola scelta di φ: la strategia attrattiva (CloserAL) ne adotta una funzione crescente, la strategia repulsiva (FurtherAL) una funzione decrescente con margine. Il livello FurtherAL è inoltre, al tempo stesso, un livello di detection: allontanare le attivazioni pulite da quelle adversarial ha l'intento di lavorare con un detector ai fini di un rilevamento più efficace, in cui il detector è capace di differenziare meglio le due attivazioni.

In aggiunta, le due strategie geometriche perseguono obiettivi incompatibili sulla stessa rappresentazione, e per questo motivo un livello CloserAL non può coesistere con livelli dotati di detector all'interno della stessa rete; le ragioni di questo vincolo, verificato in fase di costruzione del modello, sono discusse nel seguito. Ne derivano due famiglie di modelli: i modelli basati sul rilevamento, i cui livelli estesi sono di tipo DetectorLayer o FurtherAL, e i modelli basati sulla classificazione robusta (adversarial training), i cui livelli estesi sono di tipo CloserAL.

5.3.1 Architettura biforcata per detection multi-layer

Il rilevamento rappresenta una famiglia di difese particolarmente adatta al dominio NIDS, nel quale il rifiuto di un flusso sospetto è un'azione operativamente accettabile. La letteratura discussa nel Paragrafo 3.2.4 ha già argomentato che le rappresentazioni intermedie costituiscono una sorgente di segnale più ricca dell'input grezzo, ma lo ha sfruttato collegando un detector a un unico punto della rete, oppure calcolando a posteriori statistiche su una rete addestrata in modo standard. L'architettura proposta si differenzia da entrambi

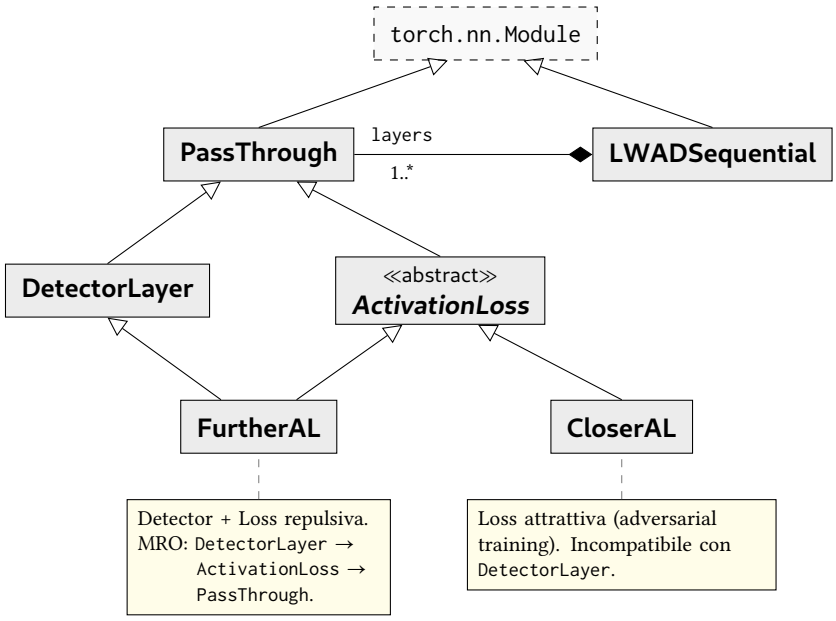


Figura 5.1. Class diagram delle estensioni dei moduli PyTorch definiti in `lwad_wrapper.py`. `LWADSequential` incapsula automaticamente ogni `nn.Module` non esteso in un `PassThrough`.

gli approcci: un detector di piccole dimensioni viene associato a ciascun livello esteso ed è addestrato congiuntamente alla backbone. Questa scelta presenta diversi vantaggi: ciascun detector si specializza sulla geometria del proprio livello, le uscite dei singoli detector permettono di localizzare il punto della rete in cui la perturbazione diventa rilevabile, contribuendo all'interpretabilità del modello, e la decisione può essere presa in modo progressivo lungo la forward pass.

Struttura del detector. Per ogni $l \in \mathcal{I}$ si introduce un detector $g^{(l)}_{\boldsymbol{\phi}_l} : \mathbb{R}^{d_l} \to \mathbb{R}$, con parametri $\boldsymbol{\phi}_l$ disgiunti da quelli della backbone, che riceve la pre-attivazione $\boldsymbol{a}^{(l)}(\boldsymbol{x})$ e ne restituisce un logit. La pre-attivazione prosegue dunque lungo due rami (Figura 5.2): verso il livello successivo della backbone, e verso il detector del suo livello. Il detector è a sua volta una piccola rete feedforward, composta da una layer normalization (opzionale) in ingresso, da un numero ridotto di livelli nascosti con attivazione ReLU e da un livello lineare finale con una sola uscita; la sua larghezza e la sua profondità sono iperparametri del modello. La quantità

$$p^{(l)}(\boldsymbol{x}) = \sigma\Big(g^{(l)}\big(\boldsymbol{a}^{(l)}(\boldsymbol{x})\big)\Big) \in (0,1),$$

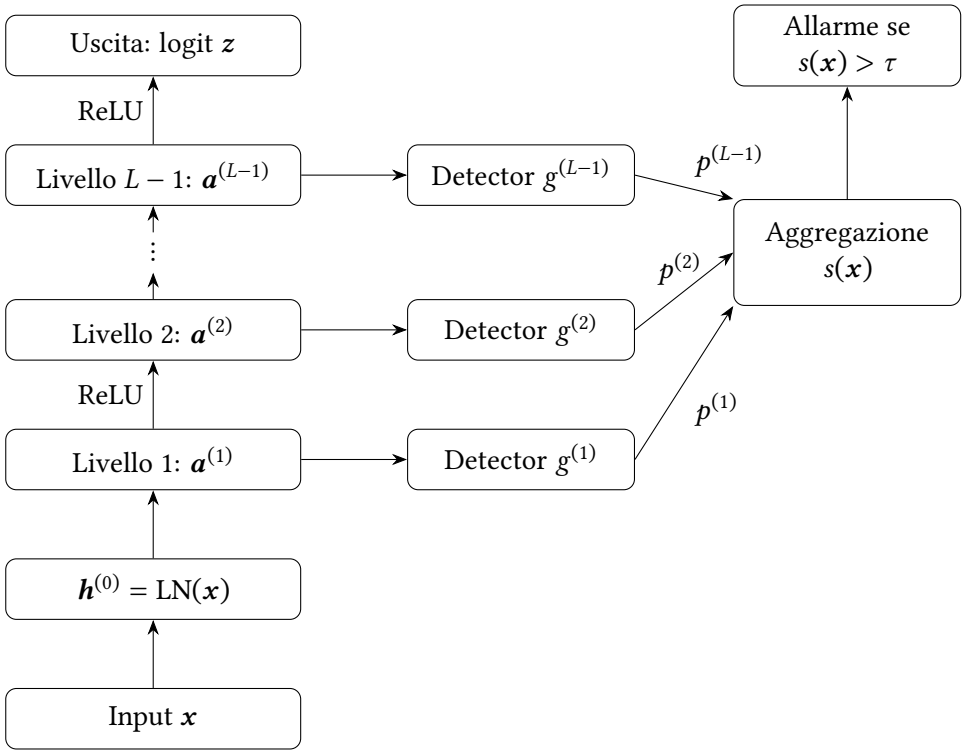


Figura 5.2. Architettura biforcata nel caso in cui tutti i livelli nascosti siano estesi. Le probabilità stimate dai detector vengono aggregate in un unico sistema di allarme.

dove σ è la funzione sigmoide, è interpretata come la stima della probabilità che il campione sia adversarial.

La normalizzazione in ingresso al detector ha una giustificazione specifica. La layer normalization è invariante rispetto a riscalature positive e traslazioni uniformi del vettore a cui è applicata, $\mathrm{LN}(c\,\boldsymbol{v} + \boldsymbol{t}) = \mathrm{LN}(\boldsymbol{v})$ per $c > 0$ e per ogni vettore costante $\boldsymbol{t}$. Il detector diventa così insensibile alla scala globale delle attivazioni, che varia durante l'addestramento della backbone e che può cambiare senza che cambi la funzione calcolata dalla rete. Il detector è quindi indotto a basare la propria decisione sulla forma del pattern di attivazione, ossia sulla sua direzione nello spazio latente, piuttosto che sulla sua ampiezza.

Obiettivo di addestramento. In fase di addestramento ogni minibatch pulito viene affiancato dalla sua versione adversarial, ottenendo un batch misto di $2n$ elementi $\tilde{\boldsymbol{x}}_j$ con indicatori $\eta_j \in \{0, 1\}$. La loss del detector al livello l è la binary cross-entropy

$$\mathcal{L}_{det}^{(l)}(\mathcal{B}) = \frac{1}{2n} \sum_{j=1}^{2n} \ell_{BCE}\Big(g^{(l)}\big(\boldsymbol{a}^{(l)}(\tilde{\boldsymbol{x}}_j)\big), \eta_j \Big)$$

e la loss complessiva di detection è $\mathcal{L}_{det}(\mathcal{B}) = \sum_{l \in \mathcal{I}} \mathcal{L}_{det}^{(l)}(\mathcal{B})$. É importante sottolineare che l'etichetta del detector non è la classe y del flusso, bensì l'indicatore η della presenza di una perturbazione adversarial: il detector risolve un task ortogonale a quello della backbone, e un flusso benigno o malevolo deve essere riconosciuto come adversarial in entrambi i casi se è stato perturbato.

Aggregazione degli score. Le stime dei singoli detector vengono combinate in uno score di allarme $s(\boldsymbol{x})$ secondo una tra due politiche:

$$s_{mean}(\boldsymbol{x}) = \frac{1}{|\mathcal{I}|} \sum_{l \in \mathcal{I}} p^{(l)}(\boldsymbol{x}), \qquad s_{max}(\boldsymbol{x}) = \max_{l \in \mathcal{I}} p^{(l)}(\boldsymbol{x}),$$

e il campione viene segnalato come adversarial se $s(\boldsymbol{x}) > \tau$, con τ soglia di decisione. Si indica con $F_\tau(\boldsymbol{x}) = \mathbb{I}[s(\boldsymbol{x}) > \tau]$ la funzione di allarme.
Le due politiche corrispondono a due diverse logiche di voto:

- La media s_{mean} realizza un "voto democratico": l'aggregazione ne riduce la varianza, e un singolo detector erroneamente confidente non è sufficiente a far scattare l'allarme, a vantaggio di una riduzione nel tasso di falsi positivi;
- Il massimo s_{max} realizza un "or logico": l'allarme scatta se almeno un detector è sufficientemente confidente, il che aumenta la sensibilità ma fa crescere i falsi positivi con il numero di detector, per cui questa politica si presta meglio a un numero contenuto di detector ben calibrati; la calibrazione della soglia sullo score aggregato, descritta di seguito, compensa in parte questo effetto.

La politica del massimo possiede inoltre una proprietà rilevante per i vincoli di latenza di un NIDS (Paragrafo 2.1.6). Non appena un detector supera la soglia, la decisione finale è determinata, e l'inferenza può essere interrotta a quel livello senza modificarne l'esito. Si ottiene così un meccanismo di uscita anticipata per ridurre il costo di inferenza delle reti profonde, che nel contesto considerato consente di scartare un flusso sospetto senza completare il calcolo della backbone.

Calibrazione della soglia. La soglia τ non viene fissata a priori, ma calibrata sul validation set al termine dell'addestramento. Dati gli score $s(\boldsymbol{x})$ dei campioni puliti e quelli dei corrispondenti campioni adversarial, la soglia scelta è quella che massimizza

l'accuratezza bilanciata del rilevamento:

$$\tau^* = \arg\max_{\tau \in G} \ \frac{1}{2}\big(\mathrm{TPR}(\tau) + \mathrm{TNR}(\tau)\big),$$

dove $\mathrm{TPR}(\tau)$ è la frazione di campioni adversarial con score superiore a τ e $\mathrm{TNR}(\tau)$ la frazione di campioni puliti con score non superiore a τ. I campioni adversarial impiegati per la calibrazione sono generati con lo stesso attacco usato in addestramento e non con quello di valutazione: il difensore non conosce l'attacco che incontrerà in esercizio, e calibrare la soglia sull'attacco di test costituirebbe una "fuga di informazioni" a favore della difesa.

5.3.2 Architetture per la manipolazione geometrica dello spazio delle attivazioni

La SLAV misura la deviazione indotta dall'attacco, ma di per sé non la modifica. Trasformando la distanza (5.1) in un termine di loss è possibile intervenire direttamente sulla geometria dello spazio delle attivazioni, in due direzioni opposte. Nella prima si vuole rendere la perturbazione irrilevante, facendo sì che il campione adversarial sia rappresentato come il corrispondente campione pulito e sia quindi classificato correttamente. Nella seconda si vuole invece rendere la perturbazione evidente, amplificando in modo controllato la differenza tra le due rappresentazioni proprio nei livelli osservati dai detector. La Figura 5.3 illustra schematicamente le due strategie.

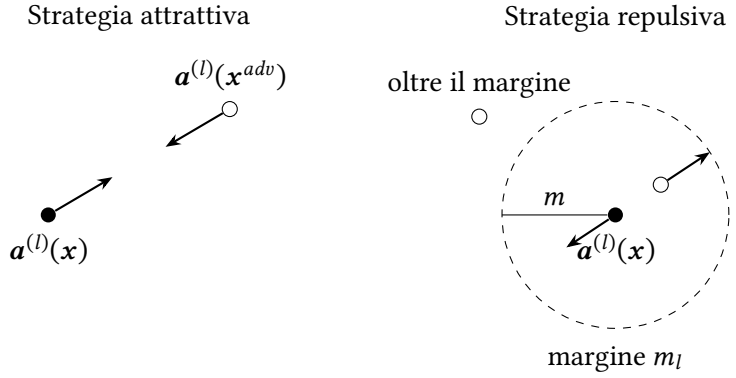


Figura 5.3. Rappresentazione schematica delle due strategie nello spazio delle attivazioni del livello l. A sinistra, la loss attrattiva avvicina tra loro le attivazioni pulite (•) e adversarial (◦), riducendone la distanza rapportata alla scala delle attivazioni pulite. A destra, la loss repulsiva le allontana, finché la stessa distanza non raggiunge il margine.

Strategia attrattiva (CloserAL): adversarial training

La strategia attrattiva adotta $\tilde{s}^{(l)} = s^{(l)}_{rel}$ e $\varphi(s) = s$ nella (5.6), per cui la loss sulle attivazioni coincide con la somma delle vulnerabilità relative dei livelli estesi:

$$\mathcal{L}_{act}(\mathcal{B}) = \mathcal{L}_{CLO}(\mathcal{B}) = \sum_{l \in \mathcal{I}} \mathrm{SLAV}^{(l)}_{rel}(\mathcal{B}).$$

Essa è combinata con l'adversarial training, ossia con una loss del task calcolata sia sui campioni puliti sia su quelli adversarial, tramite l'astrazione CloserAL. Il motivo per cui la loss opera sulla versione relativa della metrica, anziché sulla vulnerabilità assoluta (5.3), è discusso più avanti.

Il principio su cui si fonda la strategia è il seguente: se le attivazioni di un livello coincidono, coincidono anche le uscite della rete, mentre non vale il viceversa.

Questa asimmetria chiarisce perché la loss attrattiva non sia ridondante rispetto all'adversarial training. L'adversarial training richiede soltanto che il campione perturbato sia classificato correttamente, ossia che la sua rappresentazione nell'ultimo livello si trovi dalla parte corretta della frontiera decisionale; tale requisito può essere soddisfatto anche da rappresentazioni intermedie lontane da quelle del corrispondente campione pulito, magari prossime a quelle di altri campioni della stessa classe. Si ha dunque che l'adversarial training non implica necessariamente attivazioni vicine, mentre la minimizzazione di $\mathcal{L}_{CLO}$ implica attivazioni vicine, e dunque predizioni coerenti. I due termini sono complementari: l'adversarial training fornisce il bersaglio corretto per l'uscita del campione perturbato, la loss attrattiva vincola il percorso con cui tale uscita viene raggiunta.

É interessante osservare anche perché possa essere utile estendere più livelli anziché uno solo. In linea di principio sarebbe sufficiente allineare un livello iniziale per allineare tutti i successivi; nella pratica, l'allineamento è soltanto approssimato, e la deviazione residua può essere comunque amplificata in modo tanto maggiore quanto più il livello è vicino all'ingresso (cioè lontano dall'uscita). Vincolare più livelli limita la riamplificazione della perturbazione residua lungo il resto della rete.

La scelta della versione relativa della distanza nasce dall'esistenza di una soluzione degenere, che la loss attrattiva ammetterebbe se fosse formulata sulla distanza assoluta (5.1), legata alla riscalatura discussa nella Sezione 5.2.2. Moltiplicando per un fattore $c < 1$ i parametri $\boldsymbol{W}^{(l)}$ e $\boldsymbol{b}^{(l)}$ di un livello esteso e dividendo $\boldsymbol{W}^{(l+1)}$ per lo stesso fattore, la funzione calcolata

dalla rete, e con essa la loss del task, resta invariata, mentre $\mathrm{SLAV}^{(l)}$ risulta moltiplicata per c^2: la penalità potrebbe quindi essere ridotta a piacere, fino ad annullarsi al limite $c \to 0$, senza alcuna modifica della geometria delle attivazioni e senza alcun guadagno di robustezza. Come mostrato nella Sezione 5.3.3, non si tratta di una possibilità soltanto teorica: il gradiente della distanza assoluta contiene sempre una componente che spinge il livello in questa direzione, alla quale la loss del task, invariante rispetto alla riscalatura, non oppone alcuna resistenza. La distanza relativa (5.5) è invece invariante rispetto alla riscalatura, che non può quindi ridurla; affinchè anche il suo gradiente sia privo della componente di riscalatura, tuttavia, è necessario che il riferimento pulito sia lasciato libero (Sezione 5.3.3).

Resta da motivare l'incompatibilità con i detector. La loss attrattiva minimizza esattamente l'informazione che un detector sfrutta: se $s_{rel}^{(l)} \to 0$, allora le attivazioni pulite e adversarial del livello tendono a coincidere, le distribuzioni degli ingressi del detector nei due casi diventano indistinguibili e nessun classificatore che osservi $\boldsymbol{a}^{(l)}$ può separarle sufficientemente bene. I due obiettivi sono in conflitto diretto, e i modelli basati su CloserAL affidano pertanto la difesa esclusivamente alla classificazione corretta dei campioni perturbati.

Strategia repulsiva (FurtherAL): detection improvement

La strategia repulsiva è il duale della precedente: anziché nascondere la perturbazione, la amplifica in modo controllato nei livelli osservati dai detector, riducendo la sovrapposizione tra le rappresentazioni dei campioni puliti e quelle dei campioni adversarial, con l'intento di facilitare il compito di rilevamento. Essa si affianca alle proposte che modificano l'obiettivo di addestramento, per ottenere rappresentazioni latenti nelle quali i campioni adversarial siano più facilmente distinguibili [61], ma agisce localmente sui singoli livelli e con un vincolo geometrico esplicito. Il livello FurtherAL combina un detector, identico a quello della Sezione 5.3.1, con una loss repulsiva calcolata sulle stesse attivazioni.

La costruzione di una loss repulsiva richiede più cautela di quella attrattiva, perché la distanza da massimizzare non è limitata superiormente. Si considerino, data s la distanza di una coppia, tre candidati:

- $\varphi(s) = -s$. La minimizzazione di φ massimizza la distanza, ma $\partial\varphi/\partial s = -1$ ovunque: la spinta ha la stessa intensità per coppie sovrapposte e per coppie già ampiamente

separate, perché manca una nozione di separazione sufficiente. La loss non è limitata inferiormente, la sua minimizzazione diverge e il termine finisce per dominare la loss complessiva a scapito del task.

- $\varphi(s) = 1/(1+s)$. Questa volta la loss è limitata, ma $\partial\varphi/\partial s = -1/(1+s)^2$ decade quadraticamente con la separazione: di fatto la spinta si può indebolire prima che la separazione sia sufficiente.
- $\varphi(s) = \max(0, m - s)$, con margine $m > 0$. La derivata vale -1 per $s < m$ e 0 per $s \geq m$: all'interno del margine la loss si comporta come $-s$, con una spinta costante che non si indebolisce, e si annulla non appena la coppia raggiunge il margine. Il margine non modifica l'intensità della spinta, ovviando al problema del secondo candidato, e stabilisce esplicitamente quando interromperla, fornendo la separazione sufficiente che mancava al primo candidato.

La terza forma prende ispirazione dal termine relativo alle coppie dissimili della contrastive loss di [34]. Resta da stabilire su quale distanza applicare il margine. Come nella strategia attrattiva, la distanza (5.1) si rivela nuovamente inadeguata per effetto della mancata invarianza discussa precedentemente, ma questa volta nel verso opposto. Un'inflazione sufficiente della scala porta di fatto ogni coppia oltre il margine, annullando la loss insieme al suo gradiente senza che la geometria delle attivazioni sia cambiata, e senza alcun vantaggio per il detector: infatti, una riscalatura comune a tutte le attivazioni verrebbe rimossa del tutto dalla layer normalization in ingresso al detector, quando presente (Sezione 5.3.1). Il livello FurtherAL impiega pertanto, come il livello CloserAL, la distanza relativa (5.5); la loss opera così sulla stessa grandezza impiegata per la diagnosi e per la calibrazione dei margini. La loss repulsiva del livello l è quindi

$$\mathcal{L}_{FUR}^{(l)}(\mathcal{B}) = \frac{1}{n}\sum_{i=1}^{n} \max\Big(0,\ m_l - s_{rel}^{(l)}\big(\boldsymbol{x}_i, \boldsymbol{x}_i^{adv}\big)\Big), \quad \mathcal{L}_{act}(\mathcal{B}) = \mathcal{L}_{FUR}(\mathcal{B}) = \sum_{l\in\mathcal{I}} \mathcal{L}_{FUR}^{(l)}(\mathcal{B})\ .$$

Poiché il margine è applicato alla singola coppia prima della media, le coppie che hanno già superato il margine non contribuiscono al gradiente, che si concentra così sulle coppie ancora vicine, ossia sui campioni adversarial la cui rappresentazione è più simile a quella del corrispondente campione pulito: sono esattamente i campioni più difficili da rilevare, e la loss realizza di fatto una selezione implicita di quest'ultimi.

L'obiettivo della strategia repulsiva è, nelle intenzioni, asimmetrico: si vuole allontanare

la rappresentazione adversarial da quella pulita, non spostare quest'ultima, sulla quale si fonda la classificazione del task. Si potrebbe quindi pensare di trattare le attivazioni pulite come un riferimento fisso nella differenza, lasciando fluire il gradiente di quest'ultima soltanto attraverso il ramo adversarial. Come mostrato nella Sezione 5.3.3, tuttavia, poiché i pesi del livello sono condivisi tra i due rami, un simile ancoraggio non immobilizza il riferimento pulito: al contrario, lo sposta più di quanto si sposterebbe lasciandolo libero. Il gradiente fluisce quindi attraverso entrambi i rami, e la tutela della rappresentazione pulita è affidata alla loss del task, calcolata proprio sui campioni puliti, e al margine, che limita l'entità della spinta repulsiva.

Il margine deve essere commisurato alla scala naturale delle distanze relative $s_{rel}^{(l)}$, che differisce da livello a livello, proprio perché la perturbazione evolve in profondità, e da dataset a dataset. Un margine uniforme sarebbe inerte nei livelli in cui la distanza naturale lo supera già, dove la loss resterebbe identicamente nulla, e sproporzionato in quelli in cui la distanza naturale è di ordini di grandezza inferiore. Coerentemente con l'impostazione layer-wise, i margini vengono quindi definiti livello per livello come multipli della vulnerabilità relativa naturale:

$$m_l = \kappa\, \bar{s}^{(l)}, \qquad \bar{s}^{(l)} = \max\left(\mathrm{SLAV}_{rel}^{(l)}(\mathcal{P}),\ s_{min}\right), \tag{5.7}$$

dove $\mathcal{P}$ è un sottoinsieme del training set, s_{min} una piccola costante positiva che evita margini nulli e $\kappa > 0$ un fattore di scala.

La misura di $\bar{s}^{(l)}$ presenta una circolarità. Il margine è un iperparametro dell'addestramento, per cui non può essere misurato su un modello già addestrato, le cui attivazioni dipendono dal margine stesso; d'altra parte, su un modello inizializzato casualmente le distanze non sono rappresentative, sia perché l'attacco è poco efficace sia perché le attivazioni sono troppo diverse da quelle del modello addestrato. La circolarità viene risolta con una fase di warm-up: il modello viene addestrato per poche epoche con la stessa configurazione prevista, ma con peso nullo sulla loss repulsiva, così che il risultato non dipenda in alcun modo dal margine; su questo modello si generano i campioni adversarial con l'attacco di addestramento e si misura la (5.5) sui livelli estesi.

Il fattore κ ha un'interpretazione diretta. Per κ sensibilmente minore di 1 la maggior parte delle coppie si trova già oltre il margine, la loss repulsiva è in larga parte inattiva e il modello tende a coincidere con quello dotato dei soli detector; al limite $\kappa \to 0$ i due

modelli sono equivalenti (proprietà che verrà sfruttata come verifica di coerenza). Per $\kappa > 1$ a ciascun livello viene richiesto di amplificare la propria separazione naturale di un fattore κ in termini di deviazione quadratica media. Il fattore può essere fissato come iperparametro oppure selezionato con una ricerca monodimensionale su un insieme di valori candidati: per ciascun candidato il modello viene addestrato brevemente, a partire dalla stessa inizializzazione e con lo stesso ordine dei dati, così che i candidati differiscano soltanto per i margini, e viene scelto il valore che massimizza il punteggio di validazione definito nella Sezione 5.6.

5.3.3 Flusso dei gradienti

In una rete con più obiettivi, stabilire quali parametri ricevono il gradiente di quale loss è una scelta di modellazione importante quanto la definizione delle loss stesse. Per descriverla si introduce l'operatore di arresto del gradiente sg[·], che coincide con l'identità nella forward pass e ha gradiente nullo nella backward pass: esso non modifica la funzione calcolata, ma soltanto il percorso lungo cui i gradienti si propagano. L'operatore interviene in tre punti della metodologia.

Dal detector alla backbone: isolamento e cooperazione.

Il detector può ricevere in ingresso $\mathrm{sg}[\boldsymbol{a}^{(l)}]$ oppure $\boldsymbol{a}^{(l)}$. Indicando con $\boldsymbol{\theta}_b$ il vettore di parametri della backbone, con $\boldsymbol{\phi} = \{\boldsymbol{\phi}_l\}_{l \in \mathcal{I}}$ il vettore di parametri dei detector e con $\iota \in \{0, 1\}$ l'indicatore del regime isolato (in cui i detector ricevono in ingresso $\mathrm{sg}[\boldsymbol{a}^{(l)}]$), la loss complessiva $\mathcal{L}_{TOT}$ definita nella Sezione 5.5 produce i gradienti

$$\nabla_{\boldsymbol{\theta}_b} \mathcal{L}_{TOT} = \nabla_{\boldsymbol{\theta}_b} \mathcal{L}_{task} + \lambda_{act} \nabla_{\boldsymbol{\theta}_b} \mathcal{L}_{act} + (1 - \iota)\, \lambda_{det} \nabla_{\boldsymbol{\theta}_b} \mathcal{L}_{det},$$
$$\nabla_{\boldsymbol{\phi}} \mathcal{L}_{TOT} = \lambda_{det} \nabla_{\boldsymbol{\phi}} \mathcal{L}_{det}.$$

I parametri dei detector ricevono sempre e soltanto il gradiente della loss di detection, mentre la backbone lo riceve solo nel regime cooperativo.
Nel regime isolato ($\iota = 1$) il detector impara a classificare le attivazioni così come la backbone le produce, senza poterle influenzare, e la backbone viene ottimizzata esclusivamente per il task e per l'eventuale loss sulle attivazioni. Il detector si comporta come un rilevatore passivo sulle rappresentazioni: la sua accuratezza al livello l misura quanta informazione

sulla natura adversarial del campione è presente in $\boldsymbol{a}^{(l)}$ in una rete addestrata per il solo task. Il profilo di tale accuratezza lungo la profondità offre una lettura diretta, e interpretabile, del punto della rete in cui la perturbazione diventa riconoscibile.
Nel regime cooperativo ($\iota = 0$) la loss di detection si retropropaga anche nella backbone, che viene spinta a produrre attivazioni nelle quali i campioni adversarial sono più facilmente distinguibili da quelli puliti.
Con un'analogia, il detector è una telecamera di sorveglianza che osserva una stanza: nel regime isolato la telecamera impara a riconoscere gli intrusi ma non può modificare la stanza, nel regime cooperativo può anche disporre l'arredamento in modo che gli intrusi risultino più visibili. Il prezzo di questa libertà è la condivisione della capacità della backbone tra due obiettivi, con un possibile degrado dell'accuratezza del task, e una separazione che, a differenza di quella indotta dalla strategia repulsiva, non è vincolata da alcun margine. Da questo punto di vista, la strategia repulsiva può essere interpretata come un'alternativa geometrica e controllata al regime cooperativo.

Il riferimento nelle loss sulle attivazioni.

Nella distanza relativa (5.5), su cui operano le loss di entrambe le strategie, le attivazioni del campione pulito compaiono due volte: nella differenza $\boldsymbol{a}^{(l)}(\boldsymbol{x}^{adv}) - \boldsymbol{a}^{(l)}(\boldsymbol{x})$ a numeratore e, attraverso la scala $\rho^{(l)}$, a denominatore. Nella differenza esse possono essere trattate come variabile, $\boldsymbol{a}^{(l)}(\boldsymbol{x})$, oppure come riferimento fisso, $\mathrm{sg}[\boldsymbol{a}^{(l)}(\boldsymbol{x})]$; la scala, invece, è calcolata in ogni caso con il gradiente attivo, come spiegato in seguito.
L'intuizione suggerirebbe che ancorare il riferimento serva a non perturbare la rappresentazione pulita; l'analisi mostra però che, poiché i pesi del livello sono condivisi tra i due rami, l'ancoraggio non ottiene questo effetto in nessuna delle due strategie: nel caso attrattivo rende l'ottimizzazione instabile, nel caso repulsivo sposta il riferimento più di quanto accadrebbe lasciandolo libero, e converte gran parte della spinta in un'inflazione della scala delle attivazioni. L'analisi mostra inoltre che soltanto con il riferimento libero la distanza relativa svolge il compito per cui è stata introdotta.
Definiamo

- un singolo livello affine (senza costante per semplicità) $\boldsymbol{z} = \boldsymbol{W}\boldsymbol{h}$, con $\boldsymbol{W} \in \mathbb{R}^{D\times n}$;
- l'ingresso pulito $\boldsymbol{h}$ e quello adversarial $\boldsymbol{h}' = \boldsymbol{h} + \boldsymbol{\delta}$;

- l'output pulito $\boldsymbol{z} = \boldsymbol{W}\boldsymbol{h}$ e quello adversarial $\boldsymbol{z}' = \boldsymbol{W}\boldsymbol{h}' = \boldsymbol{W}\boldsymbol{h} + \boldsymbol{W}\boldsymbol{\delta}$.

La differenza tra le uscite è $\boldsymbol{z}' - \boldsymbol{z} = \boldsymbol{W}\boldsymbol{\delta}$. Su un batch composto da un solo campione, e con D nel ruolo di d_l, la distanza (5.1), la scala definita nella (5.4) e la distanza relativa (5.5) valgono quindi

$$s = \frac{1}{D}\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2, \qquad \rho = \frac{1}{D}\|\boldsymbol{W}\boldsymbol{h}\|_2^2, \qquad s_{rel} = \frac{s}{\rho} = \frac{\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2}{\|\boldsymbol{W}\boldsymbol{h}\|_2^2}.$$

L'analisi è condotta in due fasi: dapprima sul solo numeratore s, nel quale il riferimento compare attraverso la differenza e l'effetto dell'ancoraggio può essere isolato; poi sul denominatore ρ, che è ciò che distingue la distanza relativa da quella assoluta.
Per il numeratore, senza ancoraggio del campione pulito, il gradiente rispetto ai parametri è dunque

$$\frac{\partial s}{\partial \boldsymbol{W}} = \frac{2}{D}\,(\boldsymbol{W}\boldsymbol{\delta})\,\boldsymbol{\delta}^\top,$$

mentre con l'ancoraggio, indicata con $\hat{s} = \frac{1}{D}\|\boldsymbol{W}\boldsymbol{h}' - \mathrm{sg}[\boldsymbol{z}]\|_2^2$ la stessa grandezza in cui però $\boldsymbol{z}$ è trattato come costante, lo stesso calcolo, nel quale ora soltanto $\boldsymbol{W}\boldsymbol{h}'$ dipende da $\boldsymbol{W}$, fornisce

$$\frac{\partial \hat{s}}{\partial \boldsymbol{W}} = \frac{2}{D}\,(\boldsymbol{W}\boldsymbol{\delta})\,\boldsymbol{h}'^\top.$$

Poiché $\boldsymbol{h}' = \boldsymbol{h} + \boldsymbol{\delta}$, la seconda espressione si può riscrivere come

$$\frac{\partial \hat{s}}{\partial \boldsymbol{W}} = \frac{\partial s}{\partial \boldsymbol{W}} + \frac{2}{D}\,(\boldsymbol{W}\boldsymbol{\delta})\,\boldsymbol{h}^\top,$$

per cui ci si deve aspettare che i due casi differiscano tanto più quanto più $\|\boldsymbol{h}\|_2$ è grande rispetto a $\|\boldsymbol{\delta}\|_2$, ossia proprio nella situazione tipica di una perturbazione piccola.

Caso attrattivo. Si considera dapprima il solo numeratore, ponendo quindi la loss $\mathcal{L} = s$. Per i pesi, dopo un passo di discesa del gradiente con learning rate η, e posto $c = 2\eta/D$, si ha che i pesi aggiornati nel caso senza ancoraggio sono

$$\boldsymbol{W}^+ = \boldsymbol{W} - c\,(\boldsymbol{W}\boldsymbol{\delta})\,\boldsymbol{\delta}^\top;$$

moltiplicando per $\boldsymbol{\delta}$, con $\boldsymbol{\delta}^\top\boldsymbol{\delta} = \|\boldsymbol{\delta}\|_2^2$, e considerando che la distanza dopo il passo è $s^+ = \frac{1}{D}\|\boldsymbol{W}^+\boldsymbol{\delta}\|_2^2$, si ottiene

$$\boldsymbol{W}^+\boldsymbol{\delta} = \left(1 - c\|\boldsymbol{\delta}\|_2^2\right)\boldsymbol{W}\boldsymbol{\delta}, \qquad s^+ = \left(1 - c\|\boldsymbol{\delta}\|_2^2\right)^2 s,$$

e nel caso con ancoraggio, con gli stessi passaggi,

$$\hat{\boldsymbol{W}}^{+} = \boldsymbol{W} - c\,(\boldsymbol{W}\boldsymbol{\delta})\,\boldsymbol{h}'^{\top}, \qquad \hat{\boldsymbol{W}}^{+}\boldsymbol{\delta} = (1 - c\,\xi)\,\boldsymbol{W}\boldsymbol{\delta}, \qquad \hat{s}^{+} = (1 - c\,\xi)^2 s,$$

con $\xi = \boldsymbol{h}'^{\top}\boldsymbol{\delta}$. In entrambi i casi un passo di gradiente moltiplica la distanza per uno scalare, e la contrazione avviene se e solo se tale scalare ha modulo minore di uno.
Senza ancoraggio tale scenario si avvera dunque se e solo se

$$\left|1 - c\|\boldsymbol{\delta}\|_2^2\right| < 1 \;\Leftrightarrow\; 0 < c\|\boldsymbol{\delta}\|_2^2 < 2 \;\Leftrightarrow\; \eta < \frac{D}{\|\boldsymbol{\delta}\|_2^2}.$$

Poiché $\boldsymbol{\delta}$ è la propagazione di una perturbazione di norma limitata, e dunque piccola, la condizione è soddisfatta per qualunque learning rate ragionevole e la contrazione è garantita, graduale e monotona.
Con l'ancoraggio la condizione diventa $0 < c\,\xi < 2$, e per interpretarla è utile riscrivere ξ:

$$\begin{aligned}
\xi &= \boldsymbol{h}'^{\top}\boldsymbol{\delta} = \boldsymbol{h}'^{\top}(\boldsymbol{h}' - \boldsymbol{h}) = \|\boldsymbol{h}'\|_2^2 - \boldsymbol{h}'^{\top}\boldsymbol{h}, \\
\|\boldsymbol{\delta}\|_2^2 &= \|\boldsymbol{h}' - \boldsymbol{h}\|_2^2 = \|\boldsymbol{h}'\|_2^2 - 2\,\boldsymbol{h}'^{\top}\boldsymbol{h} + \|\boldsymbol{h}\|_2^2 \quad\Rightarrow\quad \boldsymbol{h}'^{\top}\boldsymbol{h} = \tfrac{1}{2}\left(\|\boldsymbol{h}'\|_2^2 - \|\boldsymbol{\delta}\|_2^2 + \|\boldsymbol{h}\|_2^2\right), \\
\xi &= \|\boldsymbol{h}'\|_2^2 - \tfrac{1}{2}\left(\|\boldsymbol{h}'\|_2^2 - \|\boldsymbol{\delta}\|_2^2 + \|\boldsymbol{h}\|_2^2\right) = \tfrac{1}{2}\left(\|\boldsymbol{h}'\|_2^2 - \|\boldsymbol{h}\|_2^2 + \|\boldsymbol{\delta}\|_2^2\right).
\end{aligned}$$

La condizione può fallire in due modi.

- L'ingresso adversarial ha un modulo sufficientemente inferiore a quello pulito:

  $$\|\boldsymbol{h}\|_2^2 - \|\boldsymbol{h}'\|_2^2 > \|\boldsymbol{\delta}\|_2^2 \;\Rightarrow\; \|\boldsymbol{h}'\|_2^2 - \|\boldsymbol{h}\|_2^2 + \|\boldsymbol{\delta}\|_2^2 < 0,$$

  quindi si ha $\xi < 0$ e il passo di gradiente aumenta la distanza che dovrebbe ridurre.
- Se invece la differenza dei moduli è positiva e grande, poiché $\|\boldsymbol{h}'\|_2 \gg \|\boldsymbol{\delta}\|_2$ il termine $c\,\xi$ può superare 2, e l'aggiornamento oltrepassa il minimo invertendo il segno della differenza, impedendo la contrazione.

L'analisi rafforza l'idea che per la strategia attrattiva il riferimento vada quindi lasciato libero, coerentemente con l'intuizione che l'obiettivo sia far incontrare le due rappresentazioni, entrambe libere di muoversi.

Caso repulsivo. Come nel caso attrattivo, l'analisi è condotta dapprima sul solo numeratore s; il ruolo del denominatore è discusso nel seguito, per entrambe le strategie.

All'interno del margine la minimizzazione di φ corrisponde a una salita del gradiente sulla distanza. Infatti, indicato con ν il learning rate,

$$\frac{\partial \varphi}{\partial \boldsymbol{W}} = \varphi'(s)\,\frac{\partial s}{\partial \boldsymbol{W}} = -\frac{\partial s}{\partial \boldsymbol{W}} \;\Rightarrow\; \boldsymbol{W}^{+} = \boldsymbol{W} - \nu\,\frac{\partial \varphi}{\partial \boldsymbol{W}} = \boldsymbol{W} + \nu\,\frac{\partial s}{\partial \boldsymbol{W}},$$

e, posto $c = 2\nu/D$, si ha nel caso senza ancoraggio

$$\boldsymbol{W}^{+} = \boldsymbol{W} + c\,(\boldsymbol{W}\boldsymbol{\delta})\,\boldsymbol{\delta}^{\top},$$

da cui, moltiplicando a destra rispettivamente per $\boldsymbol{\delta}$ e per $\boldsymbol{h}$,

$$\boldsymbol{W}^{+}\boldsymbol{\delta} = \left(1 + c\|\boldsymbol{\delta}\|_2^2\right)\boldsymbol{W}\boldsymbol{\delta}, \qquad \boldsymbol{W}^{+}\boldsymbol{h} = \boldsymbol{z} + c\,(\boldsymbol{\delta}^{\top}\boldsymbol{h})\,\boldsymbol{W}\boldsymbol{\delta},$$

e nel caso con ancoraggio

$$\hat{\boldsymbol{W}}^{+} = \boldsymbol{W} + c\,(\boldsymbol{W}\boldsymbol{\delta})\,\boldsymbol{h}'^{\top}, \qquad \hat{\boldsymbol{W}}^{+}\boldsymbol{\delta} = (1 + c\,\xi)\,\boldsymbol{W}\boldsymbol{\delta}, \qquad \hat{\boldsymbol{W}}^{+}\boldsymbol{h} = \boldsymbol{z} + c\,\omega\,\boldsymbol{W}\boldsymbol{\delta},$$

con $\omega = \boldsymbol{h}'^{\top}\boldsymbol{h}$.

Il confronto dei soli fattori di espansione della distanza, $\left(1 + c\|\boldsymbol{\delta}\|_2^2\right)^2$ senza ancoraggio e $(1 + c\,\xi)^2$ con ancoraggio, suggerirebbe che l'ancoraggio renda la spinta più incisiva, dato che $|\xi|$ può essere molto maggiore di $\|\boldsymbol{\delta}\|_2^2$. Il confronto è però incompleto: l'ancoraggio ha lo scopo di tenere fermo il riferimento, ed è dunque necessario osservare anche cosa accade all'uscita pulita. Dalle espressioni di $\boldsymbol{W}^{+}\boldsymbol{h}$ e $\hat{\boldsymbol{W}}^{+}\boldsymbol{h}$ si ricavano gli spostamenti dell'uscita pulita nei due casi,

$$\|\boldsymbol{W}^{+}\boldsymbol{h} - \boldsymbol{z}\|_2 = c\,|\boldsymbol{\delta}^{\top}\boldsymbol{h}|\,\|\boldsymbol{W}\boldsymbol{\delta}\|_2, \qquad \|\hat{\boldsymbol{W}}^{+}\boldsymbol{h} - \boldsymbol{z}\|_2 = c\,|\omega|\,\|\boldsymbol{W}\boldsymbol{\delta}\|_2\,.$$

Per Cauchy–Schwarz, $|\boldsymbol{\delta}^{\top}\boldsymbol{h}| \le \|\boldsymbol{\delta}\|_2\|\boldsymbol{h}\|_2$, quindi

$$\|\boldsymbol{W}^{+}\boldsymbol{h} - \boldsymbol{z}\|_2 \le c\,\|\boldsymbol{\delta}\|_2\|\boldsymbol{h}\|_2\,\|\boldsymbol{W}\boldsymbol{\delta}\|_2. \tag{A}$$

Espandendo $\boldsymbol{h}' = \boldsymbol{h} + \boldsymbol{\delta}$:

$$\omega = (\boldsymbol{h} + \boldsymbol{\delta})^{\top}\boldsymbol{h} = \|\boldsymbol{h}\|_2^2 + \boldsymbol{\delta}^{\top}\boldsymbol{h}.$$

Sempre per Cauchy–Schwarz, $\boldsymbol{\delta}^{\top}\boldsymbol{h} \ge -\|\boldsymbol{\delta}\|_2\|\boldsymbol{h}\|_2$, quindi

$$\omega \ge \|\boldsymbol{h}\|_2^2 - \|\boldsymbol{\delta}\|_2\|\boldsymbol{h}\|_2 = \|\boldsymbol{h}\|_2\big(\|\boldsymbol{h}\|_2 - \|\boldsymbol{\delta}\|_2\big).$$

Con l'ipotesi $\|\boldsymbol{\delta}\|_2 \leq \|\boldsymbol{h}\|_2$ (perturbazione piccola rispetto all'ingresso pulito), il membro destro è non negativo. Quindi $\omega \geq 0$, $|\omega| = \omega$ e

$$\|\hat{\boldsymbol{W}}^{+}\boldsymbol{h} - \boldsymbol{z}\|_2 \geq c\,\|\boldsymbol{h}\|_2\big(\|\boldsymbol{h}\|_2 - \|\boldsymbol{\delta}\|_2\big)\|\boldsymbol{W}\boldsymbol{\delta}\|_2. \tag{B}$$

Raccogliendo $\|\boldsymbol{\delta}\|_2\|\boldsymbol{h}\|_2$:

$$\|\boldsymbol{h}\|_2\big(\|\boldsymbol{h}\|_2 - \|\boldsymbol{\delta}\|_2\big) = \Big(\frac{\|\boldsymbol{h}\|_2}{\|\boldsymbol{\delta}\|_2} - 1\Big)\|\boldsymbol{\delta}\|_2\|\boldsymbol{h}\|_2.$$

Da (B) e dal passo precedente:

$$\|\hat{\boldsymbol{W}}^{+}\boldsymbol{h} - \boldsymbol{z}\|_2 \geq \Big(\frac{\|\boldsymbol{h}\|_2}{\|\boldsymbol{\delta}\|_2} - 1\Big)\, c\,\|\boldsymbol{\delta}\|_2\|\boldsymbol{h}\|_2\,\|\boldsymbol{W}\boldsymbol{\delta}\|_2.$$

Il fattore tra parentesi è non negativo, quindi si può applicare (A) all'ultimo prodotto senza invertire la disuguaglianza:

$$\|\hat{\boldsymbol{W}}^{+}\boldsymbol{h} - \boldsymbol{z}\|_2 \geq \Big(\frac{\|\boldsymbol{h}\|_2}{\|\boldsymbol{\delta}\|_2} - 1\Big)\|\boldsymbol{W}^{+}\boldsymbol{h} - \boldsymbol{z}\|_2.$$

L'ancoraggio, introdotto per tenere fermo il riferimento (ovvero l'uscita pulita), è quindi proprio la scelta che lo sposta di più, e di un fattore tanto maggiore quanto più la perturbazione è piccola rispetto all'ingresso pulito. Dunque il contributo del ramo pulito al gradiente non serve a spostare il riferimento, ma a compensare lo spostamento che il ramo adversarial gli imprimerebbe attraverso i pesi condivisi.

Il ruolo del denominatore. Si consideri ora la distanza relativa $s_{rel} = s/\rho$, sulla quale operano effettivamente le loss di entrambe le strategie. La scala è calcolata in ogni caso con il gradiente attivo, anche quando il riferimento nella differenza è ancorato, e dipende quindi da $\boldsymbol{W}$. Lo stesso calcolo svolto per s, con $\boldsymbol{h}$ al posto di $\boldsymbol{\delta}$, fornisce

$$\frac{\partial \rho}{\partial \boldsymbol{W}} = \frac{2}{D}\,(\boldsymbol{W}\boldsymbol{h})\,\boldsymbol{h}^{\top} = \frac{2}{D}\,\boldsymbol{z}\,\boldsymbol{h}^{\top},$$

e la regola di derivazione del prodotto dà, senza ancoraggio,

$$\frac{\partial s_{rel}}{\partial \boldsymbol{W}} = \frac{1}{\rho}\,\frac{\partial s}{\partial \boldsymbol{W}} - \frac{s}{\rho^2}\,\frac{\partial \rho}{\partial \boldsymbol{W}} = \frac{1}{\rho}\Big(\frac{\partial s}{\partial \boldsymbol{W}} - s_{rel}\,\frac{\partial \rho}{\partial \boldsymbol{W}}\Big) = \frac{2}{D\rho}\Big((\boldsymbol{W}\boldsymbol{\delta})\,\boldsymbol{\delta}^{\top} - s_{rel}\,\boldsymbol{z}\,\boldsymbol{h}^{\top}\Big).$$

Con l'ancoraggio, indicata con $\hat{s}_{rel} = \hat{s}/\rho$ la distanza relativa ancorata, la stessa regola si applica con $\hat{s}$ al posto di s nel numeratore, e poiché nel punto corrente $\hat{s} = s$ si ottiene

$$\frac{\partial \hat{s}_{rel}}{\partial \boldsymbol{W}} = \frac{1}{\rho}\Big(\frac{\partial \hat{s}}{\partial \boldsymbol{W}} - s_{rel}\,\frac{\partial \rho}{\partial \boldsymbol{W}}\Big) = \frac{2}{D\rho}\Big((\boldsymbol{W}\boldsymbol{\delta})\,\boldsymbol{h}'^{\top} - s_{rel}\,\boldsymbol{z}\,\boldsymbol{h}^{\top}\Big).$$

Ricordando che $\partial \hat{s}/\partial \boldsymbol{W} = \partial s/\partial \boldsymbol{W} + \frac{2}{D}(\boldsymbol{W}\boldsymbol{\delta})\boldsymbol{h}^{\top}$, di nuovo i due gradienti differiscono esattamente per lo stesso termine trovato per la distanza assoluta, normalizzato anch'esso per la scala delle attivazioni ρ:

$$\frac{\partial \hat{s}_{rel}}{\partial \boldsymbol{W}} = \frac{\partial s_{rel}}{\partial \boldsymbol{W}} + \frac{2}{D\rho}\,(\boldsymbol{W}\boldsymbol{\delta})\,\boldsymbol{h}^{\top}.$$

In entrambi i casi, il primo termine tra parentesi proviene dal numeratore, mentre il secondo, $-s_{rel}\,\boldsymbol{z}\,\boldsymbol{h}^{\top}$, è il contributo della scala al denominatore, identico nei due casi.

Dopo un passo di discesa con learning rate η, posto $c = 2\eta/(D\rho)$, moltiplicando a destra per $\boldsymbol{\delta}$ e per $\boldsymbol{h}$ si ottiene, senza ancoraggio,

$$\boldsymbol{W}^{+}\boldsymbol{\delta} = \big(1 - c\|\boldsymbol{\delta}\|_2^2\big)\,\boldsymbol{W}\boldsymbol{\delta} + c\,s_{rel}\,(\boldsymbol{h}^{\top}\boldsymbol{\delta})\,\boldsymbol{z}, \qquad \boldsymbol{W}^{+}\boldsymbol{h} = \big(1 + c\,s_{rel}\|\boldsymbol{h}\|_2^2\big)\,\boldsymbol{z} - c\,(\boldsymbol{\delta}^{\top}\boldsymbol{h})\,\boldsymbol{W}\boldsymbol{\delta},$$

e con l'ancoraggio

$$\hat{\boldsymbol{W}}^{+}\boldsymbol{\delta} = (1 - c\,\xi)\,\boldsymbol{W}\boldsymbol{\delta} + c\,s_{rel}\,(\boldsymbol{h}^{\top}\boldsymbol{\delta})\,\boldsymbol{z}, \qquad \hat{\boldsymbol{W}}^{+}\boldsymbol{h} = \big(1 + c\,s_{rel}\|\boldsymbol{h}\|_2^2\big)\,\boldsymbol{z} - c\,\omega\,\boldsymbol{W}\boldsymbol{\delta},$$

con ξ e ω definiti come sopra; nel caso repulsivo, che corrisponde a una salita del gradiente, con ν al posto di η, tutti i termini che contengono c cambiano segno. Tornando al caso attrattivo, rispetto alla distanza assoluta, i vettori aggiornati contengono un termine lungo $\boldsymbol{z}$, dovuto alla scala e identico nei due casi, mentre l'ancoraggio interviene, come prima, soltanto sui coefficienti di $\boldsymbol{W}\boldsymbol{\delta}$, sostituendo $\|\boldsymbol{\delta}\|_2^2$ con ξ e $\boldsymbol{\delta}^{\top}\boldsymbol{h}$ con ω. Questo però non basta per riutilizzare le conclusioni ricavate per il solo numeratore: la distanza aggiornata non è più un multiplo scalare di quella corrente. Occorre quindi calcolare direttamente come varia s_{rel} (limitandosi ad uno sviluppo al primo ordine in c).

Nel caso attrattivo, posto $\mu = (\boldsymbol{W}\boldsymbol{\delta})^{\top}\boldsymbol{z}/\|\boldsymbol{z}\|_2^2$ ([2]) e ricordando che $\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2 = s_{rel}\|\boldsymbol{z}\|_2^2$, svilup-

[2] μ è esattamente il coefficiente della componente del vettore $(\boldsymbol{W}\boldsymbol{\delta})$ parallela a $\boldsymbol{z}$. Infatti, presi due vettori $\boldsymbol{a} = \boldsymbol{z}$ e $\boldsymbol{b} = \boldsymbol{W}\boldsymbol{\delta}$, le componenti di $\boldsymbol{b}$ parallele e perpendicolari ad $\boldsymbol{a}$ sono $\boldsymbol{b}_{\parallel} = \frac{\boldsymbol{a}\cdot\boldsymbol{b}}{\|\boldsymbol{a}\|_2^2}\boldsymbol{a}$ e $\boldsymbol{b}_{\perp} = \boldsymbol{b} - \boldsymbol{b}_{\parallel}$.

pando i quadrati delle norme e trascurando i termini in c^2 si ottiene, senza ancoraggio,

$$\begin{aligned}
\|\boldsymbol{W}^+\boldsymbol{\delta}\|_2^2 &= \underbrace{\left(1-c\|\boldsymbol{\delta}\|_2^2\right)^2\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2}_{\|\boldsymbol{a}\|^2} + \underbrace{2c\left(1-c\|\boldsymbol{\delta}\|_2^2\right)s_{rel}(\boldsymbol{h}^\top\boldsymbol{\delta})(\boldsymbol{W}\boldsymbol{\delta})^\top\boldsymbol{z}}_{2\boldsymbol{a}^\top\boldsymbol{b}} + \underbrace{c^2 s_{rel}^2(\boldsymbol{h}^\top\boldsymbol{\delta})^2\|\boldsymbol{z}\|_2^2}_{\|\boldsymbol{b}\|^2=o(c)} \\
&= (1-2c\|\boldsymbol{\delta}\|_2^2 + \underbrace{c^2\|\boldsymbol{\delta}\|_2^4}_{o(c)})\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2 + 2cs_{rel}(\boldsymbol{h}^\top\boldsymbol{\delta})(\boldsymbol{W}\boldsymbol{\delta})^\top\boldsymbol{z} - \underbrace{2c^2\|\boldsymbol{\delta}\|_2^2 s_{rel}(\boldsymbol{h}^\top\boldsymbol{\delta})(\boldsymbol{W}\boldsymbol{\delta})^\top\boldsymbol{z}}_{o(c)} + o(c) \\
&= (1-2c\|\boldsymbol{\delta}\|_2^2)\underbrace{\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2}_{sost.} + 2cs_{rel}(\boldsymbol{h}^\top\boldsymbol{\delta})\underbrace{(\boldsymbol{W}\boldsymbol{\delta})^\top\boldsymbol{z}}_{sost.} + o(c) \\
&= (1-2c\|\boldsymbol{\delta}\|_2^2)s_{rel}\|\boldsymbol{z}\|_2^2 + 2cs_{rel}(\boldsymbol{h}^\top\boldsymbol{\delta})\mu\|\boldsymbol{z}\|_2^2 + o(c) \\
&\approx s_{rel}\|\boldsymbol{z}\|_2^2\left(1-2c\|\boldsymbol{\delta}\|_2^2 + 2c\,\mu\,\boldsymbol{\delta}^\top\boldsymbol{h}\right).
\end{aligned}$$

Analogamente si ottiene

$$\|\boldsymbol{W}^+\boldsymbol{h}\|_2^2 \approx \|\boldsymbol{z}\|_2^2\left(1+2c\,s_{rel}\|\boldsymbol{h}\|_2^2 - 2c\,\mu\,\boldsymbol{\delta}^\top\boldsymbol{h}\right),$$

Nel caso con ancoraggio i passaggi sono identici: cambia solo il coefficiente di $\boldsymbol{W}\boldsymbol{\delta}$, che diventa $1-c\,\xi$, mentre $\boldsymbol{b}$ resta uguale. Si ottiene quindi

$$\begin{aligned}
\|\hat{\boldsymbol{W}}^+\boldsymbol{\delta}\|_2^2 &\approx s_{rel}\|\boldsymbol{z}\|_2^2\left(1-2c\,\xi+2c\,\mu\,\boldsymbol{\delta}^\top\boldsymbol{h}\right) \\
\|\hat{\boldsymbol{W}}^+\boldsymbol{h}\|_2^2 &\approx \|\boldsymbol{z}\|_2^2\left(1+2c\,s_{rel}\|\boldsymbol{h}\|_2^2 - 2c\,\mu\,\omega\right).
\end{aligned}$$

Dividendo, usando $1/(1+x)\approx 1-x+o(x)$, omettendo i termini $o(c)$ e ricordando che $s_{rel} = \|\boldsymbol{W}\boldsymbol{\delta}\|_2^2/\|\boldsymbol{W}\boldsymbol{h}\|_2^2$ alla fine si ottiene

$$s_{rel}^+ \approx s_{rel}\left(1-2c\,\Lambda\right), \qquad \Lambda = \|\boldsymbol{\delta}\|_2^2 - 2\mu\,\boldsymbol{\delta}^\top\boldsymbol{h} + s_{rel}\|\boldsymbol{h}\|_2^2.$$

Con l'ancoraggio i calcoli sono identici, con ξ al posto di $\|\boldsymbol{\delta}\|_2^2$ nel primo sviluppo e ω al posto di $\boldsymbol{\delta}^\top\boldsymbol{h}$ nel secondo; sostituendo poi $\xi = \|\boldsymbol{\delta}\|_2^2 + \boldsymbol{\delta}^\top\boldsymbol{h}$ e $\omega = \|\boldsymbol{h}\|_2^2 + \boldsymbol{\delta}^\top\boldsymbol{h}$ si ottiene

$$\hat{s}_{rel}^+ \approx s_{rel}\left(1-2c\,\hat{\Lambda}\right), \qquad \hat{\Lambda} = \underbrace{\|\boldsymbol{\delta}\|_2^2 - 2\mu\,\boldsymbol{\delta}^\top\boldsymbol{h} + s_{rel}\|\boldsymbol{h}\|_2^2}_{\Lambda} + \boldsymbol{\delta}^\top\boldsymbol{h} - \mu\,\|\boldsymbol{h}\|_2^2.$$

Senza ancoraggio il passo è una discesa del gradiente e la distanza diminuisce: infatti $\Lambda \geq 0$, poiché per Cauchy–Schwarz $|\boldsymbol{\delta}^\top\boldsymbol{h}| \leq \|\boldsymbol{\delta}\|_2\|\boldsymbol{h}\|_2$ e $|\mu| \leq \|\boldsymbol{W}\boldsymbol{\delta}\|_2/\|\boldsymbol{z}\|_2 = \sqrt{s_{rel}}$, dunque ne segue che

$$
\begin{aligned}
2\mu\,\boldsymbol{\delta}^\top \boldsymbol{h} &\le 2\sqrt{s_{rel}}\,\|\boldsymbol{\delta}\|_2\|\boldsymbol{h}\|_2 \\
&\Downarrow \\
\Lambda &= \|\boldsymbol{\delta}\|_2^2 - 2\mu\,\boldsymbol{\delta}^\top \boldsymbol{h} + s_{rel}\|\boldsymbol{h}\|_2^2 \\
&\ge \|\boldsymbol{\delta}\|_2^2 - 2\sqrt{s_{rel}}\,\|\boldsymbol{\delta}\|_2\|\boldsymbol{h}\|_2 + s_{rel}\|\boldsymbol{h}\|_2^2 \\
&= \left(\|\boldsymbol{\delta}\|_2 - \sqrt{s_{rel}}\,\|\boldsymbol{h}\|_2\right)^2 \ge 0.
\end{aligned}
$$

per cui $\Lambda \ge 0$ e, se la perturbazione è piccola rispetto all'ingresso pulito, e lo è quindi anche la distanza relativa, Λ è un infinitesimo.

Rispetto a Λ, $\hat{\Lambda}$ contiene in più i termini $\boldsymbol{\delta}^\top \boldsymbol{h}$ e $-\mu\|\boldsymbol{h}\|_2^2$, di segno non controllato, che possono quindi dominare gli altri, come nel caso assoluto ξ poteva dominare $\|\boldsymbol{\delta}\|_2^2$. Si ritrovano così, con condizioni diverse, i due modi di fallimento già osservati nello studio della distanza assoluta.

Nel caso repulsivo, con il segno di c invertito, si ha $s_{rel}^{+} \approx s_{rel}(1+2c\,\Lambda)$ e $\hat{s}_{rel}^{+} \approx s_{rel}\left(1+2c\,\hat{\Lambda}\right)$: senza ancoraggio la spinta non riduce mai la distanza al primo ordine (coerentemente con l'obiettivo di applicare una loss repulsiva), mentre con l'ancoraggio può ridurla. Per l'uscita pulita, sempre con il segno invertito,

$$
\boldsymbol{W}^{+}\boldsymbol{h} - \boldsymbol{z} = -c\,s_{rel}\|\boldsymbol{h}\|_2^2\,\boldsymbol{z} + c\,(\boldsymbol{\delta}^\top \boldsymbol{h})\,\boldsymbol{W}\boldsymbol{\delta}, \qquad \hat{\boldsymbol{W}}^{+}\boldsymbol{h} - \boldsymbol{z} = -c\,s_{rel}\|\boldsymbol{h}\|_2^2\,\boldsymbol{z} + c\,\omega\,\boldsymbol{W}\boldsymbol{\delta}.
$$

Il primo termine, comune ai due casi, è un multiplo di $\boldsymbol{z}$ e ne cambia soltanto la norma, non la direzione. La direzione dell'uscita pulita può essere modificata soltanto dal secondo termine, che contiene lo stesso vettore $\boldsymbol{W}\boldsymbol{\delta}$ nei due casi, moltiplicato per $c\,\boldsymbol{\delta}^\top \boldsymbol{h}$ senza ancoraggio e per $c\,\omega$ con ancoraggio: la parte dello spostamento che cambia la direzione è quindi proporzionale a $|\boldsymbol{\delta}^\top \boldsymbol{h}|$ nel primo caso e a $|\omega|$ nel secondo. Con la stessa ipotesi $\|\boldsymbol{\delta}\|_2 \le \|\boldsymbol{h}\|_2$ e gli stessi passaggi che portano da (A) e (B) al confronto,

$$
|\omega| \ge \|\boldsymbol{h}\|_2\left(\|\boldsymbol{h}\|_2 - \|\boldsymbol{\delta}\|_2\right) = \left(\frac{\|\boldsymbol{h}\|_2}{\|\boldsymbol{\delta}\|_2} - 1\right)\|\boldsymbol{\delta}\|_2\|\boldsymbol{h}\|_2 \ge \left(\frac{\|\boldsymbol{h}\|_2}{\|\boldsymbol{\delta}\|_2} - 1\right)|\boldsymbol{\delta}^\top \boldsymbol{h}|,
$$

per cui anche con la distanza relativa ritroviamo il medesimo risultato osservato nel caso della distanza assoluta: l'ancoraggio sposta la direzione del riferimento di un fattore almeno $\|\boldsymbol{h}\|_2/\|\boldsymbol{\delta}\|_2 - 1$ rispetto al riferimento libero. Il confronto riguarda però soltanto la direzione:

lo spostamento complessivo comprende anche la variazione di norma, legata anche al fattore della parte di $\boldsymbol{W\delta}$ parallela a $\boldsymbol{z}$.

Per vederlo meglio conviene dividere $\boldsymbol{W\delta}$ nella sua parte parallela e in quella ortogonale a $\boldsymbol{z}$: come detto nella definizione di μ, questo è proprio il modulo della componente di $(\boldsymbol{W\delta})$ parallela a $\boldsymbol{z}$, dunque possiamo riscrivere il vettore come

$$\boldsymbol{W\delta} = \underbrace{\mu\,\boldsymbol{z}}_{(\boldsymbol{W\delta})_{\parallel}} + \underbrace{\boldsymbol{r}}_{(\boldsymbol{W\delta})_{\perp}}, \qquad \boldsymbol{r} = \boldsymbol{W\delta} - \mu\,\boldsymbol{z}.$$

Lo spostamento complessivo comprende infatti anche la parte parallela a $\boldsymbol{z}$, con coefficienti

$$c\left(\mu\,\boldsymbol{\delta}^{\top}\boldsymbol{h} - s_{rel}\|\boldsymbol{h}\|_2^2\right) \quad \text{e} \quad c\left(\mu\,\omega - s_{rel}\|\boldsymbol{h}\|_2^2\right).$$

Il rapporto tra le due variazioni di norma non è quindi controllato dal fattore precedente, di conseguenza nemmeno la norma dello spostamento complessivo lo è.

Con la distanza assoluta questo problema non c'era, ma la variazione di norma riguarda la scala dell'uscita pulita, a cui la layer normalization in ingresso al detector è insensibile. Quello che conta per la geometria del pattern di attivazione è la direzione. Il risultato è quindi ancora valido e dice proprio che l'ancoraggio inclina il riferimento pulito più del riferimento libero.

Per capire quale ruolo svolga il contributo della scala, è utile chiedersi in che misura ciascun gradiente influenzi la scala del livello. La riscalatura della Sezione 5.2.2 corrisponde, nel modello a un livello, alla sostituzione $\boldsymbol{W} \to \gamma\boldsymbol{W}$ con $\gamma > 0$: entrambe le uscite pulite e adversarial risultano moltiplicate per γ, per cui s e ρ risultano moltiplicate per γ^2, mentre s_{rel} resta invariata. Una misura naturale della scala del livello è la norma di Frobenius

$$\|\boldsymbol{W}\|_F^2 = \sum_{j=1}^{D}\sum_{k=1}^{n} W_{jk}^2,$$

ossia la norma euclidea di $\boldsymbol{W}$ considerata come un vettore, che sotto la riscalatura diventa $\gamma\|\boldsymbol{W}\|_F$. Il prodotto scalare associato è $\langle \boldsymbol{A}, \boldsymbol{B}\rangle_F = \sum_{j,k} A_{jk}B_{jk}$, che gode delle stesse proprietà del prodotto scalare tra vettori. In particolare, dopo un passo $\boldsymbol{W}^{+} = \boldsymbol{W} - \eta\,\boldsymbol{G}$ lungo una generica direzione $\boldsymbol{G}$, sviluppando il quadrato con il prodotto scalare si ottiene

$$\|\boldsymbol{W}^{+}\|_F^2 = \langle \boldsymbol{W} - \eta\boldsymbol{G},\ \boldsymbol{W} - \eta\boldsymbol{G}\rangle_F = \|\boldsymbol{W}\|_F^2 - 2\eta\,\langle \boldsymbol{G}, \boldsymbol{W}\rangle_F + \eta^2\|\boldsymbol{G}\|_F^2. \tag{C}$$

Per passi piccoli il termine in η^2 è trascurabile, dunque la variazione della scala è determinata dal segno di $\langle \boldsymbol{G}, \boldsymbol{W} \rangle_F$: se è positivo il passo rimpicciolisce il livello, se è negativo lo ingrandisce, se è nullo lo lascia invariato. Nel caso con riferimento libero, sostituendo $\boldsymbol{G}$ con i gradienti effettivi e usando l'identità $\langle \boldsymbol{a}\boldsymbol{b}^\top, \boldsymbol{W} \rangle_F = \boldsymbol{a}^\top \boldsymbol{W} \boldsymbol{b}$, valida per qualunque coppia di vettori $\boldsymbol{a}$ e $\boldsymbol{b}$, si trova che

$$\left\langle \frac{\partial s}{\partial \boldsymbol{W}}, \boldsymbol{W} \right\rangle_F = \frac{2}{D} (\boldsymbol{W}\boldsymbol{\delta})^\top (\boldsymbol{W}\boldsymbol{\delta}) = \frac{2}{D} \|\boldsymbol{W}\boldsymbol{\delta}\|_2^2 = 2s,$$
$$\left\langle \frac{\partial \rho}{\partial \boldsymbol{W}}, \boldsymbol{W} \right\rangle_F = \frac{2}{D} \boldsymbol{z}^\top (\boldsymbol{W}\boldsymbol{h}) = \frac{2}{D} \|\boldsymbol{z}\|_2^2 = 2\rho,$$

e quindi, per la linearità del prodotto scalare,

$$\left\langle \frac{\partial s_{rel}}{\partial \boldsymbol{W}}, \boldsymbol{W} \right\rangle_F = \frac{1}{\rho} \Big(2s - s_{rel} \cdot 2\rho \Big) = \frac{1}{\rho} \Big(2s - \frac{s}{\rho} 2\rho \Big) = 0.$$

Il calcolo giustifica la scelta, anticipata in apertura, di mantenere sempre attivo il gradiente della scala. Se ρ fosse trattata come costante, il gradiente della distanza relativa si ridurrebbe al solo primo termine, $\frac{1}{\rho} \frac{\partial s}{\partial \boldsymbol{W}}$, e per la linearità del prodotto scalare

$$\left\langle \frac{1}{\rho} \frac{\partial s}{\partial \boldsymbol{W}}, \boldsymbol{W} \right\rangle_F = \frac{1}{\rho} 2s = 2\, s_{rel} > 0 \ :$$

la componente di riscalatura sarebbe la stessa della distanza assoluta, divisa per ρ, e quindi un passo di discesa rimpicciolirebbe ancora il livello.

Nel caso ancorato il contributo della scala è lo stesso, ma cambia quello del numeratore. Con la stessa identità di prima, e poiché $\boldsymbol{W}\boldsymbol{h}' = \boldsymbol{z} + \boldsymbol{W}\boldsymbol{\delta}$,

$$\left\langle \frac{\partial \hat{s}}{\partial \boldsymbol{W}}, \boldsymbol{W} \right\rangle_F = \frac{2}{D} (\boldsymbol{W}\boldsymbol{\delta})^\top (\boldsymbol{W}\boldsymbol{h}') = \frac{2}{D} (\boldsymbol{W}\boldsymbol{\delta})^\top \boldsymbol{z} + \frac{2}{D} \|\boldsymbol{W}\boldsymbol{\delta}\|_2^2 = 2s + \frac{2}{D} (\boldsymbol{W}\boldsymbol{\delta})^\top \boldsymbol{z},$$

e quindi

$$\left\langle \frac{\partial \hat{s}_{rel}}{\partial \boldsymbol{W}}, \boldsymbol{W} \right\rangle_F = \frac{1}{\rho} \Big(2s + \frac{2}{D} (\boldsymbol{W}\boldsymbol{\delta})^\top \boldsymbol{z} - s_{rel} \cdot 2\rho \Big) = \frac{1}{\rho} \Big(2s + \frac{2}{D} (\boldsymbol{W}\boldsymbol{\delta})^\top \boldsymbol{z} - 2s \Big) = \frac{2}{D\rho} (\boldsymbol{W}\boldsymbol{\delta})^\top \boldsymbol{z},$$

che in generale non è nullo, e il cui segno dipende dal prodotto di $\boldsymbol{W}\boldsymbol{\delta}$ e dell'uscita pulita $\boldsymbol{z}$.

Sostituendo nella (C) il gradiente si ottiene l'effetto esatto di un passo di aggiornamento sulla scala del livello. Usando la distanza assoluta, entrambe le strategie di loss comportano un riscalamento del livello:

- Distanza assoluta nel caso attrattivo senza ancoraggio ($\boldsymbol{G} = \partial s/\partial \boldsymbol{W}$, $c = 2\eta/D$)

$$\|G\|_F^2 = \frac{4}{D^2}\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2\|\boldsymbol{\delta}\|_2^2\,,$$

e ricordando che $4\eta s = \frac{4\eta}{D}\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2 = 2c\,\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2$ e che $\frac{4\eta^2}{D^2} = c^2$, si ha

$$\begin{aligned}\|\boldsymbol{W}^+\|_F^2 &= \|\boldsymbol{W}\|_F^2 - 4\eta\, s + \frac{4\eta^2}{D^2}\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2\|\boldsymbol{\delta}\|_2^2 \\ &= \|\boldsymbol{W}\|_F^2 - 2c\,\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2 + c^2\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2\|\boldsymbol{\delta}\|_2^2 \\ &= \|\boldsymbol{W}\|_F^2 - c\,\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2\big(2 - c\|\boldsymbol{\delta}\|_2^2\big).\end{aligned}$$

Sotto la condizione $0 < c\|\boldsymbol{\delta}\|_2^2 < 2$, che è esattamente quella che garantisce la contrazione, la scala diminuisce strettamente: ogni passo che riduce la distanza rimpicciolisce anche il livello.

- Distanza assoluta nel caso repulsivo senza ancoraggio ($\boldsymbol{G} = -\partial s/\partial \boldsymbol{W}$, $c = 2\nu/D$). Con gli stessi passaggi, e con il segno opposto nel termine di primo ordine,

$$\begin{aligned}\|\boldsymbol{W}^+\|_F^2 &= \|\boldsymbol{W}\|_F^2 + 4\nu\, s + \frac{4\nu^2}{D^2}\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2\|\boldsymbol{\delta}\|_2^2 \\ &= \|\boldsymbol{W}\|_F^2 + 2c\,\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2 + c^2\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2\|\boldsymbol{\delta}\|_2^2 \\ &= \|\boldsymbol{W}\|_F^2 + c\,\|\boldsymbol{W}\boldsymbol{\delta}\|_2^2\big(2 + c\|\boldsymbol{\delta}\|_2^2\big) > \|\boldsymbol{W}\|_F^2\ :\end{aligned}$$

ogni passo ingrandisce strettamente il livello.

Usando invece la distanza relativa, l'effetto cambia a seconda dell'ancoraggio:

- Distanza relativa senza ancoraggio. In entrambi i casi attrattivo e repulsivo, con $\boldsymbol{G} = \pm\,\partial s_{rel}/\partial \boldsymbol{W}$. Poiché $\langle \boldsymbol{G}, \boldsymbol{W}\rangle_F = 0$, si ha

$$\|\boldsymbol{W}^+\|_F^2 = \|\boldsymbol{W}\|_F^2 + \eta^2\|\boldsymbol{G}\|_F^2\ :$$

la scala può soltanto crescere, ma di una quantità proporzionale a η^2 e quindi trascurabile per passi piccoli. In particolare, nel caso attrattivo la distanza relativa non induce alcun rimpicciolimento del livello[3].

- Distanza relativa con ancoraggio. In questo caso $\langle \boldsymbol{G}, \boldsymbol{W}\rangle_F = \pm\frac{2}{D\rho}\,(\boldsymbol{W}\boldsymbol{\delta})^\top \boldsymbol{z}$, in generale non nullo: l'ancoraggio del numeratore reintroduce una componente di risca-

[3] Questo riguarda soltanto l'analisi dell'effetto del gradiente di $\mathcal{L}_{act}$, nella pratica l'effetto delle altre loss può far crescere o diminuire la scala, il risultato importante qui è che la crescita ad ogni passo è un $o(\eta)$, quindi la crescita è autolimitata e non produce instabilità significative in fase di addestramento.

latura, il cui segno non è controllato.

La componente di riscalatura è dannosa proprio in una rete profonda. Come visto nella Sezione 5.2.2, il rimpicciolimento di un livello può essere compensato esattamente dal livello successivo senza alcun effetto sulla funzione calcolata dalla rete: la loss del task non vi si oppone, e la spinta individuata, presente a ogni passo, può accumularsi, riducendo la penalità senza modificare né le predizioni né la geometria delle attivazioni. Con la distanza relativa e il riferimento libero questa spinta è assente; lo stesso vale, con il segno opposto, per l'inflazione della scala nella strategia repulsiva.

Conclusione In entrambe le strategie il riferimento viene lasciato libero e la loss opera sulla distanza relativa, e le due scelte vanno lette insieme. Il riferimento libero garantisce, nella strategia attrattiva, che nessun passo sufficientemente piccolo aumenti la distanza e, in quella repulsiva, che la direzione dell'uscita pulita venga trascinata dalla spinta, attraverso i pesi condivisi, molto meno che con l'ancoraggio. La distanza relativa elimina dal gradiente la componente di pura riscalatura, che con la distanza assoluta rimpicciolisce il livello nel caso attrattivo e lo ingrandisce in quello repulsivo, riducendo la loss senza alcun effetto sulla geometria delle attivazioni.

Gradienti in fase di attacco.

Come descritto precedentemente, al momento dell'ingresso nel detector, un'attivazione della backbone può essere staccata (sg[$\boldsymbol{a}^{(l)}$]) o meno ($\boldsymbol{a}^{(l)}$) dal grafo computazionale dei gradienti, a seconda che si voglia influenzare l'attivazione stessa in funzione del detector. L'arresto del gradiente all'ingresso dei detector è dunque uno strumento usato in fase di addestramento per decidere quali loss influenzano i parametri della backbone, e non modifica la funzione calcolata dal modello in fase di inferenza.
Un attaccante white-box adattivo vuole abbassare lo score del detector, quindi gli serve $\nabla_{\boldsymbol{x}} s(\boldsymbol{x})$: lo score però dipende dalla catena

$$\boldsymbol{x} \to \boldsymbol{a}^{(l)}(\boldsymbol{x}) \to \text{detector} \to s,$$

quindi se l'attacco calcolasse il gradiente nel regime isolato, $\boldsymbol{a}^{(l)}(\boldsymbol{x})$ sarebbe trattato come una costante all'ingresso del detector, risulterebbe $\nabla_{\boldsymbol{x}} s(\boldsymbol{x}) = \mathbf{0}$, e l'attacco diventerebbe di

fatto un PGD standard che ignora i detector.
La robustezza misurata sarebbe allora sovrastimata per effetto di un gradiente artificialmente offuscato, esattamente il fenomeno descritto da [4]. Per questo motivo, durante la generazione dell'attacco adattivo, tutti gli arresti del gradiente all'ingresso dei detector vengono rimossi e ripristinati al termine.

5.4 Definizione degli attacchi adversarial

5.4.1 Modello di minaccia

Seguendo la tassonomia richiamata nel Paragrafo 2.3.2, il modello di minaccia adottato è caratterizzato come segue. L'obiettivo dell'attaccante è la violazione dell'integrità del classificatore, ossia l'induzione di una predizione errata. La conoscenza dell'attaccante è completa (white-box): architettura, parametri della backbone e, per i modelli basati sul rilevamento, parametri dei detector e politica di aggregazione degli score. Si tratta dello scenario più sfavorevole per il difensore, in linea con una valutazione rigorosa della robustezza [7].
La capacità dell'attaccante è definita nello spazio delle feature. Sia $\boldsymbol{m} \in \{0,1\}^d$ una maschera che vale 1 in corrispondenza delle feature perturbabili e 0 altrove. L'insieme dei campioni adversarial ammissibili a partire da $\boldsymbol{x}$ è definito come

$$\Delta_\varepsilon(\boldsymbol{x}) = \Big\{ \boldsymbol{x} + \boldsymbol{\delta} \;\; : \;\; \|\boldsymbol{\delta}\|_\infty \le \varepsilon, \;\; (\mathbf{1} - \boldsymbol{m}) \odot \boldsymbol{\delta} = \mathbf{0} \Big\},$$

La maschera esclude dalla perturbazione le feature categoriche, la cui codifica ordinale introduce un ordinamento artificiale che una perturbazione continua sfrutterebbe producendo codici privi di significato, e le feature che rappresentano contatori discreti, come i conteggi dei flag TCP. Il budget ε è espresso nelle unità dello spazio standardizzato: poiché dopo il preprocessing ogni feature perturbabile ha media nulla e varianza unitaria sul training set, ε corrisponde alla stessa frazione di deviazione standard per tutte le feature. Senza questa standardizzazione un budget uniforme non avrebbe significato, dato che feature diverse assumono valori su scale che differiscono di molti ordini di grandezza.
Gli attacchi sono formulati come non mirati, massimizzando la loss di cross-entropy rispetto

all'etichetta vera:

$$\boldsymbol{x}^{adv} \approx \underset{\boldsymbol{x}' \in \Delta_\varepsilon(\boldsymbol{x})}{\arg\max}\ \ell_{CE}\big(f_{\boldsymbol{\theta}}(\boldsymbol{x}'), y\big). \tag{5.8}$$

Per un flusso malevolo, l'attacco non mirato coincide con la ricerca di un falso negativo, che è il caso di interesse operativo; in fase di valutazione, la perturbazione è applicata a tutti i campioni del test set, inclusi quelli benigni, per i quali corrisponde all'induzione di un falso positivo, e le metriche descrivono così il comportamento del modello su entrambe le classi.

5.4.2 Fast Gradient Sign Method

L'attacco FGSM risolve la (5.8) ponendo il campione adversarial come

$$\boldsymbol{x}^{adv} = \boldsymbol{x} + \varepsilon\, \boldsymbol{m} \odot \operatorname{sign}\Big(\nabla_{\boldsymbol{x}} \ell_{CE}\big(f_{\boldsymbol{\theta}}(\boldsymbol{x}), y\big)\Big).$$

Il segno della perturbazione è positivo perché si vuole aumentare la loss del task. FGSM è ottimo per un modello lineare e richiede una sola backward pass, ma in generale risulta più debole delle varianti iterative.

5.4.3 Projected Gradient Descent

L'attacco PGD invece affronta la (5.8) con una salita del gradiente proiettata. Il punto di partenza è estratto uniformemente nell'insieme ammissibile e l'iterazione ripete un passo di tipo FGSM di ampiezza α seguito da una proiezione $\Pi_{\Delta_\varepsilon(\boldsymbol{x})}$ nell'intorno ammissibile:

$$\begin{aligned} \boldsymbol{x}^{(0)} &= \boldsymbol{x} + \boldsymbol{m} \odot \boldsymbol{u}, \qquad \boldsymbol{u} \sim \mathcal{U}\big([-\varepsilon, \varepsilon]^d\big), \\ \boldsymbol{x}^{(t+1)} &= \Pi_{\Delta_\varepsilon(\boldsymbol{x})}\Big(\boldsymbol{x}^{(t)} + \alpha\, \boldsymbol{m} \odot \operatorname{sign}\big(\nabla_{\boldsymbol{x}} \ell_{CE}(f_{\boldsymbol{\theta}}(\boldsymbol{x}^{(t)}), y)\big)\Big), \qquad t = 0, \dots, T-1, \end{aligned}$$

e restituisce $\boldsymbol{x}^{adv} = \boldsymbol{x}^{(T)}$. Il passo lungo il segno del gradiente è la direzione di massima salita rispetto alla norma L_∞, mentre la partenza casuale evita di iniziare l'ottimizzazione in punti in cui il gradiente è poco informativo e diversifica le soluzioni. Il passo è fissato a $\alpha = 2\varepsilon/T$: con un numero di iterazioni T sufficiente, lo spostamento complessivo può raggiungere qualunque punto dell'insieme ammissibile. Come detto nel Paragrafo 2.3.3, PGD è considerato l'attacco al primo ordine di riferimento per la valutazione della robustezza.

5.4.4 PGD adattivo

Contro un modello dotato di detector, PGD rappresenta un attaccante che ignora l'esistenza della difesa: esso cerca di indurre un errore di classificazione senza curarsi di essere rilevato. Contro un sistema con detector, un attacco ha successo se il campione perturbato è classificato in modo errato e al tempo stesso non viene segnalato. Ciò comporta che la loss del task va massimizzata, e lo score di detection minimizzato: questi due problemi di ottimizzazione possono essere combinati definendo un unico obiettivo da massimizzare J,

$$J(\boldsymbol{x}') = \ell_{CE}\big(f_{\boldsymbol{\theta}}(\boldsymbol{x}'), y\big) - \beta\, s(\boldsymbol{x}'), \qquad \beta \geq 0,$$

nel quale la cross-entropy funge da surrogato della classificazione errata e lo score aggregato s, calcolato con la stessa politica usata dal difensore, da surrogato dell'allarme. L'attacco massimizza J con lo stesso schema iterativo di PGD (Algoritmo 3): la salita aumenta la loss del task e contemporaneamente riduce la probabilità stimata di attacco. Il peso di evasione β regola il compromesso tra i due obiettivi: per $\beta = 0$ si ritrova PGD standard, mentre valori crescenti prediligono l'elusione dei detector, eventualmente a discapito dell'efficacia sull'errore di classificazione.

Algoritmo 3 PGD adattivo

Input: campione $(\boldsymbol{x}, y)$, budget ε, passo α, iterazioni T, peso di evasione β, maschera $\boldsymbol{m}$, backbone $f_{\boldsymbol{\theta}}$ e score di detection s
Output: campione adversarial $\boldsymbol{x}^{adv}$
rimuovi gli arresti del gradiente all'ingresso dei detector
$\boldsymbol{x}^{(0)} \leftarrow \boldsymbol{x} + \boldsymbol{m} \odot \boldsymbol{u}, \quad \boldsymbol{u} \sim \mathcal{U}\big([-\varepsilon, \varepsilon]^d\big)$
for $t = 0, \dots, T-1$ **do**
  $\boldsymbol{z} \leftarrow f_{\boldsymbol{\theta}}\big(\boldsymbol{x}^{(t)}\big)$ `// logits`
  $J \leftarrow \ell_{CE}(\boldsymbol{z}, y) - \beta\, s\big(\boldsymbol{x}^{(t)}\big)$
  $\boldsymbol{x}^{(t+1)} \leftarrow \Pi_{\Delta_\varepsilon(\boldsymbol{x})}\Big(\boldsymbol{x}^{(t)} + \alpha\, \boldsymbol{m} \odot \operatorname{sign}\big(\nabla_{\boldsymbol{x}} J\big)\Big)$
end
ripristina gli arresti del gradiente all'ingresso dei detector
return $\boldsymbol{x}^{(T)}$

L'attacco adattivo è definito soltanto per i modelli dotati di detector. Per i modelli basati sulla strategia attrattiva l'unica componente decisionale è il classificatore, e PGD standard costituisce già l'attacco white-box completo contro l'intero sistema.

5.5 Dettagli di addestramento

5.5.1 Funzione di loss complessiva

La loss minimizzata durante l'addestramento combina il termine del task con i termini ausiliari prodotti dai livelli estesi:

$$\mathcal{L}_{TOT}(\mathcal{B}) = \mathcal{L}_{task}(\mathcal{B}) + \lambda_{det}\,\mathcal{L}_{det}(\mathcal{B}) + \lambda_{act}\,\mathcal{L}_{act}(\mathcal{B}),$$

dove $\mathcal{L}_{act}$ coincide con $\mathcal{L}_{CLO}$ o con $\mathcal{L}_{FUR}$ a seconda del tipo di livello, e ciascun termine ausiliario è presente soltanto se la rete contiene livelli che lo producono. Il peso del termine del task è fissato a uno senza perdita di generalità, poiché ciò che conta è il rapporto tra i pesi.

Il termine del task è una cross-entropy pesata per compensare lo sbilanciamento tra le classi dei dataset. Detto N_c il numero di campioni di classe c nel training set, il peso di ciascuna classe è $w_c = N/(2N_c)$, che assegna peso unitario a entrambe le classi quando queste sono bilanciate e amplifica il contributo della classe minoritaria altrimenti. Indicato con C l'insieme degli indici del batch sui quali la loss è calcolata, si ha

$$\mathcal{L}_{task}(\mathcal{B}) = \frac{\sum_{j \in C} w_{\tilde{y}_j}\, \ell_{CE}\big(z(\tilde{x}_j), \tilde{y}_j\big)}{\sum_{j \in C} w_{\tilde{y}_j}}.$$

La scelta di C determina se la backbone viene addestrata a essere robusta. Se C contiene i soli campioni puliti, l'addestramento del task è standard e i campioni adversarial influenzano soltanto i detector e le loss sulle attivazioni. Se C contiene l'intero batch misto – campioni puliti e campioni adversarial – la loss del task è la media tra quella sui campioni puliti e quella sui campioni adversarial, secondo la formulazione originale dell'adversarial training, e la backbone viene spinta a classificare correttamente anche i campioni perturbati. Questa scelta è indipendente dalla presenza dei detector, che apprendono in ogni caso da entrambe le versioni dei campioni. Per i modelli basati sul rilevamento si ottengono così due varianti: nella prima la backbone non è robusta e la difesa è affidata interamente al rifiuto dei campioni segnalati; nella seconda le linee di difesa sono due, la classificazione corretta e il rifiuto.

5.5.2 Configurazioni di modello

La Tabella 5.1 riassume le configurazioni che derivano dalla combinazione delle scelte descritte, e che costituiscono i termini di confronto della fase sperimentale. La configurazione con adversarial training e senza loss sulle attivazioni corrisponde a un modello CloserAL con $\lambda_{act} = 0$, ed è il termine di confronto necessario per stabilire se la loss attrattiva apporti un contributo non ridondante rispetto all'adversarial training.

Configurazione	Detector	Loss sulle attivazioni	$\mathcal{L}_{task}$ su $\boldsymbol{x}^{adv}$
Baseline non difesa	no	nessuna	no
Adversarial training	no	nessuna	sì
CloserAL	no	attrattiva	sì / no
DetectorLayer	sì	nessuna	no / sì
FurtherAL	sì	repulsiva	no / sì

Tabella 5.1. Configurazioni di modello considerate. La colonna relativa alla loss del task indica se questa è calcolata anche sui campioni adversarial: se ci sono due scelte, la prima è quella di default e la seconda è configurabile.

5.6 Metriche di valutazione

Le metriche tradizionali (accuracy, precision, recall etc...) non permettono di confrontare modelli che si difendono in modi diversi: un modello CloserAL si difende classificando correttamente i campioni perturbati, un modello con detector segnalandoli. Per rendere confrontabili le due famiglie, ogni modello viene valutato come un sistema di classificazione del task, con opzionalmente anche la classificazione del detector. Su un campione pulito il sistema ha successo se il campione è classificato correttamente e non viene segnalato, perché un falso allarme su traffico legittimo è a tutti gli effetti un errore; su un campione adversarial il sistema ha successo se il campione è classificato correttamente oppure viene segnalato, perché in quest'ultimo caso l'attacco è stato intercettato anche se il classificatore è stato ingannato. Si definiscono quindi l'accuratezza end-to-end pulita e l'accuratezza end-to-end robusta

$$\mathrm{Acc}^{e2e}_{clean} = \frac{1}{N_T}\sum_{i=1}^{N_T} \mathbb{I}\Big[f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) = y_i \ \wedge \ F_\tau(\boldsymbol{x}_i) = 0\Big],$$

$$\mathrm{Acc}^{e2e}_{rob} = \frac{1}{N_T}\sum_{i=1}^{N_T} \mathbb{I}\Big[f_{\boldsymbol{\theta}}(\boldsymbol{x}_i^{adv}) = y_i \ \vee \ F_\tau(\boldsymbol{x}_i^{adv}) = 1\Big],$$

e il punteggio congiunto

$$\text{Score} = \frac{1}{2}\left(\text{Acc}_{clean}^{e2e} + \text{Acc}_{rob}^{e2e}\right).$$

Per un modello privo di detector la funzione di allarme F_τ è identicamente nulla, e le due metriche si riconducono all'accuratezza standard e all'accuratezza robusta; per un modello con detector esse misurano l'effetto combinato di classificatore e allarme. Il punteggio congiunto è la grandezza su cui vengono ordinati gli esperimenti e condotte la selezione del modello e quella dei margini; a seconda delle priorità operative, la selezione può essere condotta anche sulla sola componente pulita o sulla sola componente robusta.

Capitolo 6

Esperimenti e Analisi dei Risultati

Tutti gli esperimenti sono eseguiti con l'implementazione disponibile pubblicamente [24]: ogni stadio corrisponde a uno script `lwad_stageN.py`, e i risultati di ciascuna run sono raccolti in file di riepilogo per dataset, dai quali sono ricavate tutte le tabelle e le figure del capitolo.
La suite e le run degli esperimenti sono eseguite su una GPU NVIDIA RTX 4000 Ada, con PyTorch 2.5.1 in modalità deterministica (algoritmi deterministici abilitati, `CUBLAS_WORKSPACE_CONFIG=:4096:8` e selezione automatica dei kernel cuDNN disabilitata).

6.1 Domande di Ricerca

La metodologia descritta definisce uno spazio di sperimentazione ampio, i cui fattori principali sono la struttura della backbone (profondità e larghezze), l'insieme $\mathcal{I}$ dei livelli estesi, la struttura dei detector, la politica di aggregazione degli score di detection, il regime del flusso dei gradienti, l'impiego dell'adversarial training, il tipo e il peso della loss sulle attivazioni, e infine il tipo di attacco con i suoi parametri. L'esplorazione di questo spazio è guidata dalle seguenti domande di ricerca.

1. **Posizione dei detector**: in quali livelli la perturbazione adversarial è più facilmente rilevabile, e conviene estendere tutti i livelli oppure soltanto alcuni? Poiché la perturbazione tende ad amplificarsi con la profondità, ci si attende che i detector dei livelli finali siano più accurati; quelli dei livelli iniziali, d'altra parte, sono gli unici che abilitano un'uscita anticipata con un risparmio significativo.
2. **Flusso dei gradienti**: la cooperazione tra detector e backbone migliora il rilevamento rispetto al regime isolato, e a quale costo per l'accuratezza del task? Il profilo di

accuratezza dei detector nel regime isolato fornisce inoltre una misura interpretabile della rilevabilità della perturbazione lungo la rete.

3. **Manipolazione geometrica**: la strategia repulsiva migliora il rilevamento rispetto ai soli detector, e la strategia attrattiva migliora la robustezza rispetto all'adversarial training standard?
4. **Approfondimento della strategia attrattiva**: dal momento che l'allineamento di un singolo livello limita la deviazione di tutti i livelli successivi, quanti livelli CloserAL sono necessari per ottenere l'allineamento desiderato, e conviene che siano consecutivi?
5. **Approfondimento della strategia repulsiva**: la separazione indotta da un livello FurtherAL si conserva nei livelli successivi? E quale beneficio deriva dall'estendere più livelli, eventualmente consecutivi?
6. **Robustezza adattiva**: il vantaggio delle difese basate sul rilevamento resiste a un attaccante che conosce i detector, al variare dei parametri d'attacco?

6.2 Stadio 1: baseline e test di V&V

Lo stadio 1 ha un duplice scopo: fissare i valori di riferimento delle configurazioni di base, rispetto ai quali saranno confrontate le varianti esplorate negli stadi successivi, e produrre i modelli addestrati sui quali la suite di verifica e validazione (V&V) esegue i propri controlli.

6.2.1 Impostazione sperimentale

Lo script `lwad_stage1.py` addestra e valuta 19 configurazioni sul dataset UNSW-NB15, ciascuna con tre seed (42, 43, 44)[1], per un totale di 57 run. Salvo diversa indicazione, ogni valore riportato nel seguito è la media sui tre seed, con la deviazione standard campionaria a pedice.

Le configurazioni, riassunte nella Tabella 6.1, sono organizzate in tre gruppi. Il gruppo `base` comprende le configurazioni di modello introdotte nella metodologia (baseline non

[1] Il numero di seed è stato limitato a tre per due ragioni. La prima è il costo computazionale: ogni run richiede in media tra due e tre minuti (Tabella 6.3), e lo stadio comprende 57 run, a cui si aggiungono i riaddestramenti eseguiti dalla suite di V&V; ogni seed aggiuntivo avrebbe quindi allungato in proporzione l'intera campagna sperimentale. La seconda è che già con tre seed la variabilità delle metriche risulta contenuta: la deviazione standard dello Score di test non supera 0.005 in 13 configurazioni su 19 e non supera 0.011 in nessuna configurazione, a eccezione delle due baseline non difese (0.023), la cui variabilità deriva dall'accuratezza sotto attacco (Tabella 6.2).

difesa, adversarial training, CloserAL, DetectorLayer e FurtherAL), a cui si aggiungono la variante di CloserAL con la loss del task calcolata sui soli campioni puliti e le varianti dei modelli con detector nelle quali la stessa loss è calcolata anche sui campioni adversarial, contraddistinte dal suffisso `-advtrain`. Il gruppo `vv` raccoglie coppie di configurazioni costruite appositamente per essere confrontate tra loro dai test di verifica. Il gruppo `mech` contiene configurazioni con backbone più profonde o con un diverso insieme di livelli estesi, necessarie per validare i meccanismi su cui si fonda la metodologia. Infine, le configurazioni con detector contraddistinte dalla presenza di `exit` nel nome sono quelle che adottano la politica di aggregazione degli score s_{max}.

Salvo diversa indicazione, la backbone ha due livelli nascosti di larghezza 64 e 32, entrambi estesi. I detector hanno due livelli nascosti di larghezza 64 e 32, preceduti da una layer normalization, operano nel regime isolato, aggregano gli score con la politica della media e hanno peso $\lambda_{det} = 1$; le loss sulle attivazioni hanno peso $\lambda_{act} = 1$, e i margini di FurtherAL sono calibrati con $\kappa = 5$ dopo tre epoche di warm-up. L'addestramento dura al più 10 epoche, con arresto anticipato dopo 3 epoche senza miglioramento dello Score di validazione, e conserva il modello con lo Score di validazione migliore. Il validation set interviene quindi soltanto nell'addestramento e nella calibrazione della soglia di allarme: tutte le metriche riportate nel seguito, Score compreso, sono misurate sul test set. Sia in addestramento sia nella valutazione di questo stadio l'attacco è PGD con budget $\varepsilon = 0.2$, $T = 20$ iterazioni e passo $\alpha = 2\varepsilon/T = 0.02$; l'attacco adattivo è impiegato all'interno della suite di V&V (Paragrafo 6.2.3). Infine, i tempi di inferenza sono misurati un campione alla volta su 1024 campioni.

6.2.2 Risultati delle configurazioni

La Tabella 6.2 riporta le metriche di test di tutte le configurazioni, compreso lo Score, calcolato sul test set come media delle due accuratezze end-to-end: è su questa grandezza che le configurazioni sono confrontate nel seguito. Per leggerla è utile ricordare che, per i modelli privi di detector, le metriche end-to-end coincidono con le accuratezze della backbone, mentre per i modelli con detector misurano l'effetto combinato di classificatore e allarme.

Esperimento	Configurazione	Larghezze	Est.	Impiego nella suite
Gruppo base				
`undefended`	Baseline non difesa: CloserAL con $\lambda_{act} = 0$, $\mathcal{L}_{task}$ sui soli campioni puliti	64, 32	(1, 2)	A1, A2, A4
`advtrain`	Adversarial training: CloserAL con $\lambda_{act} = 0$	64, 32	(1, 2)	M3, A1, A2, A4
`closer`	CloserAL con adversarial training	64, 32	1, 2	M3, V3.6, A1, A2, A4
`closer-clean`	CloserAL con $\mathcal{L}_{task}$ sui soli campioni puliti	64, 32	1, 2	A1, A2, A4
`detlayer`	DetectorLayer, $\mathcal{L}_{task}$ sui soli campioni puliti	64, 32	1, 2	V3.1, V3.2, V3.6, M2, A1–A4
`detlayer-advtrain`	DetectorLayer con adversarial training	64, 32	1, 2	A1–A4
`further`	FurtherAL, $\mathcal{L}_{task}$ sui soli campioni puliti	64, 32	1, 2	M1, M2, A1–A4
`further-advtrain`	FurtherAL con adversarial training	64, 32	1, 2	M1, A1–A4
Gruppo vv				
`actloss-zero`	FurtherAL con $\lambda_{act} = 0$, per il resto identico a `detlayer`	64, 32	1, 2	V3.1
`margin-tiny`	FurtherAL con $\kappa = 0.01$, per il resto identico a `detlayer`	64, 32	1, 2	V3.2
`repro-cold/warm`	FurtherAL su un'architettura non usata altrove, con margini calcolati (cache fredda) o riletti dalla cache (cache calda)	64, 16	2	V3.3
`exit-off/on`	FurtherAL con aggregazione s_{max}, senza e con uscita anticipata	64, 32	1, 2	V3.5
Gruppo mech				
`undefended-deep`	Baseline non difesa profonda	256, 128, 64, 32	(1, 2)	M4
`detlayer-deep`	DetectorLayer profondo, un detector per livello	256, 128, 64, 32	1–4	M5
`detlayer-wide3`	DetectorLayer a tre livelli, il solo primo esteso	128, 64, 32	1	M2
`further-wide3`	FurtherAL a tre livelli, il solo primo esteso	128, 64, 32	1	M2
`further-exit-deep`	FurtherAL profondo con aggregazione s_{max} e uscita anticipata	256, 128, 64, 32	1–4	M6

Tabella 6.1. Configurazioni dello stadio 1. Le colonne Larghezze e Est. riportano le larghezze dei livelli nascosti della backbone e i livelli estesi; tra parentesi i livelli CloserAL con $\lambda_{act} = 0$, che non contribuiscono alla loss. L'ultima colonna indica i test della suite di V&V che impiegano i checkpoint della configurazione.

	Backbone		Allarme		End-to-end		
Esperimento	$\mathrm{Acc}_{task}^{clean}$	$\mathrm{Acc}_{task}^{adv}$	TNR	TPR	$\mathrm{Acc}_{clean}^{e2e}$	Acc_{rob}^{e2e}	Score_{test}
Gruppo `base`							
`undefended`	$0.837_{\pm 0.004}$	$0.514_{\pm 0.045}$	–	–	$0.837_{\pm 0.004}$	$0.514_{\pm 0.045}$	$0.676_{\pm 0.023}$
`advtrain`	$0.825_{\pm 0.005}$	$0.811_{\pm 0.003}$	–	–	$0.825_{\pm 0.005}$	$0.811_{\pm 0.003}$	$0.818_{\pm 0.003}$
`closer`	$0.821_{\pm 0.001}$	$0.808_{\pm 0.003}$	–	–	$0.821_{\pm 0.001}$	$0.808_{\pm 0.003}$	$0.814_{\pm 0.002}$
`closer-clean`	$0.836_{\pm 0.002}$	$0.635_{\pm 0.018}$	–	–	$0.836_{\pm 0.002}$	$0.635_{\pm 0.018}$	$0.735_{\pm 0.009}$
`detlayer`	$0.861_{\pm 0.007}$	$0.395_{\pm 0.066}$	$0.995_{\pm 0.002}$	$0.996_{\pm 0.002}$	$0.857_{\pm 0.008}$	$0.999_{\pm 0.001}$	$0.928_{\pm 0.004}$
`detlayer-advtrain`	$0.825_{\pm 0.008}$	$0.816_{\pm 0.008}$	$0.975_{\pm 0.007}$	$0.966_{\pm 0.020}$	$0.807_{\pm 0.011}$	$0.979_{\pm 0.013}$	$0.893_{\pm 0.011}$
`further`	$0.863_{\pm 0.008}$	$0.150_{\pm 0.044}$	$0.995_{\pm 0.002}$	$0.997_{\pm 0.002}$	$0.860_{\pm 0.008}$	$0.999_{\pm 0.000}$	$0.930_{\pm 0.004}$
`further-advtrain`	$0.817_{\pm 0.001}$	$0.807_{\pm 0.001}$	$0.970_{\pm 0.007}$	$0.994_{\pm 0.001}$	$0.790_{\pm 0.007}$	$0.997_{\pm 0.000}$	$0.894_{\pm 0.003}$
Gruppo `vv`							
`actloss-zero`	$0.861_{\pm 0.007}$	$0.395_{\pm 0.066}$	$0.995_{\pm 0.002}$	$0.996_{\pm 0.002}$	$0.857_{\pm 0.008}$	$0.999_{\pm 0.001}$	$0.928_{\pm 0.004}$
`margin-tiny`	$0.861_{\pm 0.007}$	$0.395_{\pm 0.066}$	$0.995_{\pm 0.002}$	$0.996_{\pm 0.002}$	$0.857_{\pm 0.008}$	$0.999_{\pm 0.001}$	$0.928_{\pm 0.004}$
`repro-cold`	$0.871_{\pm 0.004}$	$0.140_{\pm 0.005}$	$0.984_{\pm 0.005}$	$0.987_{\pm 0.001}$	$0.858_{\pm 0.009}$	$0.992_{\pm 0.001}$	$0.925_{\pm 0.004}$
`repro-warm`	$0.871_{\pm 0.004}$	$0.140_{\pm 0.005}$	$0.984_{\pm 0.005}$	$0.987_{\pm 0.001}$	$0.858_{\pm 0.009}$	$0.992_{\pm 0.001}$	$0.925_{\pm 0.004}$
`exit-off`	$0.867_{\pm 0.010}$	$0.135_{\pm 0.062}$	$0.990_{\pm 0.006}$	$0.998_{\pm 0.001}$	$0.861_{\pm 0.014}$	$0.999_{\pm 0.000}$	$0.930_{\pm 0.007}$
`exit-on`	$0.867_{\pm 0.010}$	$0.135_{\pm 0.062}$	$0.990_{\pm 0.006}$	$0.998_{\pm 0.001}$	$0.861_{\pm 0.014}$	$0.999_{\pm 0.000}$	$0.930_{\pm 0.007}$
Gruppo `mech`							
`undefended-deep`	$0.863_{\pm 0.003}$	$0.440_{\pm 0.047}$	–	–	$0.863_{\pm 0.003}$	$0.440_{\pm 0.047}$	$0.651_{\pm 0.023}$
`detlayer-deep`	$0.871_{\pm 0.009}$	$0.302_{\pm 0.006}$	$0.989_{\pm 0.006}$	$0.992_{\pm 0.001}$	$0.861_{\pm 0.004}$	$0.995_{\pm 0.001}$	$0.928_{\pm 0.002}$
`detlayer-wide3`	$0.857_{\pm 0.009}$	$0.355_{\pm 0.007}$	$0.997_{\pm 0.001}$	$0.996_{\pm 0.001}$	$0.854_{\pm 0.009}$	$0.998_{\pm 0.001}$	$0.926_{\pm 0.004}$
`further-wide3`	$0.862_{\pm 0.008}$	$0.177_{\pm 0.041}$	$0.995_{\pm 0.002}$	$0.998_{\pm 0.001}$	$0.859_{\pm 0.009}$	$0.999_{\pm 0.000}$	$0.929_{\pm 0.004}$
`further-exit-deep`	$0.881_{\pm 0.012}$	$0.025_{\pm 0.011}$	$0.994_{\pm 0.005}$	$0.999_{\pm 0.001}$	$0.875_{\pm 0.009}$	$1.000_{\pm 0.000}$	$0.937_{\pm 0.005}$

Tabella 6.2. Metriche di test dello stadio 1 su UNSW-NB15. $\mathrm{Acc}_{task}^{clean}$ e $\mathrm{Acc}_{task}^{adv}$ sono le accuratezze sui campioni puliti e su quelli attaccati con PGD ($\varepsilon = 0.2$); TNR e TPR sono le frazioni di campioni puliti non segnalati e di campioni adversarial segnalati dall'allarme; lo Score è la media di $\mathrm{Acc}_{clean}^{e2e}$ e Acc_{rob}^{e2e}, anch'esso calcolato sul test set.

Modelli senza detector. La baseline non difesa raggiunge un'accuratezza pulita di 0.837, che scende a 0.514 sotto attacco, e ottiene lo Score più basso del gruppo `base` (0.676). L'adversarial training porta l'accuratezza robusta a 0.811 al prezzo di poco più di un punto di accuratezza pulita, e con uno Score di 0.818 è il migliore tra i modelli senza detector. Il modello CloserAL addestrato con adversarial training (`closer`) ottiene 0.821 e 0.808, con uno Score di 0.814, lievemente inferiore a quello dell'adversarial training: in questa configurazione la loss attrattiva non produce un guadagno misurabile sulle metriche di uscita, sebbene agisca effettivamente sulla geometria delle attivazioni, come mostra il test M3. Più significativa è la variante `closer-clean`, nella quale la loss del task è calcolata sui soli campioni puliti: la loss attrattiva, da sola, porta l'accuratezza robusta da 0.514 a 0.635 e lo Score da 0.676 a 0.735, con una perdita di accuratezza pulita di appena 0.1 punti rispetto

alla baseline non difesa.

Modelli con detector. Nei modelli `detlayer` e `further` la backbone non è robusta (accuratezza sotto attacco 0.395 e 0.150), ma l'allarme segnala oltre il 99.5% dei campioni generati con PGD con un tasso di falsi allarmi di circa lo 0.5%, e l'accuratezza end-to-end robusta raggiunge 0.999. Ne risulta uno Score di 0.928 e 0.930 rispettivamente, superiore di circa 0.11 a quello di tutti i modelli senza detector, ma tale divario potrebbe essere dovuto in gran parte alla potenza maggiore della rete in termini di parametri. DetectorLayer e FurtherAL sono indistinguibili in termini di Score, ma FurtherAL riduce l'accuratezza della backbone sotto attacco da 0.395 a 0.150: la loss repulsiva amplifica la deviazione nei livelli estesi e, propagandola verso l'uscita, rende il classificatore più fragile, coerentemente con un obiettivo orientato al rilevamento anziché alla classificazione robusta. Lo stesso andamento si osserva nelle configurazioni `wide3` (0.355 e 0.177) e nel modello profondo `further-exit-deep` (0.025).
Nelle varianti con adversarial training la backbone diventa più robusta (0.816 e 0.807), ma l'allarme produce più falsi positivi (TNR 0.975 e 0.970), che abbassano l'accuratezza end-to-end pulita a 0.807 e 0.790; in `detlayer-advtrain` si riduce anche il tasso di rilevamento (0.966). Lo Score di queste varianti (0.893 e 0.894) resta quindi inferiore a quello delle corrispondenti versioni senza adversarial training: contro PGD le due linee di difesa non si sommano, plausibilmente perché l'adversarial training riduce proprio la deviazione tra rappresentazioni pulite e adversarial su cui si basano i detector, in analogia con il conflitto discusso per la strategia attrattiva.

Configurazioni di verifica. Le configurazioni del gruppo `vv` riproducono le identità attese già a livello di metriche aggregate: `actloss-zero` e `margin-tiny` coincidono con `detlayer` in ogni metrica e in ogni deviazione standard, `repro-cold` coincide con `repro-warm` e `exit-off` con `exit-on`. I test V3.1, V3.2, V3.3 e V3.5 verificano che queste identità valgano del tutto sui checkpoint, e non soltanto sulle medie.

Grandezze operative. La Tabella 6.3 raccoglie le grandezze non legate all'accuratezza. Le soglie calibrate delle configurazioni con aggregazione s_{max} (0.86 in `exit-off/on`, 0.99 in `further-exit-deep`) sono nettamente più alte di quelle delle configurazioni che aggregano

Esperimento	Epoche	τ	t_{inf}^{clean} [ms]	t_{inf}^{adv} [ms]	Uscita anticipata (adv)	Durata [s]
Gruppo base						
undefended	$5_{\pm 0}$	–	$0.194_{\pm 0.035}$	$0.192_{\pm 0.037}$	–	$68_{\pm 4}$
advtrain	$9_{\pm 1}$	–	$0.193_{\pm 0.035}$	$0.192_{\pm 0.037}$	–	$109_{\pm 7}$
closer	$8_{\pm 1}$	–	$0.218_{\pm 0.005}$	$0.218_{\pm 0.005}$	–	$101_{\pm 8}$
closer-clean	$5_{\pm 0}$	–	$0.217_{\pm 0.005}$	$0.219_{\pm 0.005}$	–	$54_{\pm 0}$
detlayer	$9_{\pm 2}$	$0.49_{\pm 0.01}$	$0.654_{\pm 0.010}$	$0.653_{\pm 0.008}$	–	$123_{\pm 29}$
detlayer-advtrain	$9_{\pm 1}$	$0.49_{\pm 0.01}$	$0.647_{\pm 0.008}$	$0.647_{\pm 0.009}$	–	$128_{\pm 16}$
further	$7_{\pm 2}$	$0.49_{\pm 0.03}$	$0.642_{\pm 0.007}$	$0.642_{\pm 0.008}$	–	$95_{\pm 24}$
further-advtrain	$10_{\pm 0}$	$0.47_{\pm 0.03}$	$0.650_{\pm 0.008}$	$0.650_{\pm 0.008}$	–	$130_{\pm 16}$
Gruppo vv						
actloss-zero	$9_{\pm 2}$	$0.49_{\pm 0.01}$	$0.655_{\pm 0.011}$	$0.656_{\pm 0.011}$	–	$122_{\pm 50}$
margin-tiny	$9_{\pm 2}$	$0.49_{\pm 0.01}$	$0.655_{\pm 0.010}$	$0.657_{\pm 0.010}$	–	$132_{\pm 32}$
repro-cold	$10_{\pm 0}$	$0.52_{\pm 0.11}$	$0.505_{\pm 0.007}$	$0.505_{\pm 0.007}$	–	$128_{\pm 19}$
repro-warm	$10_{\pm 0}$	$0.52_{\pm 0.11}$	$0.372_{\pm 0.091}$	$0.371_{\pm 0.093}$	–	$119_{\pm 4}$
exit-off	$9_{\pm 2}$	$0.86_{\pm 0.02}$	$0.579_{\pm 0.129}$	$0.581_{\pm 0.129}$	–	$134_{\pm 34}$
exit-on	$9_{\pm 2}$	$0.86_{\pm 0.02}$	$0.848_{\pm 0.020}$	$0.481_{\pm 0.030}$	$0.997_{\pm 0.002}$	$129_{\pm 40}$
Gruppo mech						
undefended-deep	$5_{\pm 0}$	–	$0.272_{\pm 0.051}$	$0.272_{\pm 0.053}$	–	$85_{\pm 2}$
detlayer-deep	$10_{\pm 0}$	$0.50_{\pm 0.01}$	$0.912_{\pm 0.189}$	$0.912_{\pm 0.188}$	–	$192_{\pm 11}$
detlayer-wide3	$8_{\pm 1}$	$0.46_{\pm 0.23}$	$0.550_{\pm 0.007}$	$0.550_{\pm 0.008}$	–	$105_{\pm 13}$
further-wide3	$8_{\pm 0}$	$0.33_{\pm 0.17}$	$0.556_{\pm 0.007}$	$0.558_{\pm 0.008}$	–	$114_{\pm 17}$
further-exit-deep	$8_{\pm 2}$	$0.99_{\pm 0.00}$	$1.434_{\pm 0.040}$	$0.477_{\pm 0.025}$	$1.000_{\pm 0.001}$	$180_{\pm 47}$

Tabella 6.3. Grandezze operative dello stadio 1: epoche eseguite prima dell'arresto anticipato, soglia di allarme τ calibrata sul validation set, latenza media di inferenza per campione sui campioni puliti e adversarial, frazione di campioni adversarial che escono in anticipo e durata complessiva della run.

con la media (0.33–0.52), in accordo con l'osservazione che la politica del massimo fa crescere i falsi positivi con il numero di detector e richiede quindi una soglia più conservativa. In tutte le run i detector hanno un costo di inferenza rilevante: le configurazioni del gruppo base con detector richiedono 0.64–0.65 ms per campione, circa il triplo delle configurazioni senza detector (0.19–0.22 ms). Con l'uscita anticipata attiva, il 99.7% dei campioni adversarial di exit-on, e praticamente la totalità di quelli di further-exit-deep, esce prima di completare la backbone, mentre i campioni puliti che escono in anticipo, ossia i falsi allarmi, restano dell'ordine dell'1%. All'interno della stessa run il tempo di inferenza sui campioni adversarial è inferiore del 43% a quello sui campioni puliti in exit-on (0.481 contro 0.848 ms) e del 67% in further-exit-deep (0.477 contro 1.434 ms), mentre in exit-off i due tempi coincidono. I tempi di inferenza vanno invece confrontati con cautela tra run diverse: repro-cold e repro-warm producono modelli identici, eppure i loro tempi

medi sono 0.505 e 0.372 ms, probabilmente per effetto del carico della macchina durante la misura. Per questo motivo, il risparmio effettivo dell'uscita anticipata è quantificato con una misura controllata nel test M6.

6.2.3 Suite di verifica e validazione

La suite, contenuta nel modulo `libs.tests`, comprende 26 test, per i quali sono necessari tutti i 57 checkpoint dello stadio 1.
I test si dividono in due categorie. I test di verifica (prefisso V) accertano che l'implementazione sia conforme a quanto specificato dalla metodologia; i test di validazione accertano che quanto implementato raggiunga lo scopo per cui è stato progettato: che gli attacchi (prefisso A) forniscano una stima affidabile della robustezza nel caso peggiore, e che i meccanismi della metodologia (prefisso M) producano gli effetti per cui sono stati introdotti. I primi ammettono in genere criteri puntuali, i secondi criteri empirici con tolleranze.
Tutti i 16 test di verifica sono superati (233 controlli su 233). Dei test di validazione ne sono superati 8 su 10 (173 controlli su 182), discussi in dettaglio nel seguito.

Verifica dell'implementazione

Costruzione del modello e flusso dello stato. I test V1.1 e V1.2 accertano che le combinazioni incoerenti descritte nella metodologia, a partire dall'incompatibilità tra CloserAL e detector, siano rifiutate già in fase di costruzione, e non producano invece modelli silenziosamente diversi da quelli specificati. Il test V1.3 conferma che i livelli estesi non alterano la funzione calcolata dalla backbone e che ogni detector osserva la preattivazione, e non l'attivazione. Il test V1.5 verifica sperimentalmente l'equivalenza su cui si fonda l'uscita anticipata, $\max_l p^{(l)}(\boldsymbol{x}) > \tau \iff \exists\, l : p^{(l)}(\boldsymbol{x}) > \tau$: sia per DetectorLayer sia per FurtherAL, i flag prodotti con e senza uscita anticipata coincidono senza alcuna eccezione.

Attacchi. I test V2.1 e V2.2 accertano che tutti gli attacchi producano campioni ammissibili: la distanza L_∞ massima coincide con il budget ε, le feature non perturbabili restano invariate e quelle perturbabili vengono modificate. Il test V2.3 mostra che con $\beta = 0$ l'attacco adattivo coincide esattamente con PGD, mentre con $\beta = 1$ i campioni prodotti differiscono; il test V2.4 garantisce che la rimozione degli arresti del gradiente, necessaria

ID	Test	Criterio di accettazione	Controlli
Costruzione del modello e flusso dello stato			
V1.1	Vincoli di costruzione	Le combinazioni incoerenti sono rifiutate: CloserAL insieme a livelli con detector, numero di margini diverso dal numero di livelli FurtherAL, uscita anticipata con aggregazione s_{mean}, attacco adattivo e calibrazione della soglia su modelli senza detector; FurtherAL eredita da DetectorLayer e ActivationLoss nell'ordine atteso	14/14
V1.2	Vincoli delle configurazioni	Indici dei livelli estesi, larghezze, modalità di score e di aggregazione e fattore κ fuori dominio sono rifiutati	19/19
V1.3	Trasparenza dei livelli estesi	Su modelli estesi non addestrati i logit coincidono con quelli della sola backbone; ogni detector riceve $\boldsymbol{a}^{(l)}$	13/13
V1.4	Contenuto dello stato di flusso	Loss ausiliarie presenti solo con $\boldsymbol{\eta}$ noto e solo nei livelli che le producono; una detection per livello esteso	24/24
V1.5	Esattezza dell'uscita anticipata	Con s_{max} l'uscita anticipata segnala gli stessi campioni del forward completo e lascia invariati label e score dei campioni che non escono	18/18
V1.6	Calibrazione dei margini	Margini positivi pari a $\kappa\, \bar{s}^{(l)}$; la selezione sceglie il κ con Score di validazione massimo	8/8
Attacchi			
V2.1	Ammissibilità L_∞	$\|\boldsymbol{x}^{adv} - \boldsymbol{x}\|_\infty \leq \varepsilon$ per FGSM, PGD e PGD adattivo, con e senza maschera	20/20
V2.2	Rispetto della maschera	Le feature non perturbabili restano invariate e quelle perturbabili cambiano; con maschera nulla $\boldsymbol{x}^{adv} = \boldsymbol{x}$	30/30
V2.3	Riduzione a PGD	Con $\beta = 0$ l'attacco adattivo riproduce PGD esattamente; con $\beta = 1$ se ne discosta	16/16
V2.4	Arresti del gradiente	Gli arresti all'ingresso dei detector sono rimossi solo durante l'attacco e ripristinati anche in caso di eccezione; pesi e gradienti del modello restano invariati	9/9
Addestramento end-to-end			
V3.1	FurtherAL con $\lambda_{act} = 0$	Metriche, soglia e checkpoint identici esattamente a quelli di DetectorLayer	11/11
V3.2	FurtherAL con $\kappa \ll 1$	Comportamento equivalente a quello di DetectorLayer	1/1
V3.3	Cache dei margini	Due run identiche, con margini calcolati e riletti dalla cache, coincidono esattamente	16/16
V3.4	Seed e ordine di esecuzione	Seed diversi producono risultati diversi; l'ordine delle run nella suite non altera i risultati	9/9
V3.5	Uscita anticipata e metriche	Con e senza uscita anticipata metriche e checkpoint coincidono; cambia soltanto il tasso di uscita	19/19
V3.6	Ricaricamento dei checkpoint	Un modello ricaricato riproduce le metriche con tolleranza 0.005	6/6

Tabella 6.4. Test di verifica e relativo esito, espresso come numero di controlli superati sul totale.

per non sottostimare l'attaccante, non lasci effetti collaterali sul modello.

Addestramento end-to-end. I test del gruppo V3 confrontano run complete. Il test V3.1 conferma che FurtherAL con $\lambda_{act} = 0$ produce checkpoint identici a quelli di DetectorLayer, poiché ciò che li distingue è proprio l'utilizzo in fase di addestramento di $\mathcal{L}_{act} = \mathcal{L}_{FUR}$. Il test V3.2 restituisce un risultato più forte di quanto previsto dalla metodologia, che per $\kappa \ll 1$ si attendeva soltanto un comportamento prossimo a quello di DetectorLayer, poiché se i margini sono stretti la loss $\mathcal{L}_{FUR}$ è sempre nulla: i checkpoint di `margin-tiny` e `detlayer` sono esattamente identici, segno che con un margine così piccolo nessuna coppia scende mai sotto di esso, e la loss repulsiva non si attiva in alcun batch. Il test V3.3 verifica un meccanismo di caching interno alla suite per evitare di ricalcolare i margini quando possibile. Il test V3.4 accerta che il seed determini effettivamente il risultato e che l'ordine delle run nella suite non lo influenzi; per le run che ricaricano un checkpoint le metriche coincidono entro una tolleranza di 0.005, perché la valutazione non reimposta il seed dell'attacco. Il test V3.5 dimostra che l'uscita anticipata modifica soltanto la latenza di inferenza e non le performance.

Validazione degli attacchi

I test A1–A4 traducono in controlli automatici i criteri proposti da [4] e [7] per riconoscere una valutazione della robustezza falsata da gradienti offuscati o da un attacco non convergente. Nel seguito m_{task} indica l'accuratezza della backbone sui campioni attaccati e m_{e2e} l'accuratezza end-to-end robusta Acc_{rob}^{e2e}; per i modelli con detector la metrica di riferimento è m_{e2e}. Le tolleranze sono $\mathrm{TOL}_A = 0.005$ e $\mathrm{TOL}_{conv} = 0.01$. Tutte le misure sono condotte sugli stessi 2000 campioni del test set. I criteri di accettazione e gli esiti dei quattro test sono riassunti nella Tabella 6.5.

Test superati: A1, A2 e A3. Il test A1 è superato in tutti i casi (Tabelle 6.6 e 6.7): PGD è sempre almeno altrettanto efficace di FGSM, e contro i modelli con detector l'attacco adattivo a 20 passi è molto più efficace della sua versione a passo singolo, che lascia m_{e2e} tra 0.96 e 1.00. Anche il test A3 è superato in tutti i casi: con $\beta = 0$ l'attacco adattivo riproduce PGD, coerentemente con il test V2.3; con $\beta > 0$ l'accuratezza end-to-end robusta crolla da valori compresi tra 0.978 e 0.999 a 0.615 per `detlayer`, 0.523 per `further` e circa 0.82 per le

ID	Test	Criterio di accettazione	Controlli
A1	Attacco iterativo e a passo singolo	$m_{task}(\text{PGD}) \leq m_{task}(\text{FGSM}) + \text{TOL}_A$ per tutti i modelli; per i modelli con detector m_{e2e} dell'attacco adattivo a 20 passi non superiore a quello a un passo, a meno di TOL_A	36/36
A2	Monotonia rispetto al budget	Per $\varepsilon \in \{0.05, 0.1, 0.2, 0.3, 0.5, 1\}$, con 200 passi: m_{task} con PGD non crescente per tutti i modelli; m_{e2e} con l'attacco adattivo non crescente per i modelli con detector	36/37
A3	Attacco adattivo e PGD	Con $\beta = 0$ l'attacco adattivo coincide con PGD; per almeno un $\beta > 0$ vale $m_{e2e}(\text{adattivo}) < m_{e2e}(\text{PGD}) - \text{TOL}_A$; il valore che minimizza m_{e2e} definisce β^*	36/36
A4	Budget illimitato e convergenza	Con $\varepsilon = 5$ (1000 passi) $m \leq 0.05$; all'ε di addestramento $m(40 \text{ passi}, 1 \text{ partenza}) - m(200 \text{ passi}, 5 \text{ partenze}) \leq \text{TOL}_{conv}$	41/49

Tabella 6.5. Test di validazione degli attacchi e relativo esito, espresso come numero di controlli superati sul totale.

Esperimento	FGSM	PGD, 20 passi	PGD, 200 × 5	PGD, $\varepsilon = 5$
`undefended`	$0.543_{\pm 0.044}$	$0.516_{\pm 0.049}$	$0.507_{\pm 0.052}$	$0.000_{\pm 0.000}$
`advtrain`	$0.808_{\pm 0.004}$	$0.804_{\pm 0.002}$	$0.801_{\pm 0.003}$	$0.010_{\pm 0.011}$
`closer`	$0.804_{\pm 0.004}$	$0.800_{\pm 0.002}$	$0.797_{\pm 0.002}$	$0.006_{\pm 0.004}$
`closer-clean`	$0.640_{\pm 0.014}$	$0.630_{\pm 0.018}$	$0.621_{\pm 0.018}$	$0.000_{\pm 0.000}$

Tabella 6.6. Modelli senza detector: accuratezza della backbone sotto attacco (m_{task}) al crescere della forza dell'attacco. Salvo l'ultima colonna, $\varepsilon = 0.2$; 200 × 5 indica 200 passi e 5 partenze casuali con $\alpha = 0.01$, e l'attacco con $\varepsilon = 5$ impiega 1000 passi.

			Adattivo, $\varepsilon = 0.2$				Adattivo
Esperimento	β^*	PGD	1 passo	20 passi	40 × 1	200 × 5	$\varepsilon = 5$
`detlayer`	2, 5, 5	$0.999_{\pm 0.001}$	$0.973_{\pm 0.013}$	$0.615_{\pm 0.028}$	$0.589_{\pm 0.035}$	$0.558_{\pm 0.035}$	$0.006_{\pm 0.011}$
`detlayer-advtrain`	0.5, 0.5, 1	$0.978_{\pm 0.012}$	$0.984_{\pm 0.011}$	$0.817_{\pm 0.013}$	$0.817_{\pm 0.012}$	$0.813_{\pm 0.013}$	$0.008_{\pm 0.000}$
`further`	2, 2, 5	$0.999_{\pm 0.001}$	$0.962_{\pm 0.008}$	$0.523_{\pm 0.027}$	$0.423_{\pm 0.044}$	$0.318_{\pm 0.038}$	$0.001_{\pm 0.001}$
`further-advtrain`	50, 50, 50	$0.998_{\pm 0.000}$	$0.999_{\pm 0.001}$	$0.816_{\pm 0.005}$	$0.814_{\pm 0.003}$	$0.806_{\pm 0.001}$	$0.036_{\pm 0.022}$

Tabella 6.7. Modelli con detector: accuratezza end-to-end robusta (m_{e2e}) al crescere della forza dell'attacco. β^* è riportato per ciascuno dei tre seed. Le colonne 1 passo e 20 passi impiegano l'attacco di base; 40 × 1 e 200 × 5 indicano passi e partenze casuali con $\alpha = 0.01$; l'attacco con $\varepsilon = 5$ impiega 1000 passi e tutti i β della griglia.

		ε					
Esperimento	Metrica	0.05	0.1	0.2	0.3	0.5	1
`undefended`	m_{task}, PGD	0.762	0.694	0.509	0.290	0.031	0.008
`advtrain`	m_{task}, PGD	0.813	0.810	0.803	0.787	0.615	0.135
`closer`	m_{task}, PGD	0.813	0.810	0.799	0.782	0.628	0.101
`closer-clean`	m_{task}, PGD	0.787	0.734	0.624	0.459	0.150	0.008
`detlayer`	m_{task}, PGD	0.715	0.617	0.392	0.166	0.009	0.004
	m_{e2e}, adattivo	0.722	0.654	0.577	0.522	0.402	0.275
`detlayer-advtrain`	m_{task}, PGD	0.816	0.815	0.808	0.786	0.573	0.100
	m_{e2e}, adattivo	0.820	0.818	0.816	0.812	0.792	0.582
`further`	m_{task}, PGD	0.671	0.469	0.144	0.016	0.005	0.000
	m_{e2e}, adattivo	0.691	0.600	0.395	0.252	0.129	0.072
`further-advtrain`	m_{task}, PGD	0.807	0.806	0.799	0.743	0.263	0.071
	m_{e2e}, adattivo	0.810	0.808	0.811	0.809	0.796	0.638

Tabella 6.8. Test A2: robustezza al variare del budget ε (media sui tre seed), con $\alpha = 0.01$ e 200 passi. Per i modelli con detector l'attacco adattivo impiega il β^* selezionato a $\varepsilon = 0.2$.

due varianti con adversarial training. Infine, nel test A2 la robustezza è non crescente in ε in tutti i 24 casi con PGD e in 11 dei 12 casi con l'attacco adattivo (Tabella 6.8). L'unica eccezione riguarda `further-advtrain` con seed 44, in cui m_{e2e} passa da 0.8080 a $\varepsilon = 0.1$ a 0.8135 a $\varepsilon = 0.2$ e 0.3: un aumento di 0.0055, appena superiore alla tolleranza TOL_A, pari a 11 campioni su 2000, dunque si considera il test superato.

Test parzialmente superato: A4. Nel test A4, con budget illimitato, l'attacco azzera la robustezza di tutti i modelli in 23 casi su 24. Nel complesso, quindi, l'attacco adattivo è monotono rispetto al budget e, con budget illimitato, elude i detector e inganna il classificatore praticamente su tutti i campioni: non emergono dunque i segni di gradienti offuscati, che in [4] si associano ad una falsa robustezza.

Il controllo di convergenza di A4 fallisce invece in alcuni casi per i modelli con soli detector (Tabella 6.7), mentre è sempre superato dai modelli senza detector, e dalle due varianti con adversarial training. É evidente però che questi fallimenti siano dovuti a ottimi locali e alla natura discreta dell'algoritmo: infatti l'attacco più forte riduce ulteriormente m_{e2e}, segno che i gradienti restano informativi e che l'attacco a una sola partenza si arresta in ottimi locali dovuti al passo finito della salita del gradiente, e non che la difesa li offusca.

Le robustezze adattive dei modelli con detector misurate in questo stadio vanno quindi

considerate limiti superiori, e l'analisi del caso peggiore dello stadio 5 impiegherà un attacco con più passi.

Validazione dei meccanismi

ID	Test	Criterio di accettazione	Controlli
M1	Attività della loss repulsiva	Una parte delle coppie è sempre sotto margine per attivare l'effetto della loss	6/6
M2	FurtherAL amplifica s_{rel}	SLAV_{rel} nei livelli FurtherAL maggiore che in DetectorLayer	3/3
M3	CloserAL riduce s_{rel}	SLAV_{rel} di `closer` minore di `advtrain`	5/5
M4	Amplificazione con la profondità	SLAV_{rel} dell'ultimo livello maggiore di quella del primo	3/3
M5	Profilo dei detector nel regime isolato	AUC almeno 0.75	5/5
M6	Risparmio dell'uscita anticipata	Uscita al primo detector più rapida del forward completo	2/2

Tabella 6.9. Test di validazione dei meccanismi e relativo esito.

La loss repulsiva. Tutti i test di validazione dei meccanismi sono superati (Tabella 6.9). I test M1 e M2 accertano che FurtherAL agisca come previsto. Il test M1 mostra che la loss repulsiva è attiva. Il test M2 verifica che la separazione ottenuta sia reale e non un'inflazione di scala: nei livelli FurtherAL la SLAV_{rel} è 8.1 e 4.6 volte quella di DetectorLayer, mentre la SLAV assoluta cresce soltanto di 2.1 e 1.4 volte. Nelle configurazioni `wide3`, dove è esteso il solo primo livello, il rapporto è 13.3 nel livello esteso e si mantiene pari a 3.3 e 2.4 nei due livelli successivi, non estesi: la separazione indotta da FurtherAL si conserva lungo la rete, pur attenuandosi, e questo costituisce una prima risposta alla domanda di ricerca 5.

La loss attrattiva. Il test M3 confronta `closer` con `advtrain`. In entrambi i livelli CloserAL la SLAV_{rel} diminuisce in modo significativo, di 3.9 e di 29.6 volte, mantenendo all'incirca la stessa accuratezza. La loss attrattiva modifica dunque la geometria delle attivazioni come previsto, anche se, come osservato nel Paragrafo 6.2.2, in questa configurazione ciò non si traduce in un significativo guadagno di robustezza rispetto all'adversarial training.

Effetto della profondità. Il test M4 conferma sperimentalmente l'amplificazione della perturbazione con la profondità (Tabella 6.10): nel modello non difeso a quattro livelli la

	`undefended-deep` (M4)		`detlayer-deep` (M5)
Livello	SLAV_{rel}	Guadagno	AUC del detector
1	$0.056_{\pm 0.001}$	–	$0.9995_{\pm 0.0004}$
2	$0.115_{\pm 0.015}$	$1.42_{\pm 0.09}$	$0.9980_{\pm 0.0006}$
3	$0.188_{\pm 0.015}$	$1.28_{\pm 0.04}$	$0.9897_{\pm 0.0045}$
4	$0.328_{\pm 0.047}$	$1.32_{\pm 0.15}$	$0.9375_{\pm 0.0255}$

Tabella 6.10. Effetto della profondità. A sinistra la SLAV_{rel} del modello non difeso a quattro livelli e il guadagno $\sqrt{\mathrm{SLAV}_{rel}^{(l)}/\mathrm{SLAV}_{rel}^{(l-1)}}$ tra livelli consecutivi; a destra l'AUC dei detector del modello DetectorLayer a quattro livelli nel regime isolato.

SLAV_{rel} cresce di quasi sei volte dal primo all'ultimo livello, e tutti i guadagni misurati sono maggiori di uno. Il test M5 mostra invece che, nel regime isolato, tutti i detector separano con grande efficacia i campioni puliti da quelli adversarial, ben oltre la soglia di accettazione di 0.75, ma la loro AUC diminuisce con la profondità, da 0.9995 nel primo livello a 0.9375 nell'ultimo. Poiché nel regime isolato i detector non modificano la backbone, questo profilo costituisce la misura della rilevabilità della perturbazione lungo la rete prevista dalla domanda di ricerca 2. Il risultato però contraddice l'ipotesi formulata nella domanda di ricerca 1, secondo la quale i detector dei livelli finali sarebbero stati i più accurati: questo confronto suggerisce che la crescita della deviazione relativa lungo la rete non si traduce in una maggiore rilevabilità, probabilmente poiché la deviazione amplificata porta le attivazioni abbastanza distanti da generare confusione lungo diverse frontiere decisionali.

Punto di uscita	Campioni	Completo [ms]	Uscita anticipata [ms]	Sola backbone [ms]	Risparmio
Primo detector	500	2.017	1.193	0.833	43.2%
Quarto detector	3	1.999	3.169	0.812	−61.1%
Nessuna uscita	497	2.029	3.277	0.844	−60.3%

Tabella 6.11. Test M6: tempo di inferenza per campione di `further-exit-deep` su 500 campioni puliti e 500 adversarial, 30 ripetizioni. I tempi sono mediane; il risparmio è la mediana, sui campioni del gruppo, della riduzione relativa rispetto al forward completo con i detector.

Uscita anticipata. Il test M6 misura in condizioni controllate il risparmio dell'uscita anticipata (Tabella 6.11). Tutti i campioni adversarial escono in anticipo, e per i 500 campioni che escono al primo detector il risparmio mediano rispetto al forward completo è del 43.2%. La misura rende evidenti anche due costi. Il primo è il costo dei detector: il

forward completo con quattro detector richiede circa 2.4 volte il tempo della sola backbone. Il secondo è il costo dei controlli intermedi sui campioni che non escono, il cui tempo cresce di circa il 60% rispetto al forward completo. Dai tempi si deduce che l'uscita anticipata riduca la latenza media soltanto se almeno il 60% circa del traffico viene segnalato dal primo detector: in un traffico in cui i campioni adversarial sono rari essa rappresenta quindi un costo netto, e il suo impiego va valutato in funzione dello scenario operativo e di un'implementazione più ottimizzata dei controlli intermedi.

6.2.4 Sintesi dello stadio 1

L'esito della suite consente di trarre quattro conclusioni. La prima riguarda l'implementazione: tutti i test di verifica sono superati, inclusi i confronti tra run, per cui le differenze osservate d'ora in avanti tra le configurazioni sono attribuibili alla metodologia e non a effetti collaterali del codice. La seconda riguarda i meccanismi: la loss repulsiva è attiva e produce una separazione reale e persistente tra rappresentazioni pulite e adversarial, la loss attrattiva riduce la deviazione relativa senza soluzioni degeneri, la perturbazione si amplifica con la profondità e il detector più accurato nel regime isolato è quello del primo livello. La terza riguarda le metriche di riferimento: contro PGD i modelli con detector addestrati senza adversarial training raggiungono sul test set Score compresi tra 0.925 e 0.937, contro 0.818 dell'adversarial training e 0.814 di CloserAL, mentre l'aggiunta dell'adversarial training ai modelli con detector abbassa lo Score a circa 0.89.
La quarta conclusione riguarda la valutazione adattiva: le robustezze adattive misurate in questo stadio sono limiti superiori. Il confronto definitivo tra le due famiglie è quindi rinviato all'analisi del caso peggiore dello stadio 5.

6.3 Stadio 2: esplorazione delle architetture

Lo stadio 1 ha fissato i valori di riferimento su un'unica architettura, una backbone a due livelli nascosti entrambi estesi, e su un unico dataset. Lo stadio 2 esplora invece lo spazio delle architetture, con un duplice scopo. Il primo è conoscitivo: misurare, su tutti e quattro i dataset, l'effetto della profondità e delle larghezze della backbone, della posizione dei livelli estesi e della dimensione dei detector, fornendo una prima risposta alle domande di ricerca 1, 3, 4 e 5. Il secondo è operativo: selezionare, per ogni tipo di modello e per

ogni attacco, tre architetture di dimensione crescente, denominate mini, medium e high, sulle quali sono condotti gli stadi successivi. Affinché i confronti di quegli stadi siano significativi, le architetture di uno stesso livello devono essere confrontabili tra loro in termini di numero di parametri; il Paragrafo 6.3.2 precisa i dettagli.

6.3.1 Impostazione sperimentale

Lo script `lwad_stage2.py` addestra e valuta 121 configurazioni su ciascuno dei quattro dataset, UNSW-NB15, CIC-IDS2017, CSE-CIC-IDS2018 e CTU-13, indicati nelle tabelle con le abbreviazioni UNSW, CIC17, CSE18 e CTU13, per un totale di 484 run, tutte con un unico seed[2]. Le configurazioni appartengono a cinque famiglie, ottenute combinando il tipo di modello con l'attacco: DetectorLayer e FurtherAL sono addestrati e valutati sia con PGD sia con PGD adattivo, mentre CloserAL è addestrato e valutato con il solo PGD, che per un modello privo di detector costituisce già l'attacco white-box completo.
A eccezione dell'architettura, l'impostazione generale degli esperimenti coincide con quella dello stadio 1.

Larghezze della backbone	Detector	Livelli estesi	Config.
DetectorLayer e FurtherAL, con PGD			
(32, 16), (64, 32), (128, 64)	(32, 16), (64, 32)	{1}, {2}, {1, 2}	18
(128, 64, 16), (256, 128, 32)	(64, 32)	{1}, {3}, {1, 3}, {1, 2, 3}	8
DetectorLayer e FurtherAL, con PGD adattivo			
(48, 16), (80, 32), (160, 64)	(32, 16), (64, 32)	{1}, {2}, {1, 2}	18
(192, 64, 16), (256, 128, 32)	(64, 32)	{1}, {3}, {1, 3}, {1, 2, 3}	8
CloserAL, con PGD			
(48, 16), (80, 32), (160, 64)	–	{1}, {2}, {1, 2}	9
(192, 64, 16), (256, 128, 32)	–	{1}, {3}, {1, 3}, {1, 2, 3}	8

Tabella 6.12. Architetture esplorate dello stadio 2. Ogni riga è un prodotto cartesiano dei valori elencati; le 26 configurazioni di ciascuno dei due blocchi dei modelli con detector sono ripetute per DetectorLayer e FurtherAL, per un totale di $4 \times 26 + 17 = 121$ configurazioni per dataset. I livelli sono numerati a partire da 1 come nello stadio 1.

La griglia, riassunta nella Tabella 6.12, è composta da tre blocchi: le architetture con detector valutate con PGD, quelle con detector valutate con PGD adattivo e quelle senza detector. Le backbone di CloserAL sono più larghe di quelle dei modelli con detector

[2]La scelta di un solo seed è dettata dal costo della campagna: le 484 run hanno richiesto complessivamente circa 20 ore di computazione, e l'impiego di tre seed, come nello stadio 1, l'avrebbe triplicato. Si è preferito esplorare una griglia ampia con un seed piuttosto che una griglia ristretta con più seed; la variabilità tra seed misurata nello stadio 1 è usata come riferimento per stabilire quali differenze siano significative.

con PGD, poiché non dispongono dei parametri dei detector: le loro larghezze sono state scelte in modo che il numero di parametri copra lo stesso intervallo coperto dai modelli con detector, e le architetture delle due famiglie possano essere accoppiate a parità di parametri a fini comparativi. I modelli con detector valutati con PGD adattivo adottano invece le stesse backbone di CloserAL, come richiesto dal secondo vincolo di confrontabilità (Paragrafo 6.3.2).

Numero di parametri. Indicate con $d_0 = d$ la dimensione dell'ingresso, con $d_1, \dots, d_{L-1}$ le larghezze dei livelli nascosti e con $d_L = 2$ il numero di logit, il numero di parametri di un modello è

$$P = 2d + \sum_{l=1}^{L} (d_{l-1} + 1)\, d_l + \sum_{l \text{ est.}} P_{det}(d_l), \qquad P_{det}(d_l) = 2d_l + \sum_{j=1}^{J+1} (k_{j-1} + 1)\, k_j, \tag{6.1}$$

dove $k_0 = d_l, k_1, \dots, k_J$ sono le larghezze dei livelli nascosti del detector e $k_{J+1} = 1$; i termini $2d$ e $2d_l$ sono i parametri $\boldsymbol{\gamma}$ e $\boldsymbol{\beta}$ delle layer normalization in ingresso alla backbone e ai detector. Nel seguito i valori numerici di P sono riportati per UNSW-NB15 ($d = 42$).

6.3.2 Criterio di selezione

Metrica. Per ogni configurazione si dispone dello Score di test su ciascuno dei quattro dataset. La grandezza su cui sono ordinate e confrontate le configurazioni è la sua media sui dataset,

$$\bar{S} = \frac{1}{4} \sum_{k=1}^{4} \text{Score}_{test}^{(k)},$$

che pesa allo stesso modo i quattro dataset. Poiché un valore elevato della media può dipendere da un solo dataset, nelle scelte si osservano anche i valori sui singoli dataset.

Tolleranza. Nello stadio 1 la deviazione standard dello Score di test tra i seed non ha superato 0.005 in 13 configurazioni su 19. Con un solo seed, due configurazioni con PGD i cui $\bar{S}$ differiscono per meno di $\delta = 0.005$ sono considerate equivalenti, e la scelta tra esse ricade su quella con meno parametri o che meglio rispetta i vincoli seguenti. Contro l'attacco adattivo la variabilità tra seed è sensibilmente maggiore (nello stadio 1 la deviazione standard dell'accuratezza end-to-end robusta arriva a 0.044), per cui nelle famiglie adattive sono considerate soltanto le differenze ampie oppure coerenti tra i dataset.

Vincoli di confrontabilità. Oltre alle prestazioni, la selezione tiene conto di tre criteri di confrontabilità

1. Parametri simili a parità di attacco. Modelli diversi valutati con lo stesso attacco devono avere un numero di parametri dello stesso ordine di grandezza: CloserAL, DetectorLayer e FurtherAL con PGD; DetectorLayer e FurtherAL con PGD adattivo. Tra configurazioni con prestazioni equivalenti si preferisce quella il cui numero di parametri è più vicino a quello delle altre famiglie dello stesso livello.
2. Backbone comune tra attacchi di potenza diversa. I modelli con detector valutati con PGD adattivo adottano la stessa backbone di CloserAL, e hanno quindi più parametri per effetto della presenza dei detector. Il confronto tra un modello con detector attaccato con PGD adattivo e CloserAL attaccato con PGD – ciascuno contro l'attacco white-box completo per la propria difesa – misura così il contributo dei parametri dedicati al rilevamento a parità di classificatore.
3. Stessa architettura per DetectorLayer e FurtherAL. Quando le prestazioni lo consentono, i due modelli con detector condividono l'architettura a parità di livello e di attacco, così che la loro differenza si riduca alla sola loss repulsiva: FurtherAL con $\lambda_{act} = 0$ coincide infatti con DetectorLayer (test V3.1).

Classi delle architetture. Le architetture selezionate sono pensate per avere tre taglie, mini, medium e high. L'architettura mini è la più piccola con prestazioni vicine alle migliori della famiglia; la medium deve migliorare la mini in modo apprezzabile; la high impiega una backbone molto grande della griglia, così da permettere negli stadi successivi esperimenti su architetture di dimensione relativamente grande. Mini e medium hanno due livelli nascosti e la high ne ha tre, per cui i livelli differiscono anche in profondità.

6.3.3 Effetto dei fattori architetturali

Posizione dei livelli estesi. È il fattore di gran lunga più influente, e fornisce una prima risposta alla domanda di ricerca 1. Con PGD le prestazioni dipendono dalla presenza di un detector sul primo livello: estendere soltanto l'ultimo livello costa fino a 0.07 di $\bar{S}$ (DetectorLayer a tre livelli), con la perdita più marcata su CTU-13, dove supera 0.11. Il risultato è coerente con il test M5 dello stadio 1, nel quale il detector del primo livello era

	PGD		PGD adattivo	
Fattore	DetectorLayer	FurtherAL	DetectorLayer	FurtherAL
Backbone a due livelli: larghezze				
(32, 16) / (48, 16)	0.9364	0.9440	0.8686	0.9121
(64, 32) / (80, 32)	0.9393	0.9474	0.8614	0.9180
(128, 64) / (160, 64)	0.9462	0.9498	0.8314	0.9259
Backbone a due livelli: larghezze dei detector				
(32, 16)	0.9389	0.9461	0.8537	0.9180
(64, 32)	0.9423	0.9480	0.8539	0.9194
Backbone a due livelli: livelli estesi				
{1}	0.9460	0.9489	0.8352	0.9203
{2}	0.9302	0.9443	0.8625	0.9105
{1, 2}	0.9457	0.9481	0.8637	0.9254
Backbone a tre livelli: larghezze				
(128, 64, 16) / (192, 64, 16)	0.9320	0.9466	0.8460	0.9304
(256, 128, 32)	0.9345	0.9495	0.8361	0.9259
Backbone a tre livelli: livelli estesi				
{1}	0.9499	0.9508	0.8491	0.9381
{3}	0.8846	0.9367	0.8067	0.9159
{1, 3}	0.9501	0.9525	0.8611	0.9317
{1, 2, 3}	0.9483	0.9521	0.8473	0.9270

Tabella 6.13. Effetti dei fattori architetturali sui modelli con detector: $\bar{S}$ medio sulle configurazioni che condividono il valore indicato del fattore. Nelle backbone a tre livelli i detector hanno sempre dimensione (64, 32).

il più accurato nel regime isolato. Aggiungere detector oltre il primo, invece, non porta benefici: in tutte le 16 combinazioni della griglia la differenza tra il solo primo livello e l'estensione completa resta entro circa 0.005, cioè entro la tolleranza.

Contro l'attacco adattivo il quadro dipende dal modello. Per FurtherAL, nelle backbone a due livelli, estendere entrambi i livelli è in media la scelta migliore, con un vantaggio di 0.005 sul solo primo livello, e di 0.015 sul solo secondo, coerente su tutti e quattro i dataset. Una spiegazione plausibile è che la loss repulsiva complichi la creazione di un'unica perturbazione in grado di eludere, contemporaneamente, detector che osservano rappresentazioni a profondità diverse, ciascuno specializzato sulla geometria del proprio livello. Nelle backbone a tre livelli, invece, a FurtherAL basta un solo detector sul primo livello, che è in media la scelta migliore: è una prima indicazione, per la domanda di ricerca 5, che la separazione indotta da un livello FurtherAL si conservi nei livelli successivi, in accordo con quanto osservato nel test M2 dello stadio 1.

Larghezza e profondità. Con PGD $\bar{S}$ cresce con la larghezza della backbone per entrambi i modelli DetectorLayer e FurtherAL, e lo stesso vale per FurtherAL contro l'attacco adattivo; DetectorLayer adattivo fa eccezione. La dimensione dei detector ha un effetto minore e non costante. Un terzo livello nascosto non porta miglioramenti apprezzabili, probabilmente perché l'amplificazione della perturbazione in profondità aggiunge difficoltà ai modelli. Le configurazioni migliori, tranne che per DetectorLayer adattivo, impiegano comunque la backbone più larga.

6.3.4 Risultati per famiglia

Larghezze	Est.	P	UNSW	CIC17	CSE18	CTU13	$\bar{S}$
(48, 16)	{1}	2 966	0.8181	0.9833	0.8744	0.7245	**0.8501**
(48, 16)	{2}	2 966	0.8133	0.9820	0.8751	0.7243	0.8487
(48, 16)	{1, 2}	2 966	0.8145	0.9810	0.8721	0.7208	0.8471
(80, 32)	{1}	6 182	0.8139	0.9854	0.8757	0.7294	0.8511
(80, 32)	{2}	6 182	0.8139	0.9843	0.8780	0.7258	0.8505
(80, 32)	{1, 2}	6 182	0.8142	0.9838	0.8760	0.7235	0.8494
(160, 64)	{1}	17 398	0.8138	0.9876	0.8783	0.7379	0.8544
(160, 64)	{2}	17 398	0.8139	0.9878	0.8810	0.7384	0.8553
(160, 64)	{1, 2}	17 398	0.8248	0.9877	0.8778	0.7343	**0.8562**
(192, 64, 16)	{1}	21 766	0.8141	0.9879	0.8820	0.7389	0.8557
(192, 64, 16)	{3}	21 766	0.8136	0.9877	0.8842	0.7386	0.8560
(192, 64, 16)	{1, 3}	21 766	0.8142	0.9873	0.8807	0.7212	0.8509
(192, 64, 16)	{1, 2, 3}	21 766	0.8105	0.9868	0.8815	0.7223	0.8503
(256, 128, 32)	{1}	48 182	0.8153	0.9888	0.8784	0.7413	**0.8560**
(256, 128, 32)	{3}	48 182	0.8103	0.9888	0.8787	0.7374	0.8538
(256, 128, 32)	{1, 3}	48 182	0.8106	0.9879	0.8837	0.7349	0.8543
(256, 128, 32)	{1, 2, 3}	48 182	0.8101	0.9881	0.8835	0.7365	0.8546

Tabella 6.14. CloserAL con PGD: Score di test per dataset e media $\bar{S}$ delle 17 configurazioni. P è il numero di parametri per UNSW-NB15 ($d = 42$). In grassetto le architetture selezionate.

CloserAL. Come mostra la Tabella 6.14, le prestazioni di CloserAL dipendono pochissimo dall'architettura: $\bar{S}$ resta compreso tra 0.847 e 0.856 in tutte le 17 configurazioni. A ogni larghezza estendere il solo primo livello è la scelta migliore o equivalente alla migliore, mentre aggiungere livelli CloserAL non porta vantaggi apprezzabili e, a eccezione della backbone (160, 64), riduce leggermente $\bar{S}$. Per la domanda di ricerca 4, in questa impostazione allineare il primo livello è quindi sufficiente, e vincolare anche i livelli più profondi non limita in modo misurabile la riamplificazione della deviazione residua. Una

causa plausibile è che il peso complessivo della loss attrattiva cresca con il numero di livelli CloserAL, sbilanciando il compromesso con la loss del task; l'aspetto è ripreso nello stadio 4, che esplora λ_{act}. CTU-13 è infine il dataset critico per CloserAL: lo Score di test resta tra 0.72 e 0.74 e l'accuratezza robusta tra 0.67 e 0.69, mentre l'accuratezza end-to-end robusta dei modelli con un detector sul primo livello supera 0.98 contro PGD. Anche su UNSW-NB15 lo Score è basso, tra 0.810 e 0.825, in linea con quello del modello `closer` dello stadio 1 (0.814).

				PGD	
Larghezze	Detector	Est.	P	DetectorLayer	FurtherAL
(32, 16)	(32, 16)	{1}	3 687	**0.9434**	**0.9448**
(32, 16)	(32, 16)	{2}	3 143	0.9184	0.9355
(32, 16)	(32, 16)	{1, 2}	4 808	0.9399	0.9459
(32, 16)	(64, 32)	{1}	6 311	0.9432	0.9475
(32, 16)	(64, 32)	{2}	5 255	0.9301	0.9440
(32, 16)	(64, 32)	{1, 2}	9 544	0.9437	0.9463
(64, 32)	(32, 16)	{1}	7 735	0.9460	0.9464
(64, 32)	(32, 16)	{2}	6 647	0.9225	0.9444
(64, 32)	(32, 16)	{1, 2}	9 400	0.9449	0.9483
(64, 32)	(64, 32)	{1}	11 383	0.9461	0.9495
(64, 32)	(64, 32)	{2}	9 271	0.9303	0.9476
(64, 32)	(64, 32)	{1, 2}	15 672	0.9462	0.9483
(128, 64)	(32, 16)	{1}	18 903	**0.9482**	**0.9540**
(128, 64)	(32, 16)	{2}	16 727	0.9381	0.9472
(128, 64)	(32, 16)	{1, 2}	21 656	0.9492	0.9489
(128, 64)	(64, 32)	{1}	24 599	0.9492	0.9511
(128, 64)	(64, 32)	{2}	20 375	0.9418	0.9470
(128, 64)	(64, 32)	{1, 2}	31 000	0.9507	0.9507
(128, 64, 16)	(64, 32)	{1}	25 543	0.9474	0.9491
(128, 64, 16)	(64, 32)	{3}	18 151	0.8858	0.9337
(128, 64, 16)	(64, 32)	{1, 3}	28 776	0.9482	0.9518
(128, 64, 16)	(64, 32)	{1, 2, 3}	35 177	0.9468	0.9519
(256, 128, 32)	(64, 32)	{1}	67 255	0.9524	0.9526
(256, 128, 32)	(64, 32)	{3}	52 471	0.8835	0.9398
(256, 128, 32)	(64, 32)	{1, 3}	71 544	**0.9521**	**0.9532**
(256, 128, 32)	(64, 32)	{1, 2, 3}	82 169	0.9498	0.9523

Tabella 6.15. Modelli con detector con PGD: $\bar{S}$ di ciascuna configurazione. P è il numero di parametri per UNSW-NB15 ($d = 42$) ed è lo stesso per DetectorLayer e FurtherAL. In grassetto le architetture selezionate.

DetectorLayer e FurtherAL con PGD. Il paesaggio delle prestazioni è saturo (Tabella 6.15): tutte le configurazioni che estendono il primo livello hanno $\bar{S}$ compreso tra 0.940

e 0.954. FurtherAL supera DetectorLayer in 24 configurazioni su 26, e nelle altre due i modelli si equivalgono. Nelle configurazioni che estendono il primo livello il vantaggio è piccolo, cioè entro o al limite della tolleranza, e precisa il risultato dello stadio 1, dove su UNSW-NB15 i due modelli risultavano indistinguibili (0.928 e 0.930); nelle configurazioni prive di detector sul primo livello il vantaggio sale fino a 0.056, coerentemente con la minore sensibilità di FurtherAL alla posizione dei detector. Pur trattandosi in gran parte di differenze piccole, la coerenza del segno su quasi tutta la griglia costituisce una prima indicazione, per la domanda di ricerca 3, che la loss repulsiva migliori sistematicamente il rilevamento.

				PGD adattivo	
Larghezze	Detector	Est.	P	DetectorLayer	FurtherAL
(48, 16)	(32, 16)	{1}	5 175	0.8788	0.9020
(48, 16)	(32, 16)	{2}	4 087	0.8035	0.9213
(48, 16)	(32, 16)	{1, 2}	6 296	0.8997	0.9059
(48, 16)	(64, 32)	{1}	8 311	0.8777	0.9119
(48, 16)	(64, 32)	{2}	6 199	0.8519	0.9120
(48, 16)	(64, 32)	{1, 2}	11 544	**0.9003**	**0.9198**
(80, 32)	(32, 16)	{1}	9 479	0.7976	0.9142
(80, 32)	(32, 16)	{2}	7 847	0.8844	0.9124
(80, 32)	(32, 16)	{1, 2}	11 144	0.8888	0.9300
(80, 32)	(64, 32)	{1}	13 639	0.8494	0.9226
(80, 32)	(64, 32)	{2}	10 471	0.8472	0.8975
(80, 32)	(64, 32)	{1, 2}	17 928	0.9011	0.9316
(160, 64)	(32, 16)	{1}	23 415	0.8241	0.9333
(160, 64)	(32, 16)	{2}	20 151	0.8930	0.9140
(160, 64)	(32, 16)	{1, 2}	26 168	**0.8138**	**0.9293**
(160, 64)	(64, 32)	{1}	30 135	0.7838	0.9376
(160, 64)	(64, 32)	{2}	23 799	0.8952	0.9058
(160, 64)	(64, 32)	{1, 2}	36 536	0.7788	0.9357
(192, 64, 16)	(64, 32)	{1}	36 615	0.8195	0.9347
(192, 64, 16)	(64, 32)	{3}	24 999	0.8304	0.9194
(192, 64, 16)	(64, 32)	{1, 3}	39 848	0.8983	0.9404
(192, 64, 16)	(64, 32)	{1, 2, 3}	46 249	0.8357	0.9272
(256, 128, 32)	(64, 32)	{1}	67 255	**0.8787**	**0.9415**
(256, 128, 32)	(64, 32)	{3}	52 471	0.7830	0.9123
(256, 128, 32)	(64, 32)	{1, 3}	71 544	0.8238	0.9231
(256, 128, 32)	(64, 32)	{1, 2, 3}	82 169	0.8588	0.9269

Tabella 6.16. Modelli con detector con PGD adattivo: $\bar{S}$ di ciascuna configurazione. Le backbone coincidono con quelle di CloserAL; P è il numero di parametri per UNSW-NB15 ($d = 42$) ed è lo stesso per DetectorLayer e FurtherAL. In grassetto le architetture selezionate.

		Backbone		Allarme		End-to-end		
Attacco	Modello	$\mathrm{Acc}_{task}^{clean}$	$\mathrm{Acc}_{task}^{adv}$	TNR	TPR	$\mathrm{Acc}_{clean}^{e2e}$	Acc_{rob}^{e2e}	Score_{test}
PGD	CloserAL	0.870	0.835	–	–	0.870	0.835	0.853
	DetectorLayer	0.900	0.333	0.985	0.984	0.887	0.989	0.938
	FurtherAL	0.902	0.078	0.996	0.995	0.899	0.996	0.947
PGD adattivo	DetectorLayer	0.900	0.683	0.839	0.532	0.753	0.947	0.850
	FurtherAL	0.902	0.232	0.953	0.889	0.861	0.983	0.922

Tabella 6.17. Metriche di test medie di ciascuna famiglia, su tutte le configurazioni e dataset.

DetectorLayer e FurtherAL con PGD adattivo. Contro l'attacco adattivo (Tabella 6.16) la dispersione delle prestazioni è molto più ampia e il divario tra i due modelli si allarga: FurtherAL supera DetectorLayer in tutte le 26 configurazioni, in media di 0.072. La Tabella 6.17 ne chiarisce l'origine. In DetectorLayer il rilevamento cede, con poco più della metà dei campioni adversarial segnalati, e i detector addestrati sui campioni adattivi producono inoltre più falsi allarmi, che abbassano l'accuratezza end-to-end pulita a 0.753. FurtherAL conserva invece in larga parte sia la specificità sia la sensibilità dell'allarme (TNR 0.953 e TPR 0.889).

Il risultato va letto con cautela: l'attacco adattivo impiega un peso di evasione fisso ($\beta = 1$), 20 passi e una sola partenza, mentre lo stadio 1 ha mostrato, nei test A3 e A4, che il caso peggiore richiede di ottimizzare β e di aumentare i passi, e che in quelle condizioni FurtherAL perdeva più di DetectorLayer (con 200 passi e cinque partenze l'accuratezza end-to-end robusta scendeva a 0.318 per `further` contro 0.558 per `detlayer`): la risposta alla domanda di ricerca 6 è quindi rinviata all'analisi del caso peggiore dello stadio 5.

6.3.5 Architetture selezionate

La Figura 6.1 riassume le 121 configurazioni nel piano tra numero di parametri e $\bar{S}$, mentre le Tabelle 6.18 e 6.19 riportano le architetture selezionate, il loro numero di parametri e le loro metriche. Le scelte sono motivate di seguito.

CloserAL. Le backbone di CloserAL definiscono i tre livelli.

- La mini è la (48, 16) con il solo primo livello esteso, la più piccola della griglia e la migliore delle tre varianti a quella larghezza;
- La medium è la (160, 64) con entrambi i livelli estesi, che è anche la configurazione

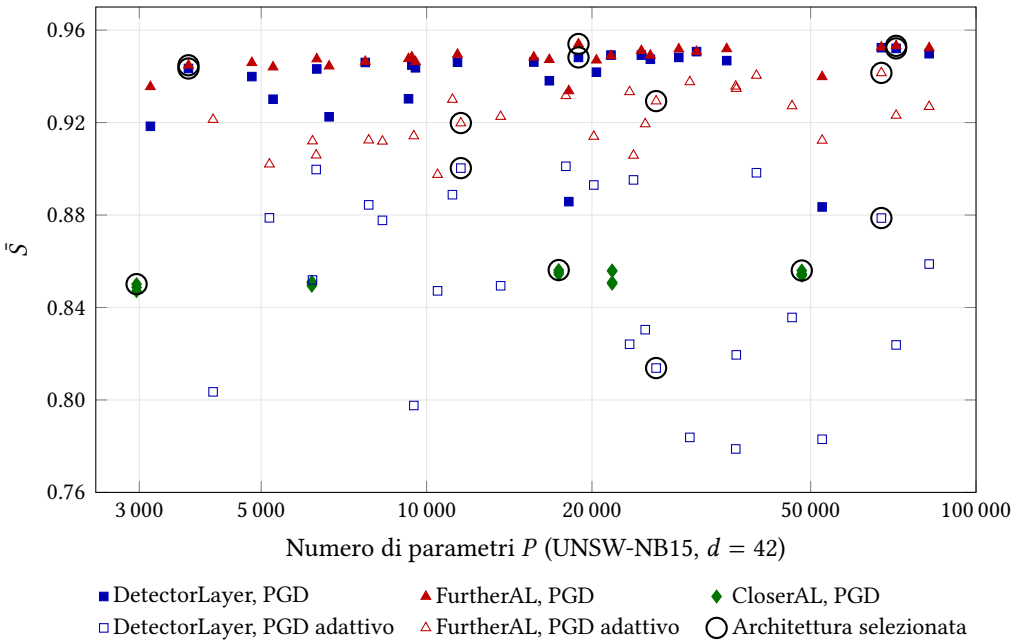


Figura 6.1. Score di test medio $\bar{S}$ in funzione del numero di parametri, per tutte le 121 configurazioni dello stadio 2.

con $\bar{S}$ più alto della famiglia e migliora la mini di 0.006 in media e su tutti e quattro i dataset. Con il solo primo livello esteso, invece, la stessa backbone migliora la mini di 0.004. Le due varianti sono equivalenti tra loro e hanno lo stesso ordine di grandezza nel numero di parametri, ma l'estensione completa offre un ulteriore vantaggio: consente allo stadio 4, che esplora λ_{act}, di verificare se la leggera penalizzazione osservata aggiungendo livelli CloserAL dipenda dal peso complessivo della loss attrattiva (Paragrafo 6.3.4).

- La high è la (256, 128, 32) con il solo primo livello esteso, equivalente alla migliore della famiglia.

DetectorLayer e FurtherAL con PGD. I due modelli condividono l'architettura a tutti i livelli.

- La mini, con backbone (32, 16) e detector (32, 16) sul primo livello, è la più piccola configurazione che estende il primo livello: detector più grandi non la migliorano in modo significativo.
- La medium ha backbone (128, 64) e detector (32, 16) sul primo livello: la configurazione migliora mini su tutti e quattro i dataset, sia per DetectorLayer che FurtherAL;

detector più grandi o più numerosi sulla stessa backbone non portano vantaggi apprezzabili e allontanerebbero il numero di parametri da quello della medium di CloserAL.

- La high ha backbone (256, 128, 32), detector (64, 32) e livelli estesi $\{1, 3\}$: su questa backbone è la configurazione migliore per FurtherAL ed è equivalente alla migliore per DetectorLayer. Tra le due varianti, equivalenti per entrambi i modelli, la regola dei parametri favorirebbe il solo primo livello; si è preferita la coppia $\{1, 3\}$, al costo di circa il 6% di parametri in più, perché è l'unica architettura con PGD dotata di più di un detector: senza di essa, contro PGD, né il confronto delle politiche di aggregazione dello stadio 3 né l'impiego di più livelli FurtherAL (domanda di ricerca 5) avrebbero un soggetto sperimentale.

Quanto ai parametri, ai livelli mini e medium i modelli con detector sono molto vicini a CloserAL. Lo scarto è maggiore alla high, poiché il solo detector del primo livello, che osserva un livello largo 256, conta circa 19 000 parametri. In entrambi i casi lo squilibrio difficilmente è decisivo, perché nelle zone interessate le prestazioni di CloserAL crescono molto lentamente in funzione dei parametri.

DetectorLayer e FurtherAL con PGD adattivo. Per il secondo criterio le backbone sono quelle di CloserAL, e per il terzo i due modelli condividono l'architettura a tutti i livelli. Per le backbone (48, 16) e (160, 64) la dimensione dei detector e i livelli estesi sono scelti in base ai risultati della Tabella 6.16 e alle metriche della Tabella 6.13.
Si preferisce l'estensione completa al solo primo livello: per FurtherAL è in media la scelta migliore, mentre per DetectorLayer migliora il solo primo livello di 0.029, su ciascuno dei quattro dataset, ed è equivalente al solo secondo.

- La mini ha detector (64, 32) su entrambi i livelli della backbone (48, 16), la configurazione migliore o equivalente alla migliore su questa backbone per entrambi i modelli: rispetto ai detector (32, 16) le prestazioni di DetectorLayer si equivalgono, mentre FurtherAL migliora su ciascuno dei quattro dataset.
- La medium ha detector (32, 16) su entrambi i livelli della backbone (160, 64). Su questa backbone DetectorLayer raggiunge il $\bar{S}$ più alto con il solo secondo livello esteso, ma segnala meno di un terzo dei campioni adversarial, mentre con un detector sul primo

livello produce molti falsi allarmi. Poiché nessuna configurazione della backbone (160, 64) migliora la mini di DetectorLayer, si mantiene l'estensione completa, equivalente al solo primo livello per entrambi i modelli, con la quale la medium migliora la mini di FurtherAL. Quanto alla dimensione dei detector, i (32, 16) sono migliori di 0.035 per DetectorLayer, mentre per FurtherAL le due dimensioni si equivalgono, e i detector più piccoli riducono il numero di parametri.

- La high ha detector (64, 32) sul solo primo livello della backbone (256, 128, 32), per entrambi i modelli.

Livello	Larghezze	Detector	Est.	$P(d)$	P
Attacco PGD: CloserAL					
mini	(48, 16)	–	$\{1\}$	$50d + 866$	2 966
medium	(160, 64)	–	$\{1, 2\}$	$162d + 10\,594$	17 398
high	(256, 128, 32)	–	$\{1\}$	$258d + 37\,346$	48 182
Attacco PGD: DetectorLayer e FurtherAL					
mini	(32, 16)	(32, 16)	$\{1\}$	$34d + 2\,259$	3 687
medium	(128, 64)	(32, 16)	$\{1\}$	$130d + 13\,443$	18 903
high	(256, 128, 32)	(64, 32)	$\{1, 3\}$	$258d + 60\,708$	71 544
Attacco PGD adattivo: DetectorLayer e FurtherAL					
mini	(48, 16)	(64, 32)	$\{1, 2\}$	$50d + 9\,444$	11 544
medium	(160, 64)	(32, 16)	$\{1, 2\}$	$162d + 19\,364$	26 168
high	(256, 128, 32)	(64, 32)	$\{1\}$	$258d + 56\,419$	67 255

Tabella 6.18. Architetture selezionate. $P(d)$ è il numero di parametri in funzione della dimensione dell'ingresso secondo la (6.1) e P il suo valore per UNSW-NB15 ($d = 42$). Per i modelli con detector l'architettura è la stessa per DetectorLayer e FurtherAL, e le backbone dei modelli adattivi coincidono con quelle di CloserAL allo stesso livello.

6.3.6 Sintesi dello stadio 2

L'esplorazione consente di trarre tre conclusioni. La prima riguarda la posizione dei detector (domande di ricerca 1 e 5): il primo livello è indispensabile, e contro PGD un solo detector in quella posizione è sufficiente; contro l'attacco adattivo estendere più livelli è vantaggioso per FurtherAL nelle backbone a due livelli, mentre in quelle a tre livelli gli basta il primo livello, segno che la separazione che induce si propaga lungo la rete, come già suggerito dal test M2 dello stadio 1. La seconda riguarda CloserAL (domanda di ricerca 4): le prestazioni dipendono poco dall'architettura, e un solo livello CloserAL sul primo livello nascosto è la scelta migliore o equivalente alla migliore. La terza riguarda il confronto tra i modelli con detector (domanda di ricerca 3): FurtherAL supera o eguaglia DetectorLayer

	Score$_{test}$				Media sui dataset		
Livello	UNSW	CIC17	CSE18	CTU13	$\bar{S}$	Acc$^{e2e}_{clean}$	Acc$^{e2e}_{rob}$
CloserAL, PGD							
mini	0.8181	0.9833	0.8744	0.7245	0.8501	0.867	0.833
medium	0.8248	0.9877	0.8778	0.7343	0.8562	0.874	0.838
high	0.8153	0.9888	0.8784	0.7413	0.8560	0.872	0.840
DetectorLayer, PGD							
mini	0.9297	0.9943	0.9430	0.9064	0.9434	0.889	0.997
medium	0.9340	0.9962	0.9447	0.9177	0.9482	0.899	0.998
high	0.9399	0.9969	0.9443	0.9273	0.9521	0.907	0.998
FurtherAL, PGD							
mini	0.9235	0.9963	0.9489	0.9105	0.9448	0.891	0.998
medium	0.9433	0.9969	0.9545	0.9214	0.9540	0.910	0.998
high	0.9380	0.9971	0.9481	0.9297	0.9532	0.908	0.998
DetectorLayer, PGD adattivo							
mini	0.9047	0.9795	0.8552	0.8617	0.9003	0.833	0.967
medium	0.6622	0.8845	0.8413	0.8673	0.8138	0.645	0.982
high	0.8507	0.9112	0.8498	0.9032	0.8787	0.796	0.961
FurtherAL, PGD adattivo							
mini	0.8794	0.9823	0.9437	0.8737	0.9198	0.851	0.988
medium	0.9079	0.9770	0.9321	0.9001	0.9293	0.867	0.992
high	0.9086	0.9960	0.9489	0.9123	0.9415	0.889	0.994

Tabella 6.19. Metriche di test delle architetture selezionate: Score per dataset e, mediati sui quattro dataset, $\bar{S}$ e accuratezze end-to-end.

in tutte le configurazioni, di poco contro PGD e nettamente contro l'attacco adattivo; se il vantaggio resiste a un attaccante più forte (domanda di ricerca 6) è verificato nello stadio 5.

6.4 Stadio 3: flusso dei gradienti e modalità di detection

Lo stadio 2 ha fissato le architetture con il meccanismo di detection di default: detector nel regime isolato e aggregazione tramite la media. Lo stadio 3 le mantiene invariate e ne esplora il meccanismo di detection, cioè il regime del flusso dei gradienti dai detector alla backbone, isolato o cooperativo, e la politica di aggregazione degli score, s_{mean} oppure s_{max}, così da rispondere alla domanda di ricerca 2, e fissare per ogni famiglia e taglia il meccanismo con cui le architetture entrano nello stadio 4.

6.4.1 Impostazione sperimentale

Lo script `lwad_stage3.py` addestra ciascuna architettura della Tabella 6.18 con tutte le combinazioni dei due fattori. Il massimo è sempre accompagnato dall'uscita anticipata,

che per il test V3.5 ne modifica soltanto la latenza, e con un solo detector le due politiche coincidono: si hanno quindi due configurazioni per le architetture con un detector e quattro per quelle con due, per un totale di 36. CloserAL, privo di detector, è soltanto riaddestrato per disporre di valori di riferimento su più seed. Le 39 configurazioni sono addestrate sui quattro dataset con i seed 42, 43 e 44, per un totale di 468 run; il resto dell'impostazione coincide con lo stadio 2.

Nel seguito $\bar{S}$ è la media sui dataset e sui seed dello Score di test. Con PGD la sua deviazione standard tra i seed non supera 0.003, e la tolleranza $\delta = 0.005$ resta adeguata; con l'attacco adattivo arriva a 0.087, e si considerano solo le differenze ampie rispetto alla dispersione o coerenti tra i dataset.

		UNSW		CIC17		CSE18		CTU13		$\bar{S}$	
Taglia	Aggr.	isol.	coop.	isol.	coop.	isol.	coop.	isol.	coop.	isolato	cooperativo
DetectorLayer, PGD											
mini	media	0.927	0.930	0.994	0.996	0.946	0.949	0.905	0.904	$\mathbf{0.9431}_{\pm 0.001}$	$0.9446_{\pm 0.000}$
medium	media	0.930	0.937	0.996	0.997	0.947	0.947	0.920	0.919	$\mathbf{0.9485}_{\pm 0.002}$	$0.9500_{\pm 0.002}$
high	media	0.933	0.930	0.997	0.997	0.947	0.948	0.923	0.927	$\mathbf{0.9500}_{\pm 0.002}$	$0.9503_{\pm 0.003}$
high	max	0.937	0.930	0.996	0.997	0.948	0.948	0.919	0.927	$0.9501_{\pm 0.002}$	$0.9504_{\pm 0.003}$
DetectorLayer, PGD adattivo											
mini	media	0.792	0.803	0.862	0.914	0.829	0.937	0.859	0.836	$0.8353_{\pm 0.087}$	$\mathbf{0.8723}_{\pm 0.038}$
mini	max	0.866	0.833	0.977	0.979	0.890	0.935	0.861	0.848	$0.8987_{\pm 0.004}$	$0.8989_{\pm 0.003}$
medium	media	0.814	0.829	0.919	0.984	0.845	0.941	0.879	0.882	$0.8644_{\pm 0.045}$	$\mathbf{0.9089}_{\pm 0.009}$
medium	max	0.773	0.787	0.972	0.985	0.890	0.939	0.879	0.869	$0.8786_{\pm 0.019}$	$0.8952_{\pm 0.025}$
high	media	0.822	0.827	0.930	0.984	0.877	0.935	0.899	0.898	$0.8819_{\pm 0.004}$	$\mathbf{0.9112}_{\pm 0.017}$
FurtherAL, PGD											
mini	media	0.928	0.933	0.996	0.997	0.950	0.947	0.912	0.906	$\mathbf{0.9464}_{\pm 0.002}$	$0.9457_{\pm 0.001}$
medium	media	0.936	0.932	0.997	0.997	0.949	0.947	0.922	0.917	$\mathbf{0.9509}_{\pm 0.003}$	$0.9482_{\pm 0.002}$
high	media	0.931	0.934	0.997	0.997	0.948	0.949	0.929	0.928	$\mathbf{0.9510}_{\pm 0.002}$	$0.9522_{\pm 0.002}$
high	max	0.933	0.932	0.997	0.997	0.951	0.949	0.929	0.928	$0.9523_{\pm 0.003}$	$0.9516_{\pm 0.003}$
FurtherAL, PGD adattivo											
mini	media	0.887	0.873	0.982	0.990	0.945	0.942	0.879	0.862	$\mathbf{0.9233}_{\pm 0.003}$	$0.9165_{\pm 0.005}$
mini	max	0.883	0.860	0.984	0.991	0.940	0.942	0.880	0.859	$0.9218_{\pm 0.003}$	$0.9127_{\pm 0.011}$
medium	media	0.884	0.908	0.982	0.994	0.932	0.946	0.903	0.886	$\mathbf{0.9255}_{\pm 0.006}$	$0.9334_{\pm 0.002}$
medium	max	0.892	0.904	0.994	0.993	0.931	0.940	0.906	0.885	$0.9304_{\pm 0.007}$	$0.9305_{\pm 0.004}$
high	media	0.909	0.884	0.996	0.994	0.944	0.946	0.911	0.908	$\mathbf{0.9401}_{\pm 0.001}$	$0.9327_{\pm 0.008}$

Tabella 6.20. Score di test dello stadio 3 nei due regimi del flusso dei gradienti, isolato e cooperativo, per taglia e politica di aggregazione (Aggr.): media sui tre seed per ciascun dataset e $\bar{S}$, con la deviazione standard delle medie per seed. In grassetto le configurazioni selezionate.

6.4.2 Regime del flusso dei gradienti

DetectorLayer con PGD. Contro PGD il regime è ininfluente (Tabella 6.20 e Figura 6.2): il cooperativo è avanti di non più di 0.0015, entro la tolleranza, perché già nel regime isolato

il rilevamento è saturo, con un TPR di 0.996 (Tabella 6.21). Lascia però una traccia sulle rappresentazioni: la SLAV_{rel} del primo livello triplica e l'accuratezza della backbone sotto attacco si dimezza, da 0.325 a 0.168. È lo stesso fenomeno osservato in FurtherAL nello stadio 1: libera di raggiungere la backbone, la loss di detection allontana le attivazioni adversarial da quelle pulite, come una loss repulsiva implicita ma priva del margine.

DetectorLayer con PGD adattivo. Qui il quadro si ribalta: il regime cooperativo migliora $\bar{S}$ di 0.029–0.045 con la media e di 0.017 con il massimo per la medium, mentre per la mini con il massimo i due regimi si equivalgono (high ha solo un detector quindi le due politiche si equivalgono). Nel regime isolato i detector vengono in buona parte elusi, con un TPR medio di 0.652, mentre in quello cooperativo l'allarme segnala il 92% dei campioni adversarial e produce meno falsi allarmi: per riprendere l'analogia della metodologia, la telecamera che può disporre l'arredamento rende visibili anche gli intrusi che cercano di nascondersi, e la SLAV_{rel} del primo livello cresce infatti di un ordine di grandezza. Fa eccezione CTU-13, dove i detector isolati resistono già all'attacco (TPR tra 0.81 e 0.91) e i regimi si equivalgono o prevale quello isolato, probabilmente perché le poche feature perturbabili lasciano all'attaccante poco spazio per ingannare classificatore e detector insieme.

Il costo per il task. Con il regime cooperativo, l'accuratezza pulita della backbone non cambia con PGD e cala leggermente con l'attacco adattivo, mentre quella end-to-end pulita sale da 0.749 a 0.804 grazie ai minori falsi allarmi. Il prezzo vero è la backbone sotto attacco adattivo, che scende da 0.690 a 0.375: la difesa passa dal classificatore all'allarme, e contro un attaccante capace di eludere anche i detector cooperativi la seconda linea sarebbe più debole.

FurtherAL. Per FurtherAL il regime è invece ininfluente con entrambi gli attacchi: con PGD le differenze restano entro 0.003, benché la cooperazione quadruplichi la SLAV_{rel} del primo livello, mentre questo non succede nel caso adattivo. Ad ogni modo, le metriche confermano ulteriormente l'ottima gestione dell'attacco white-box completo fino a questo punto degli esperimenti. Già nel regime isolato, infatti, i detector di FurtherAL resistono all'attacco adattivo, con un TPR medio di 0.930, e la cooperazione aggiunge pochi punti di

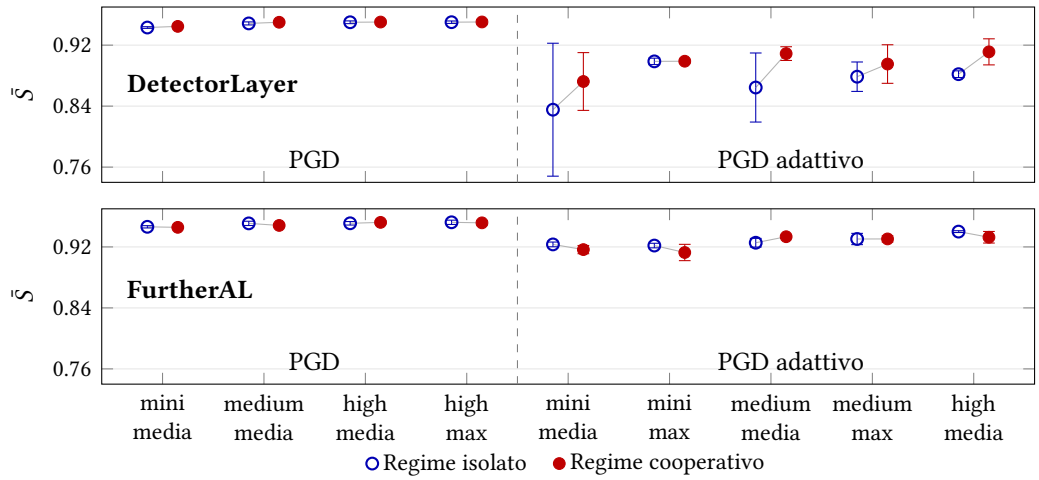


Figura 6.2. $\bar{S}$ di ciascuna configurazione nei due regimi del flusso dei gradienti, per taglia e politica di aggregazione. Le barre indicano la deviazione standard delle medie per seed.

		Backbone		Allarme			End-to-end		
Modello	Regime	$\text{Acc}_{task}^{clean}$	Acc_{task}^{adv}	TNR	TPR	$\text{SLAV}_{rel}^{(1)}$	Acc_{clean}^{e2e}	Acc_{rob}^{e2e}	Score_{test}
PGD									
DetectorLayer	isolato	0.900	0.325	0.997	0.996	0.056	0.897	0.997	0.947
	coop.	0.900	0.168	0.998	0.997	0.182	0.898	0.998	0.948
FurtherAL	isolato	0.903	0.082	0.998	0.997	1.089	0.901	0.998	0.949
	coop.	0.901	0.055	0.998	0.997	4.323	0.899	0.998	0.949
PGD adattivo									
DetectorLayer	isolato	0.902	0.690	0.835	0.652	0.026	0.749	0.972	0.861
	coop.	0.893	0.375	0.903	0.918	0.253	0.804	0.991	0.897
FurtherAL	isolato	0.905	0.244	0.958	0.930	1.033	0.867	0.992	0.930
	coop.	0.896	0.198	0.962	0.961	0.717	0.863	0.992	0.928

Tabella 6.21. Metriche di test medie nei due regimi, per le configurazioni con aggregazione tramite la media, su tutte le taglie e i dataset. $\text{SLAV}_{rel}^{(1)}$ è la vulnerabilità relativa del primo livello, esteso in tutte le architetture.

TPR senza aumentare la separazione. La separazione esplicita della loss repulsiva rende ridondante quella implicita della cooperazione, coerentemente con la lettura della strategia repulsiva come alternativa controllata al regime cooperativo.

Il confronto tra i modelli precisa la domanda di ricerca 3. Con PGD il piccolo vantaggio di FurtherAL rispetto a DetectorLayer nel regime isolato (0.9495 contro 0.9472) si dimezza quando DetectorLayer coopera (0.9483). Con l'attacco adattivo FurtherAL resta invece nettamente avanti: anche nel regime isolato supera DetectorLayer cooperativo a ogni taglia (0.930 contro 0.897 in media), segno che contro un attaccante che conosce i detector la separazione vincolata dal margine è più efficace di quella, priva di vincoli, indotta dalla

loss di detection.

6.4.3 Politica di aggregazione degli score

Con PGD. L'aggregazione non entra nell'addestramento, ma soltanto nella calibrazione della soglia dei detector. Sulla high media e massimo si equivalgono. L'uscita anticipata intercetta il 99.7% dei campioni adversarial, ma non ne riduce la latenza: su questi il tempo resta circa quello della media, mentre sui campioni puliti cresce del 50% circa per il costo dei controlli intermedi, già misurato nel test M6.

Con PGD adattivo. Qui lo score aggregato entra nell'obiettivo dell'attaccante, e quindi nell'addestramento. Per DetectorLayer nel regime isolato la politica del massimo è sempre avanti, mentre nel regime cooperativo il massimo resta avanti per la mini (0.027) e passa indietro per la medium (0.014), mentre per FurtherAL le due politiche si equivalgono entro 0.005. L'uscita anticipata non riduce la latenza nemmeno qui, e l'aggregazione resta un fattore di secondo ordine rispetto al regime.

6.4.4 Configurazioni selezionate

Per avere un vincitore per ciascun modello e attacco, il meccanismo è scelto uniformemente sulle taglie, confrontando $\bar{S}$ su ognuna; tra combinazioni equivalenti si preferisce, nell'ordine, quella in cui l'allarme intercetta davvero l'attacco, la media, i cui campioni puliti non sono rallentati dai controlli intermedi del massimo, e il regime isolato, che conserva una backbone più robusta.
Con PGD, e per FurtherAL anche con l'attacco adattivo, le quattro combinazioni si equivalgono, e si sceglie la media con il regime isolato. Per DetectorLayer contro l'attacco adattivo il regime cooperativo supera nettamente quello isolato, mentre con il massimo supera quello con la media soltanto di 0.004 in media sulle taglie: si sceglie quindi il cooperativo con la media. Il vincitore è quindi la media con il regime isolato, con la sola eccezione di DetectorLayer contro l'attacco adattivo, che richiede la cooperazione (Tabella 6.22).

6.4.5 Sintesi dello stadio 3

Lo stadio consente di trarre tre conclusioni. La prima risponde alla domanda di ricerca 2: la cooperazione tra detector e backbone agisce come una loss repulsiva implicita e priva

	mini		medium		high	
Modello	Mecc.	$\bar{S}$	Mecc.	$\bar{S}$	Mecc.	$\bar{S}$
PGD						
DetectorLayer	media, isolato	0.9431	media, isolato	0.9485	media, isolato	0.9500
FurtherAL	media, isolato	0.9464	media, isolato	0.9509	media, isolato	0.9510
CloserAL	–	0.8490	–	0.8547	–	0.8561
PGD adattivo						
DetectorLayer	media, coop.	0.8723	media, coop.	0.9089	media, coop.	0.9112
FurtherAL	media, isolato	0.9233	media, isolato	0.9255	media, isolato	0.9401

Tabella 6.22. Meccanismi di detection selezionati, come politica di aggregazione e regime del flusso dei gradienti, e relativo $\bar{S}$; per CloserAL non si applica nessuno dei due fattori.

di margine, utile soltanto dove manca una separazione esplicita e l'attaccante conosce i detector. Per DetectorLayer è ininfluente contro PGD e decisiva contro l'attacco adattivo, dove porta il TPR da 0.65 a 0.92 e $\bar{S}$ da 0.861 a 0.897, a un costo trascurabile sull'accuratezza pulita ma con una backbone sotto attacco più fragile; per FurtherAL è ridondante con entrambi gli attacchi. La seconda riguarda la domanda di ricerca 3: contro PGD il piccolo vantaggio di FurtherAL si dimezza quando DetectorLayer coopera, mentre contro l'attacco adattivo FurtherAL resta avanti di 0.032 in media anche senza cooperazione. La terza riguarda l'aggregazione, di secondo ordine, e l'uscita anticipata, che in architetture così poco profonde non riduce mai la latenza: la media è quindi adottata ovunque. Resta aperta la domanda di ricerca 6: contro l'attaccante white-box completo per la propria difesa, DetectorLayer (0.872–0.911) e FurtherAL (0.923–0.940) restano avanti a CloserAL (0.849–0.856), ma con un peso di evasione fisso, da superare nello stadio 5.

6.5 Stadio 4: ottimizzazione della loss complessiva

Gli stadi 2 e 3 hanno fissato architetture e meccanismi di detection, lasciando ai valori di default i pesi della loss complessiva. Lo stadio 4 li esplora, insieme al calcolo della loss del task anche sui campioni adversarial, per misurare il contributo alla robustezza di ciascun meccanismo (domande di ricerca dalla 2 alla 5) e fissare la configurazione con cui ogni modello entra nello stadio 5.

6.5.1 Impostazione sperimentale

Lo script `lwad_stage4.py` esplora, per ogni configurazione della Tabella 6.22, i pesi delle loss che esse impiegano (Tabella 6.23). Nei modelli con detector la loss del task è calcolata

sui soli campioni puliti oppure anche su quelli adversarial; la scelta è incrociata con $\lambda_{det} \in \{0.5, 1, 2\}$ per DetectorLayer con l'attacco adattivo, l'unico nel regime cooperativo, e con $\lambda_{act} \in \{0.5, 1, 2\}$ per FurtherAL, il cui fattore $\kappa \in \{3, 5, 8\}$ è esplorato con i soli $\lambda_{act} \in \{1, 2\}$. CloserAL esplora $\lambda_{act} \in \{0.5, 1, 2\}$ ed è affiancato dall'adversarial training ($\lambda_{act} = 0$) a fini comparativi. Negli altri modelli, che operano nel regime isolato, il gradiente della loss di detection non raggiunge la backbone e λ_{det} riscala soltanto quello dei detector: per questo λ_{det} non è esplorato.

Le 96 configurazioni per dataset della Tabella 6.23 sono addestrate con il solo seed 42, come nello stadio 2, per 384 run. $\langle \bar{S} \rangle$ è la media di $\bar{S}$ sulle tre taglie; la tolleranza resta $\delta = 0.005$ con PGD, mentre con l'attacco adattivo contano solo le differenze ampie o coerenti tra taglie e dataset.

		PGD				PGD adattivo			
x^{adv}	Pesi	mini	medium	high	$\langle \bar{S} \rangle$	mini	medium	high	$\langle \bar{S} \rangle$
DetectorLayer									
no	$\lambda_{det} = 0.5$	–	–	–	–	0.9020	0.8876	0.9254	0.9050
no	$\lambda_{det} = 1$	**0.9434**	**0.9481**	**0.9521**	**0.9479**	**0.9078**	**0.9113**	**0.9306**	**0.9166**
no	$\lambda_{det} = 2$	–	–	–	–	0.9082	0.8957	0.9160	0.9066
sì	$\lambda_{det} = 0.5$	–	–	–	–	0.8078	0.8717	0.8543	0.8446
sì	$\lambda_{det} = 1$	0.9105	0.9244	0.9266	0.9205	*0.8779*	*0.8912*	*0.8617*	*0.8769*
sì	$\lambda_{det} = 2$	–	–	–	–	0.8923	0.8933	0.8357	0.8738
FurtherAL									
no	$\kappa = 3, \lambda_{act} = 1$	0.9459	0.9501	0.9531	0.9497	0.9020	0.9115	0.9378	0.9171
no	$\kappa = 3, \lambda_{act} = 2$	0.9463	0.9494	0.9532	0.9496	0.9101	0.9280	0.9454	0.9278
no	$\kappa = 5, \lambda_{act} = 0.5$	0.9449	0.9499	0.9533	0.9494	0.9139	0.9240	0.9398	0.9259
no	$\kappa = 5, \lambda_{act} = 1$	**0.9448**	**0.9540**	**0.9532**	**0.9507**	0.9198	0.9293	0.9415	0.9302
no	$\kappa = 5, \lambda_{act} = 2$	0.9454	0.9483	0.9536	0.9491	**0.9274**	**0.9325**	**0.9457**	**0.9352**
no	$\kappa = 8, \lambda_{act} = 1$	0.9445	0.9534	0.9529	0.9503	0.9158	0.9294	0.9429	0.9294
no	$\kappa = 8, \lambda_{act} = 2$	0.9453	0.9447	0.9512	0.9471	0.9292	0.9339	0.9406	0.9346
sì	$\kappa = 5, \lambda_{act} = 0.5$	0.9191	0.9295	0.9322	0.9269	0.8641	0.9056	0.9252	0.8983
sì	$\kappa = 5, \lambda_{act} = 1$	0.9201	0.9313	0.9343	0.9286	0.8832	0.9069	0.9241	0.9047
sì	$\kappa = 5, \lambda_{act} = 2$	0.9215	0.9303	0.9320	0.9279	*0.9000*	*0.9122*	*0.9193*	*0.9105*
CloserAL									
sì	$\lambda_{act} = 0$ (adv. training)	0.8504	0.8558	0.8557	0.8540	–	–	–	–
sì	$\lambda_{act} = 0.5$	0.8506	0.8543	0.8554	0.8534	–	–	–	–
sì	$\lambda_{act} = 1$	**0.8501**	**0.8562**	**0.8559**	**0.8541**	–	–	–	–
sì	$\lambda_{act} = 2$	0.8506	0.8542	0.8558	0.8535	–	–	–	–

Tabella 6.23. $\bar{S}$ delle configurazioni dello stadio 4 per taglia, e media $\langle \bar{S} \rangle$ sulle tre taglie. La colonna x^{adv} indica se la loss del task è calcolata anche sui campioni adversarial; i pesi non indicati hanno il valore di default ($\lambda_{det} = \lambda_{act} = 1$, $\kappa = 5$). In grassetto le configurazioni selezionate, in corsivo le varianti di controllo che le affiancano nello stadio 5.

6.5.2 Loss del task sui campioni adversarial

È il fattore più influente: calcolare la loss del task anche sui campioni adversarial peggiora $\bar{S}$ in tutti i 30 confronti a parità di modello, pesi e taglia della Tabella 6.23 e, con PGD, anche in ciascuno dei 48 confronti sui singoli dataset. Come per le varianti `-advtrain` dello stadio 1, la backbone diventa robusta, ma perde circa tre punti di accuratezza pulita e l'allarme peggiora (Tabella 6.24): l'adversarial training avvicina le rappresentazioni pulite e adversarial, come una loss attrattiva implicita, e la $\mathrm{SLAV}^{(1)}_{rel}$ cala di almeno la metà. Nello stadio 1, però, contro l'attacco adattivo più forte le varianti con adversarial training restavano robuste, con un'accuratezza end-to-end robusta intorno a 0.81, contro 0.558 e 0.318 di `detlayer` e `further` (Tabella 6.7), e la validità di questo effetto è da confermare nello stadio 5.

		Backbone		Allarme			End-to-end		
Modello	Configurazione	$\mathrm{Acc}^{clean}_{task}$	$\mathrm{Acc}^{adv}_{task}$	TNR	TPR	$\mathrm{SLAV}^{(1)}_{rel}$	$\mathrm{Acc}^{e2e}_{clean}$	Acc^{e2e}_{rob}	Score_{test}
PGD									
DetectorLayer	default	0.900	0.329	0.997	0.996	0.054	0.898	0.998	0.948
	$\boldsymbol{x}^{adv}$	0.872	0.837	0.979	0.966	0.027	0.857	0.984	0.920
FurtherAL	default	0.905	0.077	0.997	0.997	1.185	0.903	0.998	0.951
	$\boldsymbol{x}^{adv}$	0.874	0.829	0.986	0.984	0.485	0.864	0.993	0.929
CloserAL	$\lambda_{act} = 0$	0.872	0.836	–	–	0.028	0.872	0.836	0.854
	default	0.871	0.837	–	–	0.006	0.871	0.837	0.854
PGD adattivo									
DetectorLayer	default	0.897	0.359	0.938	0.910	0.272	0.841	0.992	0.917
	$\lambda_{det} = 2$	0.891	0.328	0.923	0.930	0.263	0.823	0.990	0.907
	$\boldsymbol{x}^{adv}$	0.874	0.823	0.870	0.770	0.054	0.770	0.984	0.877
FurtherAL	default	0.904	0.242	0.961	0.925	1.014	0.869	0.992	0.930
	$\lambda_{act} = 2$	0.906	0.197	0.971	0.942	1.108	0.879	0.991	0.935
	$\lambda_{act} = 2$, $\kappa = 8$	0.902	0.156	0.973	0.955	1.778	0.877	0.992	0.935
	$\lambda_{act} = 2$, $\boldsymbol{x}^{adv}$	0.875	0.830	0.945	0.956	0.402	0.831	0.989	0.910

Tabella 6.24. Metriche di test medie su taglie e dataset. Sono indicati i soli valori diversi dal default; $\boldsymbol{x}^{adv}$ indica la loss del task calcolata anche sui campioni adversarial, e per CloserAL $\lambda_{act} = 0$ è l'adversarial training.

6.5.3 Peso della loss di detection

Nel regime cooperativo λ_{det} regola quanto la telecamera dell'analogia può spostare l'arredamento. Con $\lambda_{det} = 0.5$ la cooperazione è più debole e $\bar{S}$ cala a ogni taglia; con $\lambda_{det} = 2$ la separazione non cresce più, mentre la loss di detection sottrae capacità al task e i falsi allarmi aumentano. Il default è il migliore alla medium e alla high ed equivalente al migliore alla mini (Figura 6.3 a); fa eccezione CTU-13, dove $\bar{S}$ cala di poco al crescere del peso, in

accordo con lo stadio 3, in cui su questo dataset la cooperazione non portava vantaggi. Con la loss del task anche sui campioni adversarial, invece, $\langle \bar{S} \rangle$ cresce con λ_{det} da 0.845 a 0.877 e poi si stabilizza, verosimilmente perché la spinta della detection deve compensare l'attrazione indotta dall'adversarial training. Per la domanda di ricerca 2, la cooperazione è quindi utile a un'intensità intermedia, con un costo per il task che cresce con il peso.

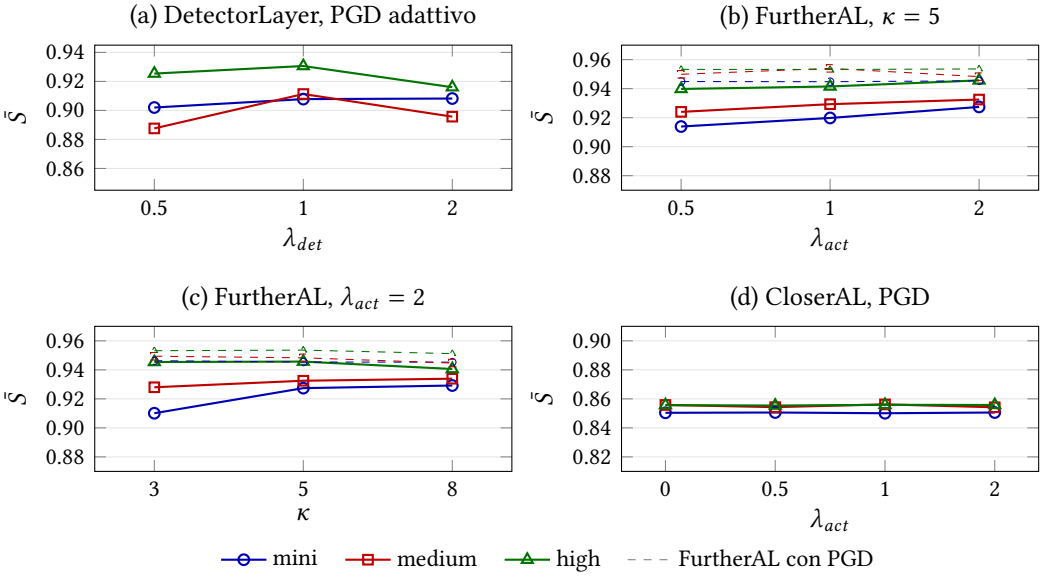


Figura 6.3. $\bar{S}$ in funzione dei pesi della loss, per taglia: (b) e (c) riportano FurtherAL con PGD adattivo e, tratteggiato, con PGD; in (d) $\lambda_{act} = 0$ è l'adversarial training. Nei modelli con detector la loss del task è calcolata sui soli campioni puliti.

6.5.4 Peso e margine della loss repulsiva

Con PGD il rilevamento è saturo e i pesi non contano (Figura 6.3 b e c). Contro l'attacco adattivo, invece, con $\kappa = 5$ lo Score medio $\bar{S}$ cresce con λ_{act} a ogni taglia. A differenza della cooperazione, la loss repulsiva rafforza l'allarme senza perdite per il task, la cui accuratezza pulita della backbone resta ferma. Il fattore κ fissa l'entità della separazione, che cresce quasi in proporzione al margine (con $\lambda_{act} = 2$ la $\mathrm{SLAV}^{(1)}_{rel}$ vale 0.76, 1.11 e 1.78 per $\kappa = 3$, 5 e 8). Con $\kappa = 3$ la separazione è insufficiente e $\langle \bar{S} \rangle$ cala, mentre $\kappa = 8$ si equivale a $\kappa = 5$ ma rende più fragile la backbone sotto attacco: una separazione di circa cinque volte quella naturale è sufficiente. Anche il peso appare vicino alla saturazione, perché con $\kappa = 8$ passare da $\lambda_{act} = 1$ a 2 non migliora più la high. Con la loss del task anche sui campioni adversarial, infine, aumentare λ_{act} recupera parte della separazione persa ($\mathrm{SLAV}^{(1)}_{rel}$ da 0.20

a 0.40) e dello Score, da 0.898 a 0.910, senza raggiungere la variante con la sola loss pulita. Per la domanda di ricerca 5, con il solo primo livello esteso la separazione dei livelli successivi segue il margine imposto al primo: con PGD, nel secondo livello della mini la SLAV_{rel} vale 2.3, 3.0 e 5.4 per $\kappa = 3$, 5 e 8, contro 0.27 di DetectorLayer, e nella high contro l'attacco adattivo il terzo livello passa da 1.11 a 1.86 con $\lambda_{act} = 1$, contro 0.86 di DetectorLayer cooperativo. La separazione si conserva quindi lungo la rete, come nel test M2.

6.5.5 Peso della loss attrattiva

Per CloserAL λ_{act} non ha effetti misurabili (Figura 6.3 d). Per la seconda parte della domanda di ricerca 3 la risposta sembra quindi negativa: avvicinare le rappresentazioni non aggiunge nulla all'adversarial training, ma il giudizio finale è lasciato allo stadio 5. Quanto alla domanda di ricerca 4, nella medium, l'unica con due livelli CloserAL, la SLAV_{rel} del secondo livello scende di circa trenta volte già con $\lambda_{act} = 0.5$, senza effetti su $\bar{S}$: l'ipotesi dello stadio 2 sul peso complessivo della loss attrattiva non trova quindi conferma. Con solo il primo livello esteso, infine, l'allineamento si propaga attenuandosi con la profondità.

6.5.6 Configurazioni selezionate

Come nello stadio 3, i pesi sono scelti uniformemente sulle taglie: vince la combinazione con il $\langle \bar{S} \rangle$ più alto. Tutti i modelli conservano i valori di default (Tabella 6.25), tranne FurtherAL contro l'attacco adattivo, che adotta $\lambda_{act} = 2$.

Varianti di controllo. Lo Score misurato con l'attacco di addestramento penalizza la loss del task sui campioni adversarial, ma non dice come si comporti la backbone quando i detector vengono elusi: in quel caso l'accuratezza end-to-end robusta si riduce a quella della backbone sotto attacco, che contro PGD adattivo vale 0.36 per il DetectorLayer selezionato e 0.20 per il FurtherAL selezionato, contro 0.82–0.83 delle varianti con adversarial training (Tabella 6.24). Poiché nello stadio 1 proprio queste varianti reggevano meglio l'attacco adattivo più forte, nello stadio 5 i modelli addestrati con l'attacco adattivo, a potenze diverse, sono valutati in entrambe le versioni della loss del task. Le varianti con adversarial training mantengono gli stessi pesi delle loss dei vincitori: ogni coppia differisce così per la sola loss del task, e il confronto nel caso peggiore dirà se la seconda linea di difesa compensa lo

Score perso con l'attacco di addestramento. Allo stesso modo, il solo adversarial training affianca CloserAL come termine di confronto per la loss attrattiva.

						$\bar{S}$		
Modello	Attacco	x^{adv}	λ_{det}	λ_{act}	κ	mini	medium	high
DetectorLayer	PGD	no	(1)	–	–	0.9434	0.9481	0.9521
FurtherAL	PGD	no	(1)	1	5	0.9448	0.9540	0.9532
CloserAL	PGD	sì	–	1	–	0.8501	0.8562	0.8559
Adv. training	PGD	sì	–	0	–	0.8504	0.8558	0.8557
DetectorLayer	PGD adattivo	no	1	–	–	0.9078	0.9113	0.9306
FurtherAL	PGD adattivo	no	(1)	2	5	0.9274	0.9325	0.9457
Varianti di controllo								
DetectorLayer	PGD adattivo	sì	1	–	–	0.8779	0.8912	0.8617
FurtherAL	PGD adattivo	sì	(1)	2	5	0.9000	0.9122	0.9193

Tabella 6.25. Configurazioni della loss selezionate per lo stadio 5, comuni alle tre taglie di ciascun modello e attacco, e relativo $\bar{S}$, insieme alle varianti di controllo che le affiancano; x^{adv} ha lo stesso significato della Tabella 6.23. Tra parentesi i pesi ininfluenti nel regime isolato; i meccanismi di detection sono quelli della Tabella 6.22.

6.5.7 Sintesi dello stadio 4

Lo stadio consente di trarre diverse conclusioni. Nei modelli con detector conviene calcolare la loss del task sui soli campioni puliti, perché l'adversarial training entra in conflitto con il rilevamento; il suo valore come seconda linea di difesa è verificato nello stadio 5, che per i modelli addestrati con l'attacco adattivo valuta entrambe le versioni della loss del task. Si hanno inoltre delle prime risposte alle domande di ricerca 2 e 3: la cooperazione è utile a un'intensità intermedia, mentre la loss repulsiva rafforza l'allarme senza costi per il task; con i pesi selezionati FurtherAL si conferma migliore di DetectorLayer, mentre al contrario la loss attrattiva non aggiunge nulla all'adversarial training. Infine, per quanto riguarda le domande di ricerca 4 e 5, l'allineamento di CloserAL si attenua con la profondità, mentre la separazione di FurtherAL si propaga ed è regolabile dal primo livello. Resta aperta la domanda di ricerca 6, oggetto dello stadio 5.

6.6 Stadio 5: analisi del caso peggiore

Gli stadi precedenti hanno selezionato i modelli valutandoli con lo stesso attacco usato in addestramento, a 20 passi e, per l'attacco adattivo, con peso di evasione $\beta = 1$. Lo stadio 5 li sottopone ad attacchi più forti e a budget diversi, per rispondere alla domanda di ricerca

6: il vantaggio delle difese basate sul rilevamento resiste a un attaccante che conosce i detector e ne sceglie liberamente i parametri?

6.6.1 Impostazione sperimentale

Lo script `lwad_stage5.py` non addestra alcun modello: ogni run ricarica un checkpoint dello stadio 4 e cambia soltanto l'attacco, lasciando la soglia dei detector a quella calibrata in addestramento, come per una difesa in esercizio che non conosce l'attacco che riceverà. Per ogni dataset si valutano i 15 modelli selezionati (Tabella 6.25), le 6 varianti di controllo con la loss del task anche sui campioni adversarial e i 3 modelli con il solo adversarial training, per un totale di 24 modelli.

Al budget di addestramento $\varepsilon = 0.2$ si cerca l'attacco più efficace al variare del numero di passi T, del passo α e, contro i modelli con detector, del peso di evasione β, mentre le curve ripetono PGD e l'attacco adattivo per altri quattro budget (Tabella 6.26). Si hanno così 792 run per dataset, 3168 in totale.

Gruppo	Attacco	ε	T	α	β	Run
Ricerca	FGSM	0.2	1	–	–	1
	PGD	0.2	20, 100	$2\varepsilon/T$, $\varepsilon/5$	–	4
	PGD adattivo*	0.2	20, 100	$2\varepsilon/T$, $\varepsilon/5$	1, 5, 10, 100	16
Curva	PGD	0.1, 0.3, 0.5, 1	100	$2\varepsilon/T$	–	4
	PGD adattivo*	0.1, 0.3, 0.5, 1	100	$2\varepsilon/T$	1, 5, 10, 100	16

Tabella 6.26. Attacchi dello stadio 5 e numero di run per modello. Gli attacchi contrassegnati da * sono eseguiti soltanto contro i modelli con detector.

6.6.2 Il caso peggiore al budget di addestramento

Il caso peggiore è sempre relativo a un budget: senza un limite, ogni modello può essere ingannato (test A4 dello stadio 1), e il minimo su tutti i budget dipenderebbe soltanto dal più grande considerato. Per ogni modello e dataset, il caso peggiore al budget di addestramento $\varepsilon = 0.2$ è quindi la minima accuratezza end-to-end robusta tra tutti gli attacchi ammissibili con quel budget. Tra questi rientrano gli attacchi con $\varepsilon = 0.1$, perché ogni perturbazione di ampiezza 0.1 rispetta anche il vincolo 0.2, mentre quelli con budget maggiore corrispondono a un attaccante più forte e sono analizzati a parte nel Paragrafo 6.6.4. Tutti i valori sono medie sui quattro dataset.

La Tabella 6.27 confronta, per ogni modello e taglia, l'attacco di addestramento con il caso peggiore.

			Acc^{e2e}_{rob}		Score		Caso peggiore	
Modello	Taglia	Acc^{e2e}_{clean}	addestr.	peggiore	addestr.	peggiore	TPR	Acc^{adv}_{task}
Addestrati con PGD								
DetectorLayer	mini	0.889	0.997	0.497	0.943	0.693	0.056	0.451
	medium	0.899	0.998	0.471	0.948	0.685	0.031	0.452
	high	0.907	0.998	0.763	0.952	0.835	0.070	0.727
FurtherAL	mini	0.891	0.998	0.291	0.945	0.591	0.065	0.254
	medium	0.910	0.998	0.354	0.954	0.632	0.100	0.284
	high	0.908	0.998	0.384	0.953	0.646	0.091	0.312
CloserAL	mini	0.867	0.834	0.833	0.850	0.850	–	0.833
	medium	0.874	0.838	0.837	0.856	0.856	–	0.837
	high	0.872	0.840	0.839	0.856	0.855	–	0.839
Adv. training	mini	0.870	0.831	0.830	0.850	0.850	–	0.830
	medium	0.874	0.837	0.836	0.856	0.855	–	0.836
	high	0.872	0.839	0.838	0.856	0.855	–	0.838
Addestrati con PGD adattivo								
DetectorLayer	mini	0.823	0.992	0.873	0.908	0.848	0.249	0.722
	medium	0.828	0.994	0.844	0.911	0.836	0.207	0.696
	high	0.872	0.990	0.817	0.931	0.845	0.253	0.661
FurtherAL	mini	0.867	0.988	0.639	0.927	0.753	0.221	0.466
	medium	0.874	0.991	0.676	0.932	0.775	0.201	0.549
	high	0.897	0.994	0.717	0.946	0.807	0.296	0.541
DetectorLayer $\boldsymbol{x}^{adv}$	mini	0.773	0.983	0.888	0.878	0.830	0.208	0.853
	medium	0.793	0.989	0.913	0.891	0.853	0.208	0.859
	high	0.745	0.979	0.925	0.862	0.835	0.255	0.866
FurtherAL $\boldsymbol{x}^{adv}$	mini	0.814	0.986	0.897	0.900	**0.855**	0.206	0.858
	medium	0.832	0.992	0.924	0.912	**0.878**	0.452	0.858
	high	0.848	0.990	0.911	0.919	**0.880**	0.318	0.861

Tabella 6.27. Attacco di addestramento e caso peggiore al budget $\varepsilon = 0.2$, in media sui dataset. TPR e Acc^{adv}_{task} si riferiscono all'attacco che realizza il caso peggiore; $\boldsymbol{x}^{adv}$ indica le varianti di controllo con la loss del task calcolata anche sui campioni adversarial. In grassetto il miglior Score nel caso peggiore per ogni taglia.

Modelli senza detector. Per CloserAL e per il solo adversarial training il caso peggiore coincide con l'attacco di addestramento: PGD a 100 passi non aggiunge nulla a quello a 20 passi, FGSM è leggermente più debole e, come mostra la Figura 6.5, la robustezza decresce con regolarità al crescere del budget.

Modelli con detector addestrati con PGD. PGD non li scalfisce mai, mentre l'attacco adattivo fa crollare l'accuratezza end-to-end robusta da 0.997 a valori tra 0.471 e 0.763 per DetectorLayer e tra 0.291 e 0.384 per FurtherAL, con un TPR mai superiore a 0.1: i detector hanno imparato a riconoscere la firma di un attaccante che li ignora, non la perturbazione

in quanto tale. Una volta elusi, conta solo la backbone, che in FurtherAL la loss repulsiva ha reso la più fragile: lo Score nel caso peggiore scende tra 0.685 e 0.835 per DetectorLayer e tra 0.591 e 0.646 per FurtherAL, sotto CloserAL a ogni taglia.

Modelli con detector addestrati con l'attacco adattivo. La caduta dell'accuratezza end-to-end robusta è minore, arrivando a 0.817–0.873 per DetectorLayer e a 0.639–0.717 per FurtherAL, ma basta a portare lo Score al livello di CloserAL: tra 0.836 e 0.848 per DetectorLayer, e tra 0.753 e 0.807 per FurtherAL. L'ordine tra i due si inverte rispetto allo stadio 4: FurtherAL, avanti con l'attacco di addestramento, resta indietro a ogni taglia.

Varianti con adversarial training. Sono i modelli più robusti dello stadio, con un'accuratezza end-to-end robusta nel caso peggiore tra 0.888 e 0.925 per DetectorLayer e tra 0.897 e 0.924 per FurtherAL, contro 0.833–0.839 di CloserAL: la backbone riesce a classificare correttamente tra l'85 e l'87% dei campioni attaccati, e la seconda linea di difesa ipotizzata nello stadio 4 si dimostra efficace. Per FurtherAL il vantaggio si traduce nello Score più alto nel caso peggiore a ogni taglia, pari a 0.855, 0.878 e 0.880, superando di nuovo CloserAL. Per DetectorLayer, invece, il guadagno sulla componente robusta è annullato dai falsi allarmi, con un'accuratezza pulita tra 0.745 e 0.793, e lo Score (0.830–0.853) resta in linea con quello della variante senza adversarial training (0.836–0.848).

6.6.3 Effetto dei parametri dell'attacco

La Figura 6.4 mostra come i parametri dell'attacco adattivo incidono sull'accuratezza end-to-end robusta al budget di addestramento. Contro PGD e FGSM, invece, i parametri non contano: qualunque siano il numero di passi e il passo, i modelli con detector restano prossimi alla robustezza piena e quelli senza detector al valore dell'attacco di addestramento.

Passo. È il parametro decisivo. Con il passo grande $\alpha = \varepsilon/5$, 100 passi non aggiungono nulla ai 20 dell'attacco di addestramento; con $\alpha = 2\varepsilon/T$ l'accuratezza scende già a 20 passi e crolla a 100, dove il passo è dieci volte più fine. Per eludere un detector, verosimilmente, la perturbazione deve restare appena sotto la soglia di uno score che satura rapidamente, e un passo grosso la scavalca; contro il solo classificatore il passo è invece ininfluente.

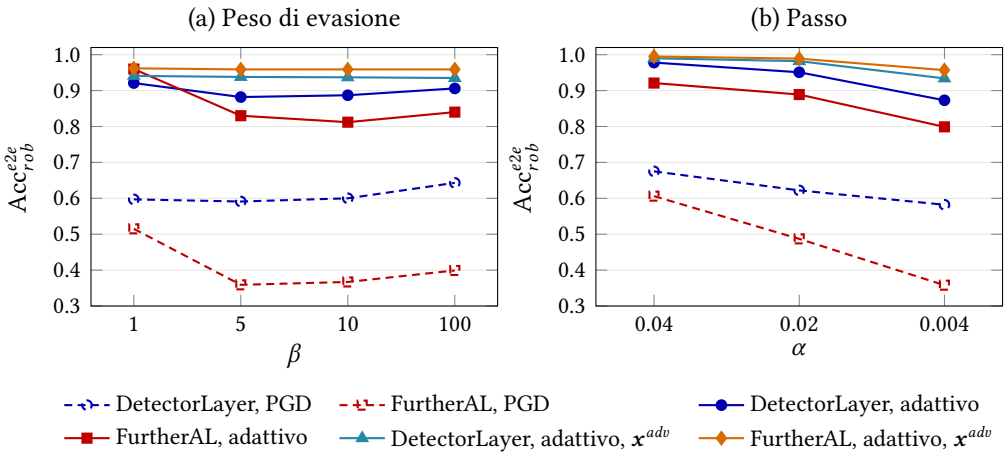


Figura 6.4. Accuratezza end-to-end robusta dei modelli con detector contro l'attacco adattivo con $\varepsilon = 0.2$, in media su taglie e dataset; la legenda indica l'attacco di addestramento. (a) Al variare di β, con $T = 100$ e $\alpha = 2\varepsilon/T$. (b) Al variare del passo, considerando il β peggiore per ciascun modello: $\alpha = \varepsilon/5 = 0.04$ con $T = 100$, e $\alpha = 2\varepsilon/T$ con $T = 20$ e $T = 100$.

Peso di evasione. β regola un compromesso: al suo crescere il TPR scende, perché l'attacco si concentra sull'elusione, ma l'accuratezza della backbone sale, perché la loss del task conta meno. Il valore peggiore dipende quindi dal modello, e nessun β è il più efficace per tutti: per DetectorLayer addestrato con PGD basta $\beta = 1$, perché l'allarme è eluso già così, mentre per FurtherAL servono valori tra 5 e 10. Il caso più significativo è FurtherAL addestrato con l'attacco adattivo: con il $\beta = 1$ dell'attacco di addestramento l'accuratezza robusta è 0.960, con $\beta = 5$ scende a 0.830, perché l'allarme viene eluso molto più spesso. La separazione ampia di FurtherAL porta verosimilmente gli score dei detector in saturazione, dove il loro gradiente si annulla, e soltanto un peso maggiore rende l'elusione competitiva con la loss del task: il vantaggio di FurtherAL negli stadi 3 e 4 rifletteva quindi in parte la debolezza dell'attacco di addestramento. Per le varianti con adversarial training invece, β risulta quasi ininfluente.

6.6.4 Robustezza al variare del budget

La Figura 6.5 riporta l'accuratezza end-to-end robusta al variare del budget, con PGD e con l'attacco adattivo, considerando per quest'ultimo il β peggiore a ogni ε.

Modelli senza detector. La robustezza decresce con regolarità, come nel test A2 dello stadio 1 (Tabella 6.8). Fino a $\varepsilon = 0.3$ CloserAL e il solo adversarial training si equivalgono,

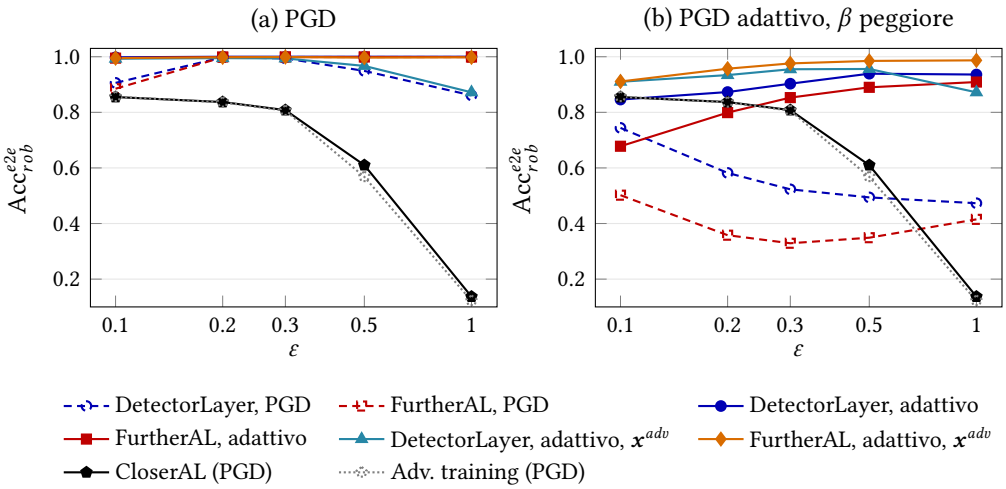


Figura 6.5. Accuratezza end-to-end robusta al variare del budget, con $T = 100$ e $\alpha = 2\varepsilon/T$, in media su taglie e dataset; la legenda indica il modello e il suo attacco di addestramento. (a) Contro PGD. (b) Contro l'attacco adattivo con il β peggiore per ciascun modello; CloserAL e il solo adversarial training, privi di detector, sono riportati con PGD, che per loro è l'attacco white-box completo.

ma oltre il budget di addestramento CloserAL degrada più lentamente. La deviazione relativa del primo livello, ridotta dalla loss attrattiva, resta infatti contenuta anche lontano dal budget di addestramento. Per la seconda parte della domanda di ricerca 3, la loss attrattiva rende più graduale la perdita di robustezza quando l'attaccante dispone di un budget maggiore: entrambe le difese degradano, ma CloserAL più lentamente.

Modelli con detector contro PGD. I detector intercettano PGD anche con budget molto superiori a quello di addestramento, con un'accuratezza robusta di almeno 0.949 fino a $\varepsilon = 0.5$ e di 0.861–0.872 per DetectorLayer a $\varepsilon = 1$. Il punto debole è sotto il budget di addestramento: a $\varepsilon = 0.1$ i modelli addestrati con PGD scendono a 0.906 e 0.885, verosimilmente perché le perturbazioni più piccole sono meno riconoscibili.

Modelli con detector contro l'attacco adattivo. I modelli addestrati con PGD si comportano come modelli non difesi. Per quelli addestrati con l'attacco adattivo, invece, l'accuratezza misurata cresce con ε. Non si tratta però di robustezza reale ovviamente: l'insieme ammissibile di raggio 0.1 è contenuto in quelli di raggio maggiore, e il caso peggiore non può quindi migliorare al crescere del budget. La causa sta nella definizione del passo. Nella formulazione di PGD adottata nella metodologia il passo è fissato a

$\alpha = 2\varepsilon/T$, in modo che in T iterazioni lo spostamento complessivo possa percorrere l'intera regione ammissibile, da un vertice dell'ipercubo a quello opposto, qualunque sia il budget. Il passo cresce quindi in proporzione a ε: con 100 passi vale 0.002 a $\varepsilon = 0.1$, mentre vale 0.02 a $\varepsilon = 1$, e la ricerca diventa dieci volte più grossolana. Contro un classificatore senza detector questo non conta, ma contro un allarme la perturbazione deve fermarsi appena sotto la soglia, e un passo più ampio la scavalca più facilmente: al crescere del budget la precisione dell'attacco cala, e con essa la sua efficacia. A parità di passo ($\alpha = 0.02$), infatti, l'accuratezza a $\varepsilon = 1$ è simile a quella a $\varepsilon = 0.2$ con 20 passi, a conferma che contro questi modelli conta il passo più del budget. Anche considerando il minimo cumulato, oltre $\varepsilon = 0.3$ tutti restano sopra CloserAL, che crolla a 0.610 e 0.137, e le varianti con adversarial training lo sono su tutta la curva: con budget ampi il rilevamento è l'unica difesa che regge.

6.6.5 Sintesi dello stadio 5

Lo stadio risponde alla domanda di ricerca 6, e la risposta dipende dall'attacco di addestramento. Per i modelli con detector addestrati con PGD il vantaggio non resiste: l'attacco adattivo elude i detector nella maggior parte dei casi. I modelli hanno quindi bisogno di vedere un attacco per imparare a resistervi. Per quelli addestrati con l'attacco adattivo infatti il vantaggio si annulla. Inoltre, FurtherAL con la loss del task anche sui campioni adversarial ha lo Score più alto nel caso peggiore (0.855–0.880), sopra CloserAL a ogni taglia, perché la backbone robusta compensa i campioni che sfuggono all'allarme, e anche sopra DetectorLayer.

Contro i detector il passo dell'attacco è decisivo, mentre il β peggiore varia da modello a modello; i detector restano efficaci oltre il budget di addestramento ma sono vulnerabili alle perturbazioni più piccole.

6.7 Risposte alle domande di ricerca

1. Posizione dei detector. Contrariamente all'ipotesi iniziale, la perturbazione è più facilmente rilevabile nei livelli iniziali. Nel modello non difeso la deviazione relativa cresce di quasi sei volte dal primo all'uall'ultimoltimo livello, ma nel regime isolato l'AUC dei detector scende da 0.9995 nel primo livello a 0.94 nell'ultimo: l'amplificazione della deviazione non si traduce in una maggiore rilevabilità, probabilmente perché porta le

attivazioni abbastanza lontano da generare confusione lungo diverse frontiere decisionali. Sul piano operativo il primo livello è indispensabile: senza un detector in quella posizione lo Score perde fino a 0.07, e oltre 0.11 su CTU-13. Contro PGD un solo detector sul primo livello è sufficiente, mentre contro l'attacco adattivo estendere più livelli giova nelle backbone a due livelli, presumibilmente perché una stessa perturbazione deve eludere contemporaneamente detector specializzati su profondità diverse. Il primo livello è anche quello che abilita l'uscita anticipata con il risparmio maggiore, ma l'uscita anticipata non ha mai ridotto la latenza media: il risparmio del 43% sui campioni segnalati dal primo detector è compensato dal costo dei controlli intermedi sugli altri, e conviene soltanto se viene segnalato almeno il 60% circa del traffico, quindi in contesti altamente adversarial.

2. Flusso dei gradienti. Il regime isolato fornisce, ai fini dell'interpretabilità, la misura della rilevabilità descritta al punto precedente, poiché in esso i detector osservano la backbone senza modificarla. La cooperazione agisce invece come una loss repulsiva implicita e priva di margine: è ininfluente contro PGD, ridondante per FurtherAL, che possiede già una separazione esplicita, e decisiva per DetectorLayer contro l'attacco adattivo, dove porta il TPR da 0.65 a 0.92 e lo Score da 0.861 a 0.897. Non ci sono ripercussioni sull'accuratezza pulita, quasi invariata, ma sulla backbone sotto attacco, che diventa più fragile, e il suo costo cresce con l'intensità della cooperazione: il peso della loss di detection rende al meglio a un valore intermedio, oltre il quale la capacità condivisa con il task fa aumentare i falsi allarmi. Sul piano formale, l'analisi del flusso dei gradienti ha inoltre mostrato che nelle loss sulle attivazioni il riferimento pulito va lasciato libero, e che l'attacco adattivo deve rimuovere ogni arresto del gradiente all'ingresso dei detector, pena una robustezza sovrastimata.

3. Manipolazione geometrica. Contro l'attacco di addestramento la strategia repulsiva migliora il rilevamento rispetto ai soli detector: FurtherAL supera o eguaglia DetectorLayer in tutte le architetture esplorate, rafforzando l'allarme senza costi per l'accuratezza del task. Nel caso peggiore, però, il vantaggio si inverte (0.778 contro 0.843): gli score saturi di FurtherAL rendono debole l'attacco di addestramento, con peso di evasione unitario, e la backbone, resa a sua volta più fragile dalla repulsione, non offre una seconda linea di difesa. La strategia repulsiva migliora quindi il rilevamento in senso stretto, ma per

tradurre questo vantaggio nel caso peggiore richiede una backbone robusta: combinata con l'adversarial training, essa ottiene lo Score più alto nel caso peggiore. Quanto alla strategia attrattiva, la risposta è negativa entro il budget di addestramento: a ogni peso e a ogni dimensione della rete, CloserAL e il solo adversarial training si equivalgono, benché la loss attrattiva riduca fino a sette volte la deviazione relativa del primo livello. Oltre quel budget, invece, CloserAL degrada più lentamente, con un'accuratezza robusta superiore in 11 casi su 12: la loss attrattiva non rafforza la robustezza ottenuta dall'adversarial training, ma la estende oltre il budget su cui è stata ottenuta.

4. Approfondimento della strategia attrattiva. Un solo livello CloserAL, sul primo livello nascosto, è sufficiente: lo Score dipende pochissimo dall'architettura, restando tra 0.847 e 0.856 in tutte le configurazioni esplorate, e aggiungere livelli CloserAL, consecutivi o meno, non porta vantaggi apprezzabili e nella maggior parte delle architetture lo riduce leggermente. L'ipotesi che questa penalizzazione derivi dal peso complessivo della loss attrattiva non trova conferma, perché dimezzarne il peso non modifica lo Score. L'allineamento del primo livello si propaga ai successivi attenuandosi con la profondità, dimezzando la deviazione del secondo livello nella rete più piccola ma riducendo soltanto del 18% quella del terzo nella più grande; anche l'allineamento diretto dei livelli profondi, che ne riduce la deviazione di circa trenta volte, non si traduce in un guadagno di robustezza.

5. Approfondimento della strategia repulsiva. La separazione indotta da FurtherAL si conserva e si propaga lungo la rete. Nei livelli non estesi la deviazione relativa è molte volte superiore a quella di DetectorLayer e proporzionale al margine imposto alla loss: nel secondo livello della rete più piccola vale 2.3, 3.0 e 5.4 per $\kappa = 3$, 5 e 8, contro 0.27 di DetectorLayer. Ne segue che nelle backbone a tre livelli basta un solo livello FurtherAL, sul primo, mentre in quelle a due livelli estenderli entrambi porta un piccolo vantaggio contro l'attacco adattivo. Il margine più efficace è intorno a cinque volte la separazione naturale: uno più piccolo è insufficiente, uno più grande non migliora lo Score e rende la backbone più fragile.

6. Robustezza adattiva. La risposta dipende dall'attacco con cui i modelli sono addestrati. Per i modelli con detector addestrati contro PGD il vantaggio non resiste: l'attacco

adattivo elude i detector in oltre il 90% dei casi, e lo Score nel caso peggiore scende a 0.738 per DetectorLayer e a 0.623 per FurtherAL. I detector avevano imparato a riconoscere la firma di un attaccante che li ignora, non la perturbazione in quanto tale. Per i modelli addestrati con l'attacco adattivo il vantaggio si annulla, con 0.843 per DetectorLayer e 0.778 per FurtherAL. Resiste invece la combinazione dei due meccanismi: FurtherAL con la loss del task calcolata anche sui campioni adversarial raggiunge 0.871, sopra l'adversarial training a ogni dimensione della rete, con un'accuratezza robusta nel caso peggiore di 0.911. Quanto ai parametri dell'attacco, contro i detector il passo è decisivo, perché l'elusione richiede una ricerca fine appena sotto la soglia dell'allarme, mentre il peso di evasione più efficace varia da modello a modello. Al variare del budget, infine, i detector restano efficaci con perturbazioni molto più ampie di quelle di addestramento, ma sono vulnerabili a quelle con un passo più piccolo.

Capitolo 7

Conclusioni e sviluppi futuri

Questa tesi ha studiato la robustezza dei NIDS basati su reti neurali profonde da una prospettiva layer-wise, trattando ogni livello nascosto come un punto in cui la deviazione indotta da un attacco adversarial può essere misurata, rilevata e modellata. Il capitolo riassume le risposte alle domande di ricerca emerse dai cinque stadi della sperimentazione, ne trae alcune considerazioni complessive e discute i limiti del lavoro e le sue possibili estensioni.

7.1 Considerazioni finali

Il quadro che emerge dalla sperimentazione è quello di due famiglie di difese complementari più che concorrenti. Valutate con l'attacco di addestramento, le difese basate sul rilevamento appaiono nettamente superiori alla classificazione robusta; nel caso peggiore, invece, nessuna delle due prevale da sola, e i due meccanismi coprono regioni diverse dello spazio degli attacchi. La classificazione robusta tiene contro le perturbazioni piccole ma crolla quando il budget cresce; il rilevamento, al contrario, intercetta le perturbazioni ampie ma lascia sfuggire quelle sotto il budget di addestramento. Un modello che li combina, con detector e loss repulsiva sui livelli iniziali e una backbone addestrata anche sui campioni adversarial, ottiene le migliori prestazioni lungo tutta la curva del budget.

Ne deriva un'indicazione metodologica che va oltre le architetture proposte: la selezione di una difesa basata sul rilevamento non può fondarsi sull'attacco con cui è stata addestrata. Nello stadio 4 la scelta guidata dall'attacco di addestramento escludeva infatti proprio la configurazione che si è rivelata la migliore nel caso peggiore. Una valutazione affidabile richiede un attaccante adattivo i cui parametri, in particolare il passo e il peso di evasione,

siano esplorati e non fissati.

Il lavoro è stato guidato da due esigenze tenute deliberatamente insieme. Da un lato si è cercato di condurre la sperimentazione sulla scala più ampia possibile, con cinque stadi, quattro dataset, circa 1400 addestramenti e oltre 3000 valutazioni, e una suite di verifica e validazione che garantisse che le differenze osservate fossero attribuibili alla metodologia e non all'implementazione. Dall'altro si è cercata la precisione in ciò che era possibile spiegare analiticamente, per offrire un contributo di interpretabilità: le invarianze della metrica SLAV, la necessità della distanza relativa e del riferimento libero, il ruolo degli arresti del gradiente durante l'attacco sono stati approfonditi formalmente, anziché scelti per tentativi. Ciò che non si prestava a una derivazione analitica, come la rilevabilità lungo la profondità, l'effetto della cooperazione o il comportamento nel caso peggiore, è stato lasciato all'analisi sperimentale, interpretata a sua volta attraverso le grandezze layer-wise introdotte. In questo modo è stato possibile spiegare, oltre che misurare, molti dei risultati ottenuti: perché l'adversarial training entri in conflitto con i detector, perché un attacco con passo grossolano non riesca a eluderli, perché la separazione di FurtherAL si propaghi lungo la rete.

7.2 Limiti e sviluppi futuri

Nella lettura dei risultati vanno tenuti presenti due limiti: diversi stadi impiegano un solo seed, per cui le differenze contro l'attacco adattivo sono state interpretate soltanto quando ampie o coerenti tra dimensioni della rete e dataset, e i valori nel caso peggiore dei modelli con detector sono limiti superiori, che un attacco con passo ancora più fine potrebbe abbassare. Da questi limiti, e dai risultati ottenuti, discendono le seguenti direzioni di sviluppo.

Vincoli di dominio e budget per feature. Lo spettro degli attacchi realizzabili dipende fortemente dalla natura delle feature. In questo studio il caso peggiore è stato calcolato su un insieme di perturbazioni più ampio di quello realizzabile, escludendo soltanto le feature categoriche e discrete: molte possibilità lasciate all'attaccante non sono applicabili nella realtà, ma l'approccio conservativo lascia un margine positivo sulla bontà dei risultati, poiché la vulnerabilità può risultare al più sovrastimata. Un primo sviluppo

consiste nel costruire una suite di attacchi che tenga conto dei vincoli intra-feature, come la non negatività e l'interezza dei conteggi, e inter-feature, come le relazioni aritmetiche tra grandezze dipendenti. Un secondo sviluppo, complementare, riguarda il budget: anziché un unico ε uniforme sulle feature standardizzate, si propone di quantificare budget diversi per feature diverse, che nel dominio possono essere più o meno perturbabili. Le grandezze controllate direttamente dal mittente di un flusso, ad esempio, sono più facilmente manipolabili di quelle determinate dalla risposta della vittima o dalle condizioni della rete. La maschera binaria delle feature perturbabili si generalizzerebbe così in un vettore di budget, e l'insieme ammissibile da un ipercubo a un iperrettangolo, i cui lati potrebbero essere ricavati dall'analisi del dominio o dalla variabilità naturale di ciascuna feature.

Flusso dei gradienti nelle reti profonde. L'analisi formale del Capitolo 5 è condotta su un singolo livello affine e su un singolo campione. Estenderla a reti profonde, nelle quali i gradienti delle loss di livelli diversi si compongono attraverso i jacobiani dei livelli successivi e la riscalatura di un livello può essere compensata dai seguenti, avrebbe richiesto uno sforzo considerevole, non giustificato rispetto all'obiettivo primario della tesi. I risultati sperimentali suggeriscono però che un'analisi di questo tipo sarebbe fruttuosa: la propagazione della separazione di FurtherAL e l'attenuazione dell'allineamento di CloserAL con la profondità sono fenomeni osservati ma non ancora spiegati formalmente, così come l'interazione tra la cooperazione dei detector e le loss sulle attivazioni di livelli diversi.

Detector con una visione più ampia dello spazio latente. Ogni detector proposto osserva le attivazioni di un solo livello, e gli score vengono combinati soltanto a valle. Un'ulteriore architettura potrebbe impiegare un detector che osservi contemporaneamente più livelli, se non l'intero spazio latente, eventualmente insieme all'ingresso: se le rappresentazioni di un singolo livello non bastano a distinguere un campione adversarial da uno pulito, il confronto tra l'ingresso e la sua rappresentazione, o tra rappresentazioni a profondità diverse, potrebbe farlo. I risultati dello stadio 2, in cui estendere più livelli rendeva più difficile l'elusione contro l'attacco adattivo, suggeriscono che una visione congiunta possa rendere ancora più arduo trovare un'unica perturbazione che inganni tutti i punti di osservazione in cui un tale detector è innestato.

Addestramento contro più attaccanti. L'analisi del caso peggiore ha mostrato che i detector si specializzano sull'attacco di addestramento, con scarsa generalizzazione: quelli addestrati con PGD standard non resistono all'attacco adattivo, e anche quelli addestrati con l'attacco adattivo sono vulnerabili a passi più fini, a pesi di evasione maggiori e a budget inferiori. Un'estensione naturale consiste nell'addestrare la rete contro più attaccanti, combinando FGSM, PGD e PGD adattivo con budget, numero di passi, ampiezza del passo e peso di evasione diversi, in modo che resista a tutti. Il limite di questo approccio è il costo: per ogni attaccante aggiunto l'insieme dei campioni adversarial cresce, sia in addestramento sia, soprattutto, in valutazione, dove ogni modello va confrontato con l'intero spettro degli attacchi.

Estensione di modelli già addestrati. L'implementazione è stata progettata per rendere semplice la costruzione di un modello robusto: tutti i livelli estesi derivano da un'unica astrazione che inoltra lo stato del flusso, e ogni modulo non esteso viene incapsulato automaticamente, così che una rete possa essere arricchita sostituendo soltanto i livelli di interesse, senza modificare gli altri né la funzione che essi calcolano. Questa natura del framework permette senza difficoltà di estendere una rete già addestrata, e quindi di irrobustire un modello già in esercizio senza riaddestrarlo da zero. Nel caso del rilevamento, è sufficiente innestare i detector nei livelli desiderati e addestrare soltanto questi ultimi, congelando la backbone: si tratta della forma più restrittiva del regime isolato, nel quale la backbone non soltanto non riceve il gradiente della loss di detection, ma non viene aggiornata affatto. Nel caso della classificazione robusta, la backbone può essere sottoposta a un fine-tuning con adversarial training e con la loss attrattiva di CloserAL sui livelli scelti. Una valutazione sistematica di queste modalità, confrontandone costo e robustezza con quelli dell'addestramento da zero, completerebbe il quadro delle possibili applicazioni di questo lavoro.

Ringraziamenti

Alla mia famiglia Un ringraziamento speciale alla mia famiglia, grazie alla quale ho conosciuto il privilegio più grande: poter scegliere con piena libertà la strada da percorrere nella mia vita. Voi avete sempre creduto in me con ineguagliabile fiducia, e mi avete forgiato per essere la persona che sono oggi, pertanto io vivrò per rendervi fieri della persona che diventerò domani.

Ai miei professori Vorrei esprimere la più sincera gratitudine al mio relatore il Dott. Gabriele Russo Russo per la guida preziosa e il supporto durante tutto questo percorso. Un grazie di cuore anche al mio correlatore il Prof. Giuseppe Bianchi per le preziose opportunità, per il primo articolo scritto in 8 ore, l'ispirazione e per aver creduto nelle mie potenzialità anche prima che lo facessi io.
E una menzione d'onore al Professor Alessandro Pellegrini per aver suscitato in me l'estro della curiosità, con il suo fervore, il suo genio e il suo carisma magnetico mi ha illuminato.

Alla mia storica comitiva Sono in questa vita da 25 anni, e ormai da tantissimo voi ci siete dentro, arrivando fino ad oggi ad emanare la mia essenza e diventando una parte imprescindibile di essa.

> *"Questa parte della mia vita… questa piccola parte… si può chiamare felicità."*
>
> — *La ricerca della felicità*

A Sigma e Ugo Quando sono andato via da Sabaudia, partivo al tempo per un'avventura che mi spaventava. Come si dice, sai quello che lasci ma non sai quello che trovi, ma a saperlo prima me ne partivo con qualche anno di anticipo! Siete stati il mio prologo ad una nuova vita a Roma, e senza di voi non mi sarei mai innamorato così tanto di questa città, perchè una città senza persone come voi è solo un deserto, e invece voi l'avete trasformata nella mia oasi.

Alla mia auletta A tutti capita di incontrare persone che, viste con la coda degli occhi, dal lato opposto della strada, ci appaiono di una bellezza sorprendente, addirittura simili a dei o dee: il modo in cui si muovono, il modo in cui la luce le colpisce, suscitano riverenza e stupore, poi ad un'occhiata più ravvicinata, cavolo, che delusione, belle da lontano ma neanche lontanamente belle. L'IMPRESSIONE. Alcune persone non saranno mai attraenti quanto in quella prima impressione, da lontano, in quella luce e a quell'ora, non saranno mai più belle che in quel primo sguardo dai contorni sfumati. É fascino. É immaginazione. É una scopata. É distacco. É transitorio. E a me piace così, perchè a volte è meglio con le luci soffuse.
A pochi, invece, capita di incontrare persone che, quando le guardiamo negli occhi, quando le abbiamo proprio di fronte, nella piena luce del giorno, ci appaiono di una bellezza sorprendente: il modo in cui si muovono, la vibrazione della loro anima suscitano riverenza e stupore. La DEFINIZIONE. Poi le guardiamo da vicino e... wow. Prendiamo il volo. Belle da vicino, ancora meglio in primo piano. Alcune persone diventano più attraenti, più le guardiamo e più ci impressionano, in quella luce, a quell'ora, nel modo in cui le vediamo, e più le vediamo con chiarezza e più sono belle. É ammirazione. É letterale. É fare l'amore. É unione. É definitivo. E a me piace così, perchè con voi è meglio con le luci accese.

Ai più grossi Alle meravigliose persone che hanno azzerato ogni distanza, facendomi sentire a casa anche quando da casa ero lontano. Siete stati un'ancora nei momenti più ardui, e un'immensa gioia in quelli più belli.

> *"Ohana significa famiglia. Famiglia vuol dire che nessuno viene abbandonato o dimenticato."*
>
> — *Lilo & Stitch*

Al mio tavolo C'è stato un periodo in cui avevo smesso di sentire musica. Uno dei miei motti è che se la musica non fosse esistita sarebbe stato come se il mondo non fosse più brillato dei suoi colori, come l'avessero disegnato con una scala di grigi. La realtà è che la musica non mi serviva più per rompere il silenzio, perchè adesso avevo delle persone che mi piaceva sentire più di qualsiasi canzone, e quelle persone eravate proprio voi: voi che con le vostre chiacchierate, i vostri modi di fare e le vostre abitudini mi avete incantato e avete dato colore voi stessi a questa tela in cui ogni giorno ci muoviamo.

> *"Per alcune persone vale la pena sciogliersi."*
>
> *— Olaf, Frozen*

A Giuseppe Quando avevo qualcosa come cinque anni e con mamma e papà tornavamo a casa di sabato sera da fuori, mamma mi fece questa domanda "Amore ma lo vuoi un fratellino o una sorellina?" e io, da bravo bambino di cinque anni, figlio unico, geloso dei miei genitori... le dissi ovviamente di no. Oggi mi chiedo se il motivo di quella risposta non fosse che, in fondo, qualcuno del genere già esisteva, dovevo solo incontrarti.
D'altronde, i piani di dio sono imperscrutabili, no?
Con te ho capito una cosa: non è la freccia a cercare sempre il bersaglio, ma il bersaglio ad attirare la freccia. Mi hai reso consapevole che ciò che ci attrae nella nostra vita non è casuale, e nemmeno una coincidenza, sì dobbiamo inseguire quel che vogliamo, andare a cercarlo, ma a volte non serve che facciamo succedere le cose, le nostre anime sono infinitamente magnetiche. E così sei diventato la mia metà, una parte di me.
Sfortunatamente per il sottoscritto, non hanno ancora inventato le parole per descrivere l'affetto che provo per te, ma ci provo lo stesso. Sei una delle persone più speciali che abbia mai conosciuto, mi affascini, giorno dopo giorno sempre di più con la tua filosofia di vivere, e mentre decanti la bellezza più pura a parole, io rimango infatuato nel vederti e ascoltarti. Sei unico, sei straordinario, sei una delle più grandi gioie per i miei sensi, perchè so che se io volessi soltanto sedermi e parlare con te, tu mi ascolteresti.
Ad oggi non so dove la vita ci porterà, spero solo che i fili del nostro destino si intreccino e si aggroviglino su loro stessi, così da tenerci vicini come in questi anni di magistrale.
A te auguro di volare sempre più in alto, conquista il mondo, perchè so per certo che neanche il sole potrà bruciare le tue ali.

A nonna Adriana Nonna, la mia bellissima nonna, tu hai cresciuto questo ragazzo con infinito amore, mi hai educato e viziato ad esserci immerso, eri la persona più forte che avessi mai incontrato, e ci tenevo a dedicarti un piccolissimo verso.
Avrei desiderato che mi vedessi oggi, avrei sperato di averti al mio fianco nel mentre, a te che non ci sei più, ma in me ci sarai per sempre.

A tutti voi avrete per sempre una parte del mio cuore.

Bibliografia

[1] James P. Anderson. Computer security threat monitoring and surveillance. Technical Report Contract 79F296400, James P. Anderson Co., Fort Washington, PA, April 1980. (cited on page 5.)

[2] Maksym Andriushchenko, Francesco Croce, Nicolas Flammarion, and Matthias Hein. Square attack: a query-efficient black-box adversarial attack via random search, 2020. (cited on pages 21, 22.)

[3] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. Dos and don'ts of machine learning in computer security, 2021. (cited on pages 35, 47.)

[4] Anish Athalye, Nicholas Carlini, and David Wagner. Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples, 2018. (cited on pages 23, 31, 80, 96, 98.)

[5] Yang Bai, Yuyuan Zeng, Yong Jiang, Shu-Tao Xia, Xingjun Ma, and Yisen Wang. Improving adversarial robustness via channel-wise activation suppressing, 2022. (cited on page 33.)

[6] Wieland Brendel, Jonas Rauber, and Matthias Bethge. Decision-based adversarial attacks: Reliable attacks against black-box machine learning models, 2018. (cited on page 22.)

[7] Nicholas Carlini, Anish Athalye, Nicolas Papernot, Wieland Brendel, Jonas Rauber, Dimitris Tsipras, Ian Goodfellow, Aleksander Madry, and Alexey Kurakin. On evaluating adversarial robustness, 2019. (cited on pages 31, 80, 96.)

[8] Nicholas Carlini and David Wagner. Adversarial examples are not easily detected: Bypassing ten detection methods, 2017. (cited on pages 23, 24, 31, 33.)

[9] Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks, 2017. (cited on pages 20, 21, 31.)

[10] Evan Caville, Wai Weng Lo, Siamak Layeghy, and Marius Portmann. Anomal-e: A self-supervised network intrusion detection system based on graph neural networks. *Knowledge-Based Systems*, 258:110030, December 2022. (cited on page 30.)

[11] Pin-Yu Chen, Huan Zhang, Yash Sharma, Jinfeng Yi, and Cho-Jui Hsieh. Zoo: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models. In *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*, CCS '17, page 15–26. ACM, November 2017. (cited on page 22.)

[12] Cisco Systems. Cisco Secure Network Analytics (formerly Stealthwatch). `https://www.cisco.com/site/us/en/products/security/security-analytics/secure-network-analytics/index.html`, 2018. Accessed: 2026-09-22. (cited on page 27.)

[13] Jeremy M Cohen, Elan Rosenfeld, and J. Zico Kolter. Certified adversarial robustness via randomized smoothing, 2019. (cited on pages 23, 32.)

[14] Francesco Croce, Maksym Andriushchenko, Vikash Sehwag, Edoardo Debenedetti, Nicolas Flammarion, Mung Chiang, Prateek Mittal, and Matthias Hein. Robustbench: a standardized adversarial robustness benchmark, 2021. (cited on page 31.)

[15] Francesco Croce and Matthias Hein. Minimally distorted adversarial examples with a fast adaptive boundary attack. *CoRR*, abs/1907.02044, 2019. (cited on pages 20, 21.)

[16] Francesco Croce and Matthias Hein. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks, 2020. (cited on pages 21, 31.)

[17] CrowdStrike. CrowdStrike Falcon platform. `https://www.crowdstrike.com/it-it/platform//`, 2013. Accessed: 2026-09-22. (cited on page 27.)

[18] Xuelong Dai, Dong Wang, Xiuzhen Cheng, and Bin Xiao. Gradient-free adversarial purification with diffusion models, 2025. (cited on pages 23, 32.)

[19] Ambra Demontis, Marco Melis, Maura Pintor, Matthew Jagielski, Battista Biggio, Alina Oprea, Cristina Nita-Rotaru, and Fabio Roli. Why do adversarial attacks transfer? explaining transferability of evasion and poisoning attacks, 2019. (cited on page 21.)

[20] D.E. Denning. An intrusion-detection model. *IEEE Transactions on Software Engineering*, SE-13(2):222–232, 1987. (cited on page 5.)

[21] Yinpeng Dong, Fangzhou Liao, Tianyu Pang, Hang Su, Jun Zhu, Xiaolin Hu, and Jianguo Li. Boosting adversarial attacks with momentum, 2018. (cited on page 21.)

[22] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, CCS '17, page 1285–1298, New York, NY, USA, 2017. Association for Computing Machinery. (cited on page 6.)

[23] Reuben Feinman, Ryan R. Curtin, Saurabh Shintre, and Andrew B. Gardner. Detecting adversarial samples from artifacts, 2017. (cited on page 33.)

[24] Alessandro Finocchi. lw_aml_detector: Layer-wise adversarial machine learning detector. `https://github.com/AlessandroFinocchi/lw_aml_detector`, 2026. Repository GitHub. (cited on page 87.)

[25] Fortinet. FortiNDR: Network detection and response. `https://www.fortinet.com/it/products/network-detection-and-response`, 2022. Accessed: 2026-09-22. (cited on page 27.)

[26] Chuanpu Fu, Qi Li, Meng Shen, and Ke Xu. Realtime robust malicious traffic detection via frequency domain analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, CCS '21, page 3431–3446. ACM, November 2021. (cited on page 9.)

[27] Sunanda Gamage and Jagath Samarabandu. Deep learning methods in network intrusion detection: A survey and an objective comparison. *Journal of Network and Computer Applications*, 169:102767, 2020. (cited on pages 1, 6, 27, 28, 29, 31, 53.)

[28] S. García, M. Grill, J. Stiborek, and A. Zunino. An empirical comparison of botnet detection methods. *Computers & Security*, 45:100–123, 2014. (cited on page 43.)

[29] Patrik Goldschmidt and Daniela Chudá. Network intrusion datasets: A survey, limitations, and recommendations. *Computers & Security*, 156:104510, 2025. (cited on pages xi, 35, 36, 37, 38, 41, 43.)

[30] Zhitao Gong, Wenlu Wang, and Wei-Shinn Ku. Adversarial and clean data are not twins, 2017. (cited on page 23.)

[31] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. `http://www.deeplearningbook.org`. (cited on page 10.)

[32] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples, 2015. (cited on pages 18, 20.)

[33] Kathrin Grosse, Praveen Manoharan, Nicolas Papernot, Michael Backes, and Patrick McDaniel. On the (statistical) detection of adversarial examples, 2017. (cited on page 23.)

[34] R. Hadsell, S. Chopra, and Y. LeCun. Dimensionality reduction by learning an invariant mapping. In *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*, volume 2, pages 1735–1742, 2006. (cited on page 65.)

[35] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs, 2018. (cited on page 30.)

[36] Dan Hendrycks and Kevin Gimpel. Early methods for detecting adversarial images, 2017. (cited on page 23.)

[37] Jamal Hussain and Vanlalruata Hnamte. Deep learning based intrusion detection system: Software defined network. In *2021 Asian Conference on Innovation in Technology (ASIANCON)*, pages 1–6, 2021. (cited on page 6.)

[38] Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Logan Engstrom, Brandon Tran, and Aleksander Madry. Adversarial examples are not bugs, they are features, 2019. (cited on page 18.)

[39] Ahmad Javaid, Quamar Niyaz, Weiqing Sun, and Mansoor Alam. A deep learning approach for network intrusion detection system. In *EAI Endorsed Transactions on Security and Safety*, volume 3, 12 2015. (cited on page 28.)

[40] Mintong Kang, Dawn Song, and Bo Li. Diffattack: Evasion attacks against diffusion-based adversarial purification, 2024. (cited on pages 23, 31.)

[41] Mohammad Khalooei, Mohammad Mehdi Homayounpour, and Maryam Amirmazlaghani. Layer-wise regularized adversarial training using layers sustainability analysis (lsa) framework, 2022. (cited on page 33.)

[42] Alexey Kurakin, Ian Goodfellow, and Samy Bengio. Adversarial examples in the physical world, 2017. (cited on page 20.)

[43] Kimin Lee, Kibok Lee, Honglak Lee, and Jinwoo Shin. A simple unified framework for detecting out-of-distribution samples and adversarial attacks, 2018. (cited on page 33.)

[44] Xinjie Lin, Gang Xiong, Gaopeng Gou, Zhen Li, Junzheng Shi, and Jing Yu. Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In *Proceedings of the ACM Web Conference 2022*, WWW '22, page 633–642. ACM, April 2022. (cited on page 7.)

[45] Xinjie Lin, Gang Xiong, Gaopeng Gou, Zhen Li, Junzheng Shi, and Jing Yu. Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In *Proceedings of the ACM Web Conference 2022*, WWW '22, page 633–642. ACM, April 2022. (cited on page 30.)

[46] Wai Weng Lo, Siamak Layeghy, Mohanad Sarhan, Marcus Gallagher, and Marius Portmann. E-graphsage: A graph neural network based intrusion detection system for iot. In *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, page 1–9. IEEE, April 2022. (cited on pages 6, 30.)

[47] Mohammad Lotfollahi, Ramin Shirali Hossein Zade, Mahdi Jafari Siavoshani, and Mohammdsadegh Saberian. Deep packet: A novel approach for encrypted traffic classification using deep learning, 2018. (cited on page 29.)

[48] Xingjun Ma, Linxi Jiang, Hanxun Huang, Zejia Weng, James Bailey, and Yu-Gang Jiang. Imbalanced gradients: a subtle cause of overestimated adversarial robustness. *Machine Learning*, 113(5):2301–2326, March 2023. (cited on page 31.)

[49] Xingjun Ma, Bo Li, Yisen Wang, Sarah M. Erfani, Sudanthi Wijewickrema, Grant Schoenebeck, Dawn Song, Michael E. Houle, and James Bailey. Characterizing adversarial subspaces using local intrinsic dimensionality, 2018. (cited on page 33.)

[50] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks, 2019. (cited on pages 9, 20, 22, 32.)

[51] Chengzhi Mao, Ziyuan Zhong, Junfeng Yang, Carl Vondrick, and Baishakhi Ray. Metric learning for adversarial robustness. *CoRR*, abs/1909.00900, 2019. (cited on page 32.)

[52] Jan Hendrik Metzen, Tim Genewein, Volker Fischer, and Bastian Bischoff. On detecting adversarial perturbations, 2017. (cited on page 33.)

[53] Microsoft. What is Microsoft Sentinel? `https://www.microsoft.com/it-it/security/business/siem-and-xdr/microsoft-sentinel-siem`, 2019. Accessed: 2026-09-22. (cited on page 27.)

[54] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. *CoRR*, abs/1802.09089, 2018. (cited on pages 8, 9, 28.)

[55] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, and Pascal Frossard. Deepfool: a simple and accurate method to fool deep neural networks, 2016. (cited on page 21.)

[56] Nour Moustafa and Jill Slay. Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set). In *2015 Military Communications and Information Systems Conference (MilCIS)*, pages 1–6, 2015. (cited on page 39.)

[57] Nathiya N, Aruna S, Nikalyaa S J, and Kavya C. Deep learning based intrusion detection system using gated recurrent neural network with sigmoid activation function. In *2024 Third International Conference on Electrical, Electronics, Information and Communication Technologies (ICEEICT)*, pages 1–8, 2024. (cited on page 29.)

[58] National Cybersecurity Center of Excellence (NCCoE). Artificial intelligence: Adversarial machine learning. `https://www.nccoe.nist.gov/ai/adversarial-machine-learning`, 2024. Accessed: 2026-09-19. (cited on page 19.)

[59] Weili Nie, Brandon Guo, Yujia Huang, Chaowei Xiao, Arash Vahdat, and Anima Anandkumar. Diffusion models for adversarial purification, 2022. (cited on pages 23, 32.)

[60] Palo Alto Networks. Cortex XSIAM: AI-driven security operations platform. `https://www.paloaltonetworks.it/cortex/cortex-xsiam`, 2022. Accessed: 2026-09-22. (cited on page 27.)

[61] Tianyu Pang, Chao Du, Yinpeng Dong, and Jun Zhu. Towards robust detection of adversarial examples, 2018. (cited on pages 33, 64.)

[62] Nicolas Papernot and Patrick McDaniel. Deep k-nearest neighbors: Towards confident, interpretable and robust deep learning, 2018. (cited on page 33.)

[63] Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z. Berkay Celik, and Ananthram Swami. Practical black-box attacks against machine learning, 2017. (cited on page 21.)

[64] Nicolas Papernot, Patrick McDaniel, Xi Wu, Somesh Jha, and Ananthram Swami. Distillation as a defense to adversarial perturbations against deep neural networks, 2016. (cited on pages 21, 31.)

[65] Vern Paxson. Bro: a system for detecting network intruders in real-time. *Computer Networks*, 31(23):2435–2463, 1999. (cited on page 6.)

[66] Sotiris Pelekis, Thanos Koutroubas, Afroditi Blika, Anastasis Berdelis, Evangelos Karakolis, Christos Ntanos, Evangelos Spiliotis, and Dimitris Askounis. Adversarial machine learning: a review of methods, tools, and critical industry sectors. *Artificial Intelligence Review*, 58, 05 2025. (cited on pages 18, 34.)

[67] Samuel Pfrommer, Brendon G. Anderson, and Somayeh Sojoudi. Projected randomized smoothing for certified adversarial robustness, 2023. (cited on page 23.)

[68] David Pujol-Perich, José Suárez-Varela, Albert Cabellos-Aparicio, and Pere Barlet-Ros. Unveiling the potential of graph neural networks for robust intrusion detection, 2021. (cited on page 30.)

[69] Devi Puvvala, Gopichand Madala, Maheendra Kada, and U. Hariharan. Improved network intrusion detection system using deep learning. In *2023 7th International Conference on Electronics, Materials Engineering & Nano-Technology (IEMENTech)*, pages 1–6, 12 2023. (cited on pages 28, 29.)

[70] Aditi Raghunathan, Jacob Steinhardt, and Percy Liang. Certified defenses against adversarial examples, 2020. (cited on page 23.)

[71] Leslie Rice, Eric Wong, and J. Zico Kolter. Overfitting in adversarially robust deep learning, 2020. (cited on page 22.)

[72] Martin Roesch. Snort - lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration*, LISA '99, page 229–238, USA, 1999. USENIX Association. (cited on page 6.)

[73] Kevin Roth, Yannic Kilcher, and Thomas Hofmann. The odds are odd: A statistical test for detecting adversarial examples, 2019. (cited on page 33.)

[74] Swami Sankaranarayanan, Arpit Jain, Rama Chellappa, and Ser Nam Lim. Regularizing deep networks using efficient layerwise adversarial training, 2018. (cited on page 33.)

[75] Mohanad Sarhan, Siamak Layeghy, Nour Moustafa, and Marius Portmann. Towards a standard feature set of NIDS datasets. *CoRR*, abs/2101.11315, 2021. (cited on page 8.)

[76] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *International Conference on Information Systems Security and Privacy*, 2018. (cited on page 41.)

[77] Erfan Shayegani, Md Abdullah Al Mamun, Yu Fu, Pedram Zaree, Yue Dong, and Nael Abu-Ghazaleh. Survey of vulnerabilities in large language models revealed by adversarial attacks, 2023. (cited on page 34.)

[78] Ryan Sheatsley, Blaine Hoak, Eric Pauley, Yohan Beugin, Michael J. Weisman, and Patrick McDaniel. On the robustness of domain constraints, 2021. (cited on page 9.)

[79] Nathan Shone, Tran Nguyen Ngoc, Vu Dinh Phai, and Qi Shi. A deep learning approach to network intrusion detection. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2(1):41–50, 2018. (cited on page 28.)

[80] Insoo Sohn. Deep belief network based intrusion detection techniques: A survey. *Expert Systems with Applications*, 167:114170, 2021. (cited on pages 1, 7, 27.)

[81] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks, 2014. (cited on pages 17, 18, 21, 32.)

[82] Sara Taheri, Mahalakshmi Sabanayagam, Debarghya Ghoshdastidar, and Majid Zamani. Robustness certificates for neural networks against data poisoning and evasion attacks. *IEEE Open Journal of Control Systems*, 5:303–318, 2026. (cited on page 19.)

[83] Apostol Vassilev, Alina Oprea, Alie Fordyce, and Hyrum Anderson. Adversarial machine learning: A taxonomy and terminology of attacks and mitigations, 01 2024. (cited on page 19.)

[84] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. (cited on page 30.)

[85] R. Vinayakumar, Mamoun Alazab, K. P. Soman, Prabaharan Poornachandran, Ameer Al-Nemrat, and Sitalakshmi Venkatraman. Deep learning approach for intelligent intrusion detection system. *IEEE Access*, 7:41525–41550, 2019. (cited on page 28.)

[86] Wei Wang, Ming Zhu, Xuewen Zeng, Xiaozhou Ye, and Yiqiang Sheng. Malware traffic classification using convolutional neural network for representation learning. In *2017 International Conference on Information Networking (ICOIN)*, pages 712–717, 2017. (cited on page 29.)

[87] Yisen Wang, Difan Zou, Jinfeng Yi, James Bailey, Xingjun Ma, and Quanquan Gu. Improving adversarial robustness requires revisiting misclassified examples. In *International Conference on Learning Representations*, 2020. (cited on page 32.)

[88] Tsui-Wei Weng, Huan Zhang, Hongge Chen, Zhao Song, Cho-Jui Hsieh, Duane Boning, Inderjit S. Dhillon, and Luca Daniel. Towards fast computation of certified robustness for relu networks, 2018. (cited on page 22.)

[89] Eric Wong and J. Zico Kolter. Provable defenses against adversarial examples via the convex outer adversarial polytope, 2018. (cited on page 23.)

[90] Cihang Xie, Yuxin Wu, Laurens van der Maaten, Alan Yuille, and Kaiming He. Feature denoising for improving adversarial robustness, 2019. (cited on page 33.)

[91] Yong Xie, Weijie Zheng, Hanxun Huang, Guangnan Ye, and Xingjun Ma. Towards million-scale adversarial robustness evaluation with stronger individual attacks, 2025. (cited on page 31.)

[92] Chuanlong Yin, Yuefei Zhu, Jinlong Fei, and Xinzheng He. A deep learning approach for intrusion detection using recurrent neural networks. *IEEE Access*, 5:21954–21961, 2017. (cited on page 29.)

[93] Hongyang Zhang, Yaodong Yu, Jiantao Jiao, Eric P. Xing, Laurent El Ghaoui, and Michael I. Jordan. Theoretically principled trade-off between robustness and accuracy, 2019. (cited on page 32.)

[94] Chao Zhou, Yuan-Gen Wang, Zi jia Wang, and Xiangui Kang. l_p-norm distortion-efficient adversarial attack, 2024. (cited on page 31.)

[95] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models, 2023. (cited on page 34.)